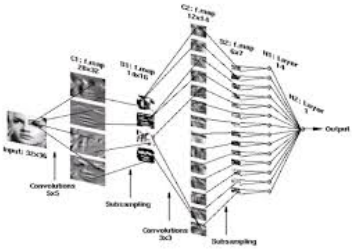
Mathematics of Machine Learning

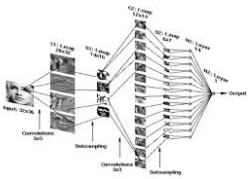
Introduction



Course Details and Topics

Outline

1. **Why are you doing this course?**
2. What we assume that you know



Orientation

- This courses addresses three cohorts of students:
  - Third year Artificial Intelligence Students
  - MSc in Machine Learning Students
  - Volunteers
- It is dual coded (AICE3001/AICE6002), with subtle difference in expectations reflected in slightly different evaluation
- Notes are on Moodle and

  https://ecs-vlc.github.io/aice3001/

Why are We Teaching You This?

- In a world where LLMs can do everything, why bother with learning mathematics?
- You need to learn to trust your own judgement
- LLMs can help us a lot, but if you don't trust yourself you are disempowered
- This course is aimed at building up trust in your understanding of how machine learning works

Why Mathematics?

- Learning machines are notoriously difficult to understand
- Mathematics allows us to reason rigorously
- Posing question mathematically and seeking mathematical solutions provide allow us to build a strong intuition—we can't cheat that much
- Mathematics also allow us to describe the world precisely
- It also provides an arsenal of tools that turn out to be hugely useful

Cracking the Code

- Mathematics is a language that takes time to master
- For some it is easy, for others it will be a struggle
- But given determination everyone can crack the code and understand what the mathematics is saying
- This comes from engaging and working with maths—this course has tutorials that should be considered compulsory not because there are marks attached, but because without doing the work you aren't going to crack the code

Starting from Different Places

- You all start from different places
- Some of you may have a degree in mathematics
- For others it may have been you least favourite subject
- I invite strong mathematicians to become even stronger—the best way to learn is through teaching others
- For students starting from a weak base, judge yourselves by your progress, not by comparing yourself to others

The Golden Rule of Mathematics

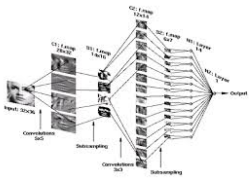
- **Understand every step**
- When you don't understand a step, stop, break the problem down, ask someone
- Mathematics uses a lot of short-cuts, which speeds-up calculations, but you need to be able to justify every step
- It takes bravery to admit you don't understand and insist that you will
- Asking "stupid" questions isn't stupid it is how we all learn

The Journey

- We are going to visit lots of aspects of mathematics that have proved fruitful in machine learning
- Some of this will be very concrete, while in other cases by look at a higher level of abstraction we can generalise methods across different application areas
- We will cover, calculus, vector spaces, convexity, optimisation, entropy, probability, statistics and much else besides
- The journey will pay you off these are the foundations of most of machine learning and will give you super-powers that are hard to acquire without mathematics

Outline

1. Why are you doing this course?
2. **What we assume that you know**



What we Assume you Know

- You should all have A-level mathematics and have done some mathematics at Uni
- You should know what a function is, e.g. $f(x) = x^2 + 3$
- You should be able to do simple algebraic manipulations
- And some elementary calculus

$$f'(x) = \frac{\mathrm{d}(x^2+3)}{\mathrm{d}x} = 2x$$

Vectors

- Ordinary vectors (what mathematicians call elements of $\mathbb{R}^n$) are much used in machine learning and should be familiar
- We will use bold symbols to represent **column vectors**, e.g.

$$\boldsymbol{x} = \begin{pmatrix} 1.2 \\ 2.3 \\ 4.7 \end{pmatrix}$$

- We denote **row vectors** as the **transpose** of column vectors $\boldsymbol{x}^\mathsf{T} = (1.2, 2.3, 4.7)$
- We care whether vectors are row of column vectors as it allows us to write equation involving vectors succinctly and unambiguously

Matrices

- Matrices are blocks of number that define linear mappings between vectors, e.g. $\mathbf{M} \in \mathbb{R}^{2\times 3}$

$$\mathbf{M} = \begin{pmatrix} 1.1 & -2.1 & 6.2 \\ 9.3 & 3.1 & -4.4 \end{pmatrix}$$

- An $m \times n$ matrix (i.e. $\mathbf{M} \in \mathbb{R}^{m\times n}$) has m rows and n columns
- An $m \times n$ matrix acting on a vector in $\mathbb{R}^n$ transforms it to $\mathbb{R}^m$ vector, e.g.

$$\mathbf{M}\boldsymbol{x} = \begin{pmatrix} 1.1 & -2.1 & 6.2 \\ 9.3 & 3.1 & -4.4 \end{pmatrix} \begin{pmatrix} 1.2 \\ 2.3 \\ 4.7 \end{pmatrix} = \begin{pmatrix} 28.63 \\ -2.39 \end{pmatrix}$$

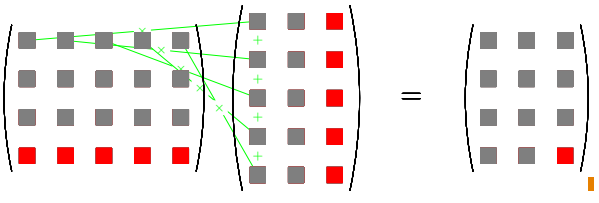
Composition of Mappings

- Matrices, $\mathbf{A}$, $\mathbf{B}$, etc. represent mappings between spaces
- Provided the matrices are the correct size we can compose these mappings, e.g. if $\mathbf{A} \in \mathbb{R}^{\ell\times m}$ and $\mathbf{B} \in \mathbb{R}^{m\times n}$ then $\mathbf{AB}$ represents the effect of
  - first applying $\mathbf{B}$ to vectors in $\mathbb{R}^n$ to obtain vectors in $\mathbb{R}^m$
  - and then applying $\mathbf{A}$ to get vectors in $\mathbb{R}^\ell$

$$\mathbf{AB}\boldsymbol{x} = \mathbf{A}(\mathbf{B}\boldsymbol{x})$$

Matrix Multiplication

- The combined mapping $\mathbf{AB}$ can be represprested by an $\ell \times n$ matrix $\mathbf{C}$ which we can compute using matrix multiplcation (rows times columns)



- Matrix multiplication is associative: $(\mathbf{AB})\mathbf{C} = \mathbf{A}(\mathbf{BC})$
- But in general not commutative $\mathbf{AB} \neq \mathbf{BA}$

Transpose

- We often encounter transposes of a matrix $\mathbf{M}^\mathsf{T}$ where we have exchanged the rows and columns: $[\mathbf{M}^\mathsf{T}]_{ij} = [\mathbf{M}]_{ji}$
- Clear $(\mathbf{M}^\mathsf{T})^\mathsf{T} = \mathbf{M}$
- It is simple question of book keeping to show that

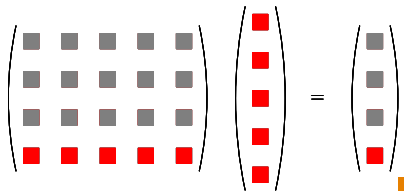
$$(\mathbf{AB})^\mathsf{T} = \mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}$$

- For a symmetric matrix $\mathbf{M} = \mathbf{M}^\mathsf{T}$
- Since 1×1 matrices are always symmetric

$$\boldsymbol{a}^\mathsf{T}\boldsymbol{b} = (\boldsymbol{a}^\mathsf{T}\boldsymbol{b})^\mathsf{T} = \boldsymbol{b}^\mathsf{T}\boldsymbol{a}$$
$$\boldsymbol{a}^\mathsf{T}\mathbf{M}\boldsymbol{b} = \boldsymbol{b}^\mathsf{T}\mathbf{M}^\mathsf{T}\boldsymbol{a}$$

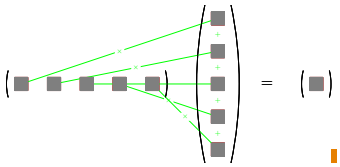
Matrix-Vector Multiplication

- We can treat a column vector as a $n \times 1$ matrix
- Then matrix-vector multiplication can be treated the same as matrix multiplication



Inner Products

- The dot or inner-product between two vectors with the same number of elements, can be written $\boldsymbol{a}^\mathsf{T}\boldsymbol{b}$

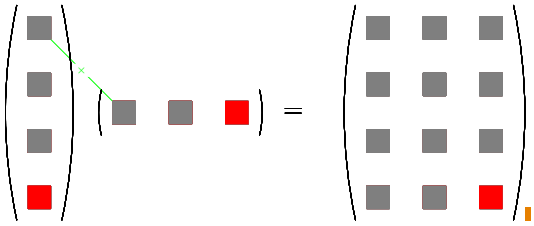


- Inner products encode angles between vectors

$$\boldsymbol{a}^\mathsf{T}\boldsymbol{b} = \|\boldsymbol{a}\|\|\boldsymbol{b}\|\cos(\theta)$$

Outer Products

- We can also define the outer-produce between any size vectors, $\boldsymbol{a}\boldsymbol{b}^\mathsf{T}$



- Notice that we need to know whether we are dealing with row or column vectors

Norms

- Norms are non-negative numbers associated with the "size" of a vector
- The norm you should be familiar with is the **Euclidean Norm** or 2-norm

$$\|\boldsymbol{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} = \sqrt{x_1^2 + x_2^2 + \ldots + x_n^2} = \sqrt{\boldsymbol{x}^\mathsf{T}\boldsymbol{x}}$$

- We will see there exist other norms (using different ways of measuring size)
- The Euclidean norm is intimately related to the inner-product, we will see other examples of this for different vector spaces

Matrices as Linear Mappings

- Matrices represent mappings between vectors

$$\boldsymbol{b} = \mathbf{M}\boldsymbol{a}$$

- The elements of $\boldsymbol{b}$ are **linear sums** of the elements of $\boldsymbol{a}$

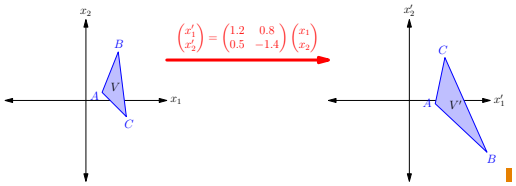
$$b_i = \sum_j M_{ij}a_j = M_{i1}a_1 + M_{i2}a_2 + \cdots + M_{in}a_n$$

- Two important consequences of this linearity are for any real c

$$\mathbf{M}(c\boldsymbol{a}) = c(\mathbf{M}\boldsymbol{a})$$
$$\mathbf{M}(\boldsymbol{a}+\boldsymbol{b}) = (\mathbf{M}\boldsymbol{a}) + (\mathbf{M}\boldsymbol{b})$$

Action Of Matrices on Volumes

- For a closed set of points corresponding to a volume, V in $\mathbb{R}^n$, treating each point as a vector, we can as what happens to the volume after mapping the vector by a matrix



- For square matrices, $\mathbf{M}$, the determinant is equal to $\det(\mathbf{M}) = \pm V'/V$
- The sign tells us if there has been a reflection

Determinants

- A consequence of the fact that $\mathbf{AB}$ denotes the composition, apply mapping $\mathbf{B}$ then apply mapping $\mathbf{A}$

$$\det(\mathbf{AB}) = \det(\mathbf{A}) \times \det(\mathbf{B})$$

- Determinants are only defined for square matrices, as non-square matrices project into different size spaces, where the volumes cannot be compared
- **Singular matrices** projects all points onto a lower dimensional space, so have determinant 0

Multivariate Functions

- In machine learning we heavily use multivariate functions $f : \mathbb{R}^n \to \mathbb{R}^m$
- Often our functions will be scalared value $f : \mathbb{R}^n \to \mathbb{R}$
- E.g. a learning machine for regression might be $\hat{f}(\boldsymbol{x},\boldsymbol{w})$ or we might have a loss function $L(\boldsymbol{w})$
- We say a function f is linear if

$$f(c\boldsymbol{x}) = cf(\boldsymbol{x}) \qquad f(\boldsymbol{x}+\boldsymbol{y}) = f(\boldsymbol{x}) + f(\boldsymbol{y})$$

Gradients

- The gradient of a function $f : \mathbb{R}^n \to \mathbb{R}$ at a point $\boldsymbol{x}^*$ is defined as the column vector

$$\nabla f(\boldsymbol{x}^*) = \begin{pmatrix} \frac{\partial f(\boldsymbol{x}^*)}{\partial x_1} \\ \frac{\partial f(\boldsymbol{x}^*)}{\partial x_2} \\ \vdots \\ \frac{\partial f(\boldsymbol{x}^*)}{\partial x_n} \end{pmatrix}$$

- The gradient vector is orthogonal to the contour plane and points in the direction in which the function increases fastest

Probability

- We will have a lot to say about probability later in this course
- To reason about events with uncertain outcomes we introduce **random variables**
- Random variables corresponds to objects that we assign numbers to depending on the outcome of a random event
- E.g. we might assign 0 to the outcome of tossing a tail and 1 for a head—this is an example of a discrete random variable
- A continuous random variable might be the temperature at 12 noon tomorrow in Hyde park

Probability Distributions

- For discrete events, we can define a **probability mass function** $\mathbb{P}(X = x)$ that the random variable takes the value x
- This is a number between 0 and 1 with $\sum_{x\in\mathcal{X}} \mathbb{P}(X = x) = 1$
- For continuous events the probability of X taking a particular value x is zero
- Instead we consider the probability density $f_X(x)$ which we can define as

$$f_X(x) = \lim_{\delta x \to 0} \frac{\mathbb{P}(x < X \leq x + \delta x)}{\delta x}$$

Expectations

- One of the most useful quantities in dealing with random numbers is to look at the average value, aka as the **expectation**, for some function $g(X)$ of the random variable

$$\mathbb{E}[g(X)] = \begin{cases} \sum_{x\in\mathcal{X}} \mathbb{P}(X = x) g(x) \\ \int f_X(x) g(x) \mathrm{d}x \end{cases}$$

- Expectations are linear operators

$$\mathbb{E}[cg(X)] = c\mathbb{E}[g(x)]$$
$$\mathbb{E}[g(X) + h(X)] = \mathbb{E}[g(X)] + \mathbb{E}[h(X)]$$

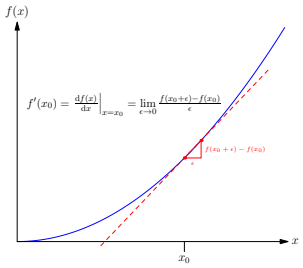
- The expectation of a constant is itself $\mathbb{E}[c] = c$

Lesson

- I've covered material that we assume you know
- If you don't, you are going to struggle following any machine learning course, so grab a maths book and revise this
- We will revisit most of these topics to gain a better understanding, but you will be lost in the other machine learning modules if you are not familiar with everything I've covered
- There are exercises we will go through in the tutorial session where you can test your proficiency

Mathematics of Machine Learning

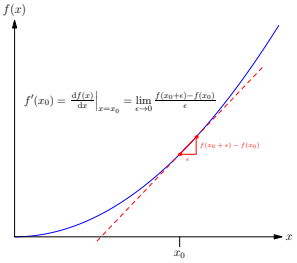
Differential Calculus



Differentiation, product and chain rules, vectors and matrices

Outline

1. **Why Calculus?**
2. Differentiation
3. Vector Calculus
4. Matrix Calculus



Why Calculus?

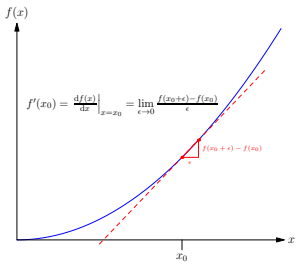
- Calculus is a fundamental tool of mathematical analysis
- In machine learning differentiation is fundamental tool in optimisation
- Integration is an essential tool in taking expectations over continuous distributions
- Both differentiation and integration crop up elsewhere

Back to Basics

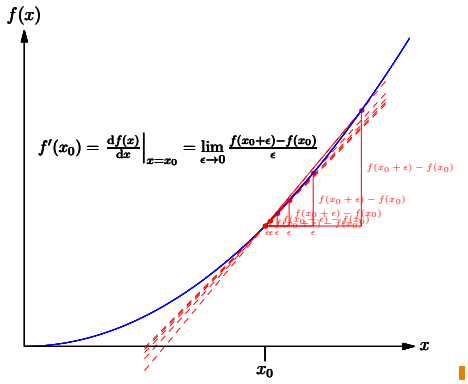
- You have all done A-level maths so should be familiar with the rules of calculus
- But, it is easy to forget the rules and sometimes we use quite sophisticated tricks
- Although the sophisticated tricks really speed up calculations, it pays to be able to understand where these tricks come from

Outline

1. Why Calculus?
2. **Differentiation**
3. Vector Calculus
4. Matrix Calculus



Differentiation



Polynomials

- $f(x) = x^2$

$$\frac{\mathrm{d}x^2}{\mathrm{d}x} = \lim_{\epsilon\to 0} \frac{(x+\epsilon)^2 - x^2}{\epsilon} = \lim_{\epsilon\to 0} \frac{(x^2+2\epsilon x+\epsilon^2) - x^2}{\epsilon}$$
$$= \lim_{\epsilon\to 0} 2x + \epsilon = 2x$$

- $(x+\epsilon)^n = (x+\epsilon)(x+\epsilon)\cdots(x+\epsilon) = x^n + n\epsilon x^{n-1} + O(\epsilon^2)$

$$\frac{\mathrm{d}x^n}{\mathrm{d}x} = \lim_{\epsilon\to 0} \frac{(x+\epsilon)^n - x^n}{\epsilon} = \lim_{\epsilon\to 0} nx^{n-1} + O(\epsilon) = nx^{n-1}$$

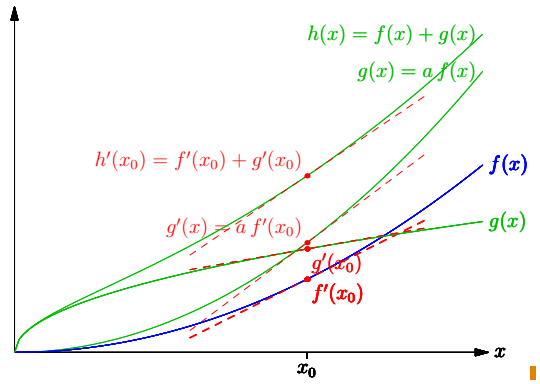
Linearity of derivatives

- Consider the derivative of $af(x) + bg(x)$

$$\frac{\mathrm{d}(af(x)+bg(x))}{\mathrm{d}x} = \lim_{\epsilon\to 0} \frac{(af(x+\epsilon)+bg(x+\epsilon)) - (af(x)+bg(x))}{\epsilon}$$
$$= \lim_{\epsilon\to 0} \frac{a(f(x+\epsilon)-f(x)) + b(g(x+\epsilon)-g(x))}{\epsilon}$$
$$= \lim_{\epsilon\to 0} a\left(\frac{f(x+\epsilon)-f(x)}{\epsilon}\right) + b\left(\frac{g(x+\epsilon)-g(x)}{\epsilon}\right)$$
$$= af'(x) + bg'(x)$$

- **Differentiation is a linear operation!**

Linearity in Pictures



Product Rule

- Recall $f(x+\epsilon) = f(x) + \epsilon f'(x) + O(\epsilon^2)$
- If $h(x) = f(x)g(x)$

$$\begin{aligned} h'(x) &= \lim_{\epsilon\to 0} \frac{f(x+\epsilon)g(x+\epsilon) - f(x)g(x)}{\epsilon} \\ &= \lim_{\epsilon\to 0} \frac{\big(f(x) + \epsilon f'(x) + O(\epsilon^2)\big)\big(g(x) + \epsilon g'(x) + O(\epsilon^2)\big) - f(x)g(x)}{\epsilon} \\ &= \lim_{\epsilon\to 0} \frac{\epsilon(f'(x)g(x) + f(x)g'(x)) + O(\epsilon^2)}{\epsilon} = f'(x)g(x) + f(x)g'(x) \end{aligned}$$

- This is the **product rule**

Chain Rule

- Recall $f(x+\epsilon) = f(x) + \epsilon f'(x) + O(\epsilon^2)$
- Let $h(x) = f(g(x))$
- Then

$$\begin{aligned} h(x+\epsilon) &= f(g(x+\epsilon)) = f\big(g(x) + \epsilon g'(x) + O(\epsilon^2)\big) \\ &= f(g(x)) + \epsilon g'(x) f'(g(x)) + O(\epsilon^2) \end{aligned}$$

- Thus

$$h'(x) = \lim_{\epsilon\to 0} \frac{h(x+\epsilon) - h(x)}{\epsilon} = g'(x) f'(g(x))$$

- This is the famous **chain rule**. Together with the product rule it means you can differentiate almost everything

More on chain rules

- We can also write the chain rule as

$$\frac{\mathrm{d}f(g(x))}{\mathrm{d}x} = \frac{\mathrm{d}f(g)}{\mathrm{d}g}\frac{\mathrm{d}g(x)}{\mathrm{d}x}$$

- Sometimes this is neater or easier to remember

$$\begin{aligned} \frac{\mathrm{d}\mathrm{e}^{\cos(x^2)}}{\mathrm{d}x} &= \frac{\mathrm{d}\mathrm{e}^{\cos(x^2)}}{\mathrm{d}\cos(x^2)}\frac{\mathrm{d}\cos(x^2)}{\mathrm{d}x^2}\frac{\mathrm{d}x^2}{\mathrm{d}x} \\ &= \mathrm{e}^{\cos(x^2)}\big(-\sin(x^2)\big)2x \\ &= -2x\sin(x^2)\mathrm{e}^{\cos(x^2)} \end{aligned}$$

Inverse functions

- Suppose $g(y) = f^{-1}(y)$ is the inverse of $f(x)$ in the sense that $g(f(x)) = f^{-1}(f(x)) = x$
- Using the chain rule

$$\frac{\mathrm{d}g(f(x))}{\mathrm{d}x} = f'(x)g'(f(x)) = 1$$

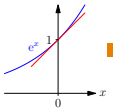
since $g(f(x)) = x$

- So $g'(f(x)) = 1/f'(x)$
- Writing $y = f(x)$ so that $x = f^{-1}(y) = g(y)$ we find $g'(y) = 1/f'(g(y))$ that is

$$\frac{\mathrm{d}g(y)}{\mathrm{d}y} = \frac{1}{f'(g(y))} \qquad \frac{\mathrm{d}f^{-1}(y)}{\mathrm{d}y} = \frac{1}{f'(f^{-1}(y))}$$

Exponentials

- Note that $a^{b+c} = a^b a^c$ (that is we multiply a together $b+c$ times)
- Now $\mathrm{e}^\epsilon \approx (1+\epsilon)$



- But $\mathrm{e}^{x+\epsilon} = \mathrm{e}^x\mathrm{e}^\epsilon = \mathrm{e}^x(1+\epsilon+O(\epsilon^2)) = \mathrm{e}^x + \epsilon\mathrm{e}^x + O(\epsilon^2)$

$$\frac{\mathrm{d}\mathrm{e}^x}{\mathrm{d}x} = \lim_{\epsilon\to 0}\frac{\mathrm{e}^{x+\epsilon} - \mathrm{e}^x}{\epsilon} = \lim_{\epsilon\to 0}\frac{\epsilon\mathrm{e}^x + O(\epsilon^2)}{\epsilon} = \mathrm{e}^x$$

Functions of Exponentials

- What about $f(x) = \mathrm{e}^{cx}$

$$\frac{\mathrm{d}\mathrm{e}^{cx}}{\mathrm{d}x} = \frac{\mathrm{d}\mathrm{e}^{cx}}{\mathrm{d}cx}\frac{\mathrm{d}cx}{\mathrm{d}x} = c\mathrm{e}^{cx}$$

- More generally using the chain rule

$$\frac{\mathrm{d}\mathrm{e}^{g(x)}}{\mathrm{d}x} = g'(x)\mathrm{e}^{g(x)}$$

- Also $a^{bc} = (a^b)^c$ (that is we multiply a together $b \times c$ times)

$$\frac{\mathrm{d}a^x}{\mathrm{d}x} = \frac{\mathrm{d}(\mathrm{e}^{\ln(a)})^x}{\mathrm{d}x} = \frac{\mathrm{d}\mathrm{e}^{\ln(a)x}}{\mathrm{d}x} = \ln(a)\mathrm{e}^{\ln(a)x} = \ln(a)a^x$$

Natural Logarithms

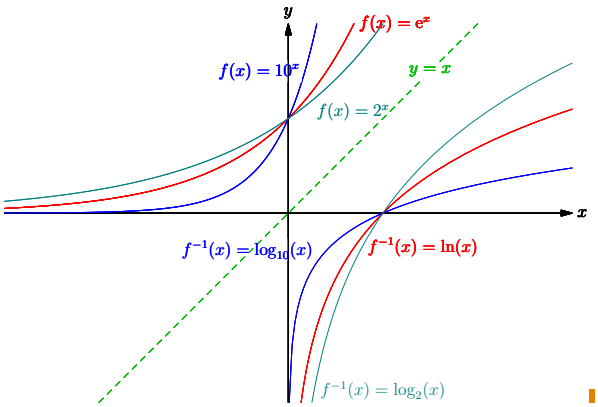
- The natural logarithm is defined as the inverse of e^x

$$\ln(\mathrm{e}^x) = x \qquad \mathrm{e}^{\ln(y)} = y$$

- Recall that if $g(y) = f^{-1}(y)$ then $g'(y) = 1/f'(g(y))$
- Consider $g(y) = \ln(y)$ and $f(x) = \mathrm{e}^x$ (with $f'(x) = \mathrm{e}^x$)

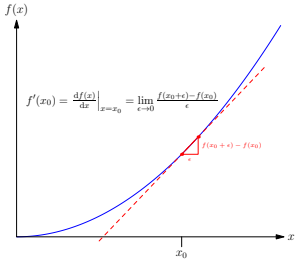
$$\frac{\mathrm{d}\ln(y)}{\mathrm{d}y} = \frac{1}{\mathrm{e}^{\ln(y)}} = \frac{1}{y}$$

Exponentials and Logarithms



Outline

1. Why Calculus?
2. Differentiation
3. **Vector Calculus**
4. Matrix Calculus



Derivatives in High Dimensions

- When working with functions $f : \mathbb{R}^n \to \mathbb{R}$ in many dimensions then there will typically be different derivative in different directions
- To compute the derivative in a direction $\boldsymbol{u} \in \mathbb{R}^n$ (where $\|\boldsymbol{u}\| = 1$) at a point $\boldsymbol{x} \in \mathbb{R}^n$ we use

$$\partial_{\boldsymbol{u}} F(\boldsymbol{x}) = \lim_{\epsilon \to 0} \frac{f(\boldsymbol{x} + \epsilon \boldsymbol{u}) - f(\boldsymbol{x})}{\epsilon}$$

- If $\boldsymbol{u} = \boldsymbol{\delta}_i = (0,\ldots,0,1,0,\ldots,0)^\mathsf{T}$ (i.e. $u_i = 1$) then

$$\frac{\partial f(\boldsymbol{x})}{\partial x_i} = \lim_{\epsilon \to 0} \frac{f(\boldsymbol{x} + \epsilon \boldsymbol{\delta}_i) - f(\boldsymbol{x})}{\epsilon}$$

Taylor

- If we expand $f(\boldsymbol{x} + \epsilon \boldsymbol{u})$ to first order in ϵ

$$f(\boldsymbol{x} + \epsilon \boldsymbol{u}) = f(\boldsymbol{x}) + \epsilon \boldsymbol{u}^\mathsf{T} \boldsymbol{g}(\boldsymbol{x}) + O(\epsilon^2)$$

  then $g_i(\boldsymbol{x}) = \frac{\partial f(\boldsymbol{x})}{\partial x_i}$
- Recall we defined the vector of first order derivatives of $f(\boldsymbol{x})$ to be the gradient

$$\boldsymbol{\nabla} f(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f(\boldsymbol{x})}{\partial x_1} \\ \frac{\partial f(\boldsymbol{x})}{\partial x_2} \\ \vdots \\ \frac{\partial f(\boldsymbol{x})}{\partial x_n} \end{pmatrix}$$

- Thus

$$f(\boldsymbol{x} + \epsilon \boldsymbol{u}) = f(\boldsymbol{x}) + \epsilon \boldsymbol{u}^\mathsf{T} \boldsymbol{\nabla} f(\boldsymbol{x}) + O(\epsilon^2)$$

  This is the start of the high-dimensional Taylor expansion

Computing Gradients 1

- We can compute the gradient by writing out $f(\boldsymbol{x})$ componentwise and performing the partial derivative with respect to x_i

$$\boldsymbol{\nabla} \boldsymbol{w}^\mathsf{T} \mathbf{M} \boldsymbol{w} = \begin{pmatrix} \frac{\partial}{\partial w_1} \\ \frac{\partial}{\partial w_2} \\ \frac{\partial}{\partial w_3} \\ \vdots \end{pmatrix} \sum_{i,j} w_i M_{ij} w_j = \begin{pmatrix} \sum_j M_{1j} w_j + \sum_i w_i M_{i1} \\ \sum_j M_{2j} w_j + \sum_i w_i M_{i2} \\ \sum_j M_{3j} w_j + \sum_i w_i M_{i3} \\ \vdots \end{pmatrix}$$
$$= \mathbf{M}\boldsymbol{w} + \mathbf{M}^\mathsf{T}\boldsymbol{w} = (\mathbf{M} + \mathbf{M}^\mathsf{T})\boldsymbol{w}$$

- It is tedious to compute these things component-wise, but when you need to understand what is going on then go back to the basics

Computing Gradients 2

- A slicker way is just to expand $f(\boldsymbol{x} + \epsilon \boldsymbol{u})$
- Consider $f(\boldsymbol{x}) = \boldsymbol{x}^\mathsf{T} \mathbf{M} \boldsymbol{x} + \boldsymbol{a}^\mathsf{T} \boldsymbol{x}$

$$\begin{aligned} f(\boldsymbol{x} + \epsilon \boldsymbol{u}) &= (\boldsymbol{x} + \epsilon \boldsymbol{u})^\mathsf{T} \mathbf{M} (\boldsymbol{x} + \epsilon \boldsymbol{u}) + \boldsymbol{a}^\mathsf{T} (\boldsymbol{x} + \epsilon \boldsymbol{u}) \\ &= f(\boldsymbol{x}) + \epsilon \left(\boldsymbol{u}^\mathsf{T} \mathbf{M} \boldsymbol{x} + \boldsymbol{x}^\mathsf{T} \mathbf{M} \boldsymbol{u} + \boldsymbol{a}^\mathsf{T} \boldsymbol{u} \right) + O(\epsilon^2) \\ &= f(\boldsymbol{x}) + \epsilon \boldsymbol{u}^\mathsf{T} \left(\mathbf{M} \boldsymbol{x} + \mathbf{M}^\mathsf{T} \boldsymbol{x} + \boldsymbol{a} \right) + O(\epsilon^2) \end{aligned}$$

  using $\boldsymbol{x}^\mathsf{T} \mathbf{M} \boldsymbol{u} = \boldsymbol{u}^\mathsf{T} \mathbf{M}^\mathsf{T} \boldsymbol{x}$ and $\boldsymbol{a}^\mathsf{T} \boldsymbol{u} = \boldsymbol{u}^\mathsf{T} \boldsymbol{a}$
- But $f(\boldsymbol{x} + \epsilon \boldsymbol{u}) = f(\boldsymbol{x}) + \epsilon \boldsymbol{u}^\mathsf{T} \boldsymbol{\nabla} f(\boldsymbol{x}) + O(\epsilon^2)$ so

$$\boldsymbol{\nabla} f(\boldsymbol{x}) = \mathbf{M} \boldsymbol{x} + \mathbf{M}^\mathsf{T} \boldsymbol{x} + \boldsymbol{a}$$

Function Composition

- The gradient tells us about the change in a function value when we perturb an input $\boldsymbol{x}$ by a small amount in the directions $\boldsymbol{u}$

$$f(\boldsymbol{x} + \epsilon \boldsymbol{u}) = f(\boldsymbol{x}) + \epsilon \boldsymbol{u}^\mathsf{T} \boldsymbol{\nabla} f(\boldsymbol{x}) + O(\epsilon^2)$$

- What happens when we compose functions: $h(\boldsymbol{x}) = f(\boldsymbol{g}(\boldsymbol{x}))$ Note that,

$$x \in \mathbb{R}^n \qquad \boldsymbol{g} : \mathbb{R}^n \to \mathbb{R}^m \qquad f : \mathbb{R}^m \to \mathbb{R} \qquad h : \mathbb{R}^n \to \mathbb{R}$$

- The vector valued function $\boldsymbol{g}(\boldsymbol{x})$ can be seen as m independent function $\boldsymbol{g}(\boldsymbol{x}) = \begin{pmatrix} g_1(\boldsymbol{x}) \\ g_2(\boldsymbol{x}) \\ \vdots \\ g_m(\boldsymbol{x}) \end{pmatrix}$

Jacobians

- Perturbing $\boldsymbol{x}$ by $\epsilon \boldsymbol{u}$

$$g_i(\boldsymbol{x} + \epsilon \boldsymbol{u}) = g_i(\boldsymbol{x}) + \epsilon \boldsymbol{u}^\mathsf{T} \boldsymbol{\nabla} g_i(\boldsymbol{x}) + O(\epsilon^2)$$

  so that

$$\boldsymbol{g}(\boldsymbol{x} + \epsilon \boldsymbol{u}) = \boldsymbol{g}(\boldsymbol{x}) + \epsilon \mathbf{J} \boldsymbol{u} + O(\epsilon^2)$$

  where $\mathbf{J}$ is an $m \times n$ matrix with components $J_{ij} = \frac{\partial g_i(\boldsymbol{x})}{\partial x_j}$
- Note that $\mathbf{J}\boldsymbol{u}$ is a column vector of size m
- Ignoring terms of order ϵ^2

$$\begin{aligned} h(\boldsymbol{x} + \epsilon \boldsymbol{u}) &= f(\boldsymbol{g}(\boldsymbol{x} + \epsilon \boldsymbol{u})) = f(\boldsymbol{g}(\boldsymbol{x}) + \epsilon \mathbf{J} \boldsymbol{u}) \\ &= f(\boldsymbol{g}(\boldsymbol{x})) + \epsilon \boldsymbol{u}^\mathsf{T} \mathbf{J}^\mathsf{T} \boldsymbol{\nabla}_{\boldsymbol{g}} f(\boldsymbol{g}(\boldsymbol{x})) \end{aligned}$$

Chain Rule in High Dimensions

- Using $h(\boldsymbol{x}) = f(\boldsymbol{g}(\boldsymbol{x}))$

$$h(\boldsymbol{x} + \epsilon \boldsymbol{u}) = h(\boldsymbol{x}) + \epsilon \boldsymbol{u}^\mathsf{T} \mathbf{J}^\mathsf{T} \nabla_{\boldsymbol{g}} f(\boldsymbol{g}(\boldsymbol{x}))$$

- We get that

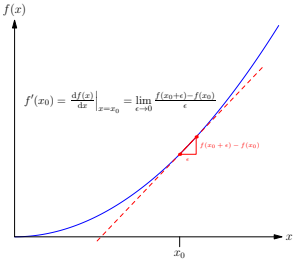
$$\nabla_{\boldsymbol{x}} h(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} f(\boldsymbol{g}(\boldsymbol{x})) = \mathbf{J}^\mathsf{T} \nabla_{\boldsymbol{g}} f(\boldsymbol{g}(\boldsymbol{x}))$$

- Componentwise

$$\frac{\partial h(\boldsymbol{x})}{\partial x_i} = \frac{\partial f(\boldsymbol{g}(\boldsymbol{x}))}{\partial x_i} = \sum_{j=1}^{m} \frac{\partial g_j(\boldsymbol{x})}{\partial x_i} \frac{\partial f(\boldsymbol{g}(\boldsymbol{x}))}{\partial g_j}$$

Outline

1. Why Calculus?
2. Differentiation
3. Vector Calculus
4. **Matrix Calculus**



Differentiating Matrices

- Often we have loss functions with respect to a matrix $\boldsymbol{W}$, e.g.

$$L(\boldsymbol{W}) = (\boldsymbol{a}^\mathsf{T} \boldsymbol{W} \boldsymbol{b} - c)^2$$

- We might want to find the minimum with respect to $\boldsymbol{W}$
- This occurs at a point $\boldsymbol{W}^*$ where $L(\boldsymbol{W})$ does not increase as we change $\boldsymbol{W}$ in any way
- That is, we seek a $\boldsymbol{W}^*$ such that, for any matrices $\mathbf{U}$

$$L(\boldsymbol{W}^* + \epsilon \mathbf{U}) - L(\boldsymbol{W}^*) = O(\epsilon^2)$$

Generalised Gradient

- We can generalise the idea of gradient to matrices

$$\frac{\partial L(\boldsymbol{W})}{\partial \boldsymbol{W}} = \begin{pmatrix} \frac{\partial L(\boldsymbol{W})}{\partial W_{11}} & \frac{\partial L(\boldsymbol{W})}{\partial W_{12}} & \cdots & \frac{\partial L(\boldsymbol{W})}{\partial W_{1m}} \\ \frac{\partial L(\boldsymbol{W})}{\partial W_{21}} & \frac{\partial L(\boldsymbol{W})}{\partial W_{22}} & \cdots & \frac{\partial L(\boldsymbol{W})}{\partial W_{2m}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial L(\boldsymbol{W})}{\partial W_{n1}} & \frac{\partial L(\boldsymbol{W})}{\partial W_{n2}} & \cdots & \frac{\partial L(\boldsymbol{W})}{\partial W_{nm}} \end{pmatrix}$$

- From an identical argument we used for vectors

$$L(\boldsymbol{W} + \epsilon \mathbf{U}) = L(\boldsymbol{W}) + \epsilon \mathrm{tr} \mathbf{U}^\mathsf{T} \frac{\partial L(\boldsymbol{W})}{\partial \boldsymbol{W}} + O(\epsilon^2)$$

where

$$\mathrm{tr} \mathbf{U}^\mathsf{T} \mathbf{G} = \sum_i \left[\mathbf{U}^\mathsf{T} \mathbf{G}\right]_{ii} = \sum_{ij} U_{ji} G_{ji} = \sum_{ij} U_{ij} G_{ij} = \langle \mathbf{U}, \mathbf{G} \rangle$$

Example

- Suppose

$$L(\boldsymbol{W}) = \left(\boldsymbol{a}^\mathsf{T} \boldsymbol{W} \boldsymbol{b} - c\right)^2$$

then

$$\begin{aligned} L(\boldsymbol{W} + \epsilon \mathbf{U}) &= \left(\boldsymbol{a}^\mathsf{T} (\boldsymbol{W} + \epsilon \mathbf{U}) \boldsymbol{b} - c\right)^2 = \left(\boldsymbol{a}^\mathsf{T} \boldsymbol{W} \boldsymbol{b} + \epsilon \boldsymbol{a}^\mathsf{T} \mathbf{U} \boldsymbol{b} - c\right)^2 \\ &= L(\boldsymbol{W}) + 2\epsilon \left(\boldsymbol{a}^\mathsf{T} \boldsymbol{W} \boldsymbol{b} - c\right)\left(\boldsymbol{a}^\mathsf{T} \mathbf{U} \boldsymbol{b}\right) + O(\epsilon^2) \end{aligned}$$

- Now

$$\boldsymbol{a}^\mathsf{T} \mathbf{U} \boldsymbol{b} = \sum_{ij} a_i U_{ij} b_j = \sum_{ij} U_{ji} a_j b_i = \mathrm{tr} \mathbf{U}^\mathsf{T} \boldsymbol{a} \boldsymbol{b}^\mathsf{T}$$

Thus $\frac{\partial L(\boldsymbol{W})}{\partial \boldsymbol{W}} = 2\left(\boldsymbol{a}^\mathsf{T} \boldsymbol{W} \boldsymbol{b} - c\right) \boldsymbol{a} \boldsymbol{b}^\mathsf{T}$

Traces

- The trace of a matrix is the sum of its diagonal elements

$$\mathrm{tr} \mathbf{A} = \mathrm{tr} \mathbf{A}^\mathsf{T} = \sum_i A_{ii}$$

- Clearly $\mathrm{tr} c\mathbf{A} = c \mathrm{tr} \mathbf{A}$
- Also $\mathrm{tr}(\mathbf{A} + \mathbf{B}) = \mathrm{tr} \mathbf{A} + \mathrm{tr} \mathbf{B}$
- We note that

$$\mathrm{tr} \mathbf{A} \mathbf{B} = \sum_{i,j} A_{ij} B_{ji} = \sum_{i,j} B_{ij} A_{ji} = \mathrm{tr} \mathbf{B} \mathbf{A}$$

- It follows that

$$\mathrm{tr} \mathbf{A} \mathbf{B} \mathbf{C} \mathbf{D} = \mathrm{tr} \mathbf{D} \mathbf{A} \mathbf{B} \mathbf{C} = \mathrm{tr} \mathbf{C} \mathbf{D} \mathbf{A} \mathbf{B} = \mathrm{tr} \mathbf{B} \mathbf{C} \mathbf{D} \mathbf{A}$$

Quick Matrix Differentiation

- Let

$$\partial_{\mathbf{U}} f(\mathbf{X}) = \lim_{\epsilon \to 0} \frac{f(\mathbf{X} + \epsilon \mathbf{U}) - f(\mathbf{X})}{\epsilon} = \mathrm{tr}\, \mathbf{U}^\mathsf{T} \frac{\partial f(\mathbf{X})}{\partial \mathbf{X}}$$

- E.g.

$$\begin{aligned} \partial_{\mathbf{U}} \mathrm{tr}\, \mathbf{A}\mathbf{X}\mathbf{B} &= \lim_{\epsilon \to 0} \frac{1}{\epsilon} \left(\mathrm{tr}\, \mathbf{A}(\mathbf{X} + \epsilon \mathbf{U})\mathbf{B} - \mathrm{tr}\, \mathbf{A}\mathbf{X}\mathbf{B}\right) \\ &= \mathrm{tr}\, \mathbf{A}\mathbf{U}\mathbf{B} = \mathrm{tr}\, \mathbf{B}^\mathsf{T} \mathbf{U}^\mathsf{T} \mathbf{A}^\mathsf{T} = \mathrm{tr}\, \mathbf{U}^\mathsf{T} \mathbf{A}^\mathsf{T} \mathbf{B}^\mathsf{T} \end{aligned}$$

thus

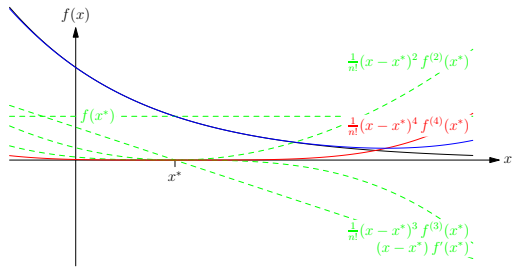
$$\frac{\partial \mathrm{tr} \mathbf{A}\mathbf{X}\mathbf{B}}{\partial \mathbf{X}} = \mathbf{A}^\mathsf{T} \mathbf{B}^\mathsf{T}$$

Summary

- With care you can differentiate most expressions
- The chain and product rule are incredibly powerful tools
- We can generalise differentiation to vectors and matrices
- There are a number of surprisingly useful results see **The Matrix Cookbook**
- When we look at **integration** it gets harder

Mathematics of Machine Learning

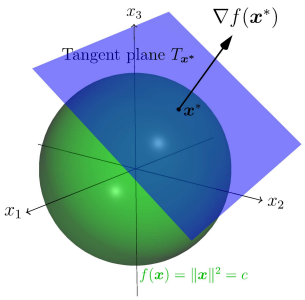
Taylor's Expansion



Convergence, remainder, high-dimensions, Hessian

Outline

1. **1-D Taylor Expansion**
2. High Dimensions
3. Hessians



Approximating Functions

- In much of machine learning we are trying to approximate functions
- There are many different ways to do this
- Depending on your application some are much better than others
- All function approximation makes some assumptions about the regularity of the function they are approximating
- Taylor expansions are polynomial approximations of a smooth function around a point

Polynomial Approximation

- Suppose we wanted to approximate a function $f(x)$ around a point x^* using a polynomial

$$f(x) \approx a_0 + a_1(x-x^*) + a_2(x-x^*)^2 + \ldots + a_n(x-x^*)^n$$

- Taking derivatives

$$f'(x) \approx a_1 + 2a_2(x-x^*) + 3a_3(x-x^*)^2 + \ldots + na_n(x-x^*)^{n-1}$$
$$f''(x) \approx 2a_2 + 6a_3(x-x^*) + \ldots + n(n-1)a_n(x-x^*)^{n-2}$$

- One way to decide on the coefficients is to set $x = x^*$ and assume equality

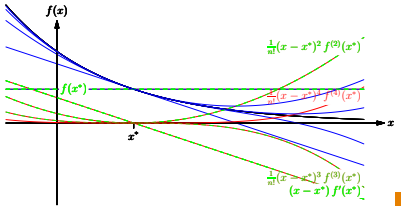
$$a_0 = f(x^*) \quad a_1 = f'(x^*) \quad a_2 = \frac{1}{2}f''(x^*) \quad a_i = \frac{1}{i!}f^{(i)}(x^*)$$

Taylor Expansion

- This gives the Taylor expansion

$$f(x) \approx f(x^*) + f'(x^*)(x-x^*) + \frac{1}{2}f''(x^*)(x-x^*)^2 + \ldots + \frac{1}{n!}f^{(n)}(x^*)(x-x^*)^n$$

- The left and right-hand side both have the same n derivatives at the point x^*



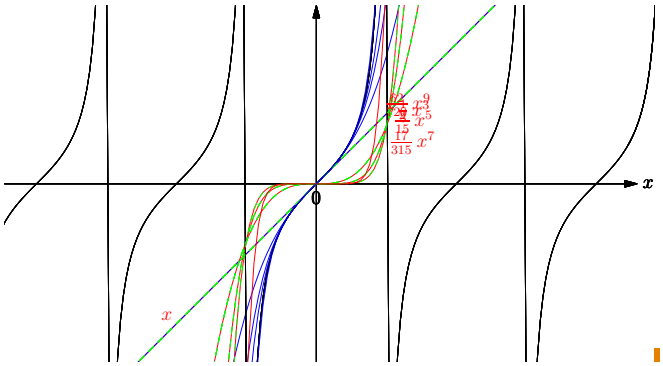
Convergence

- As we add more and more terms our local approximation becomes more accurate, provided the higher derivatives exist
- Polynomials have the nice property that they only diverge at ∞
- However, polynomials cannot capture functions that diverge for finite numbers, e.g.

$$\frac{1}{x} \qquad \frac{1}{(x-5)^2} \qquad \tan(x)$$

- This leads to a loss of global convergence

Taylor Expansion of $\tan(x)$



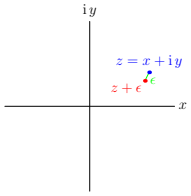
Complex Functions

- To understand convergence of Taylor series more generally we need to go into the complex plane
- We can generalise many functions by considering them as a function with a complex argument $z = x + \mathrm{i}y$
- The value of the complex function will be complex valued
- Complex functions are a huge and beautiful field (almost all of which go beyond this course)

Analytic Functions

- An important class of functions are those where for almost all z the derivative

$$\frac{\mathrm{d}f(z)}{\mathrm{d}z} = \lim_{\epsilon \to 0} \frac{f(z+\epsilon) - f(z)}{\epsilon}$$

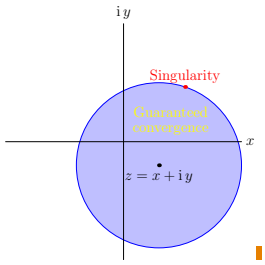


does not depend on the direction ϵ

- This includes polynomials, but also includes exponentials, trigonometric functions, logarithms, ratios of polynomials, "special functions", etc.

Guaranteed Convergence of Taylor Series

- For analytic functions, Taylor series are guaranteed to converge inside a circle that excludes singular points

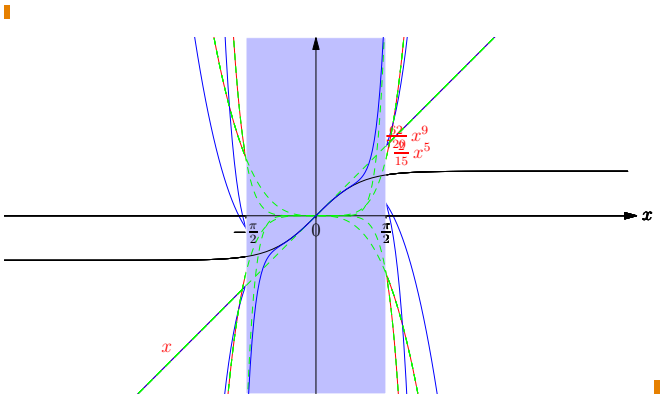


- E.g. the function $f(z) = \frac{1}{z^2+1}$ has a singularity at $z = \pm\mathrm{i}$. The Taylor series of $f(z)$ around 0 is guaranteed to converge inside a circle of radius 1

The Hyperbolic Tangent

- The hyperbolic tangent, $\tanh$, is related to the normal tangent function through $\tanh(x) = -\mathrm{i}\tan(\mathrm{i}x)$
- The function (along the real line) doesn't have any singularities
- But we know that $\tan(x)$ has singularities at $\frac{\pi}{2} + k\pi$ where k is any integer, so $\tanh(x)$ has singularities at $x = \pm\mathrm{i}\frac{\pi}{2}, \pm\mathrm{i}\frac{3\pi}{2}, \ldots$ on the imaginary axis
- Thus a Taylor expansion of $\tanh(x)$ around $x = 0$ is guaranteed to converge in a circle of radius $\pi/2$ but not necessarily for larger values of x

Taylor Expansion of $\tanh(x)$



Remainder

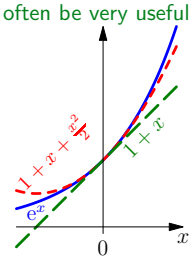
- Taylor expansions actually come with an error bound

$$f(x) = \sum_{i=0}^{n-1} \frac{(x-x^*)^i}{i!} f^{(i)}(x^*) + \frac{(x-x^*)^n}{n!} f^{(n)}(\theta_x)$$

θ_x is a number in the interval between x^* and x

- The last term, known as the **remainder**, can often be very useful

$$\mathrm{e}^x = 1 + x + \frac{x^2}{2}\mathrm{e}^{\theta_x}$$
$$\mathrm{e}^x = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!}\mathrm{e}^{\theta_x}$$



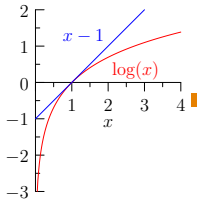
Useful Bounds

- A much used bound in mathematical analysis is $\mathrm{e}^x \geq 1 + x$
- Many other bounds can be obtained similarly, e.g. expanding $\log(x)$ around $x = 1$

$$\log(x) = (x-1) - \frac{(x-1)^2}{2}\frac{1}{\theta^2}$$

giving

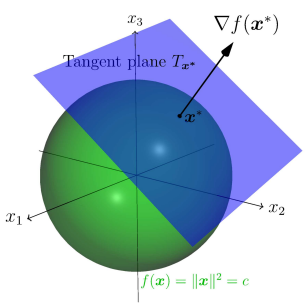
$$\log(x) \leq x - 1 \quad \text{or} \quad \log(1+x) \leq x$$



- The proof of the remainder term comes from applying multiple integrals to the term $f^{(n)}(x)$. For the sake of time I will skip this

Outline

1. 1-D Taylor Expansion
2. **High Dimensions**
3. Hessians



High Dimensional Spaces

- Just as we defined a Taylor expansion by approximating the function by a polynomial we can do the same for a function $f : \mathbb{R}^n \to \mathbb{R}$

$$f(\boldsymbol{x}) \approx c^{(0)} + \sum_{i=1}^{n} c_i^{(1)}(x_i - x_i^*) + \sum_{i,j=1}^{n} (x_i - x_i^*)c_{ij}^{(2)}(x_j - x_j^*) + \ldots$$

- We can determine the coefficients by differentiating both sides and setting $\boldsymbol{x} = \boldsymbol{x}^*$

$$f(\boldsymbol{x}^*) = c^{(0)}$$
$$\frac{\partial f(\boldsymbol{x}^*)}{\partial x_i} = c_i^{(1)}$$
$$\frac{\partial^2 f(\boldsymbol{x}^*)}{\partial x_i \partial x_j} = 2c_{ij}^{(2)} \quad \text{(choosing } c_{ij}^{(2)} = c_{ji}^{(2)}\text{)}$$

High Dimensional Taylor Expansion

- We find

$$f(\boldsymbol{x}) \approx f(\boldsymbol{x}^*) + (\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T}\boldsymbol{\nabla} f(\boldsymbol{x}^*) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T}\mathbf{H}(\boldsymbol{x} - \boldsymbol{x}^*) + \cdots$$

- Where the matrix $\mathbf{H}$ is known as the Hessian and has components

$$H_{ij} = \frac{\partial^2 f(\boldsymbol{x}^*)}{\partial x_i \partial x_j}$$

- Because

$$\frac{\partial^2 f(\boldsymbol{x})}{\partial x_i \partial x_j} = \frac{\partial^2 f(\boldsymbol{x})}{\partial x_j \partial x_i}$$

the Hessian is symmetric ($\mathbf{H} = \mathbf{H}^\mathsf{T}$)

Tangent Plane

- For smooth functions in n dimensions the equation $f(\boldsymbol{x}) = c$ defines an $(n-1)$-dimensional hyper-surface (because we have imposed one constraint)
- This can be thought of as the high dimensional analogue of a contour line
- The plane that goes through a point $\boldsymbol{x}^*$ that lines up with the contour surface is known as the tangent plane
- Away from $\boldsymbol{x}^*$ the value of the function $f(\boldsymbol{x})$ differs from the tangent plane by a quantity of order $\|\boldsymbol{x} - \boldsymbol{x}^*\|^2$

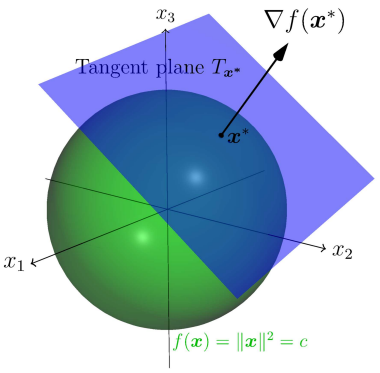
Defining the Tangent Plane

- Thus if $\boldsymbol{x}$ is in the tangent plane $T_{\boldsymbol{x}^*}$ then

$$f(\boldsymbol{x}) = f(\boldsymbol{x}^*) + O\big(\|\boldsymbol{x} - \boldsymbol{x}^*\|^2\big)$$

- Or $(\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T}\boldsymbol{\nabla} f(\boldsymbol{x}^*) = 0$
- That is the tangent plane at a point $\boldsymbol{x}^*$ is the set of points orthogonal to the gradient
- Notice that the gradient then points in the direction orthogonal to the tangent plane of the contour surface

Picture of Tangent Plane for $f(\boldsymbol{x}) = \|\boldsymbol{x}\|^2 = c$



Direction of Steepest Ascent

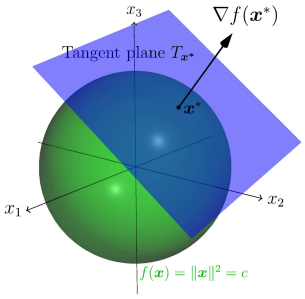
- We note that as

$$f(\boldsymbol{x}^* + \epsilon\boldsymbol{u}) = f(\boldsymbol{x}^*) + \epsilon\boldsymbol{u}^\mathsf{T}\boldsymbol{\nabla} f(\boldsymbol{x}^*) + O(\epsilon^2)$$

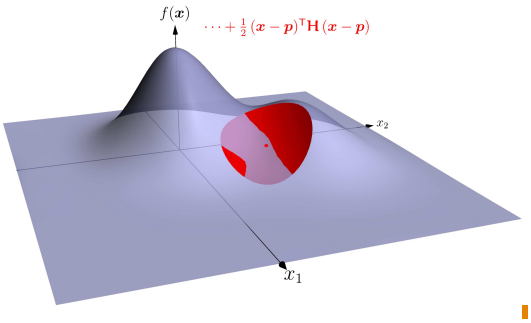
- For a step $\boldsymbol{u}$ of fixed length, the function locally grows fastest when $\boldsymbol{u}$ is in the direction $\boldsymbol{\nabla} f(\boldsymbol{x}^*)$
- Likewise when $\boldsymbol{u}$ is in the direction $-\boldsymbol{\nabla} f(\boldsymbol{x}^*)$ the function decreases as fast as possible
- This is only true locally (when $\epsilon \gg \epsilon^2$)

Outline

1. 1-D Taylor Expansion
2. High Dimensions
3. **Hessians**



2-D Taylor Expansion



The Hessian

- The second order term in the Taylor expansion tells us about the **curvature** of the function
- This is very relevant when we are at a **stationary point**, $\boldsymbol{x}^*$, where $\boldsymbol{\nabla} f(\boldsymbol{x}^*) = \mathbf{0}$ so that

$$f(\boldsymbol{x}) = f(\boldsymbol{x}^*) + \frac{1}{2}(\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T}\mathbf{H}(\boldsymbol{x} - \boldsymbol{x}^*) + O\big(\|\boldsymbol{x} - \boldsymbol{x}^*\|^3\big)$$

- If we make a perturbation $\boldsymbol{x} = \boldsymbol{x}^* + \epsilon\boldsymbol{u}$ then

$$f(\boldsymbol{x}^* + \epsilon\boldsymbol{u}) = f(\boldsymbol{x}^*) + \frac{1}{2}\epsilon^2\boldsymbol{u}^\mathsf{T}\mathbf{H}\boldsymbol{u} + O(\epsilon^3)$$

- For sufficiently small perturbation the sign of change in the function depends on the sign of $\boldsymbol{u}^\mathsf{T}\mathbf{H}\boldsymbol{u}$

Eigenvalues

- We will revisit eigen-systems, but recall that for a square matrix an eigenvector, $\boldsymbol{v}_i$, of a matrix $\mathbf{H}$ is defined such that

$$\mathbf{H}\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i$$

- For an $n \times n$ symmetric matrix (such as the Hessian), the eigenvalues are all real, and we can choose n orthonormal eigenvectors such that $\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = \delta_{ij} = [\![i = j]\!]$
- We note that

$$\boldsymbol{v}_i^\mathsf{T}\mathbf{H}\boldsymbol{v}_i = \boldsymbol{v}_i^\mathsf{T}\lambda_i\boldsymbol{v}_i = \lambda_i\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_i = \lambda_i$$

- If we perturb in a direction $\boldsymbol{v}_i$ then

$$f(\boldsymbol{x}^* + \epsilon\boldsymbol{v}_i) = f(\boldsymbol{x}^*) + \frac{1}{2}\epsilon^2\lambda_i + O(\epsilon^3)$$

General Directions

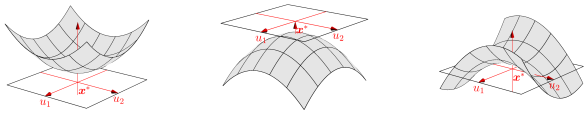
- We can write any vector $\boldsymbol{u}$ as a weighted sum of the eigenvectors (the eigenvectors form a basis) $\boldsymbol{u} = \sum_{i=1}^n c_i\boldsymbol{v}_i$
- Note that

$$\begin{aligned}\boldsymbol{u}^\mathsf{T}\mathbf{H}\boldsymbol{u} &= \left(\sum_{i=1}^n c_i\boldsymbol{v}_i^\mathsf{T}\right)\mathbf{H}\left(\sum_{j=1}^n c_j\boldsymbol{v}_j\right)\\ &= \left(\sum_{i=1}^n c_i\boldsymbol{v}_i^\mathsf{T}\right)\left(\sum_{j=1}^n c_j\lambda_j\boldsymbol{v}_j\right)\\ &= \sum_{i,j=1}^n \lambda_j c_i c_j \boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = \sum_{i=1}^n \lambda_i c_i^2\end{aligned}$$

using the orthogonality of eigenvectors $\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = \delta_{ij}$

Minima, Maxima and Saddle-Points

- If all the eigenvalues are positive, $\lambda_i > 0$, then for any $\boldsymbol{u}$ we have $\boldsymbol{u}^\mathsf{T}\mathbf{H}\boldsymbol{u} > 0$ and $\boldsymbol{x}^*$ would be a local minimum. We say $\mathbf{H}$ is **positive definite** or $\mathbf{H} \succ 0$
- Similarly if all the eigenvalues are less than zero ($\mathbf{H}$ is **negative definite**, $\mathbf{H} \prec 0$), then $\boldsymbol{x}^*$ will be a local maximum
- If there are both positive and negative eigenvalues then $\boldsymbol{x}^*$ is a saddle-point (if some eigenvalues are zero we need higher order terms to decide)



Finding Optima

- We are often given a loss function, $L(\boldsymbol{w})$, which we wish to minimise
- A sufficient condition for $\boldsymbol{w}^*$ to be a local minimum is that $\boldsymbol{\nabla}L(\boldsymbol{w}^*) = \mathbf{0}$ and $\mathbf{H} \succ 0$ where $\mathbf{H}$ is the Hessian of $L(\boldsymbol{w})$ at the point $\boldsymbol{w}^*$
- Except for a few very simple loss functions, there are likely to be many (and often a huge number) of local minima
- Unless $L(\boldsymbol{w})$ is very simple we are unlikely to be able to solve $\boldsymbol{\nabla}L(\boldsymbol{w}) = \mathbf{0}$ analytically

Gradient Descent

- We usually proceed iteratively, starting from some $\boldsymbol{w}^{(0)}$

$$\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} - r\boldsymbol{\nabla}L(\boldsymbol{w}^{(t)})$$

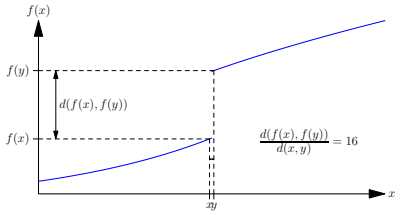
- This moves us in the direction that (locally) minimises the loss function $L(\boldsymbol{w})$
- As we get closer to the minimum $\boldsymbol{\nabla}L(\boldsymbol{w}^{(t)})$ typically becomes smaller and we take smaller steps
- However, many nasty things can happen, and we have to be careful in how we perform gradient descent
- We will revisit this when we look at optimisation

Lessons

- For functions that are smooth (at least almost everywhere) Taylor series are really useful in understanding the local behaviour of the function
- The gradient defines the direction in which a function increases fastest
- This will be orthogonal to the tangent plane, where locally the function is a constant
- The Hessian (not to be confused with the Jacobian) is the matrix of second derivatives defining the curvature away from the tangent plane

Mathematics of Machine Learning

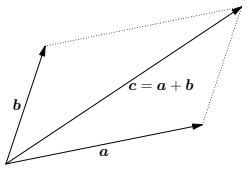
Vector Spaces



Vectors, metric spaces, norms

Outline

1. **Vector Spaces**
2. Metrics (distances)
3. Norms



Matrices, Vectors and All That

- The language of machine learning is mathematics
- Sometimes we draw pretty pictures to explain the mathematics
- Much of the mathematics we will use involves vectors, matrices and functions
- You need to master the language of mathematics, otherwise you won't understand the algorithms
- I'm going to spend this lecture and the next revising the mathematics you need to know (but I'm going to use a slightly posher language than you are probably used to)

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together $(a + b)$ and multiply together $(a \times b)$
- Formally they form an Abelian group under addition with an identity 0 and excluding 0 an Abelian group under multiplication and they are distributive

$$a \times (b + c) = a \times b + a \times c$$

- Although this sounds rather daunting don't panic. They behave like numbers. The field might be rational numbers, reals, complex numbers or something a bit more exotic (not the integers: 2 has no integer inverse)—but we will almost always consider reals

Vectors

- We often work with objects with many components (features)
- To help handle this we will use vector notation

$$\boldsymbol{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

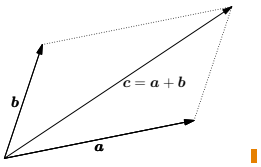
- We represent vectors by bold symbols
- All our vectors are column vectors by default
- We treat them as $n \times 1$ matrices

- We write row vectors as transposes of column vectors

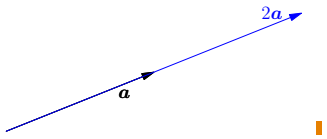
$$\boldsymbol{y}^\mathsf{T} = (y_1, y_2, \ldots, y_n)$$

Basic Vector Operations

- The basic vector operations are adding



- multiplying by a scalar (a number)



Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies

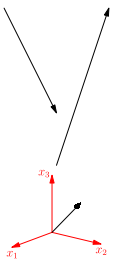
1. if $\boldsymbol{v}, \boldsymbol{w} \in \mathcal{V}$ then $a\boldsymbol{v} \in \mathcal{V}$ and $\boldsymbol{v} + \boldsymbol{w} \in \mathcal{V}$ (closure)
2. $\boldsymbol{v} + \boldsymbol{w} = \boldsymbol{w} + \boldsymbol{v}$ (commutativity of addition)
3. $(\boldsymbol{u} + \boldsymbol{v}) + \boldsymbol{w} = \boldsymbol{u} + (\boldsymbol{v} + \boldsymbol{w})$ (associativity of addition)
4. $\boldsymbol{v} + \boldsymbol{0} = \boldsymbol{v}$ (existence of additive identity $\boldsymbol{0}$)
5. $\boldsymbol{v} + (-\boldsymbol{v}) = \boldsymbol{0}$ (existence of additive inverse $-\boldsymbol{v}$)
6. $1\boldsymbol{v} = \boldsymbol{v}$ (existence of multiplicative identity 1)
7. $a(b\boldsymbol{v}) = (ab)\boldsymbol{v}$ (associativity)
8. $a(\boldsymbol{v} + \boldsymbol{w}) = a\boldsymbol{v} + a\boldsymbol{w}$ (distributive properties)
9. $(a + b)\boldsymbol{v} = a\boldsymbol{v} + b\boldsymbol{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

$\mathbb{R}^n$

- When we first learn about vectors we think of them as arrows in 3-D space
- If we centre them all at the origin then there is a one-to-one correspondence between vectors and points in space
- We call this vector space $\mathbb{R}^3$
- Any set of quantities $\boldsymbol{x} = (x_1, x_2, \ldots, x_n)^\mathsf{T}$ which satisfy the axioms above form a vector space $\mathbb{R}^n$
- Of course, we can't so easily draw pictures of high-dimensional vectors

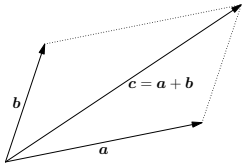


Other Vector Spaces

- Any objects that satisfy the axioms of a vector space are *vectors*—not just $\boldsymbol{v} \in \mathbb{R}^n$
- Matrices satisfy all the conditions of a vector space
- Infinite sequences form a vector space
- Functions form a vector space
  - Let $C(a,b)$ be the set of functions defined on the interval $[a,b]$
  - Note that if $f(x),g(x) \in C(a,b)$ then $\alpha f(x) \in C(a,b)$ and $f(x)+g(x) \in C(a,b)$
- Bounded vectors in $\mathbb{R}^n$ **don't** form a vector space

Outline

1. Vector Spaces
2. **Metrics (distances)**
3. Norms



Metrics

- Vector spaces become more interesting if we have a notion of distance
- We say $d(\boldsymbol{x},\boldsymbol{y})$ is a **distance** or **metric** if

1. $d(\boldsymbol{x},\boldsymbol{y}) \geq 0$ (non-negativity)
2. $d(\boldsymbol{x},\boldsymbol{y}) = 0$ iff $\boldsymbol{x} = \boldsymbol{y}$ (identity of indiscernibles)
3. $d(\boldsymbol{x},\boldsymbol{y}) = d(\boldsymbol{y},\boldsymbol{x})$ (symmetry)
4. $d(\boldsymbol{x},\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{z}) + d(\boldsymbol{z},\boldsymbol{y})$ (triangular inequality)

- There are typically many possible distances (e.g. Euclidean distance, Manhattan distance, etc.)
- If one or more of the conditions aren't satisfied we have a **pseudo-metric**

Mappings and Functions

- A function defines a mapping from one vector space to another (although the spaces might be the same), e.g.

$$f : \mathbb{R} \to \mathbb{R}$$

(f maps the reals to the reals, i.e. $f(x)$ takes a real x and gives you a new real number $y = f(x)$)

- We are often interested in functions that behave nicely
- E.g. They are continuous

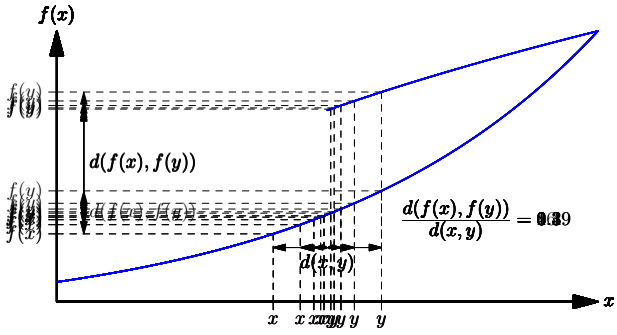
Lipschitz Function

- One way to characterise a well-behaved function, $f(x)$, is if there exists a number $K < \infty$ such that for all x and y

$$d(f(x),f(y)) \leq K\, d(x,y)$$

- This is known as a **Lipschitz condition** and the function is said to be K-Lipschitz or Lipschitz continuous
- Note that such functions cannot have any jumps (i.e. they are continuous)
- The size of K measures the limit on the amplifying effect of the function

Lipschitz Continuous Functions



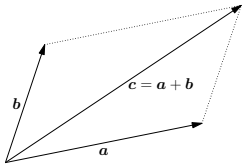
- The definition applies to any function defined in a metric space

Contractive Mappings

- An interesting class of functions are those for which $K < 1$
- These are said to be contractive mappings
- A famous theorem that applies to contractive mappings is the Banach fixed-point theorem which says there exists a unique fixed point such that $f(x) = x$, and repeatedly applying f converges to it
- This is used for example in showing that various algorithms will converge

Outline

1. Vector Spaces
2. Metrics (distances)
3. **Norms**



Norms

- Vector spaces are even more interesting with a notion of length
- **Norms** provide some measure of the size of a vector
- To formalise this we define the **norm** of an object $\boldsymbol{v}$ as $\|\boldsymbol{v}\|$ satisfying

1. $\|\boldsymbol{v}\| > 0$ if $\boldsymbol{v} \neq \boldsymbol{0}$ and $\|\boldsymbol{0}\| = 0$ (non-negativity)
2. $\|a\boldsymbol{v}\| = |a|\|\boldsymbol{v}\|$ ("linearity")
3. $\|\boldsymbol{u} + \boldsymbol{v}\| \leq \|\boldsymbol{u}\| + \|\boldsymbol{v}\|$ (triangular inequality)

- When some criteria aren't satisfied we have a **pseudo-norm**
- Norms provide a metric $d(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|$ (they are metric spaces)

Vector Norms

- The familiar vector norm is the (Euclidean) two norm

$$\|\boldsymbol{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

- Other norms exist, such as the p-norm ($p \geq 1$)

$$\|\boldsymbol{v}\|_p = \left(\sum_{i=1}^n |v_i|^p\right)^{1/p}$$

- Special cases include the 1-norm and the infinite norm

$$\|\boldsymbol{v}\|_1 = \sum_{i=1}^n |v_i| \qquad \|\boldsymbol{v}\|_\infty = \max_i |v_i|$$

- The 0-norm is a pseudo-norm as it does not satisfy condition 2

$$\|\boldsymbol{v}\|_0 = \text{number of non-zero components}$$

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\boldsymbol{w}$ explain the training data equally well
- We usually prefer a "small" $\boldsymbol{w}$, so we minimise

$$\text{loss}(\boldsymbol{w}) + \eta\|\boldsymbol{w}\|$$

- Equivalently we minimise the loss subject to $\|\boldsymbol{w}\| \leq c$, confining $\boldsymbol{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by "small"
- $\|\boldsymbol{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\boldsymbol{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\boldsymbol{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by "small"**

Matrix Norms

- We can define norms for other objects
- The norm of a matrix encodes how large the mapping is
- The Frobenius norm is defined by

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2}$$

- Many other norms exist including 1-norm, max-norm, etc.
- Some, but not all, norms satisfy the inequality

$$\|\mathbf{AB}\| \leq \|\mathbf{A}\| \times \|\mathbf{B}\|$$

Compatible Norms

- A vector and matrix norm are said to be compatible if

$$\|\mathbf{M}\boldsymbol{v}\|_b \leq \|\mathbf{M}\|_a \times \|\boldsymbol{v}\|_b$$

(Spectral and Euclidean norms are compatible)

- Norms provide quick ways to bound the maximum growth of a vector under a mapping induced by the matrix
- We will see that a measure of the sensitivity of a mapping is in terms of the ratio of its maximum effect to its minimum effect on a vector
- This is known as the **conditioning**, given by $\|\mathbf{M}\| \times \|\mathbf{M}^{-1}\|$

Why Should You Care?

- Deep learning involves multiplying the input (which we can think of as a vector $\boldsymbol{x}$) by many layers
- In CNNs we have convolutional layers and dense layers
- The effect of applying these layers can be represented by a matrix multiplication $\boldsymbol{x}_n = \mathbf{L}_n\boldsymbol{x}_{n-1}$
- We also do other things like applying ReLUs or pooling that change the magnitude, $\|\boldsymbol{x}_n\|$, of our representation
- If you are developing new architectures you want $\|\boldsymbol{x}_n\|$ neither to blow up nor vanish
- This can be controlled by carefully choosing $\|\mathbf{L}_n\|$

The Lipschitz Constant of a Network

- We can now put the Lipschitz condition and the matrix norm together
- Recall f is Lipschitz with constant K if

$$\|f(\boldsymbol{x}) - f(\boldsymbol{y})\| \leq K\|\boldsymbol{x} - \boldsymbol{y}\|$$

- If we compose two functions the constants **multiply**

$$K_{f\circ g} \leq K_f K_g$$

- A layer $\boldsymbol{x} \mapsto \mathbf{L}_n\boldsymbol{x}$ has $K = \|\mathbf{L}_n\|_2$, the **spectral norm**—this is just compatibility, $\|\mathbf{L}\boldsymbol{v}\| \leq \|\mathbf{L}\|_2\|\boldsymbol{v}\|$
- ReLUs and max-pooling never stretch distances (though they can shrink them): $K = 1$
- A network is just a composition of layers, so

$$K_{\text{net}} \leq \prod_n \|\mathbf{L}_n\|_2$$

Why Care About K?

- K tells us how much the output can move when we nudge the input

$$\|f(\boldsymbol{x} + \boldsymbol{\delta}) - f(\boldsymbol{x})\| \leq K\|\boldsymbol{\delta}\|$$

- **Adversarial examples** are tiny $\boldsymbol{\delta}$ that change the classification: a network with small K cannot be fooled this easily
- It is the same quantity that controls the signal: $K \gg 1$ and signals (and gradients) explode, $K \ll 1$ and they vanish
- Wasserstein GANs require the critic to have $K \leq 1$. One way to enforce this is **spectral normalisation**: divide each $\mathbf{L}_n$ by $\|\mathbf{L}_n\|_2$
- Bounds on how well a network generalises also depend on K
- But there is a trade-off: make K too small and the network is too smooth to fit the data

Function Norms

- Functions can also have norms, for example, if $f(x)$ is defined on some interval $\mathcal{I}$

$$\|f\|_{L^2} = \sqrt{\int_{x\in\mathcal{I}} f^2(x)\mathrm{d}x}$$

- The L^2 vector space is the set of functions for which $\|f\|_{L^2} < \infty$
- The L^1-norm is given by $\|f\|_{L^1} = \int_{x\in\mathcal{I}} |f(x)|\mathrm{d}x$
- The infinite-norm is given by $\|f\|_\infty = \max_{x\in\mathcal{I}} |f(x)|$

Norms and Metrics

- Given a norm, there exists an induced distance

$$d(\boldsymbol{x},\boldsymbol{y}) = \|\boldsymbol{x}-\boldsymbol{y}\|$$

- Any normed space has an associated metric space
- Thus we can define the Frobenius distance between matrices

$$d_F(\mathbf{A},\mathbf{B}) = \|\mathbf{A}-\mathbf{B}\|_F$$

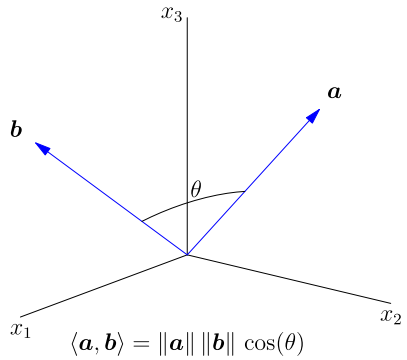
- It is easy to see that non-negativity, symmetry and the triangular inequality for distances are all inherited from the same properties of norms

Summary

- Vector spaces with a distance (metric spaces) and vector spaces with a norm (normed vector spaces) are interesting objects
- They allow you to define a topology (open/closed sets, etc.)
- You can build up ideas about connectedness, continuity, contractive maps, fixed-point theorems, . . .
- For the most part we are going to consider an even more powerful vector space that has an inner-product defined

Mathematics of Machine Learning

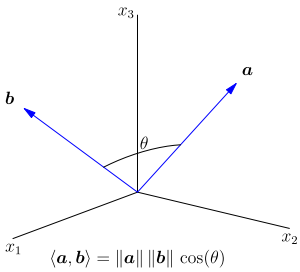
Inner Product Spaces



Inner products, operators

Outline

1. **Inner Products**
2. Operators



Recap

- We have looked at vector spaces (closed sets where we can add elements and multiply them by a scalar)
- Recall that vector spaces don't just apply to normal vectors ($\mathbb{R}^n$), but to matrices, functions, sequences, random variables, . . .
- Proper distances or metrics, $d(\boldsymbol{x},\boldsymbol{y})$, allow us to construct ideas about geometry of the vector space
- Norms, $\|\boldsymbol{x}\|$, allow us to reason about the size of a vector
- Norms induce a distance, $d(\boldsymbol{x},\boldsymbol{y}) = \|\boldsymbol{x}-\boldsymbol{y}\|$

Inner Products

- We will often consider objects with an *inner product*, $\langle , \rangle$
- For vectors in $\mathbb{R}^n$

$$\langle \boldsymbol{x},\boldsymbol{y} \rangle = \boldsymbol{x}^\mathsf{T}\boldsymbol{y} = \sum_{i=1}^n x_i y_i$$

- For functions

$$\langle f,g \rangle = \int_{x\in\mathcal{I}} f(x)g(x)\mathrm{d}x$$

- For $m \times n$ matrices

$$\langle \mathbf{A},\mathbf{B} \rangle = \mathrm{Tr}\boldsymbol{A}^\mathsf{T}\mathbf{B} = \sum_{i=1}^m\sum_{j=1}^n A_{ij}B_{ij}$$

Axioms of Inner Products

- An inner product satisfies
  1. $\langle \boldsymbol{x},\boldsymbol{x} \rangle \geq 0$ for all $\boldsymbol{x} \in \mathcal{V}$
  2. $\langle \boldsymbol{x},\boldsymbol{x} \rangle = 0$ if and only if $\boldsymbol{x} = \boldsymbol{0}$
  3. $\langle \alpha\boldsymbol{x},\boldsymbol{y} \rangle = \alpha\langle \boldsymbol{x},\boldsymbol{y} \rangle$
  4. $\langle \boldsymbol{x},\boldsymbol{y}+\boldsymbol{z} \rangle = \langle \boldsymbol{x},\boldsymbol{y} \rangle + \langle \boldsymbol{x},\boldsymbol{z} \rangle$
  5. $\langle \boldsymbol{x},\boldsymbol{y} \rangle = \langle \boldsymbol{y},\boldsymbol{x} \rangle$
- We can show that $\|\boldsymbol{x}\| = \sqrt{\langle \boldsymbol{x},\boldsymbol{x} \rangle}$ satisfies the axioms of a norm, so that an inner-product space is a normed space
- The norm associated with the inner-product for vectors in $\mathbb{R}^n$ (i.e. $\langle \boldsymbol{x},\boldsymbol{y} \rangle = \boldsymbol{x}^\mathsf{T}\boldsymbol{y}$) is the Euclidean norm $\|\boldsymbol{x}\| = \sqrt{\boldsymbol{x}^\mathsf{T}\boldsymbol{x}}$

Cauchy-Schwarz Inequality

- One of the most important results of inner-product spaces, known as the **Cauchy-Schwarz inequality** is that

$$\langle \boldsymbol{x},\boldsymbol{y} \rangle^2 \leq \langle \boldsymbol{x},\boldsymbol{x} \rangle \langle \boldsymbol{y},\boldsymbol{y} \rangle = \|\boldsymbol{x}\|^2\|\boldsymbol{y}\|^2$$

- Or

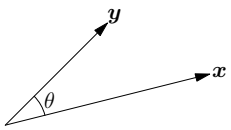
$$|\langle \boldsymbol{x},\boldsymbol{y} \rangle| \leq \|\boldsymbol{x}\|\,\|\boldsymbol{y}\|$$

- This is a very general result so for example

$$\left|\int f(x)g(x)\mathrm{d}x\right| \leq \sqrt{\left(\int f^2(x)\mathrm{d}x\right)\left(\int g^2(x)\mathrm{d}x\right)}$$

Angles Between Vectors

- A natural interpretation of the inner product is in providing a measure of the angle between vectors



$$\langle \boldsymbol{x},\boldsymbol{y} \rangle = \boldsymbol{x}^\mathsf{T}\boldsymbol{y} = \|\boldsymbol{x}\| \, \|\boldsymbol{y}\| \cos(\theta)$$

- Vectors are orthogonal if $\langle \boldsymbol{x},\boldsymbol{y} \rangle = 0$
- We can extend this idea to functions

$$\langle f(x),g(x) \rangle = \int_{x\in\mathcal{I}} f(x)g(x)\mathrm{d}x = \|f(x)\|\,\|g(x)\|\cos(\theta)$$

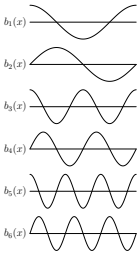
- Note that $\sin(x)$ and $\cos(x)$ are orthogonal in the interval $[0,2\pi]$

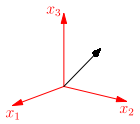
Basis Functions

- Any set of vectors $\{\boldsymbol{b}_i | i = 1,\ldots\}$ that span the space can be used as a basis or coordinate system
- The simplest and most useful case is when the vectors are orthogonal and normalised (i.e. $\|\boldsymbol{b}_i\| = 1$)
- In $\mathbb{R}^3$ we could use $\boldsymbol{b}_1 = \begin{pmatrix}1\\0\\0\end{pmatrix}$, $\boldsymbol{b}_2 = \begin{pmatrix}0\\1\\0\end{pmatrix}$, $\boldsymbol{b}_3 = \begin{pmatrix}0\\0\\1\end{pmatrix}$
- This is not unique as we can rotate our basis vectors
- For an orthonormal basis we can write any vector as $\hat{\boldsymbol{x}} = \begin{pmatrix}\boldsymbol{x}^\mathsf{T}\boldsymbol{b}_1\\ \boldsymbol{x}^\mathsf{T}\boldsymbol{b}_2\\ \boldsymbol{x}^\mathsf{T}\boldsymbol{b}_3\end{pmatrix}$

Orthogonal Functions

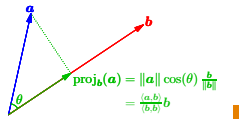
- For functions we can use any ortho-normal set of functions as a basis
- The most familiar are the Fourier functions $\sin(n\theta)$ and $\cos(n\theta)$
- Any function in $C(0,2\pi)$ can be represented by a point $\boldsymbol{f} = \begin{pmatrix} \langle f(x), b_0(x)\rangle \\ \langle f(x), b_1(x)\rangle \\ \vdots \end{pmatrix}$
- There might be an infinite number of components
- This is analogous to points in $\mathbb{R}^n$ (for large n)





Projection Operator

- Given a vector $\boldsymbol{a}$ we can project it onto a vector $\boldsymbol{b}$

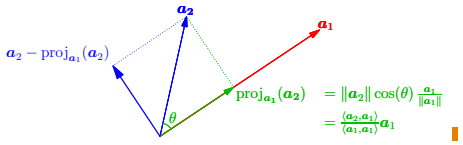


- Using $\langle \boldsymbol{a},\boldsymbol{b}\rangle = \|\boldsymbol{a}\|\,\|\boldsymbol{b}\|\cos(\theta)$ and $\langle \boldsymbol{b},\boldsymbol{b}\rangle = \|\boldsymbol{b}\|^2$
- A defining feature of a projection operator is

$$\text{proj}_{\boldsymbol{b}}(\text{proj}_{\boldsymbol{b}}(\boldsymbol{a})) = \text{proj}_{\boldsymbol{b}}\left(\frac{\langle \boldsymbol{a},\boldsymbol{b}\rangle}{\langle \boldsymbol{b},\boldsymbol{b}\rangle}\boldsymbol{b}\right)$$
$$= \left\langle \frac{\langle \boldsymbol{a},\boldsymbol{b}\rangle}{\langle \boldsymbol{b},\boldsymbol{b}\rangle}\boldsymbol{b}, \boldsymbol{b}\right\rangle \frac{\boldsymbol{b}}{\langle \boldsymbol{b},\boldsymbol{b}\rangle} = \frac{\langle \boldsymbol{a},\boldsymbol{b}\rangle}{\langle \boldsymbol{b},\boldsymbol{b}\rangle}\boldsymbol{b} = \text{proj}_{\boldsymbol{b}}(\boldsymbol{a})$$

Constructing an Orthonormal Basis

- Given a set of linearly independent vectors $(\boldsymbol{a}_1,\boldsymbol{a}_2,\ldots\boldsymbol{a}_n)$ we can construct an orthonormal basis



- We can iterate this

$$\boldsymbol{u}_1 = \boldsymbol{a}_1 \qquad \boldsymbol{e}_1 = \frac{\boldsymbol{u}_1}{\|\boldsymbol{u}_1\|}$$
$$\boldsymbol{u}_2 = \boldsymbol{a}_2 - \text{proj}_{\boldsymbol{u}_1}(\boldsymbol{a}_2) \qquad \boldsymbol{e}_2 = \frac{\boldsymbol{u}_2}{\|\boldsymbol{u}_2\|}$$
$$\boldsymbol{u}_3 = \boldsymbol{a}_3 - \text{proj}_{\boldsymbol{u}_1}(\boldsymbol{a}_3) - \text{proj}_{\boldsymbol{u}_2}(\boldsymbol{a}_3) \qquad \boldsymbol{e}_3 = \frac{\boldsymbol{u}_3}{\|\boldsymbol{u}_3\|}$$

Gram-Schmidt Orthogonalisation

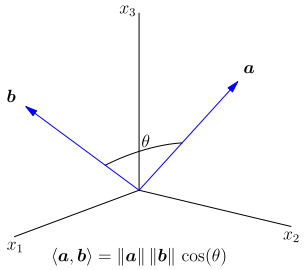
- The algorithm in the last slide is known as **Gram-Schmidt**
- We can perform it on any inner-product space
- For vectors in $\mathbb{R}^n$ it is prone to accumulate rounding errors and is not the method of choice
- Numerical packages use "Householder transformations" to perform orthogonalisation
- Nevertheless, it is nice to know that we can form an orthonormal basis

Algebraic Structure

- We have gone to these lengths as we want to show that many properties of vectors are shared by other objects (matrices, functions, etc.)
- The notions of distance (geometry), norms (size of vectors) and inner products (angles between vectors) provides a very rich set of concepts
- Vectors form the backbone of objects we will use repeatedly in machine learning
- The next piece of the jigsaw is to understand how we can transform these objects

Outline

1. Inner Products
2. **Operators**



Operators

- In machine learning we are interested in transforming our input vectors into some output predictions
- To accomplish this we will apply some mapping or operators on the vector $\mathcal{T} : \mathcal{V} \to \mathcal{V}'$
- This says that $\mathcal{T}$ maps some object $\boldsymbol{x} \in \mathcal{V}$ to an object $\boldsymbol{y} = \mathcal{T}[\boldsymbol{x}]$ in a new vector space $\mathcal{V}'$
- This new vector space may or may not be the same as the original vector space
- Our objects may be any object in a vector space such as a function

Linear Operators

- Operators are in general very complicated, but a particularly nice set of operators are linear operators
- $\mathcal{T}$ is a linear operator if
  1. $\mathcal{T}[a\boldsymbol{x}] = a\mathcal{T}[\boldsymbol{x}]$
  2. $\mathcal{T}[\boldsymbol{x}+\boldsymbol{y}] = \mathcal{T}[\boldsymbol{x}] + \mathcal{T}[\boldsymbol{y}]$
- For normal vectors ($\boldsymbol{x} \in \mathbb{R}^n$) the most general linear operation is

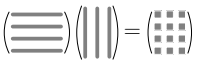
$$\mathcal{T}[\boldsymbol{x}] = \mathbf{M}\boldsymbol{x}$$

where $\mathbf{M}$ is a matrix

Matrix multiplication

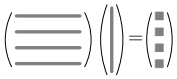
- For an $\ell \times m$ matrix $\mathbf{A}$ and an $m \times n$ matrix $\mathbf{B}$ we can compute the $(\ell \times n)$ product, $\mathbf{C} = \mathbf{AB}$, such that

$$C_{ij} = \sum_{k=1}^{m} A_{ik} B_{kj}$$



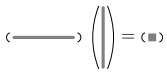
- Treating the vector $\boldsymbol{x}$ as an $n \times 1$ matrix then

$$\boldsymbol{y} = \mathbf{A}\boldsymbol{x} \quad \Rightarrow \quad y_i = \sum_j A_{ij} x_j$$



- Using the same matrix notation we can define the inner product as

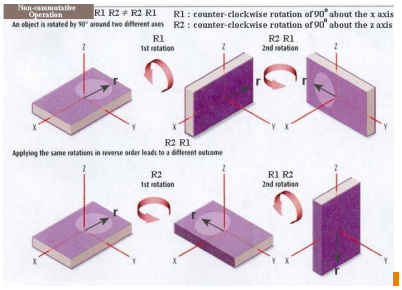
$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \boldsymbol{x}^\mathsf{T} \boldsymbol{y} = \sum_{i=1}^{n} x_i y_i$$



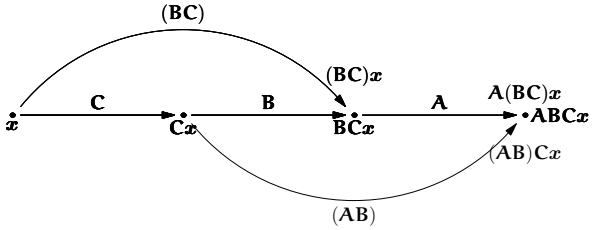
Non-commutativity

- In general $\mathbf{AB} \neq \mathbf{BA}$

$$\begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \quad \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & -1 & 0 \\ 0 & 0 & -1 \\ 1 & 0 & 0 \end{pmatrix}$$



Associativity of Mappings



- For all $\boldsymbol{x}$ we have $\mathbf{A}(\mathbf{BC})\boldsymbol{x} = (\mathbf{AB})\mathbf{C}\boldsymbol{x}$
- This implies $\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C}$

Kernels

- The equivalent of a matrix for functions (i.e. a linear operator) is known as a kernel $K(x,y)$

$$g(x) = \mathcal{T}[f] = \int_{y \in \mathcal{I}} K(x,y) f(y) \mathrm{d}y$$

- Our domain does not need to be one dimensional, e.g.

$$g(\boldsymbol{x}) = \mathcal{T}[f] = \int_{\boldsymbol{y} \in \mathcal{I}} K(\boldsymbol{x},\boldsymbol{y}) f(\boldsymbol{y}) \mathrm{d}\boldsymbol{y}$$

- We shall soon see examples of high-dimensional kernels

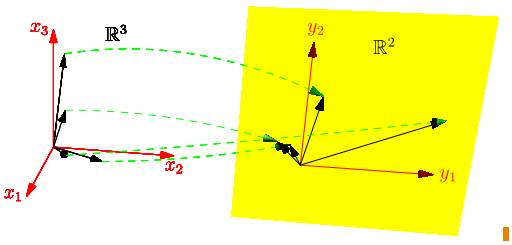
Kernels in Machine Learning

- Kernels are used heavily in machine learning
- In kernel methods such as SVM, SVR, Kernel-PCA
- They are also used in Gaussian Processes
- In all these cases we consider symmetric, positive semi-definite kernels
- Sometimes they can be interpreted as covariance between random functions

$$K(\boldsymbol{x},\boldsymbol{y}) = \mathbb{E}_{f \sim \mathcal{P}}\big[\big(f(\boldsymbol{x}) - \mu(\boldsymbol{x})\big)\big(f(\boldsymbol{y}) - \mu(\boldsymbol{y})\big)\big]$$

General Linear Mappings

- In general a linear operator will map vectors between different vector spaces
- E.g. $\mathbb{R}^3 \to \mathbb{R}^2$



Square Matrices

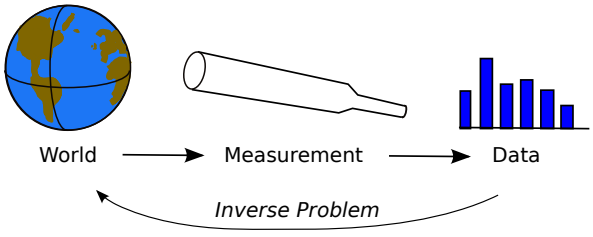
- We will spend a lot of time on operators that map from a vector space onto itself $\mathcal{T} : \mathcal{V} \to \mathcal{V}$
- For vectors in $\mathbb{R}^n$ such linear operators are represented by square matrices
- When there is a one-to-one mapping then we have a unique inverse
- We will study such mappings in detail in the next lecture

Summary

- Mathematics is the language of machine learning and you have to get used to it
- Mathematics is like programming, if you don't understand the syntax and you can't write it down then it's meaningless
- We've taken a high level view of inner product spaces and operators, this will pay us back later as we look at kernel methods

Mathematics of Machine Learning

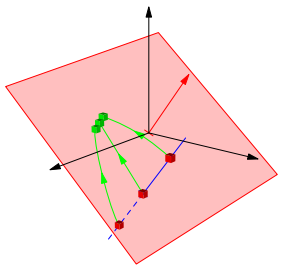
Understand Mappings



Least Squares, Under-constrained Systems, Ill-conditioning

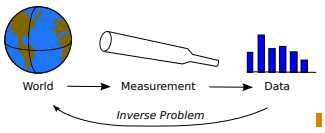
Outline

1. **Least Squares**
2. Under-constrained Systems
3. Ill-conditioning



Transforming Data

- In the last lecture we spent time developing a sophisticated view of vector spaces and operators
- At a mathematical level machine learning can be viewed as performing an inverse mapping



- Although our mappings are not necessarily linear in either direction we learn a lot by understanding linear operators

Inverse Problems

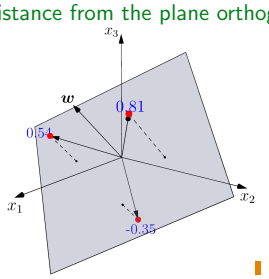
- Given m observations $\{(\boldsymbol{x}_k,y_k)|k=1,\ldots,m\}$, find the p unknowns $\boldsymbol{w}=(w_1,w_2,\ldots,w_p)^\mathsf{T}$ such that $\boldsymbol{x}_k^\mathsf{T}\boldsymbol{w}=y_k$
- Define the *design matrix* as the matrix of feature vectors

$$\mathbf{X}=\begin{pmatrix}\boldsymbol{x}_1^\mathsf{T}\\ \boldsymbol{x}_2^\mathsf{T}\\ \vdots\\ \boldsymbol{x}_m^\mathsf{T}\end{pmatrix}=\begin{pmatrix}x_{11}&x_{12}&\cdots&x_{1p}\\ x_{21}&x_{22}&\cdots&x_{2p}\\ \vdots&\vdots&\ddots&\vdots\\ x_{m1}&x_{m2}&\cdots&x_{mp}\end{pmatrix}$$

- and the target vector $\boldsymbol{y}=(y_1,y_2,\cdots,y_m)^\mathsf{T}$
- Then $\boldsymbol{y}=\mathbf{X}\boldsymbol{w}$ and if $m=p$ (and $\mathbf{X}$ is invertible) $\boldsymbol{w}=\mathbf{X}^{-1}\boldsymbol{y}$

Linear Regression

- $\boldsymbol{x}_k^\mathsf{T}\boldsymbol{w}$ depends on distance from the plane orthogonal to $\boldsymbol{w}$



- If $m>p$ then $\mathbf{X}$ isn't square so doesn't have an inverse
- Worse, unless the data is exact, $\boldsymbol{y}\approx\mathbf{X}\boldsymbol{w}\Rightarrow$ no exact "solution"
- Problem solved by Gauss to predict the orbit of the asteroid Ceres

Linear Least Squares

- The error of input pattern $\boldsymbol{x}_k$ is

$$\epsilon_k=\boldsymbol{x}_k^\mathsf{T}\boldsymbol{w}-y_k$$

- We can define the error vector

$$\boldsymbol{\epsilon}=\mathbf{X}\boldsymbol{w}-\boldsymbol{y}$$

(note that $\epsilon_k=\boldsymbol{x}_k^\mathsf{T}\boldsymbol{w}-y_k$)

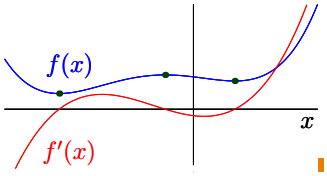
- The squared loss

$$L(\boldsymbol{w}|\mathcal{D})=\sum_{k=1}^m\left(\boldsymbol{x}_k^\mathsf{T}\boldsymbol{w}-y_k\right)^2=\sum_{k=1}^m\epsilon_k^2=\|\boldsymbol{\epsilon}\|^2$$

- Minimising this loss is known as the least squares problem

Finding a Minimum

- At a minimum of a one-dimensional function, $f(x)$, we have $f'(x)=0$



- At a minimum of an n-dimensional function $f(\boldsymbol{x})$ we have the set of equations

$$\frac{\partial f(\boldsymbol{x})}{\partial x_i}=0\quad\forall i=1,\ldots,n$$

Least Squares Solution

- The least squares solution is given by

$$\begin{aligned}\nabla L(\boldsymbol{w}|\mathcal{D})&=\nabla\|\boldsymbol{\epsilon}\|^2=\nabla\|\mathbf{X}\boldsymbol{w}-\boldsymbol{y}\|^2\\&=\nabla\left(\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w}-2\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y}+\boldsymbol{y}^\mathsf{T}\boldsymbol{y}\right)\\&=2\left(\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w}-\mathbf{X}^\mathsf{T}\boldsymbol{y}\right)=0\end{aligned}$$

- Or

$$\boldsymbol{w}=\left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}=\mathbf{X}^+\boldsymbol{y}$$

- $\mathbf{X}^+=\left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}\mathbf{X}^\mathsf{T}$ is known as the pseudo-inverse

Missing Bits of the Mathematics

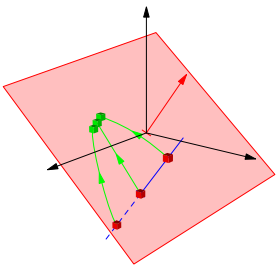
- Note that $\|\boldsymbol{a}\|^2 = \boldsymbol{a}^\mathsf{T}\boldsymbol{a} = \sum_i a_i^2$

$$\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2 = (\mathbf{X}\boldsymbol{w} - \boldsymbol{y})^\mathsf{T}(\mathbf{X}\boldsymbol{w} - \boldsymbol{y}) = (\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T} - \boldsymbol{y}^\mathsf{T})(\mathbf{X}\boldsymbol{w} - \boldsymbol{y})$$
$$= \boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w} - 2\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y} + \boldsymbol{y}^\mathsf{T}\boldsymbol{y}$$

- Where we have used $\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y} = \boldsymbol{y}^\mathsf{T}\mathbf{X}\boldsymbol{w}$, $\sum_{i,j} w_i X_{ji} y_j = \sum_{i,j} y_i X_{ij} w_j$
- Also $\nabla \boldsymbol{w}^\mathsf{T}\mathbf{M}\boldsymbol{w} = \mathbf{M}\boldsymbol{w} + \mathbf{M}^\mathsf{T}\boldsymbol{w} = (\mathbf{M} + \mathbf{M}^\mathsf{T})\boldsymbol{w}$
- If $\mathbf{M} = \mathbf{M}^\mathsf{T}$ (i.e. $\mathbf{M}$ is symmetric) then $\nabla \boldsymbol{w}^\mathsf{T}\mathbf{M}\boldsymbol{w} = 2\mathbf{M}\boldsymbol{w}$
- As $(\mathbf{AB})^\mathsf{T} = \mathbf{B}^\mathsf{T}\mathbf{A}^\mathsf{T}$ then $(\mathbf{X}^\mathsf{T}\mathbf{X})^\mathsf{T} = \mathbf{X}^\mathsf{T}\mathbf{X}^{\mathsf{TT}} = \mathbf{X}^\mathsf{T}\mathbf{X}$ and $\mathbf{X}^\mathsf{T}\mathbf{X}$ is symmetric

Outline

1. Least Squares
2. **Under-constrained Systems**
3. Ill-conditioning

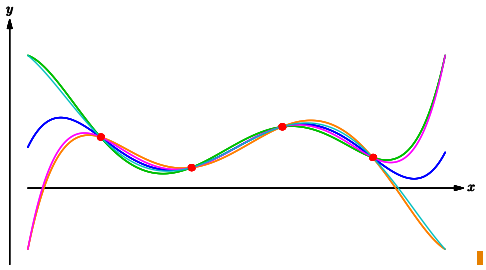


Solving Inverse Problems

- Gauss showed us how to solve **over-constrained** problems (we have more observations than parameters)
- We seek a solution which isn't necessarily exact but minimises an error
- But, what if we have more parameters than observations?
- That is, we are **under-constrained**
- Note that in some directions you might be over-constrained and in other directions under-constrained
- This is very typical of most machine learning problems

Under-constrained Systems

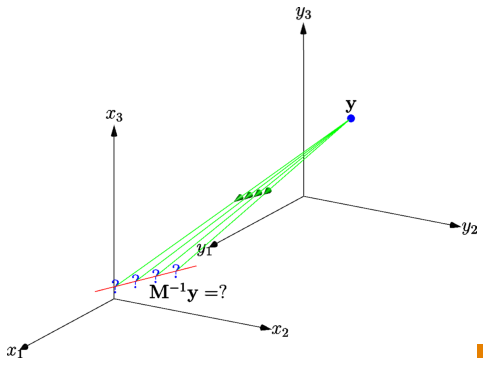
- If we have fewer data points than parameters then there will be multiple solutions, e.g. fitting $y = w_0 + w_1 x + \cdots + w_5 x^5$ to 4 points



- Every curve fits the data exactly, so which one should we choose?

What is the Inverse?

- Many points can map to the same point



Under-constrained Systems

- The system is **under-constrained**
- We have more unknowns than equations
- The inverse is not unique
- Solving the inverse problem ($\boldsymbol{w} = \left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$) is said to be **ill-posed**
- The inverse $\left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}$ doesn't exist
- If we have a complicated learning machine and not sufficient data we often end up with an ill-posed inverse problem (there are lots of sets of parameters that explain the data)

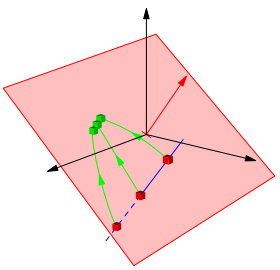
Handling Under-constrained Systems

- The issue with under-constrained systems is not that there is no solution
- There is a whole family of solutions, so which one should we pick
- William of Ockham in the 14th century postulated that the "simplest solution" is likely to be the correct solution
- How we determine the simplest solution is a bit of a black art
- However, for linear regression, adding an L_2 regulariser makes the solution unique

$$L_{full} = L_{fit}(\boldsymbol{w}) + \eta\|\boldsymbol{w}\|_2^2$$

Outline

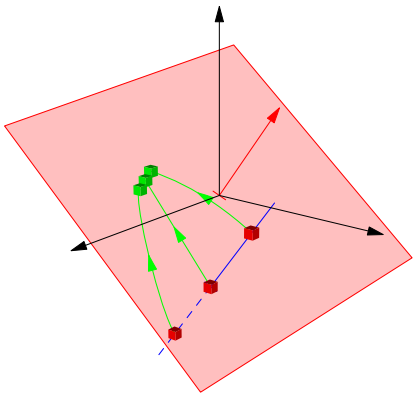
1. Least Squares
2. Under-constrained Systems
3. **Ill-conditioning**



Ill-Conditioning

- Singular matrices are rare (although they occur when we don't have enough data), but matrices that are close to being singular are common
- If a matrix is close to singular it is ill-conditioned
- Ill-conditioned matrices have some eigenvalues that are tiny compared with the largest
- All points get squashed towards a lower-dimensional subspace (a plane in 3-D)
- Large matrices are very often ill-conditioned

Ill-Conditioned Matrices



A Worked Example

- Two data points and two features that are almost the same

$$\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 1.001 \end{pmatrix}, \qquad \boldsymbol{y} = \begin{pmatrix} 2 \\ 2.001 \end{pmatrix} \quad \Rightarrow \quad \boldsymbol{w} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

- Change y_2 by 0.001 (i.e. 0.05%)

$$\boldsymbol{y} = \begin{pmatrix} 2 \\ 2.002 \end{pmatrix} \quad \Rightarrow \quad \boldsymbol{w} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$$

- The prediction for a new input $\boldsymbol{x} = (1,0)^\mathsf{T}$ jumps from 1 to 0
- The eigenvalues of $\mathbf{X}$ are about 2 and 0.0005
- $\boldsymbol{w}$ moved along $(-1,1)^\mathsf{T}$, the direction with the tiny eigenvalue, which the data hardly constrains

Ill-Conditioning in ML

- Ill-conditioning in machine learning occurs when a very small change in the learning data causes a large change in the predictions of the learning machine
- In linear regression the matrix $\mathbf{X}^\mathsf{T}\mathbf{X}$ is typically ill-conditioned when we have roughly as many data points as parameters
- Much of machine learning is concerned with making learning machines better conditioned
- Adding regularisers is one approach to achieve this

Where Does Ill-Conditioning Come From?

- Too little data: about as many data points as parameters
- Nearly collinear features, e.g. floor area and number of rooms, or height measured in both cm and inches
  - ⋆ The columns of $\mathbf{X}$ are nearly parallel so the data cannot tell us how to share the weight between them
- Polynomial features $x, x^2, \ldots, x^9$ for $x \in [0,1]$ all look alike: the eigenvalues of $\mathbf{X}^\mathsf{T}\mathbf{X}$ span about 13 orders of magnitude
- Features on very different scales (income in pounds and age in years)—easily fixed by standardising the features

Least Squares in Practice

- $\boldsymbol{w} = \left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$ is for understanding, not for computing
- Never compute $\left(\mathbf{X}^\mathsf{T}\mathbf{X}\right)^{-1}$; use a least squares solver such as `np.linalg.lstsq(X, y)`, which works with $\mathbf{X}$ directly
- Forming $\mathbf{X}^\mathsf{T}\mathbf{X}$ squares the ratio of largest to smallest eigenvalues: 10^8 becomes 10^{16}, beyond the $\sim$16 significant figures of a double
- Standardise features (zero mean, unit variance) before fitting
- When $m < p$, `lstsq` returns the solution with the smallest $\|\boldsymbol{w}\|$—one version of the "simplest" solution (we will see why when we meet the SVD)

Three Regimes

	$m > p$ over-constrained	$m = p$	$m < p$ under-constrained
Exact fit?	usually not	yes	yes, many
Unique?	yes (usually)	yes (usually)	no
Method	least squares	$\boldsymbol{w} = \mathbf{X}^{-1}\boldsymbol{y}$	pick "simplest"

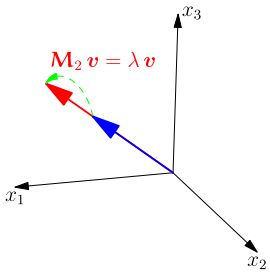
- Real problems mix these: over-constrained in some directions and (nearly) under-constrained in others
- The nearly under-constrained directions make the problem ill-conditioned

Summary

- Linear mappings are commonly used in machine learning algorithms such as regression
- Systems can be over-constrained, under-constrained or ill-conditioned—often all at once
- We will often meet the pseudo-inverse involving inverting $\mathbf{X}^\mathsf{T}\mathbf{X}$
- Inverting an ill-conditioned $\mathbf{X}^\mathsf{T}\mathbf{X}$ amplifies noise in the data

Mathematics of Machine Learning

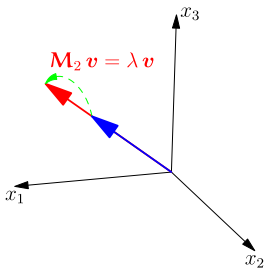
Eigensystems



Eigenvectors, Orthogonal Matrices, Eigenvector Decomposition, Rank

Outline

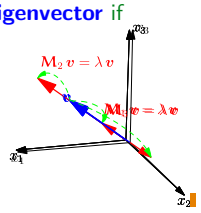
1. **Eigenvectors**
2. Orthogonal Matrices
3. Eigen Decomposition
4. Low Rank Approximation
5. Non-Normal Matrices



Eigenvector equation

- Eigen-systems help us to understand mappings
- A non-zero vector $\boldsymbol{v}$ is said to be an **eigenvector** if

$$\mathbf{M}\boldsymbol{v} = \lambda\boldsymbol{v}$$



- $\mathbf{M}$ is square (i.e. $n\times n$)
- Where the number λ is the **eigenvalue**
- Eigenvalues play a fundamental role in understanding operators

Symmetric Matrices

- If $\mathbf{M}$ is an $n\times n$ **symmetric** matrix then it has n real orthogonal eigenvectors with real eigenvalues
- We denote the i^{th} eigenvector by $\boldsymbol{v}_i$ and the corresponding eigenvalue by λ_i so that

$$\mathbf{M}\boldsymbol{v}_i = \lambda_i\boldsymbol{v}_i$$

- Orthogonal means that if $i \neq j$ then

$$\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = 0$$

- (We can always normalise eigenvectors if we want)

Proof of Orthogonality

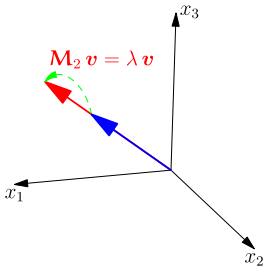
- $\left(\mathbf{M}\boldsymbol{v}_i = \lambda_i\boldsymbol{v}_i\right)^\mathsf{T}$ implies $\boldsymbol{v}_i^\mathsf{T}\mathbf{M}^\mathsf{T} = \lambda_i\boldsymbol{v}_i^\mathsf{T}$
- When $\mathbf{M}$ is symmetric then $\mathbf{M}\boldsymbol{v}_i = \lambda_i\boldsymbol{v}_i \Rightarrow \boldsymbol{v}_i^\mathsf{T}\mathbf{M} = \lambda_i\boldsymbol{v}_i^\mathsf{T}$
- Consider two eigenvectors $\boldsymbol{v}_i$ and $\boldsymbol{v}_j$ of $\mathbf{M}$

$$\begin{aligned}\boldsymbol{v}_i^\mathsf{T}\mathbf{M}\boldsymbol{v}_j &= (\boldsymbol{v}_i^\mathsf{T}\mathbf{M})\boldsymbol{v}_j = \lambda_i\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j \\ &= \boldsymbol{v}_i^\mathsf{T}(\mathbf{M}\boldsymbol{v}_j) = \lambda_j\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j\end{aligned}$$

- So either $\lambda_i = \lambda_j$ or $\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j = 0$
- If $\lambda_i = \lambda_j$ then any linear combination of $\boldsymbol{v}_i$ and $\boldsymbol{v}_j$ is an eigenvector $(\mathbf{M}(a\boldsymbol{v}_i + b\boldsymbol{v}_j) = \lambda_i(a\boldsymbol{v}_i + b\boldsymbol{v}_j))$ So I can choose two eigenvectors that are orthogonal to each other.

Outline

1. Eigenvectors
2. **Orthogonal Matrices**
3. Eigen Decomposition
4. Low Rank Approximation
5. Non-Normal Matrices



Orthogonal Matrices

- We can construct an **orthogonal** matrix $\mathbf{V}$ from the normalised eigenvectors $(\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_i = 1)$

$$\mathbf{V} = (\boldsymbol{v}_1, \boldsymbol{v}_2, \cdots, \boldsymbol{v}_n)$$

- Matrix $\mathbf{V}$ is an $n\times n$ matrix
- Because the vectors $\boldsymbol{v}_i$ are orthogonal and normalised

$$\mathbf{V}^\mathsf{T}\mathbf{V} = \begin{pmatrix} \boldsymbol{v}_1^\mathsf{T}\boldsymbol{v}_1 & \boldsymbol{v}_1^\mathsf{T}\boldsymbol{v}_2 & \cdots & \boldsymbol{v}_1^\mathsf{T}\boldsymbol{v}_n \\ \boldsymbol{v}_2^\mathsf{T}\boldsymbol{v}_1 & \boldsymbol{v}_2^\mathsf{T}\boldsymbol{v}_2 & \cdots & \boldsymbol{v}_2^\mathsf{T}\boldsymbol{v}_n \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{v}_n^\mathsf{T}\boldsymbol{v}_1 & \boldsymbol{v}_n^\mathsf{T}\boldsymbol{v}_2 & \cdots & \boldsymbol{v}_n^\mathsf{T}\boldsymbol{v}_n \end{pmatrix} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} = \mathbf{I}$$

The Other Way Around

- We have shown that $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{I}$
- Thus multiply both sides on the left by $\mathbf{V}$

$$\mathbf{V}\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{V}$$

- $\mathbf{V}$ will have an inverse (we show this on the next slide), $\mathbf{V}^{-1}$, such that $\mathbf{V}\mathbf{V}^{-1} = \mathbf{I}$
- Multiplying the equation on the right by $\mathbf{V}^{-1}$

$$\begin{aligned}(\mathbf{V}\mathbf{V}^\mathsf{T})\mathbf{V}\mathbf{V}^{-1} &= \mathbf{V}\mathbf{V}^{-1} \\ \mathbf{V}\mathbf{V}^\mathsf{T} &= \mathbf{I}\end{aligned}$$

- Note that, $\mathbf{V}^{-1} = \mathbf{V}^\mathsf{T}$ (definition of orthogonal matrix)

Invertible Matrices

- A matrix, $\mathbf{M}$, will be singular (not invertible) if there exists a vector $\boldsymbol{x}$ ($\neq \mathbf{0}$) such that

$$\mathbf{M}\boldsymbol{x} = \mathbf{0}$$

- Now if there exists such a vector such that $\mathbf{V}\boldsymbol{x} = \mathbf{0}$ then multiplying by $\mathbf{V}^\mathsf{T}$ we get

$$\mathbf{V}^\mathsf{T}\mathbf{V}\boldsymbol{x} = \mathbf{V}^\mathsf{T}\mathbf{0}$$
$$\boldsymbol{x} = \mathbf{0}$$

since $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{I}$

- Thus $\mathbf{V}$ is invertible

Rotations

- Orthogonal matrices satisfy $\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{I}$
- As a consequence they define rotations (and possibly a reflection)
- Consider a vector $\boldsymbol{x}$ and $\boldsymbol{x}' = \mathbf{V}\boldsymbol{x}$, now

$$\|\boldsymbol{x}'\|_2^2 = \boldsymbol{x}'^\mathsf{T}\boldsymbol{x}' = (\mathbf{V}\boldsymbol{x})^\mathsf{T}(\mathbf{V}\boldsymbol{x}) = \boldsymbol{x}^\mathsf{T}\mathbf{V}^\mathsf{T}\mathbf{V}\boldsymbol{x} = \boldsymbol{x}^\mathsf{T}\boldsymbol{x} = \|\boldsymbol{x}\|_2^2$$

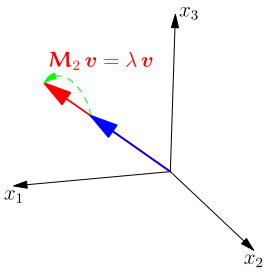
- Similarly if additionally $\boldsymbol{y}' = \mathbf{V}\boldsymbol{y}$ then

$$\langle \boldsymbol{x}', \boldsymbol{y}' \rangle = (\mathbf{V}\boldsymbol{x})^\mathsf{T}(\mathbf{V}\boldsymbol{y}) = \boldsymbol{x}^\mathsf{T}\mathbf{V}^\mathsf{T}\mathbf{V}\boldsymbol{y} = \boldsymbol{x}^\mathsf{T}\boldsymbol{y} = \langle \boldsymbol{x}, \boldsymbol{y} \rangle = \|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2 \cos(\theta)$$

- Rotations and reflections preserve lengths and angles

Outline

1. Eigenvectors
2. Orthogonal Matrices
3. **Eigen Decomposition**
4. Low Rank Approximation
5. Non-Normal Matrices



Matrix Decomposition

- Taking the matrix of eigenvectors, $\mathbf{V}$, then

$$\mathbf{M}\mathbf{V} = \mathbf{M}(\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_n) = (\lambda_1\boldsymbol{v}_1, \lambda_2\boldsymbol{v}_2, \ldots, \lambda_n\boldsymbol{v}_n) = \mathbf{V}\boldsymbol{\Lambda}$$

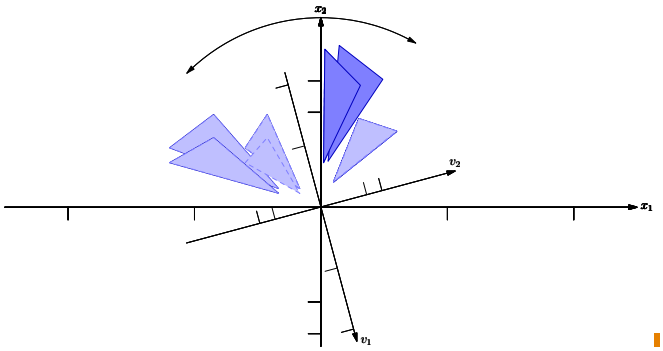
- where $\boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n) = \begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{pmatrix}$
- Now

$$\mathbf{M} = \mathbf{M}\mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}$$

- Very important *similarity transform*

Mappings by Symmetric Matrices

$$\mathbf{M} = \begin{pmatrix} 0.83 & -0.31 \\ -0.31 & 1.9 \end{pmatrix} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T} = \begin{pmatrix} \cos(-75°) & -\sin(-75°) \\ \sin(-75°) & \cos(-75°) \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 0.75 \end{pmatrix} \begin{pmatrix} \cos(75°) & -\sin(75°) \\ \sin(75°) & \cos(75°) \end{pmatrix}$$



Inverses

- For any symmetric invertible matrix

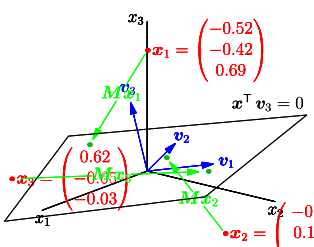
$$\mathbf{M} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T} \qquad \mathbf{M}^{-1} = \mathbf{V}\boldsymbol{\Lambda}^{-1}\mathbf{V}^\mathsf{T}$$

- Where $\boldsymbol{\Lambda}^{-1} = \mathrm{diag}(\frac{1}{\lambda_1}, \frac{1}{\lambda_2}, \ldots, \frac{1}{\lambda_n}) = \begin{pmatrix} \frac{1}{\lambda_1} & 0 & \cdots & 0 \\ 0 & \frac{1}{\lambda_2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{\lambda_n} \end{pmatrix}$
- Since

$$\mathbf{M}\mathbf{M}^{-1} = (\mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T})(\mathbf{V}\boldsymbol{\Lambda}^{-1}\mathbf{V}^\mathsf{T}) = \mathbf{V}\boldsymbol{\Lambda}(\mathbf{V}^\mathsf{T}\mathbf{V})\boldsymbol{\Lambda}^{-1}\mathbf{V}^\mathsf{T}$$
$$= \mathbf{V}\boldsymbol{\Lambda}\boldsymbol{\Lambda}^{-1}\mathbf{V}^\mathsf{T} = \mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{I}$$

- I.e. small eigenvalues become large eigenvalues and vice versa

Ill-Conditioning Again



$$\boldsymbol{M} = \begin{pmatrix} 0.80 & 0.01 & 0.25 \\ 0.01 & 0.85 & 0.48 \\ 0.25 & 0.48 & 0.5 \end{pmatrix} = \boldsymbol{V} \begin{pmatrix} 0.8 & 0 & 0 \\ 0 & 1.25 & 0 \\ 0 & 0 & 0.1 \end{pmatrix} \boldsymbol{V}^\mathsf{T}$$

$$\boldsymbol{M}^{-1} = \begin{pmatrix} 1.9 & 1.17 & -2.1 \\ 1.17 & 3.34 & -3.8 \\ -2.1 & -3.8 & 6.8 \end{pmatrix} = \boldsymbol{V} \begin{pmatrix} 1.25 & 0 & 0 \\ 0 & 0.8 & 0 \\ 0 & 0 & 10 \end{pmatrix} \boldsymbol{V}^\mathsf{T}$$

Condition Number

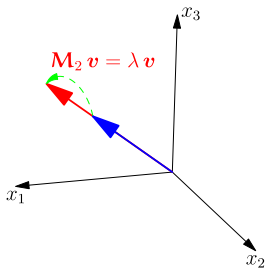
- Taking matrix inverses can be inherently unstable
- Any small error can be amplified by taking the inverse
- The stability of the inverse depends on the ratio of smallest eigenvalue to the largest eigenvalue (i.e. the biggest possible amplification compared to the smallest)
- For symmetric matrices the largest $|\lambda|$ is the spectral norm, $\|\mathbf{M}\|_2$, from the vector spaces lecture
- The condition number is given by

$$\|\mathbf{M}\|_2 \times \|\mathbf{M}^{-1}\|_2 = \frac{|\lambda_{\max}|}{|\lambda_{\min}|}$$

- Large condition number implies very ill-conditioned

Outline

1. Eigenvectors
2. Orthogonal Matrices
3. Eigen Decomposition
4. **Low Rank Approximation**
5. Non-Normal Matrices



Rank of a Matrix

- The rank of a symmetric matrix, $\mathbf{M}$, is the number of non-zero eigenvalues
- The space spanned by the eigenvectors $\boldsymbol{v}_a$, $\boldsymbol{v}_b$, etc. with zero eigenvalue forms a **null space**
- Any vector in the null space will get mapped to the zero vector

$$\mathbf{M}(a\boldsymbol{v}_a + b\boldsymbol{v}_b + \cdots) = \mathbf{0}$$

- A square matrix is said to be **rank deficient** if it has any eigenvectors with eigenvalue equal to 0
- This happens when the columns of the matrix are not linearly independent

"Inverting" Rank Deficient Matrices

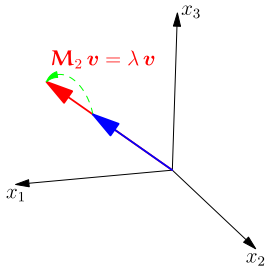
- Rank deficient matrices are non-invertible (i.e. we don't know the vector $\boldsymbol{x}$ such that $\mathbf{M}\boldsymbol{x} = \boldsymbol{b}$) as we don't know the component of the $\boldsymbol{x}$ in the null space
- Although we don't know $\boldsymbol{x}$ we can find a vector, $\boldsymbol{x}$, that satisfies $\mathbf{M}\boldsymbol{x} = \boldsymbol{b}$ (provided one exists)
- Given a symmetric $n \times n$ matrix with k non-zero eigenvalues λ_1, λ_2, ..., λ_k we can construct a "pseudo-inverse" $\mathbf{M}^+$ as $\mathbf{V}\boldsymbol{\Lambda}^+\mathbf{V}^\mathsf{T}$ where $\boldsymbol{\Lambda}^+ = \mathrm{diag}(\frac{1}{\lambda_1}, \frac{1}{\lambda_2}, \ldots, \frac{1}{\lambda_k}, 0, \ldots, 0)$
- This finds the vector $\boldsymbol{x}$ with no component in the null space (it is the solution with the smallest norm)
- This and $(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}$ are special cases of the pseudo-inverse we meet with SVD

Low Rank Approximation

- Recall that matrices with large and small eigenvalues are ill-conditioned so the inverse has the potential to greatly amplify any measurement error
- One work around is to set all small eigenvalues to zero and use the pseudo-inverse
- Setting small eigenvalues to zero reduces the rank of the matrix and is an example of a low rank approximation
- Low rank approximations are much used to obtain approximate models for arrays of data (we will revisit this when we look at SVD)

Outline

1. Eigenvectors
2. Orthogonal Matrices
3. Eigen Decomposition
4. Low Rank Approximation
5. **Non-Normal Matrices**



Non-Normal Matrices

- Symmetric matrices are part of a class of matrices known as **normal matrices**, which satisfy $\mathbf{M}\mathbf{M}^\dagger = \mathbf{M}^\dagger\mathbf{M}$ (for real matrices $\mathbf{M}^\dagger = \mathbf{M}^\mathsf{T}$)
- Hermitian matrices, $\mathbf{H}$, are complex matrices that equal their conjugate transpose: $\mathbf{H} = (\mathbf{H}^\mathsf{T})^* = \mathbf{H}^\dagger$
- Hermitian matrices are also normal; they are important in quantum theory, less so in machine learning
- Normal matrices are exactly those with a full set of orthonormal (possibly complex) eigenvectors
- Most non-symmetric real matrices are not normal (rotations are an exception); their eigenvectors are more confusing

Asymmetric Matrices

- For $n \times n$ real symmetric matrices we are guaranteed there are n orthogonal real eigenvectors
- For non-symmetric matrices the eigenvectors will generally not be orthogonal, they won't necessarily be real
- We are not even guaranteed there will be n linearly independent eigenvectors
- Matrices which don't have n linearly independent eigenvectors are said to be **defective**, e.g. $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ has only one
- Randomly generated matrices will typically not be defective, but nevertheless they can arise in certain situations

Characteristic Equation

- Since our eigenvectors are defined by $\mathbf{M}\boldsymbol{v} = \lambda\boldsymbol{v}$ then

$(\mathbf{M} - \lambda\mathbf{I})\boldsymbol{v} = \mathbf{0}$ has a solution $\boldsymbol{v} \neq \mathbf{0}$ only if $\det(\mathbf{M} - \lambda\mathbf{I}) = 0$

- This is known as the characteristic equation and leads to an n^{th} order polynomial in λ
- An n^{th} order polynomial has n roots (counting repeats), which are in general complex
- Thus even for real (non-symmetric) matrices we can get complex eigenvalues

Complex Conjugate Pairs

- If $\mathbf{M}$ is real and λ is complex then $\boldsymbol{v}$ must be complex too, otherwise $\mathbf{M}\boldsymbol{v}$ would be real but $\lambda\boldsymbol{v}$ would not
- Furthermore

$$\mathbf{M}\boldsymbol{v} = \lambda\boldsymbol{v} \qquad \Longrightarrow \qquad \mathbf{M}\boldsymbol{v}^* = \lambda^*\boldsymbol{v}^*$$

- So that eigenvectors and their corresponding eigenvalues exist in complex conjugate pairs

Left and Right Eigenvectors

- For symmetric matrices if $\boldsymbol{v}$ is a right-hand eigenvector in the sense that $\mathbf{M}\boldsymbol{v} = \lambda\boldsymbol{v}$ then $\boldsymbol{v}^\mathsf{T}$ is a left-hand eigenvector

$$\boldsymbol{v}^\mathsf{T}\mathbf{M} = \lambda\boldsymbol{v}^\mathsf{T}$$

- Earlier we used this property to show that the eigenvectors for symmetric matrices are orthogonal
- For asymmetric real matrices the left-hand eigenvectors $\boldsymbol{u}^\mathsf{T}$ are different from the right-hand eigenvectors, although they share the same eigenvalues (they have the same characteristic equation)

$$\mathbf{M}\boldsymbol{v} = \lambda\boldsymbol{v} \qquad\qquad \boldsymbol{u}^\mathsf{T}\mathbf{M} = \lambda\boldsymbol{u}^\mathsf{T}$$

- As before, $\boldsymbol{u}_i^\mathsf{T}\boldsymbol{v}_j = 0$ if $\lambda_i \neq \lambda_j$, but in general $\boldsymbol{v}_i^\mathsf{T}\boldsymbol{v}_j \neq 0$

Eigen-Decomposition

- For a non-symmetric matrix, $\mathbf{M}$, provided it is not defective, we can decompose the matrix as

$$\mathbf{M} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{-1}$$

- $\boldsymbol{\Lambda}$ is a diagonal matrix of eigenvalues
- $\mathbf{V}$ is a matrix whose columns are the right-hand eigenvectors—in general it is not an orthogonal matrix
- $\mathbf{V}^{-1}$ is a matrix whose rows are the left-hand eigenvectors (scaled so that $\boldsymbol{u}_i^\mathsf{T}\boldsymbol{v}_i = 1$)

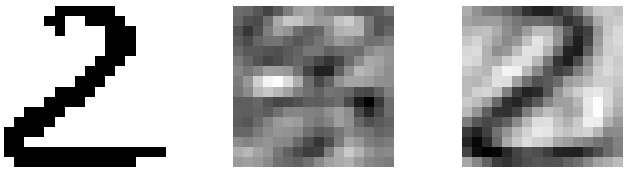
Summary

- Linear mappings are commonly used in machine learning algorithms such as regression
- We can understand symmetric operators by looking at their eigenvectors
- Any symmetric matrix can be decomposed as $\mathbf{M} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T}$
  - where $\mathbf{V}$ is an orthogonal matrix whose columns are the eigenvectors
  - and $\boldsymbol{\Lambda}$ is a diagonal matrix of the eigenvalues
- This decomposition allows us to understand inverse mappings
- Non-symmetric matrices are more complex—we use the eigen-decomposition less because of this

Mathematics of Machine Learning

Principal Component Analysis (PCA)

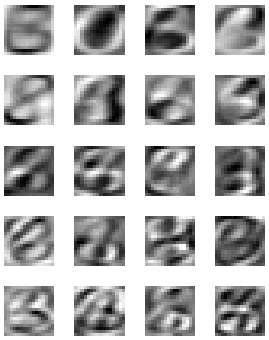
1.6 -1.1 -1.6 2.1 -0.52 2.8 0.72 0.7 -0.68 -0.41 -1.4 -1.5 -0.54 -0.62 1.3 -1.4 -0.27 0.74 0.77 -1



Covariance matrices, dimensionality reduction, PCA, Duality

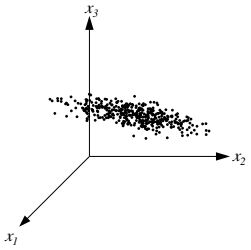
Outline

1. **Covariance Matrices**
2. Principal Component Analysis
3. Duality



Spread of Data

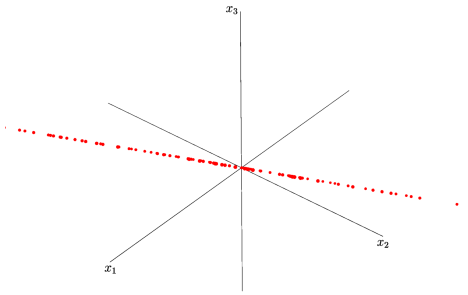
- Often data varies significantly in only some directions



- Reduce dimensions by projecting onto low dimensional subspace with maximum variation

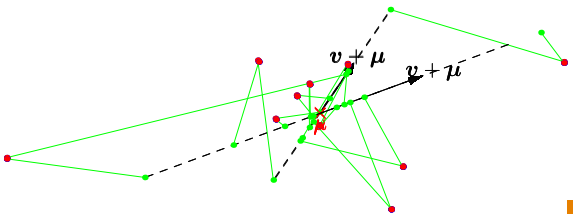
Looking is not Enough

Can't spot low dimensional data by looking at numbers



Dimensionality Reduction

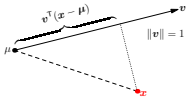
- Often helpful to consider only directions where data varies significantly
- Want to find directions along which data has its greatest variation



Direction of Maximum Variation

- Look for the vector $\boldsymbol{v}$ with $\|\boldsymbol{v}\|^2 = 1$ to maximise

$$\sigma^2 = \frac{1}{m-1}\sum_{i=1}^{m}\left(\boldsymbol{v}^\mathsf{T}(\boldsymbol{x}_i - \boldsymbol{\mu})\right)^2$$



- This is a constrained optimisation problem
- Solve by maximising Lagrangian

$$\mathcal{L} = \frac{1}{m-1}\sum_{k=1}^{m}\left(\boldsymbol{v}^\mathsf{T}(\boldsymbol{x}_k - \boldsymbol{\mu})\right)^2 - \lambda\left(\|\boldsymbol{v}\|^2 - 1\right)$$

- λ is a Lagrange multiplier

Direction of Maximum Variation

- Expanding the Lagrangian

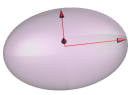
$$\begin{aligned}\mathcal{L} &= \frac{1}{m-1}\sum_{k=1}^{m}\left(\boldsymbol{v}^\mathsf{T}(\boldsymbol{x}_k - \boldsymbol{\mu})\right)^2 - \lambda\left(\|\boldsymbol{v}\|^2 - 1\right)\\ &= \frac{1}{m-1}\sum_{k=1}^{m}\left(\boldsymbol{v}^\mathsf{T}(\boldsymbol{x}_k - \boldsymbol{\mu})(\boldsymbol{x}_k - \boldsymbol{\mu})^\mathsf{T}\boldsymbol{v}\right) - \lambda\left(\|\boldsymbol{v}\|^2 - 1\right)\\ &= \boldsymbol{v}^\mathsf{T}\left(\frac{1}{m-1}\sum_{k=1}^{m}(\boldsymbol{x}_k - \boldsymbol{\mu})(\boldsymbol{x}_k - \boldsymbol{\mu})^\mathsf{T}\right)\boldsymbol{v} - \lambda\left(\|\boldsymbol{v}\|^2 - 1\right)\\ &= \boldsymbol{v}^\mathsf{T}\mathbf{C}\boldsymbol{v} - \lambda\left(\boldsymbol{v}^\mathsf{T}\boldsymbol{v} - 1\right)\end{aligned}$$

- Extrema of the Lagrangian

$$\nabla\mathcal{L} = 2(\mathbf{C}\boldsymbol{v} - \lambda\boldsymbol{v}) = 0 \qquad \Rightarrow \qquad \mathbf{C}\boldsymbol{v} = \lambda\boldsymbol{v}$$

Direction of Maximum Variation

- The eigenvectors are directions that are extrema of the variance



- The variance in direction $\boldsymbol{v}$ is equal to

$$\begin{aligned}\sigma^2 &= \frac{1}{m-1}\sum_{i=1}^{m}\left(\boldsymbol{v}^\mathsf{T}(\boldsymbol{x}_i - \boldsymbol{\mu})\right)^2\\ &= \boldsymbol{v}^\mathsf{T}\mathbf{C}\boldsymbol{v} = \lambda\boldsymbol{v}^\mathsf{T}\boldsymbol{v} = \lambda\end{aligned}$$

- The variance is maximised by the eigenvector with the maximum eigenvalue

Covariance Matrix

- The **covariance matrix** is defined as

$$\mathbf{C} = \frac{1}{m-1}\sum_{k=1}^{m}(\boldsymbol{x}_k - \boldsymbol{\mu})(\boldsymbol{x}_k - \boldsymbol{\mu})^{\mathsf{T}}$$

- The components C_{ij} measure how the i^{th} and j^{th} components co-vary

$$C_{ij} = \frac{1}{m-1}\sum_{k=1}^{m}(x_{ik} - \mu_i)(x_{jk} - \mu_j)$$

- C.f. covariance of random variables

$$\mathrm{Cov}(X,Y) = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])]$$

Outer Product

- Remember that the outer-product of two vectors is defined as

$$\boldsymbol{x}\boldsymbol{y}^{\mathsf{T}} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}\begin{pmatrix} y_1 & y_2 & \cdots & y_n \end{pmatrix} = \begin{pmatrix} x_1y_1 & x_1y_2 & \cdots & x_1y_n \\ x_2y_1 & x_2y_2 & \cdots & x_2y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_ny_1 & x_ny_2 & \cdots & x_ny_n \end{pmatrix}$$

- C.f. Inner product

$$\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y} = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix}\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = x_1y_1 + x_2y_2 + \cdots + x_ny_n$$

Matrix Form

- The covariance matrix is

$$\mathbf{C} = \frac{1}{m-1}\sum_{k=1}^{m}(\boldsymbol{x}_k - \boldsymbol{\mu})(\boldsymbol{x}_k - \boldsymbol{\mu})^{\mathsf{T}}$$

- Define the matrix

$$\mathbf{X} = \frac{1}{\sqrt{m-1}}(\boldsymbol{x}_1 - \boldsymbol{\mu}, \boldsymbol{x}_2 - \boldsymbol{\mu}, \cdots \boldsymbol{x}_m - \boldsymbol{\mu})$$

- We can write the covariance matrix as

$$\mathbf{C} = \mathbf{X}\mathbf{X}^{\mathsf{T}}$$

Properties of Covariance Matrix

- The **quadratic form** of a vector and matrix is defined as

$$\boldsymbol{v}^{\mathsf{T}}\mathbf{M}\boldsymbol{v}$$

- The quadratic form of a covariance matrix is non-negative for any vector

$$\boldsymbol{v}^{\mathsf{T}}\mathbf{C}\boldsymbol{v} = \boldsymbol{v}^{\mathsf{T}}\mathbf{X}\mathbf{X}^{\mathsf{T}}\boldsymbol{v} = \boldsymbol{u}^{\mathsf{T}}\boldsymbol{u} = \|\boldsymbol{u}\|^2 \geq 0$$

where $\boldsymbol{u} = \mathbf{X}^{\mathsf{T}}\boldsymbol{v}$

- Matrices with non-negative quadratic forms are known as **positive semi-definite**

Eigenvalue Decomposition

- The eigenvectors of $\mathbf{C}$ with the largest eigenvalues are known as the **principal components**
- The eigenvalues are all greater than or equal to zero
- Recall an eigenvector $\boldsymbol{v}$ satisfies the equation

$$\mathbf{C}\boldsymbol{v} = \lambda\boldsymbol{v}$$

- Multiplying both sides by $\boldsymbol{v}^{\mathsf{T}}$

$$\boldsymbol{v}^{\mathsf{T}}\mathbf{C}\boldsymbol{v} = \lambda\boldsymbol{v}^{\mathsf{T}}\boldsymbol{v} = \lambda\|\boldsymbol{v}\|^2$$

but $\boldsymbol{v}^{\mathsf{T}}\mathbf{C}\boldsymbol{v} \geq 0$ and $\|\boldsymbol{v}\|^2 > 0$ so

$$\lambda = \frac{\boldsymbol{v}^{\mathsf{T}}\mathbf{C}\boldsymbol{v}}{\|\boldsymbol{v}\|^2} \geq 0$$

Surface Defined by Matrix

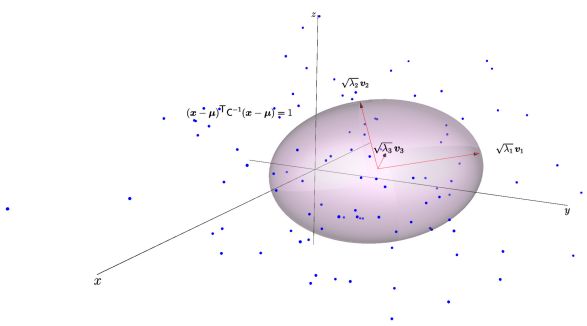
- The set of vectors $\boldsymbol{x}$ such that

$$\boldsymbol{x}^{\mathsf{T}}\mathbf{C}^{-1}\boldsymbol{x} = 1$$

defines a surface

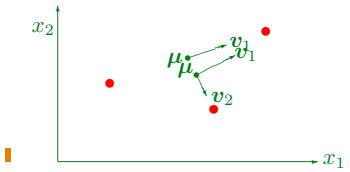
- The surface is an ellipsoid, $\mathcal{E}$
- The eigenvectors point in the direction of the principal axes of the ellipsoid
- The radii of the principal axes are equal to the square root of the eigenvalues

Ellipsoid and Eigen Space



Spanning Input Space

- A covariance matrix will have a zero eigenvalue only if there is no variation in the direction of the corresponding eigenvector
- A covariance matrix will have zero eigenvalues if the number of patterns are less than or equal to the number of dimensions
- A covariance matrix formed from $p+1$ patterns that do not all lie in a hyperplane (i.e. the p differences $\boldsymbol{x}_k - \boldsymbol{x}_1$ are linearly independent) will have no zero eigenvalues

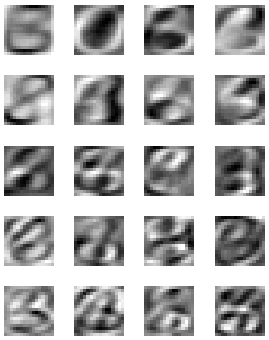


Positive Definite

- Matrices with no zero eigenvalues are called **full rank** matrices (as opposed to rank deficient)
- Full rank matrices are invertible, rank deficient matrices are singular and non-invertible
- Full rank covariance matrices have only positive eigenvalues and are said to be **positive definite**
- We would expect that when $m > p$ the covariance matrix will be positive definite unless there are some symmetries that linearly constrain the patterns

Outline

1. Covariance Matrices
2. **Principal Component Analysis**
3. Duality



Principal Component Analysis

- PCA occurs as follows
  - Construct the covariance matrix
  - Find the eigenvalues and eigenvectors
  - Keep the eigenvectors with the largest eigenvalues (principal components)
  - Project the inputs into the space spanned by the principal components
- We then use the projected inputs as inputs to our learning machine

Projection Matrix

- To project the inputs construct the projection matrix

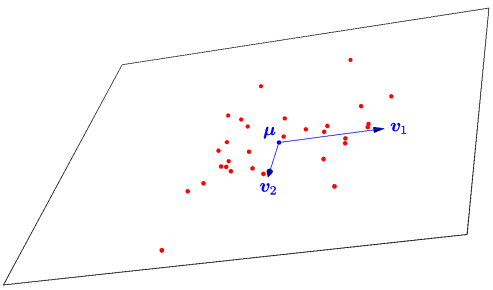
$$\mathbf{P} = \begin{pmatrix} \boldsymbol{v}_1^{\mathsf{T}} \\ \boldsymbol{v}_2^{\mathsf{T}} \\ \vdots \\ \boldsymbol{v}_k^{\mathsf{T}} \end{pmatrix}$$

- $k < p$ is the number of principal components we keep
- Given a p-dimensional input pattern $\boldsymbol{x}$ we can construct a k-dimensional representation $\boldsymbol{z}$

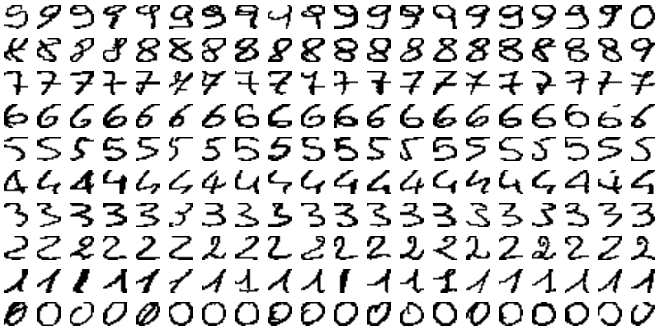
$$\boldsymbol{z} = \mathbf{P}(\boldsymbol{x} - \boldsymbol{\mu})$$

- Use $\boldsymbol{z}$ as our new inputs

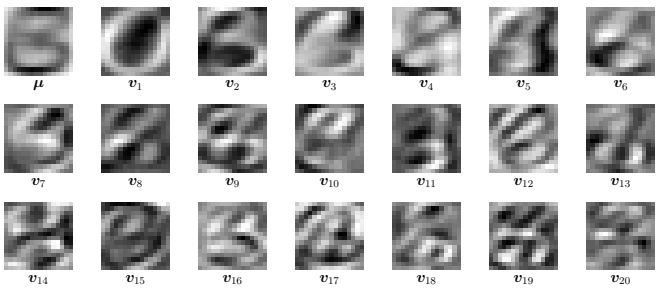
Subspace Projection



Hand Written Digits



Eigenvectors



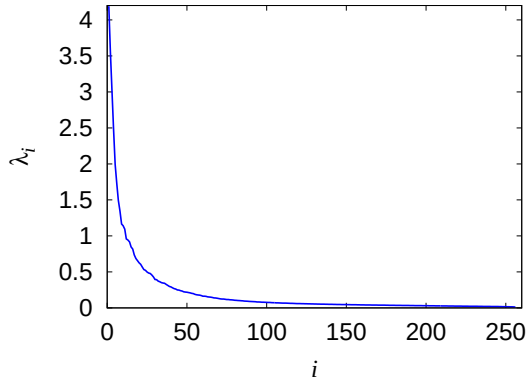
Reconstruction

- Projecting into a subspace of eigenvectors can be seen as approximating the inputs by

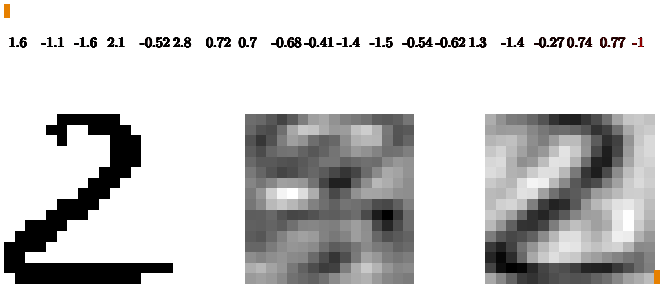
$$\hat{\boldsymbol{x}}_i = \boldsymbol{\mu} + \sum_{j=1}^{k} z_j^i \boldsymbol{v}_j, \qquad z_j^i = \boldsymbol{v}_j^{\mathsf{T}}(\boldsymbol{x}_i - \boldsymbol{\mu}), \qquad \|\boldsymbol{v}_j\| = 1$$

- Principal component analysis projects the data into the subspace of dimension k with the minimal approximation error $\mathbb{E}\left[\|\hat{\boldsymbol{x}}_i - \boldsymbol{x}_i\|^2\right]$
- The loss of "energy" (or squared error) is equal to the sum of the eigenvalues in the directions that are ignored

Eigenvalues for Digits

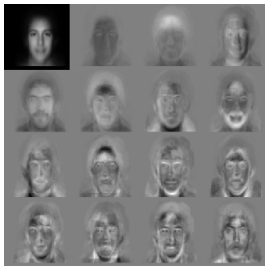


Reconstruction from Eigenvectors



Outline

1. Covariance Matrices
2. Principal Component Analysis
3. **Duality**



PCA for Images

- An image often contains around $p = 256 \times 256 = 64k$ pixels
- In standard PCA we would create an $p \times p$ matrix with over 4×10^9 elements
- This is intractable
- m images span at most a $m-1$ dimensional subspace
- Usually this subspace will be much smaller than the space of all images $m \ll p$

Dual Matrix

- The covariance $\mathbf{C} = \mathbf{X}\mathbf{X}^\mathsf{T}$ is a $p \times p$ matrix
- Consider the $m \times m$ matrix $\mathbf{D} = \mathbf{X}^\mathsf{T}\mathbf{X}$
- Suppose $\boldsymbol{v}$ is an eigenvector of $\mathbf{D}$

$$\begin{aligned} \mathbf{D}\boldsymbol{v} &= \lambda \boldsymbol{v} \\ \mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{v} &= \lambda \boldsymbol{v} \\ \mathbf{X}\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{v} &= \lambda \mathbf{X}\boldsymbol{v} \\ \mathbf{C}\mathbf{X}\boldsymbol{v} = \lambda \mathbf{X}\boldsymbol{v} \quad &\Rightarrow \quad \mathbf{C}\boldsymbol{u} = \lambda \boldsymbol{u} \end{aligned}$$

- $\boldsymbol{u} = \mathbf{X}\boldsymbol{v}$ (and $\boldsymbol{v} \propto \mathbf{X}^\mathsf{T}\boldsymbol{u}$)

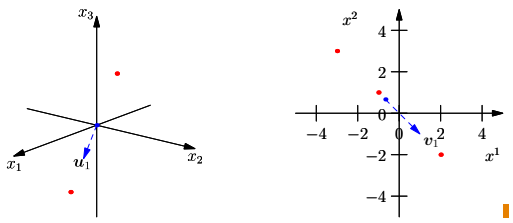
Dual Matrix

- Matrices $\mathbf{C} = \mathbf{X}\mathbf{X}^\mathsf{T}$ and $\mathbf{D} = \mathbf{X}^\mathsf{T}\mathbf{X}$ have the same non-zero eigenvalues
- Can use the dual $m \times m$ matrix $\mathbf{D}$ to find eigenvalues and eigenvectors of $\mathbf{C}$
- Note that $\mathbf{D} = \mathbf{X}^\mathsf{T}\mathbf{X}$ has components $D_{kl} \propto (\boldsymbol{x}_k - \boldsymbol{\mu})^\mathsf{T}(\boldsymbol{x}_l - \boldsymbol{\mu})$
- Takes $O(p \times m \times m)$ time to construct $\mathbf{D}$
- We work in a "dual space" which is the space spanned by the examples

What Does a Subspace Look Like?

- Consider $\boldsymbol{y}^1 = \begin{pmatrix} 2 \\ 4 \\ 5 \end{pmatrix}$, $\boldsymbol{y}^2 = \begin{pmatrix} 8 \\ 6 \\ 1 \end{pmatrix}$ with mean $\boldsymbol{\mu} = \begin{pmatrix} 5 \\ 5 \\ 3 \end{pmatrix}$
- Subtracting the mean $\boldsymbol{x}^i = \boldsymbol{y}^i - \boldsymbol{\mu}$ we can construct matrix

$$\mathbf{X} = \begin{pmatrix} x_1^1 & x_1^2 \\ x_2^1 & x_2^2 \\ x_3^1 & x_3^2 \end{pmatrix} = \begin{pmatrix} -3 & 3 \\ -1 & 1 \\ 2 & -2 \end{pmatrix}$$

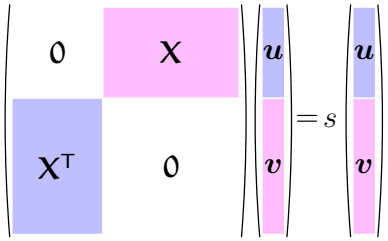


Summary

- PCA allows us to reduce the dimensionality of the inputs
- We project the inputs into a sub-space where the data varies the most
- We can work in either the original space ($\mathbf{X}\mathbf{X}^\mathsf{T}$) or the dual space ($\mathbf{X}^\mathsf{T}\mathbf{X}$)
- When we have many more features than examples (i.e. $p \gg m$) then it is more efficient working in the dual space
- We will see examples of dual spaces again when we look at SVMs

Mathematics of Machine Learning

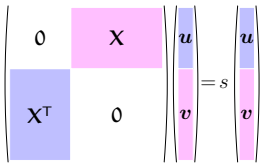
Singular Value Decomposition (SVD)



Singular Valued Decomposition, SVD, general linear maps

Outline

1. **Singular Value Decomposition**
2. General Linear Mappings
3. Linear Regression Revisited



Singular Value Decomposition

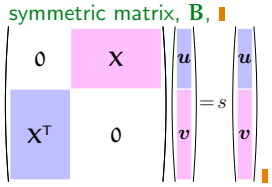
- One of the most important results in linear algebra is that any matrix $\mathbf{X}$ can be decomposed as

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$$

- If $\mathbf{X}$ is an $m \times p$ matrix then $\mathbf{U}$ is an $m \times m$ orthogonal matrix and $\mathbf{V}$ is a $p \times p$ orthogonal matrix i.e. $\mathbf{U}\mathbf{U}^\mathsf{T} = \mathbf{U}^\mathsf{T}\mathbf{U} = \mathbf{I}$, $\mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{I}$
- And $\mathbf{S}$ is an $m \times p$ matrix with non-zero diagonal elements $S_{ii} = s_i$ known as the **singular values**
- Where did that come from?

Eigen Decomposition

- Consider an arbitrary $m \times p$ matrix $\mathbf{X}$, and construct the $(m+p) \times (m+p)$ symmetric matrix, $\mathbf{B}$,



$\binom{\boldsymbol{u}}{\boldsymbol{v}}$ is an eigenvector of $\mathbf{B}$ with eigenvalue s

- We observe that

$$\mathbf{X}\boldsymbol{v} = s\boldsymbol{u} \qquad \mathbf{X}^\mathsf{T}\boldsymbol{u} = s\boldsymbol{v}$$
$$\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{v} = s\mathbf{X}^\mathsf{T}\boldsymbol{u} = s^2\boldsymbol{v} \qquad \mathbf{X}\mathbf{X}^\mathsf{T}\boldsymbol{u} = s\mathbf{X}\boldsymbol{v} = s^2\boldsymbol{u}$$

Eigenvectors

- Note that as $\mathbf{X}\boldsymbol{v} = s\boldsymbol{u}$ and $\mathbf{X}^\mathsf{T}\boldsymbol{u} = s\boldsymbol{v}$ then

$$\mathbf{X}(-\boldsymbol{v}) = (-s)\boldsymbol{u} \qquad \mathbf{X}^\mathsf{T}\boldsymbol{u} = (-s)(-\boldsymbol{v})$$

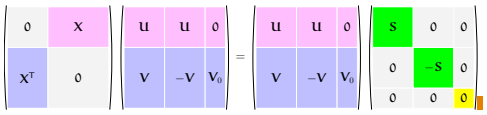
if $\binom{\boldsymbol{u}}{\boldsymbol{v}}$ is an eigenvector of $\mathbf{B}$ with eigenvalue s then so is $\binom{\boldsymbol{u}}{-\boldsymbol{v}}$ with eigenvalue $-s$

- If $m < p$ then $\mathbf{X}^\mathsf{T}\mathbf{X}$ is not full rank so some eigenvalues are zero
- As a consequence $p - m$ vectors exist such that $\mathbf{X}\boldsymbol{v} = 0$
- The eigenvalues and eigenvectors are

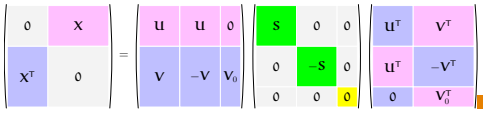
$$m \times \left(s_i, \binom{\boldsymbol{u}_i}{\boldsymbol{v}_i}\right) \quad m \times \left(-s_i, \binom{\boldsymbol{u}_i}{-\boldsymbol{v}_i}\right) \quad (p-m) \times \left(0, \binom{0}{\boldsymbol{v}_k}\right)$$

Matrix Decomposition

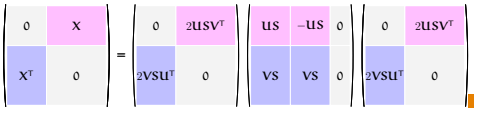
- Stacking the eigenvectors into a matrix



- Since the vectors $\binom{\boldsymbol{u}_i}{\boldsymbol{v}_i}$ are eigenvectors of a symmetric matrix they form an orthogonal matrix if they are normalised.
- Multiply on the right by the transpose of the orthogonal matrix



Normalisation Subtlety



- Multiplying out we have

$$\mathbf{X} = 2\mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T} \qquad \mathbf{X}^\mathsf{T} = 2\mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T}$$

- Now the vectors $\boldsymbol{u}_i$ and $\boldsymbol{v}_i$ form orthogonal sets as they satisfy

$$\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{v} = s^2\boldsymbol{v} \qquad \mathbf{X}\mathbf{X}^\mathsf{T}\boldsymbol{u} = s^2\boldsymbol{u}$$

- But they are not normalised (since $\binom{\boldsymbol{u}_i}{\boldsymbol{v}_i}$ is normalised). If we define $\tilde{\mathbf{U}} = \sqrt{2}\mathbf{U}$ and $\tilde{\mathbf{V}} = \sqrt{2}\mathbf{V}$ we find

$$\mathbf{X} = \tilde{\mathbf{U}}\mathbf{S}\tilde{\mathbf{V}}^\mathsf{T} \qquad \mathbf{X}^\mathsf{T} = \tilde{\mathbf{V}}\mathbf{S}^\mathsf{T}\tilde{\mathbf{U}}^\mathsf{T}$$

SVD

- Any matrix, $\mathbf{X}$, can be written as $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$
  - $\mathbf{U}$, $\mathbf{V}$ are orthogonal matrices
  - $\mathbf{S} = \text{diag}(s_1, s_2, \ldots, s_r)$ with $r = \min(m, p)$
- s_i can always be chosen to be non-negative and are known as **singular values**
- If $\mathbf{X}$ is symmetric and positive semi-definite (e.g. a covariance matrix) then $\mathbf{U} = \mathbf{V}$ and $\mathbf{S} = \mathbf{\Lambda}$ (the matrix of eigenvalues)—SVD is the same as eigen-decomposition (for a general symmetric matrix $s_i = |\lambda_i|$)
- Singular value decomposition applies to both square and non-square matrices—they describe general linear mappings

Spectrum of a Matrix

- The set of singular values (when ordered) is often referred to as the (singular value) spectrum of a matrix
- The largest singular value, s_{max}, provides a measure of the maximum change in the two norm of a vector

$$\mathbf{X}\boldsymbol{v}_{max} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}\boldsymbol{v}_{max} = s_{max}\boldsymbol{u}_{max}$$

  where $\boldsymbol{u}_{max}$ and $\boldsymbol{v}_{max}$ are the columns of the matrix $\mathbf{U}$ and $\mathbf{V}$ corresponding to s_{max} (note $\|\boldsymbol{v}_i\| = \|\boldsymbol{u}_i\| = 1$)
- $s_{max} = \|\mathbf{X}\|_2$ is known as the **spectral norm** or **2 norm** of a matrix satisfying

$$\|\mathbf{X}\boldsymbol{x}\|_2 \leq s_{max}\|\boldsymbol{x}\|_2$$

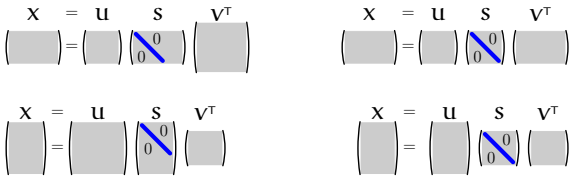
  with equality when $\boldsymbol{x} = \boldsymbol{v}_{max}$

Finding SVD

- Most libraries will compute the SVD for you
- In principle we can choose the smaller of the two matrices $\mathbf{X}\mathbf{X}^\mathsf{T}$ and $\mathbf{X}^\mathsf{T}\mathbf{X}$ and compute its eigenvalues (libraries avoid this: it squares the condition number)
- The singular values are the square root of the eigenvalues (notice that $\mathbf{X}\mathbf{X}^\mathsf{T}$ and $\mathbf{X}^\mathsf{T}\mathbf{X}$ are both positive semi-definite so the eigenvalues will be non-negative)
- We can then compute the $\mathbf{U}$ matrix or $\mathbf{V}$ matrix by multiplying through by $\mathbf{X}$ or $\mathbf{X}^\mathsf{T}$ ($\mathbf{U} = \mathbf{X}\mathbf{V}\mathbf{S}^{-1}$ and $\mathbf{V} = \mathbf{X}^\mathsf{T}\mathbf{U}\mathbf{S}^{-1}$)
- In practice to perform PCA most people subtract the mean from their data and then perform SVD

Economical Forms of SVD

- Often the rows or columns of the orthogonal matrices $\mathbf{U}$ and $\mathbf{V}$ that are not associated with a singular value are ignored

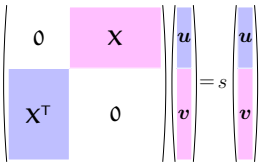


- In numpy (which returns a vector $\mathtt{s}$ and $\mathbf{V}^\mathsf{T}$)

```
U, s, Vt = np.linalg.svd(X)     # full: U is m x m
# economical: U is m x r
U, s, Vt = np.linalg.svd(X, full_matrices=False)
```


Outline

1. Singular Value Decomposition
2. **General Linear Mappings**
3. Linear Regression Revisited



General Matrix

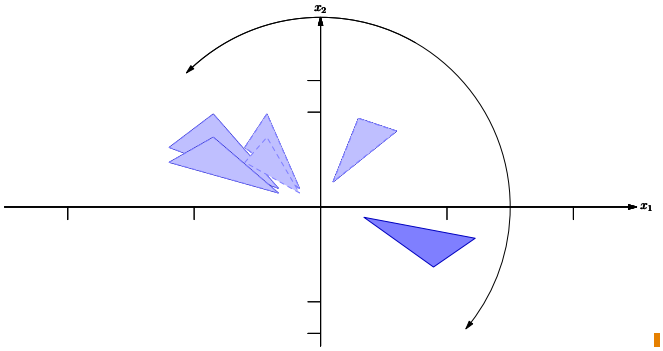
- Recall that we can compute the SVD for any matrix, $\mathbf{X}$
- As matrices describe the most general linear mapping

$$\boldsymbol{v} \to \mathcal{T}[\boldsymbol{v}] = \mathbf{X}\boldsymbol{v}$$

- We can use SVD to understand any linear mapping
- Thus any linear mapping can be seen as a rotation followed by a squashing or expansion independently in each coordinate followed by another rotation

Matrices

$$\mathbf{M} = \begin{pmatrix} -0.45 & 1.9 \\ -0.77 & -0.025 \end{pmatrix} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T} = \begin{pmatrix} \cos(-175°) & -\sin(-175°) \\ \sin(-175°) & \cos(-175°) \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 0.75 \end{pmatrix} \begin{pmatrix} \cos(75°) & -\sin(75°) \\ \sin(75°) & \cos(75°) \end{pmatrix}$$



Determinants

- The determinant, $|\mathbf{M}|$ of a matrix $\mathbf{M}$ is defined for square matrices
- It describes the change in volume under the mapping
- Now for any two matrices $|\mathbf{A}\mathbf{B}| = |\mathbf{A}||\mathbf{B}|$
- Thus $|\mathbf{M}| = |\mathbf{U}||\mathbf{S}||\mathbf{V}^\mathsf{T}|$
- For an orthogonal matrix $|\mathbf{U}| = \pm 1$ since $\mathbf{U}\mathbf{U}^\mathsf{T} = \mathbf{I} \Rightarrow |\mathbf{U}\mathbf{U}^\mathsf{T}| = |\mathbf{I}| \Rightarrow |\mathbf{U}||\mathbf{U}^\mathsf{T}| = 1$ or $|\mathbf{U}|^2 = 1$
- Thus

$$|\mathbf{M}| = \pm|\mathbf{S}| = \pm\prod_i s_i$$

Non-Square Matrices

- When the matrices are non-square then the singular value matrix will either
  - ⋆ Squash some directions to zero
  - ⋆ Introduce new dimensions orthogonal to the vector

- The rank of an arbitrary matrix is the number of non-zero singular values (also number of linearly independent rows or columns)

Duality Revisited

- If $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$ then

$$\begin{aligned}\mathbf{C} &= \mathbf{X}\mathbf{X}^\mathsf{T} & \mathbf{D} &= \mathbf{X}^\mathsf{T}\mathbf{X}\\ &= \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}\mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T} & &= \mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T}\mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}\\ &= \mathbf{U}(\mathbf{S}\mathbf{S}^\mathsf{T})\mathbf{U}^\mathsf{T} & &= \mathbf{V}(\mathbf{S}^\mathsf{T}\mathbf{S})\mathbf{V}^\mathsf{T}\end{aligned}$$

- If $\mathbf{X}$ is an $m \times p$ matrix then $\mathbf{S}\mathbf{S}^\mathsf{T}$ is an $m \times m$ diagonal matrix with elements $S_{ii}^2 = s_i^2$
- $\mathbf{S}^\mathsf{T}\mathbf{S}$ is a $p \times p$ matrix with elements $S_{ii}^2 = s_i^2$
- $\mathbf{U}$ and $\mathbf{V}$ are matrices of eigenvectors for $\mathbf{C}$ and $\mathbf{D}$
- The eigenvalues of $\mathbf{X}\mathbf{X}^\mathsf{T}$ and $\mathbf{X}^\mathsf{T}\mathbf{X}$ are $\lambda_i = S_{ii}^2 = s_i^2$

$\mathbf{S}\mathbf{S}^\mathsf{T}$ and $\mathbf{S}^\mathsf{T}\mathbf{S}$

$$\mathbf{S} = \begin{pmatrix} s_1 & 0 & \cdots & 0 & 0 & \cdots & 0\\ 0 & s_2 & \cdots & 0 & 0 & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & s_m & 0 & 0\cdots & 0 \end{pmatrix}$$

$$\mathbf{S}^\mathsf{T}\mathbf{S} = \begin{pmatrix} s_1^2 & 0 & \cdots & 0 & 0 & \cdots & 0\\ 0 & s_2^2 & \cdots & 0 & 0 & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & s_m^2 & 0 & 0\cdots & 0\\ 0 & 0 & \cdots & 0 & 0 & 0\cdots & 0\\ \vdots & \vdots & \cdots & \vdots & \vdots & \vdots\cdots & \vdots\\ 0 & 0 & \cdots & 0 & 0 & 0\cdots & 0 \end{pmatrix}$$

$$\mathbf{S}\mathbf{S}^\mathsf{T} = \begin{pmatrix} s_1^2 & 0 & \cdots & 0\\ 0 & s_2^2 & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & s_m^2 \end{pmatrix}$$

Having A Go

- It's really easy to verify this in Python

```
import numpy as np
X = np.random.rand(3, 2)
U, s, Vt = np.linalg.svd(X)      # s: vector, Vt: V transposed
S = np.zeros_like(X)             # rebuild the 3 x 2 matrix S
np.fill_diagonal(S, s)
U @ S @ Vt                       # should equal X
U[:, 0] @ U[:, 1]                # columns of U are orthogonal
U.T @ U, U @ U.T                 # both the identity
L, Ua = np.linalg.eigh(X @ X.T) # eigenvalues, ascending
S @ S.T                          # the same values: s squared
```

- Test yourself!

Outline

1. Singular Value Decomposition
2. General Linear Mappings
3. **Linear Regression Revisited**

$$\begin{pmatrix} 0 & \mathbf{X}\\ \mathbf{X}^\mathsf{T} & 0 \end{pmatrix}\begin{pmatrix} \boldsymbol{u}\\ \boldsymbol{v} \end{pmatrix} = s\begin{pmatrix} \boldsymbol{u}\\ \boldsymbol{v} \end{pmatrix}$$

Linear Regression

- Given a set of data $\mathcal{D} = \{(\boldsymbol{x}_k, y_k) | k = 1, 2, \ldots, m\}$
- In linear regression we try to fit a linear model

$$f(\boldsymbol{x}|\boldsymbol{w}) = \boldsymbol{x}^\mathsf{T}\boldsymbol{w}$$

- Which we fit by minimising the squared error loss

$$L(\boldsymbol{w}) = \sum_{k=1}^{m} (f(\boldsymbol{x}_k|\boldsymbol{w}) - y_k)^2$$

Matrix Form

- In matrix form we write $L(\boldsymbol{w}) = \|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$

$$\mathbf{X} = \begin{pmatrix} \boldsymbol{x}_1^\mathsf{T}\\ \boldsymbol{x}_2^\mathsf{T}\\ \vdots\\ \boldsymbol{x}_m^\mathsf{T} \end{pmatrix} \qquad \boldsymbol{y} = \begin{pmatrix} y_1\\ y_2\\ \vdots\\ y_m \end{pmatrix}$$

- Then $\nabla L(\boldsymbol{w}^*) = 0$ implies

$$\boldsymbol{w}^* = (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} = \mathbf{X}^+\boldsymbol{y}$$

- This is known as the pseudo-inverse

Using SVD

- Using $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$ then

$$\begin{aligned}\mathbf{X}^+ &= (\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}\\ &= (\mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{S}\mathbf{V}^\mathsf{T})^{-1}\mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T}\\ &= \mathbf{V}(\mathbf{S}^\mathsf{T}\mathbf{S})^{-1}\mathbf{V}^\mathsf{T}\mathbf{V}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T}\\ &= \mathbf{V}(\mathbf{S}^\mathsf{T}\mathbf{S})^{-1}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T} = \mathbf{V}\mathbf{S}^+\mathbf{U}^\mathsf{T}\end{aligned}$$

- If $m > p$

$$\mathbf{X}^\mathsf{T} = \left(\begin{array}{cccccccc} | & | & | & | & | & | & | & |\\ | & | & | & | & | & | & | & | \end{array}\right), \mathbf{S}^\mathsf{T} = \begin{pmatrix} s_1 & 0 & 0 & \cdots & 0 & 0 & \cdots & 0\\ 0 & s_2 & 0 & \cdots & 0 & 0 & \cdots & 0\\ 0 & 0 & s_3 & \cdots & 0 & & 0\cdots & 0\\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & 0 & \cdots & s_p & 0 & 0\cdots & 0 \end{pmatrix}$$

Pseudo-Inverse of S

$$\mathbf{S}^\mathsf{T}\mathbf{S} = \begin{pmatrix} s_1^2 & 0 & \cdots & 0\\ 0 & s_2^2 & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & s_p^2 \end{pmatrix}, \quad (\mathbf{S}^\mathsf{T}\mathbf{S})^{-1} = \begin{pmatrix} s_1^{-2} & 0 & \cdots & 0\\ 0 & s_2^{-2} & \cdots & 0\\ \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & \cdots & s_p^{-2} \end{pmatrix}$$

$$\mathbf{S}^+ = (\mathbf{S}^\mathsf{T}\mathbf{S})^{-1}\mathbf{S}^\mathsf{T} = \begin{pmatrix} s_1^{-1} & 0 & 0 & \cdots & 0 & 0 & \cdots & 0\\ 0 & s_2^{-1} & 0 & \cdots & 0 & 0 & \cdots & 0\\ 0 & 0 & s_3^{-1} & \cdots & 0 & & 0\cdots & 0\\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots\\ 0 & 0 & 0 & \cdots & s_p^{-1} & 0 & 0\cdots & 0 \end{pmatrix}$$

Moore–Penrose Pseudo-Inverse

- $(\mathbf{X}^\mathsf{T}\mathbf{X})^{-1}\mathbf{X}^\mathsf{T}$ needs every $s_i > 0$: it fails if $m < p$ or the columns of $\mathbf{X}$ are dependent
- The **Moore–Penrose** pseudo-inverse, $\mathbf{X}^+ = \mathbf{V}\mathbf{S}^+\mathbf{U}^\mathsf{T}$, just skips the zero singular values: $S^+_{ii} = 1/s_i$ if $s_i \neq 0$, and 0 otherwise
- **Over-constrained** ($m > p$, all $s_i > 0$): nothing changes, $\boldsymbol{w} = \mathbf{X}^+\boldsymbol{y}$ minimises $\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2$
- **Under-constrained** ($m < p$): infinitely many $\boldsymbol{w}$ fit the data, and $\mathbf{X}^+\boldsymbol{y}$ picks the one with the smallest $\|\boldsymbol{w}\|$
  - If $s_i = 0$ then $\mathbf{X}\boldsymbol{v}_i = \mathbf{0}$: the data say nothing about $\boldsymbol{w}$ along $\boldsymbol{v}_i$, so that component is set to zero
- In numpy: `np.linalg.pinv` or `np.linalg.lstsq`

Ill-Conditioned Data Matrix

- Recall that

$$\boldsymbol{w}^* = \mathbf{X}^+\boldsymbol{y} = \mathbf{V}\mathbf{S}^+\mathbf{U}^\mathsf{T}\boldsymbol{y}$$

- If any of the singular values of $\mathbf{X}$ are small then $\mathbf{S}^+$ will magnify components in that direction
- Any errors in the target $\boldsymbol{y}$ will be magnified
- This leads to poor weights

Regularisation

- Consider linear regression with a regulariser

$$\begin{aligned}\mathcal{L}(\boldsymbol{w}) &= \|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2 + \eta\|\boldsymbol{w}\|^2 \\ &= \boldsymbol{w}^\mathsf{T}(\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I})\boldsymbol{w} - 2\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y} + \boldsymbol{y}^\mathsf{T}\boldsymbol{y}\end{aligned}$$

- Thus

$$\nabla\mathcal{L}(\boldsymbol{w}) = 2(\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I})\boldsymbol{w} - 2\mathbf{X}^\mathsf{T}\boldsymbol{y}$$

- and $\nabla\mathcal{L}(\boldsymbol{w}^*) = 0$ gives

$$\boldsymbol{w}^* = (\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$$

Regularisation Continued

- Using $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$

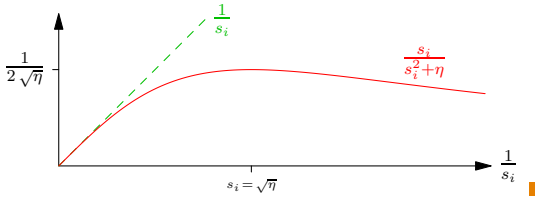
$$\begin{aligned}\boldsymbol{w}^* &= (\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I})^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y} \\ &= \mathbf{V}(\mathbf{S}^\mathsf{T}\mathbf{S} + \eta\mathbf{I})^{-1}\mathbf{S}^\mathsf{T}\mathbf{U}^\mathsf{T}\boldsymbol{y}\end{aligned}$$

- where

$$(\mathbf{S}^\mathsf{T}\mathbf{S} + \eta\mathbf{I})^{-1}\mathbf{S}^\mathsf{T} = \begin{pmatrix} \frac{s_1}{s_1^2+\eta} & 0 & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & \frac{s_2}{s_2^2+\eta} & 0 & \cdots & 0 & 0 & \cdots & 0 \\ 0 & 0 & \frac{s_3}{s_3^2+\eta} & \cdots & 0 & & 0\cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & \frac{s_p}{s_p^2+\eta} & 0 & 0\cdots & 0 \end{pmatrix}$$

Effect of Regularisation

- Without regularisation if $s_i = 0$ the problem would be ill-posed (even $\mathbf{S}^+$ does not exist since s_i^{-1} would be ill defined) and if s_i is small then $\mathbf{S}^+$ is ill-conditioned
- Using $\hat{\mathbf{S}}^+ = (\mathbf{S}^\mathsf{T}\mathbf{S} + \eta\mathbf{I})^{-1}\mathbf{S}^\mathsf{T}$ instead of $\mathbf{S}^+$ then

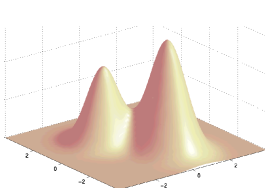
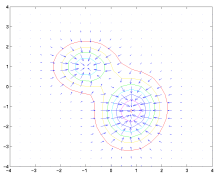


- Regularisation makes the machine much more stable (reduces the variance)

Summary

- Any matrix can be decomposed as $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$ where
  - $\mathbf{U}$ and $\mathbf{V}$ are orthogonal (rotation matrices)
  - $\mathbf{S} = \mathrm{diag}(s_1,\ldots,s_r)$ is a diagonal matrix of non-negative singular values
- This describes the most general linear transform
- The transform exploits the duality between $\mathbf{X}\mathbf{X}^\mathsf{T}$ and $\mathbf{X}^\mathsf{T}\mathbf{X}$
- In linear regression the pseudo-inverse involves the reciprocal of the singular values, which can lead to poor generalisation
- Regularisation improves the conditioning of the "inverse" matrix

Mathematics of Machine Learning
Optimisation

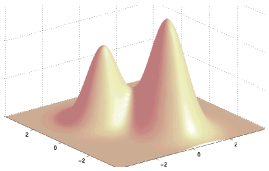



$$z = \mathrm{e}^{-(x+1)^2-(y-1)^2} + 0.6\mathrm{e}^{-(x-1)^2-0.5(y+1)^2+0.1(x-3)(y-3)}$$

Gradient descent, quadratic minima, differing length scales

Outline

1. **Motivation**
2. Gradient Descent
3. Why Gradient Descent is Difficult

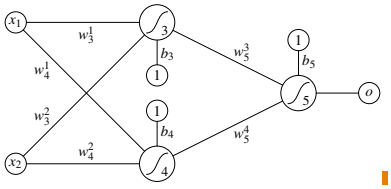


ML = Optimisation

- Many learning machines can be thought of as functions of the form
$$\hat{y} = f(\boldsymbol{x}|\boldsymbol{w})$$
(or more generally $\hat{\boldsymbol{y}} = \boldsymbol{f}(\boldsymbol{x}|\boldsymbol{w})$)
- Given an input pattern (set of features) $\boldsymbol{x}$ the learning machine makes a prediction $\hat{y}$
- We try to choose the parameters $\boldsymbol{w}$ so that the predictions are good
- In practice training a learning machine comes down to optimising some loss function

MLP

- We can depict a neural network such as an MLP by a diagram



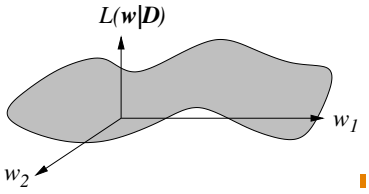
- Stands for the function ($\hat{y} = f(\boldsymbol{x}|\boldsymbol{w})$)
$$\hat{y} = g(w_5^3 g(w_3^1 x_1 + w_3^2 x_2 + b_3) + w_5^4 g(w_4^1 x_1 + w_4^2 x_2 + b_4) + b_5)$$
where, for example, $g(V) = \frac{1}{1+\mathrm{e}^{-V}}$

Training

- Given a (labelled) training dataset
$$\mathcal{D} = \{(\boldsymbol{x}_k, y_k) | k = 1, \ldots, m\}$$
- We define an error or loss function that we want to minimise
$$L(\boldsymbol{w}|\mathcal{D}) = \frac{1}{m}\sum_{k=1}^{m}\left(f(\boldsymbol{x}_k|\boldsymbol{w}) - y_k\right)^2$$
- We then use the machine with the weights $\boldsymbol{w}^*$ which minimise $L(\boldsymbol{w}|\mathcal{D})$

Computing Gradients

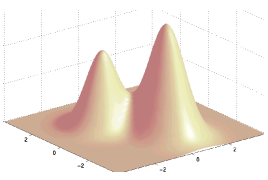
- $L(\boldsymbol{w}|\mathcal{D})$ is a complex function of the weights $\boldsymbol{w}$



- To minimise $L(\boldsymbol{w}|\mathcal{D})$ we compute the gradient $\nabla L(\boldsymbol{w}|\mathcal{D})$
- For neural networks an efficient algorithm for computing the gradient is known as back-propagation of errors (*back-prop*)

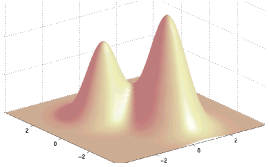
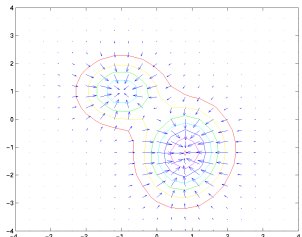
Outline

1. Motivation
2. **Gradient Descent**
3. Why Gradient Descent is Difficult



Gradient Optimisation

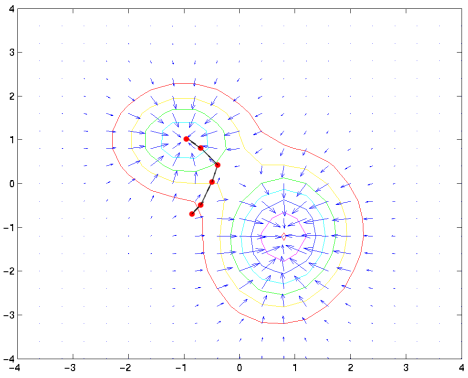
- At a maximum or minimum (or a saddle point) $\nabla L(\boldsymbol{w}|\mathcal{D}) = \boldsymbol{0}$
- E.g.
$$L = \mathrm{e}^{-(x+1)^2-(y-1)^2} + 0.6\mathrm{e}^{-(x-1)^2-0.5(y+1)^2+0.1(x-3)(y-3)}$$

Gradient Descent

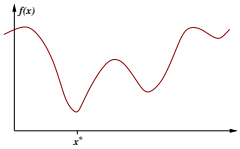
- For a simple function $L(\boldsymbol{w}|\mathcal{D})$ we can solve $\nabla L(\boldsymbol{w}|\mathcal{D}) = \mathbf{0}$ explicitly. E.g. the linear perceptron.
- For non-linear functions we usually can't solve this set of simultaneous equations.
- We can find a maximum or minimum **iteratively**.
- If we know the gradient then we can follow the gradient.
  - Maximisation: $\boldsymbol{w} \rightarrow \boldsymbol{w}' = \boldsymbol{w} + r\nabla L(\boldsymbol{w}|\mathcal{D})$
  - Minimisation: $\boldsymbol{w} \rightarrow \boldsymbol{w}' = \boldsymbol{w} - r\nabla L(\boldsymbol{w}|\mathcal{D})$

Hill-Climbing



What Goes Right

- Almost all minima are quadratic (Morse's theorem).



- Taylor expanding around a minimum x^*

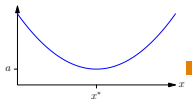
$$\begin{aligned} f(x) &= f(x^*) + (x-x^*)f'(x^*) + \frac{1}{2}(x-x^*)^2 f''(x^*) + \cdots \\ &= f(x^*) + \frac{1}{2}(x-x^*)^2 f''(x^*) + \frac{1}{3!}(x-x^*)^3 f'''(x^*) + \cdots \end{aligned}$$

- If $x - x^*$ is sufficiently small the higher order terms are negligible.

Newton's Method

- If we were in a quadratic minimum

$$f(x) = a + \frac{b}{2}(x-x^*)^2$$



- then

$$f'(x) = b(x-x^*), \qquad f''(x) = b$$

- so

$$x - x^* = \frac{f'(x)}{b} = \frac{f'(x)}{f''(x)}$$

- or

$$x^* = x - \frac{f'(x)}{f''(x)}$$

Newton's Method

- This is Newton's method.
- For non-quadratic functions Newton's method converges **quadratically** provided we are sufficiently close to a minimum.
- If we are at a distance $x - x^* = \epsilon$ from the minimum then after one iteration we will be a distance of order ϵ^2, after two iterations of order ϵ^4, etc.
- If we are too far from a minimum we might go anywhere!
- We should follow the gradient until we are near the minimum.

Taylor's Expansion in High Dimensions

- We can generalise these results to many dimensions.
- The Taylor expansion of a function $f(\boldsymbol{x})$ about $\boldsymbol{x}_0$

$$f(\boldsymbol{x}) = f(\boldsymbol{x}_0) + (\boldsymbol{x}-\boldsymbol{x}_0)^\mathsf{T}\nabla f(\boldsymbol{x}_0) + \frac{1}{2}(\boldsymbol{x}-\boldsymbol{x}_0)^\mathsf{T}\mathbf{H}(\boldsymbol{x}-\boldsymbol{x}_0) + \cdots$$

  where $\mathbf{H}$ is the **Hessian** matrix with elements

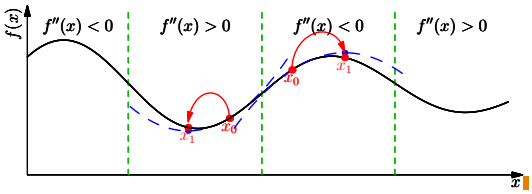
$$H_{ij} = \frac{\partial^2 f(\boldsymbol{x}_0)}{\partial x_i \partial x_j}$$

- Newton's method in high dimension is

$$\boldsymbol{x}^* = \boldsymbol{x} - \mathbf{H}^{-1}\nabla f(\boldsymbol{x})$$

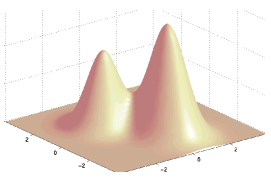
Using the Second Derivative

- If we are optimising n parameters the Hessian is an $n \times n$ matrix.
- It is time-consuming to compute (and prone to errors when coding)—for deep learning it is impossible even to store the Hessian.
- Away from minima they can be misleading



Outline

1. Motivation
2. Gradient Descent
3. **Why Gradient Descent is Difficult**



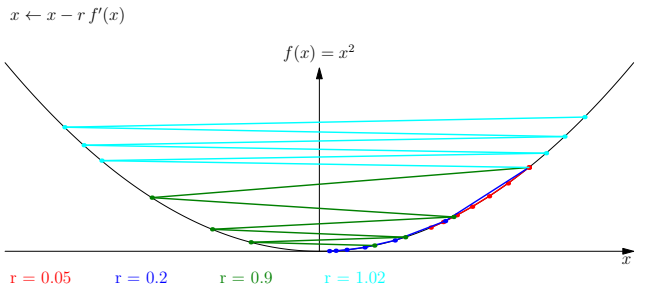
Step Size

- Gradient descent

$$\boldsymbol{x}' = \boldsymbol{x} - r\nabla f(\boldsymbol{x})$$

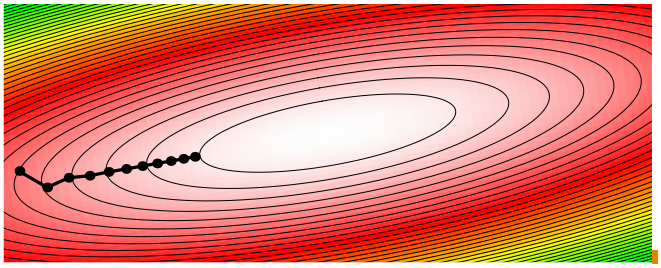
- Need to choose the learning rate or step size, r
- Too small a step takes lots of time
- Too large a step takes you away from a minimum

Step Size



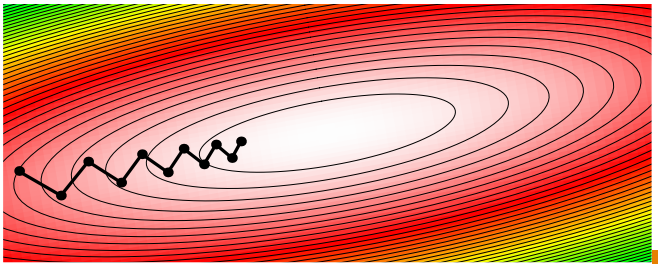
Higher Dimensions

- In higher dimensions the problem is that there are some directions you need to move a long way



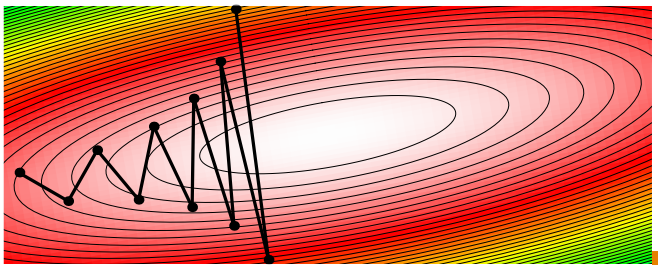
Getting There Quicker

- Increasing the step size speeds up convergence, but the direction of steepest descent doesn't point to the minimum



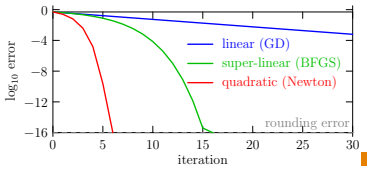
More Haste Less Speed

- Increasing the step size, just a little further, increases the rate of convergence in one direction, but . . .



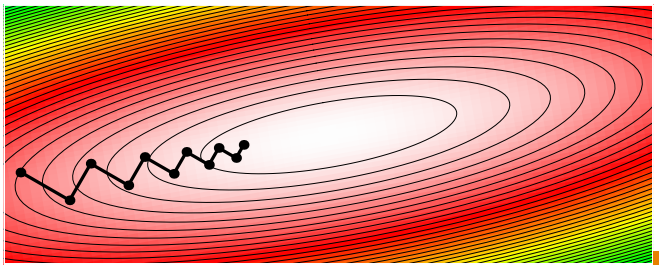
Linear and Quadratic Convergence

- Near a minimum, along each eigenvector of the Hessian, a gradient step multiplies the error by $(1 - r\lambda_i)$
- Stable only if $r < 2/\lambda_{\max}$, so the flattest direction shrinks by at best $1 - 2\lambda_{\min}/\lambda_{\max} = 1 - 2/\kappa$ per step (κ is the condition number)
- A constant factor per step is **linear** convergence; Newton squares the error: **quadratic** convergence



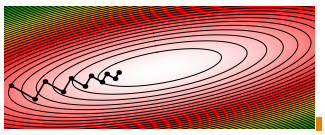
Line Minimisation

- We can systematically seek the minimum along a line of the gradient



Zig-Zag

- Note that in narrow valleys line minimisation zigzags (each step is at right angles to the last)



- If we computed the Hessian and used Newton's method we would jump straight to the minimum if we were in a quadratic potential
- However computing the Hessian is time consuming and misleading if we are not in a quadratic potential (i.e. far from the optimum)

Better Optimisation Algorithms

- Good optimisation algorithms use second-order (curvature) information without computing the full Hessian
- E.g. Conjugate gradient
  - ⋆ Performs Line Minimisation
  - ⋆ Uses gradient, but does not go along it
  - ⋆ For a quadratic minimum in d dimensions it reaches the minimum in d steps
- E.g. Levenberg-Marquardt
  - ⋆ Used on least squares problem only
  - ⋆ Uses linear approximation of function to approximate Hessian
  - ⋆ Adapts from hill-climbing to Newton method
  - ⋆ Avoids line-minimisation

BFGS and L-BFGS

- BFGS (Broyden–Fletcher–Goldfarb–Shanno) is the most widely used quasi-Newton method
- It builds an approximation $\mathbf{B} \approx \mathbf{H}^{-1}$ from how the gradient changes from one step to the next

$$\mathbf{B}\big(\boldsymbol{\nabla} f(\boldsymbol{x}_{t+1}) - \boldsymbol{\nabla} f(\boldsymbol{x}_t)\big) = \boldsymbol{x}_{t+1} - \boldsymbol{x}_t$$

- Steps $\boldsymbol{x}_{t+1} = \boldsymbol{x}_t - \alpha \mathbf{B} \boldsymbol{\nabla} f(\boldsymbol{x}_t)$, with a line search for α
- Converges **super-linearly** without ever computing the Hessian
- $\mathbf{B}$ is $N \times N$; **L-BFGS** keeps only the last ten or so steps instead, so its memory is $O(N)$

Levenberg-Marquardt

- Want to minimise $\|\boldsymbol{\epsilon}(\boldsymbol{w})\|^2$ where $\epsilon_i(\boldsymbol{w}) = f(\boldsymbol{x}_i|\boldsymbol{w}) - y_i$
- Use linear approximation

$$\epsilon_i(\boldsymbol{w}) \approx \epsilon_i(\boldsymbol{w}^{(k)}) + (\boldsymbol{w} - \boldsymbol{w}^{(k)})^\mathsf{T} \boldsymbol{\nabla} \epsilon_i(\boldsymbol{w}^{(k)})$$

  with $\boldsymbol{\nabla} \epsilon_i(\boldsymbol{w}^{(k)}) = \boldsymbol{\nabla} f(\boldsymbol{x}_i|\boldsymbol{w}^{(k)})$
- Solve quadratic minimisation of approximate error $\operatorname{argmin}_{\boldsymbol{w}} L_{approx}(\boldsymbol{w})$ with $\mathbf{J} = \boldsymbol{\nabla}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)})$

$$\begin{aligned} L_{approx}(\boldsymbol{w}) &= \|\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)}) + \mathbf{J}(\boldsymbol{w} - \boldsymbol{w}^{(k)})\|^2 \\ &= \boldsymbol{\epsilon}(\boldsymbol{w}^{(k)})^\mathsf{T}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)}) + 2(\boldsymbol{w} - \boldsymbol{w}^{(k)})^\mathsf{T}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)}) \\ &\quad + (\boldsymbol{w} - \boldsymbol{w}^{(k)})^\mathsf{T}\mathbf{J}^\mathsf{T}\mathbf{J}(\boldsymbol{w} - \boldsymbol{w}^{(k)}) \end{aligned}$$

Trust Region

- Solution given by $\boldsymbol{\nabla}_{\boldsymbol{w}} L_{approx}(\boldsymbol{w}) = 0$ gives

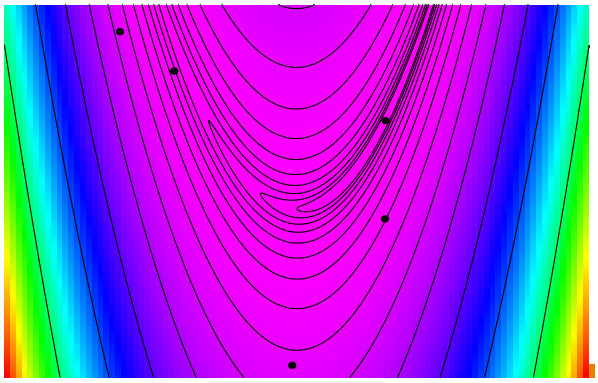
$$\boldsymbol{w}^{(k+1)} = \boldsymbol{w}^{(k)} - \big(\mathbf{J}^\mathsf{T}\mathbf{J}\big)^{-1}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)})$$

- Can lead us in the wrong direction
- Instead use $\boldsymbol{w}^{(k+1)} = \operatorname{argmin}_{\boldsymbol{w}} L_{approx}(\boldsymbol{w}) + \nu\|\boldsymbol{w} - \boldsymbol{w}^{(k)}\|^2$

$$\boldsymbol{w}^{(k+1)} = \boldsymbol{w}^{(k)} - \big(\mathbf{J}^\mathsf{T}\mathbf{J} + \nu\mathbf{I}\big)^{-1}\mathbf{J}^\mathsf{T}\boldsymbol{\epsilon}(\boldsymbol{w}^{(k)})$$

- ν limits the step size
- If predicted reduction in error is accurate then reduce ν, else if predicted reduction in error is very poor increase ν

Rosenbrock: $\epsilon_1 = 10(x_2 - x_1^2)$, $\epsilon_2 = 1 - x_1$



Where Are These Used?

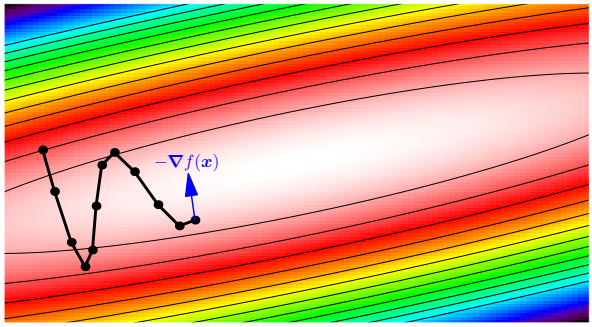
- For up to thousands of parameters (L-BFGS: millions), with gradients computed from all the data
- **Logistic regression**: scikit-learn's default solver is L-BFGS; Newton's method for generalised linear models
- **Gaussian processes**: fitting the kernel hyper-parameters with L-BFGS
- **Curve fitting** (non-linear least squares): Levenberg–Marquardt, e.g. SciPy's `curve_fit`
- **Support vector machines**: a quadratic programme (later lectures)
- With millions to billions of parameters and mini-batches of data we need different methods: next lecture

Summary

- There are some **non-gradient methods** (Nelder Mead, evolutionary strategies, Powell's method), but in very high dimensions these are not very competitive
- There are **gradient methods** (first order methods) that suffer from the problem of having to choose a single step size with conflicting requirements in different directions; they converge **linearly**
- **Newton's method** (a second order method) requires computing the Hessian matrix, converges **quadratically**, but can take you in the wrong direction if you are not sufficiently close to a minimum
- There exist a number of **quasi-Newton methods** (BFGS, conjugate gradient, Levenberg-Marquardt, etc.) that approximate Newton's method often without explicitly computing the Hessian

Mathematics of Machine Learning

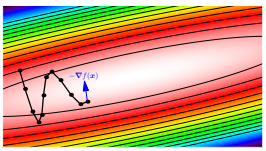
Stochastic Gradient Descent



SGD, momentum, step size, RMSProp, Adam, learning-rate schedules

Outline

1. **SGD**
2. Momentum
3. Loss Landscapes



The Next Step

- In most machine learning problems we design a network and a loss function
- The rest requires simple optimisation
- Although Newton and Quasi-Newton methods ensure rapid convergence for many tasks this apparent advantage is illusory
- Since the rise of deep learning, gradient descent and its variants have become dominant

Mini-Batch Learning

- Rather than computing the gradient of the loss function, e.g.

$$\nabla L_{\mathcal{D}}(\boldsymbol{w}) = \frac{1}{|\mathcal{D}|}\nabla \sum_{(\boldsymbol{x},y)\in\mathcal{D}} L\big(f(\boldsymbol{x}|\boldsymbol{w}),y\big)$$

- We often compute an approximation

$$\nabla L_{\mathcal{B}}(\boldsymbol{w}) = \frac{1}{|\mathcal{B}|}\nabla \sum_{(\boldsymbol{x},y)\in\mathcal{B}} L\big(f(\boldsymbol{x}|\boldsymbol{w}),y\big)$$

- where $\mathcal{B}\subset\mathcal{D}$ is a randomly sampled subset of the training set
- Usually $|\mathcal{B}| \ll |\mathcal{D}|$

Stochastic Gradient Descent

- Stochastic gradient descent (SGD) follows the gradient of mini-batches
- Computationally this is efficient because it is much quicker to compute $\nabla L_{\mathcal{B}}(\boldsymbol{w})$ than $\nabla L_{\mathcal{D}}(\boldsymbol{w})$
- The batch gradients add noise (are stochastic) as we are only sampling the dataset
- This noise might help escape local-minima (but I'm not sure there is compelling evidence for this)
- Taking many smaller steps of approximate gradients reduces the likelihood of divergence

Automatic Differentiation

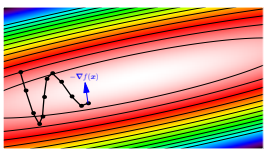
- For gradient descent we need to compute the gradient
- For most of my life this was just a pain
- You guys have it easy, modern deep learning frameworks do this for you automatically
- This is a *game changer*! We are no longer afraid of large models
- We can differentiate through algorithms allowing us to train incredibly sophisticated networks

Convergence

- Asymptotic convergence of gradient descent (and SGD) is slower than quasi-Newton methods
- But there are three reasons why we don't really care
  - ⋆ For complex models we spend a lot of time before we are close to a quadratic minimum where asymptotic convergence kicks in
  - ⋆ These days we often use ReLUs (rectified linear units) which are not differentiable at 0. The convergence we discussed depends on a Taylor expansion that assumes a smooth (twice differentiable) loss
  - ⋆ We want to minimise the generalisation error, but are only minimising a poor surrogate, namely the training error

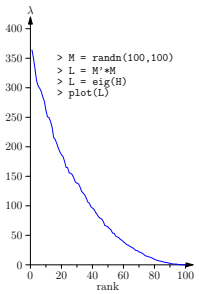
Outline

1. SGD
2. **Momentum**
3. Loss Landscapes



Recap: One Step Size, Many Curvatures

- The Hessian has a spectrum of eigenvalues: some directions are steep, others nearly flat
- One step size r must satisfy $r < 2/\lambda_{\max}$, or we diverge (and quickly get NaN errors)
- The flat directions then shrink by at best $1 - 2/\kappa$ per step
- This seems hopelessly slow, yet plain SGD (nearly always with momentum) is widely used in deep learning



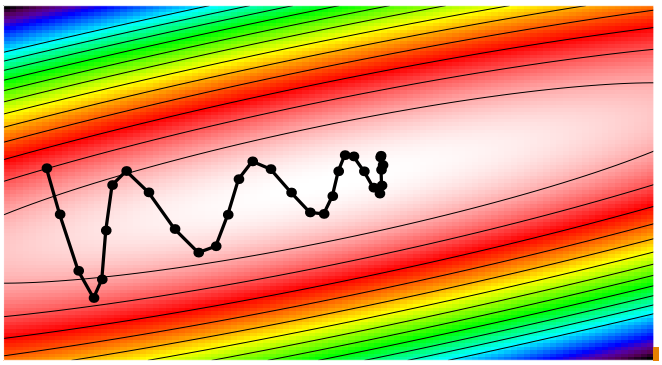
Momentum

- In high dimensions we zig-zag:
  - stepping consistently in directions with low curvature
  - jumping backwards and forwards past the minimum in directions with high curvature
- By introducing "momentum" we can increase our steps in low curvature directions and decrease them in high curvature directions

$$\boldsymbol{v}(t+1) = \gamma \boldsymbol{v}(t) - r\boldsymbol{\nabla} f(\boldsymbol{x}(t))$$
$$\boldsymbol{x}(t+1) = \boldsymbol{x}(t) + \boldsymbol{v}(t+1)$$

$0 < \gamma < 1$ sets how much velocity is kept ($1 - \gamma$ acts as friction)

Gradient Descent with Momentum



Nesterov Momentum

- Nesterov's variant takes the gradient where momentum is about to carry us

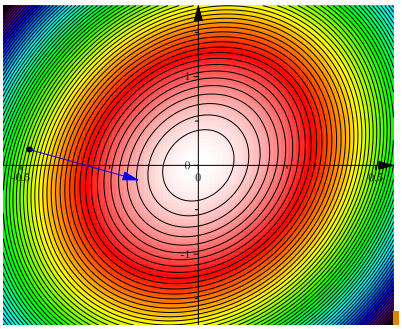
$$\boldsymbol{v}(t+1) = \gamma \boldsymbol{v}(t) - r\boldsymbol{\nabla} f\big(\boldsymbol{x}(t) + \gamma \boldsymbol{v}(t)\big)$$
$$\boldsymbol{x}(t+1) = \boldsymbol{x}(t) + \boldsymbol{v}(t+1)$$

- With well-tuned momentum the number of steps needed scales as $\sqrt{\kappa}$ rather than κ
- The convergence is still **linear**: momentum fixes the condition number, not the order of convergence
- On convex problems Nesterov reaches error $O(1/t^2)$ rather than $O(1/t)$, the best any gradient method can do (much more in COMP6260, Optimisation for Machine Learning)

Adaptive Methods

- A major difficulty of high dimensional optimisation is the existence of different scales
- That is, there are some directions where we have to move a lot (low curvature), while other directions we have to make small steps
- In adaptive methods we use a dynamic algorithm to rescale the step size of each parameter (weight)
- We can think of this as a "regularisation" that makes our basins of attraction more spherical

Rescaling Co-ordinates



RMSProp

- Rescale each coordinate by the typical size of its gradient
- Keep a running average of the squared gradients

$$S_i(t+1) = (1-\gamma) S_i(t) + \gamma \left(\frac{\partial L_{\mathcal{B}}(\boldsymbol{w}(t))}{\partial w_i(t)} \right)^2$$

- and divide the step by its square root

$$w_i(t+1) = w_i(t) - \frac{r}{\sqrt{S_i(t+1)} + \epsilon} \frac{\partial L_{\mathcal{B}}(\boldsymbol{w}(t))}{\partial w_i(t)}$$

- Steep directions take small steps and flat ones large steps: each weight moves about r per step, whatever the scale of the loss

Adding in Momentum

- RMSProp has no momentum (one argument is that rescaling has already "regularised the landscape")
- Nevertheless we can learn a momentum term

$$M_i(t+1) = (1-\beta) M_i(t) + \beta \frac{\partial L_{\mathcal{B}}(\boldsymbol{w}(t))}{\partial w_i(t)}$$
$$S_i(t+1) = (1-\gamma) S_i(t) + \gamma \left(\frac{\partial L_{\mathcal{B}}(\boldsymbol{w}(t))}{\partial w_i(t)} \right)^2$$

- Starting from $M_i(0) = S_i(0) = 0$ biases these averages towards zero at first, which we correct

$$\hat{M}_i(t+1) = \frac{M_i(t+1)}{1-(1-\beta)^{t+1}} \qquad \hat{S}_i(t+1) = \frac{S_i(t+1)}{1-(1-\gamma)^{t+1}}$$

ADAM

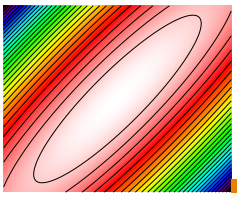
- Adaptive Moment Estimation (Adam) adapts the scale of the parameters, but also uses momentum
- Update weights

$$w_i(t+1) = w_i(t) - \frac{r}{\sqrt{\hat{S}_i(t+1)} + \epsilon}\hat{M}_i(t+1)$$

- Adam and its variants are very successful in deep learning: a network often learns with Adam when it would not with SGD
- **AdamW** applies weight decay directly to the weights, $w_i \leftarrow w_i - r\eta w_i$, rather than through the loss, where Adam's rescaling would weaken it; the default for transformers

Covariant Equations

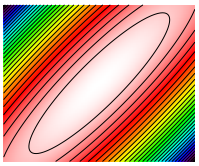
- Vector arithmetic, (matrix multiplication, addition, multiplying by a scale, computing gradients) has a covariant property that it is invariant to the coordinate system
- That is we can translate and rotate our coordinates and get the same result
- That is not true of element-wise multiplication of vectors
- Many ML algorithms do this (including RMSProp and Adam), but they aren't invariant if we rotate our coordinates



Can't rescale coordinates

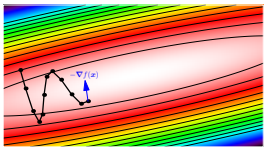
Correlated Weights

- Correlated weights will lead to skewed valleys
- Adaptive methods would be more efficient if they were rotationally invariant
- However, this would slow down the algorithm
- ADAM is a compromise between speed of implementation and effectiveness



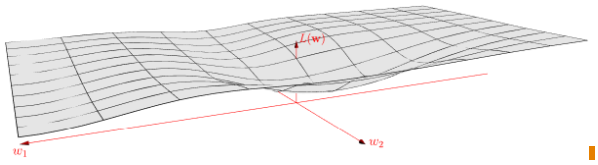
Outline

1. SGD
2. Momentum
3. **Loss Landscapes**



Loss Landscape

- A useful concept for understanding optimisation is the loss landscape
- For every value of the weights $\boldsymbol{w}$ there is some loss associated with it
- Note that this landscape is very high dimensional, so our intuition can be a bit misleading
- The landscape is also huge



Global and Local Minima

- Our objective is to minimise our loss
- This would mean finding the global minimum
- For non-convex losses no efficient algorithm is guaranteed to find the global minimum
- The best we can hope is to find a local minimum by doing gradient descent
- In practice, our landscapes are so large that we are probably unable to find even a local minimum

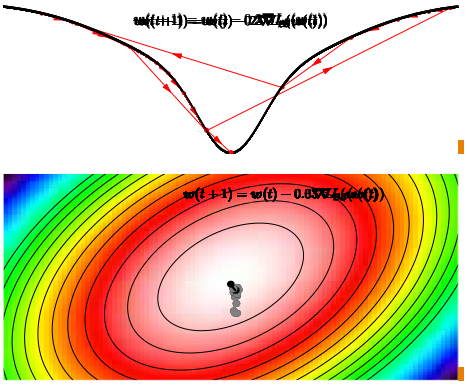
Noisy Optimisation

- Because we are using mini-batches we aren't actually following the correct gradients
- If we reduce our step size then our average gradient is closer to the true gradient
- Thus our optimisation is inherently noisy
- At least for deep learning there is no evidence that the weights stop changing even after learning for a huge amount of time

Valleys in Valleys in Valleys

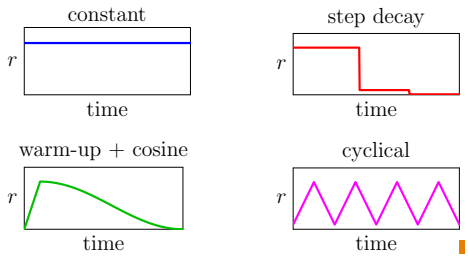
- My view of the loss landscape is that we have valleys inside bigger valleys inside bigger valleys, and so on
- If our noise is high (we use large step size), we can navigate away from local valleys and move towards the big valley
- But, we can't explore the small valleys
- Often when the rate of improvement slows down, practitioners will reduce their learning rate by a factor of 10 and the rate of improvement jumps up
- Some practitioners cycle through raising and lowering their learning rates which may speed up convergence

Reducing the Step Size



Learning-Rate Schedules

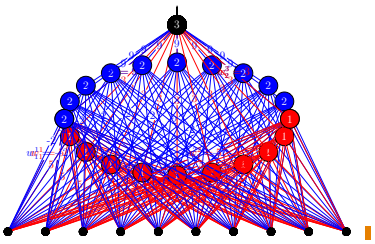
- Large r early: noisy steps cross the small valleys and find the big ones
- Small r late: settle into a valley



- A short warm-up stops the first steps diverging before Adam's running averages have settled; common for transformers

Symmetries

- When training neural networks (deep or shallow) there are typically many symmetries
- Examples come from permuting the neurons (or filters in CNNs) within a layer, together with their weights



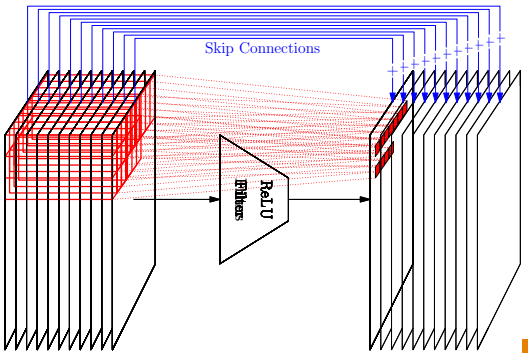
More Symmetries

- With n hidden nodes there are $n!$ symmetries
- For odd activation functions such as $\tanh$, multiplying all the input and output weights of a node by -1 gives the same network
- There are also continuous symmetries. For linear networks with two layers performing a mapping $\boldsymbol{W}_2\boldsymbol{W}_1$ then for any invertible matrix $\mathbf{U}$ (where $\mathbf{U}\mathbf{U}^{-1}=\mathbf{I}$)

$$\boldsymbol{W}_2\boldsymbol{W}_1 = \boldsymbol{W}_2\mathbf{I}\boldsymbol{W}_1 = \boldsymbol{W}_2(\mathbf{U}\mathbf{U}^{-1})\boldsymbol{W}_1 = (\boldsymbol{W}_2\mathbf{U})(\mathbf{U}^{-1}\boldsymbol{W}_1)$$

  so a network with weights $\boldsymbol{W}_2' = \boldsymbol{W}_2\mathbf{U}$ and $\boldsymbol{W}_1' = \mathbf{U}^{-1}\boldsymbol{W}_1$ will perform the same mapping
- There will be a huge space of equivalent networks

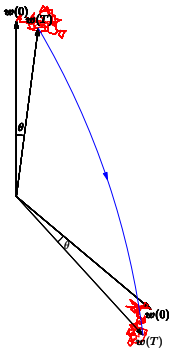
Skip Connections



- Breaks permutation symmetry. Used in Resnets, transformer, etc.

Symmetries

- The landscape potentially has a huge manifold with the same losses (neutral network)
- Find solution close to start
- We should think of optimisation more as a process of travelling than a process of arriving
- Of course even if we do minimise our loss we are not guaranteed to minimise our generalisation performance

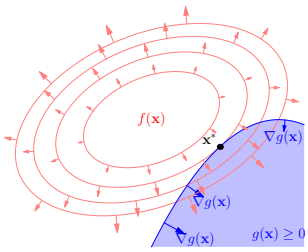


Lessons

- SGD together with automatic differentiation has revolutionised machine learning
- There are a number of techniques to get the step size right: momentum (and Nesterov), adaptive step sizes (RMSProp), Adam and AdamW, learning-rate schedules
- Still need to understand that we are exploring a huge and complex loss landscape
- What works is problem specific (as always)

Mathematics of Machine Learning

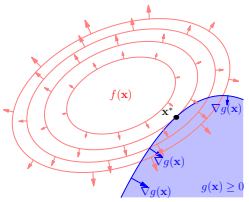
Constrained Optimisation



Lagrangians, Inequalities, KKT, Linear Programming, Quadratic Programming, Duality

Outline

1. **Constrained Optimisation**
2. Inequalities
3. Duality



Optimisation with Constraints

- There are a number of important applications where we wish to minimise an objective function subject to inequality constraints
- A prominent example of this is support vector machines
- More generally there are a large number of kernel models that involve constraints
- However, constraints are ubiquitous in machine learning (e.g. in Wasserstein GANs)

Solving Constrained Optimisation Problems

- Suppose we have a problem

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{subject to } g(\boldsymbol{x}) = 0$$

- A standard procedure is to define the Lagrangian

$$\mathcal{L}(\boldsymbol{x},\alpha) = f(\boldsymbol{x}) - \alpha g(\boldsymbol{x})$$

  where α is known as a Lagrange multiplier
- In the extended space $(\boldsymbol{x},\alpha)$ we have to solve

$$\max_{\alpha}\min_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x},\alpha)$$

Conditions on Optimum

- The optimisation problem is

$$\max_{\alpha}\min_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x},\alpha) \quad \text{where} \quad \mathcal{L}(\boldsymbol{x},\alpha) = f(\boldsymbol{x}) - \alpha g(\boldsymbol{x})$$

- Assuming differentiability

$$\boldsymbol{\nabla}_{\boldsymbol{x}}\mathcal{L}(\boldsymbol{x},\alpha) = \boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}) - \alpha \boldsymbol{\nabla}_{\boldsymbol{x}} g(\boldsymbol{x}) = 0$$
$$\frac{\partial \mathcal{L}}{\partial \alpha} = -g(\boldsymbol{x}) = 0$$

- The second condition is just the constraint
- But what about first condition: $\boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}) = \alpha \boldsymbol{\nabla}_{\boldsymbol{x}} g(\boldsymbol{x})$?

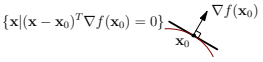
Note on Gradients

- Note that for any function $f(\boldsymbol{x})$ we can Taylor expand around $\boldsymbol{x}_0$

$$f(\boldsymbol{x}) = f(\boldsymbol{x}_0) + (\boldsymbol{x}-\boldsymbol{x}_0)^{\mathsf{T}}\boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}_0) + \frac{1}{2}(\boldsymbol{x}-\boldsymbol{x}_0)^{\mathsf{T}}\mathbf{H}(\boldsymbol{x}-\boldsymbol{x}_0) + \ldots$$

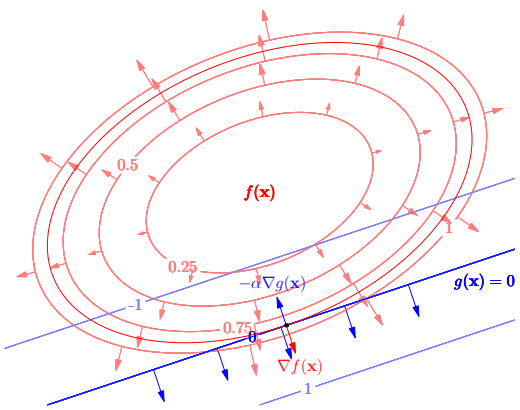
  where $\mathbf{H}$ is the matrix of second derivatives known as the Hessian
- If we consider the set of points perpendicular to $\boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}_0)$ which go through $\boldsymbol{x}_0$ (the tangent plane), these will have values

$$f(\boldsymbol{x}) = f(\boldsymbol{x}_0) + O(\|\boldsymbol{x}-\boldsymbol{x}_0\|^2)$$



  thus $\boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x})$ is always orthogonal to the contour lines

Constrained Optima



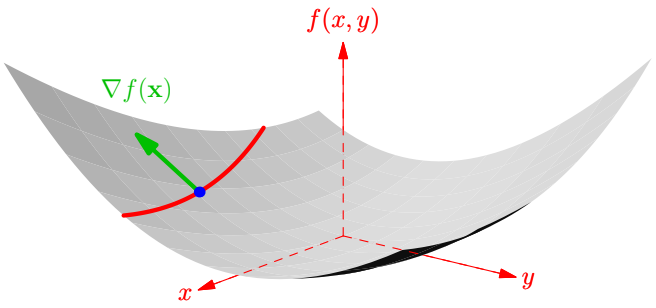
Example

- Minimise $f(\boldsymbol{x}) = x^2 + 2y^2 - xy$
- Subject to $g(\boldsymbol{x}) = x - 2y - 3 = 0$
- Writing $\mathcal{L} = f(\boldsymbol{x}) - \alpha g(\boldsymbol{x})$
- Condition for minima is $\boldsymbol{\nabla}_{\boldsymbol{x}}\mathcal{L} = 0$

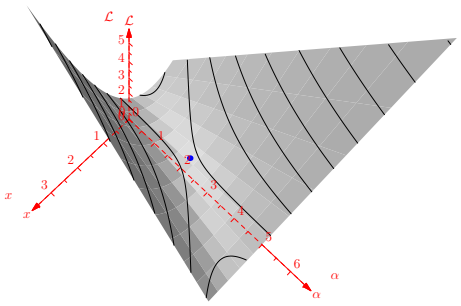
$$\boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}) = \begin{pmatrix} 2x - y \\ -x + 4y \end{pmatrix} = \alpha \boldsymbol{\nabla}_{\boldsymbol{x}} g(\boldsymbol{x}) = \alpha \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

  and $\frac{\partial \mathcal{L}}{\partial \alpha} = -g(\boldsymbol{x}) = -x + 2y + 3 = 0$
- Solving simultaneous equations gives the minimum at $(x,y) = (\frac{3}{4}, -\frac{9}{8})$ with $\alpha = \frac{21}{8}$

Surface



Saddle-Point $y = -9/8$



Multiple Constraints

- Given an optimisation problem with multiple constraints

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{subject to } g_k(\boldsymbol{x}) = 0 \text{ for } k = 1, 2, \ldots, m$$

- We introduce multiple Lagrange multipliers

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\alpha}) = f(\boldsymbol{x}) - \sum_{k=1}^{m} \alpha_k g_k(\boldsymbol{x})$$

- The condition for an optimum is $\boldsymbol{\nabla}_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x}, \boldsymbol{\alpha}) = 0$ which implies

$$\boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}) = \sum_{k=1}^{m} \alpha_k \boldsymbol{\nabla}_{\boldsymbol{x}} g_k(\boldsymbol{x})$$

plus the original constraints $\frac{\partial \mathcal{L}(\boldsymbol{x}, \boldsymbol{\alpha})}{\partial \alpha_k} = -g_k(\boldsymbol{x}) = 0$

Example

- Minimise $f(\boldsymbol{x}) = x^2 + 2y^2 + 5z^2 - xy - xz$ subject to $g_1(\boldsymbol{x}) = x - 2y - z - 3 = 0$ and $g_2(\boldsymbol{x}) = 2x + 3y + z - 2 = 0$
- Writing $\mathcal{L}(\boldsymbol{x}, \boldsymbol{\alpha}) = f(\boldsymbol{x}) - \alpha_1 g_1(\boldsymbol{x}) - \alpha_2 g_2(\boldsymbol{x})$
- Condition for minima is $\boldsymbol{\nabla}_{\boldsymbol{x}} \mathcal{L} = 0$ or $\boldsymbol{\nabla}_{\boldsymbol{x}} f(\boldsymbol{x}) = \sum_{k=1}^{2} \alpha_k \boldsymbol{\nabla}_{\boldsymbol{x}} g_k(\boldsymbol{x})$

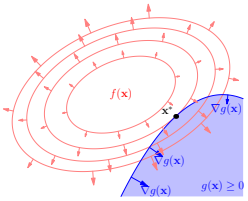
$$\begin{pmatrix} 2x - y - z \\ -x + 4y \\ 10z - x \end{pmatrix} = \alpha_1 \begin{pmatrix} 1 \\ -2 \\ -1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 2 \\ 3 \\ 1 \end{pmatrix}$$

and $\frac{\partial \mathcal{L}}{\partial \alpha_i} = -g_i(\boldsymbol{x}) = 0$

- Solving simultaneous equations gives the minimum at $(\frac{37}{20}, -\frac{11}{20}, -\frac{1}{20})$ with $\alpha_1 = 3$ and $\alpha_2 = \frac{13}{20}$

Outline

1. Constrained Optimisation
2. **Inequalities**
3. Duality



Inequality Constraints

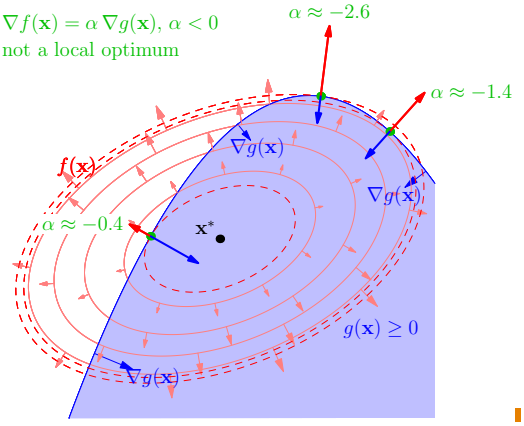
- Suppose we have the problem

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{subject to } g(\boldsymbol{x}) \geq 0$$

- Looks much more complicated, but
- Only two things can happen
  - ⋆ Either a minimum, $\boldsymbol{x}^*$, of $f(\boldsymbol{x})$ satisfies $g(\boldsymbol{x}^*) > 0$
    - ∗ We then have an unconstrained optimisation problem
  - ⋆ Otherwise, it satisfies $g(\boldsymbol{x}^*) = 0$
    - ∗ We have a constrained optimisation problem

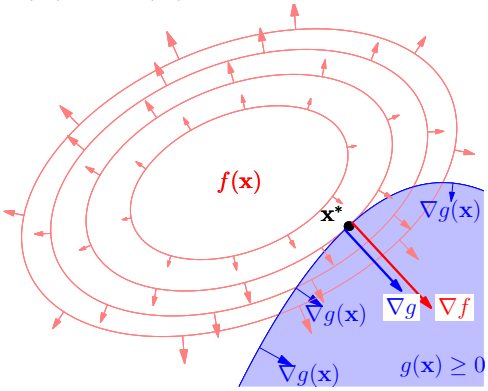
Inside Region

$\nabla f(\mathbf{x}) = \alpha \nabla g(\mathbf{x})$, $\alpha < 0$
not a local optimum



On the Boundary

$\nabla f(\mathbf{x}^*) = \alpha \nabla g(\mathbf{x}^*)$, $\alpha \approx 1.4 > 0$



KKT Conditions

- To minimise $f(\boldsymbol{x})$ subject to $g(\boldsymbol{x}) \geq 0$

$$\mathcal{L}(\boldsymbol{x},\alpha) = f(\boldsymbol{x}) - \alpha g(\boldsymbol{x})$$

- Then $\nabla_{\boldsymbol{x}}\mathcal{L} = 0$ or

$$\nabla_{\boldsymbol{x}}\mathcal{L} = \nabla_{\boldsymbol{x}} f(\boldsymbol{x}) - \alpha \nabla_{\boldsymbol{x}} g(\boldsymbol{x}) = 0$$

- with $\alpha \geq 0$ and $\alpha g(\boldsymbol{x}) = 0$: either $\alpha = 0$ (the constraint plays no part) or $\alpha > 0$ and $g(\boldsymbol{x}) = 0$ (the solution is on the boundary)
- These conditions are known as the Karush-Kuhn-Tucker conditions

Many Inequalities

- Given the problem

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{subject to } g_k(\boldsymbol{x}) \geq 0 \text{ for } k = 1,2,\ldots,m$$

- We introduce multiple Lagrange multipliers

$$\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha}) = f(\boldsymbol{x}) - \sum_{k=1}^{m} \alpha_k g_k(\boldsymbol{x})$$

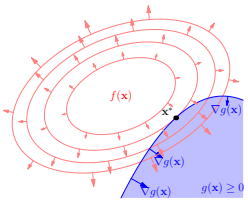
- The condition for an optimum is

$$\nabla_{\boldsymbol{x}} f(\boldsymbol{x}) = \sum_{k=1}^{m} \alpha_k \nabla_{\boldsymbol{x}} g_k(\boldsymbol{x})$$

- Plus, for each k, $\alpha_k \geq 0$ and $\alpha_k g_k(\boldsymbol{x}) = 0$

Outline

1. Constrained Optimisation
2. Inequalities
3. **Duality**



Solving the Lagrangian for x

- Consider minimising a function $f(\boldsymbol{x})$ subject to a set of constraints $g_i(\boldsymbol{x}) = 0$ or $g_i(\boldsymbol{x}) \geq 0$
- We can consider this a double optimisation problem

$$\max_{\boldsymbol{\alpha}} \min_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha}) = \max_{\boldsymbol{\alpha}} \min_{\boldsymbol{x}} \left(f(\boldsymbol{x}) - \sum_i \alpha_i g_i(\boldsymbol{x}) \right)$$

where $\alpha_i \geq 0$ for each inequality constraint

Dual Problem

- If $f(\boldsymbol{x})$ and $g_i(\boldsymbol{x})$ are simple we can sometimes find a set of variables $\boldsymbol{x}^*(\boldsymbol{\alpha})$ that minimises the Lagrangian

$$\nabla_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x}^*(\boldsymbol{\alpha}),\boldsymbol{\alpha}) = 0$$

- This leaves us with the **dual problem**

$$\max_{\boldsymbol{\alpha}} \mathcal{L}(\boldsymbol{x}^*(\boldsymbol{\alpha}),\boldsymbol{\alpha})$$

- If we had an inequality constraint $g_i(\boldsymbol{x}) \geq 0$ then we would have the additional constraint in the dual problem $\alpha_i \geq 0$

Linear Programming

- In linear programming we minimise a linear objective function $\boldsymbol{c}^{\mathsf{T}}\boldsymbol{x}$ subject to linear constraints $\boldsymbol{g}(\boldsymbol{x}) = \mathbf{M}\boldsymbol{x} - \boldsymbol{b} = 0$ (or $\boldsymbol{g}(\boldsymbol{x}) \geq \mathbf{0}$ with $\boldsymbol{x} \geq \mathbf{0}$)
- The Lagrangian becomes

$$\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha}) = \boldsymbol{c}^{\mathsf{T}}\boldsymbol{x} - \boldsymbol{\alpha}^{\mathsf{T}}(\mathbf{M}\boldsymbol{x} - \boldsymbol{b})$$

- An equivalent way of writing the Lagrangian is

$$\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha}) = \boldsymbol{b}^{\mathsf{T}}\boldsymbol{\alpha} - \boldsymbol{x}^{\mathsf{T}}(\mathbf{M}^{\mathsf{T}}\boldsymbol{\alpha} - \boldsymbol{c})$$

- An entirely equivalent interpretation is that we maximise an objective function $\boldsymbol{b}^{\mathsf{T}}\boldsymbol{\alpha}$ subject to constraints $\mathbf{M}^{\mathsf{T}}\boldsymbol{\alpha} - \boldsymbol{c} = 0$ (or $\mathbf{M}^{\mathsf{T}}\boldsymbol{\alpha} - \boldsymbol{c} \leq \mathbf{0}$ with $\boldsymbol{\alpha} \geq \mathbf{0}$)

Linear Programming Example

- Suppose we eat potatoes and rice and we want to ensure that we get enough vitamin A and C

	Potatoes	Rice	Daily Requirement
Vitamin A	3	5	20
Vitamin C	5	2	24
Price	5	4	

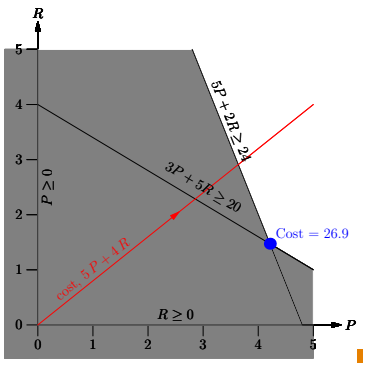
- We want to buy P kg potatoes and R kg of rice as cheaply as possible subject to fulfilling our vitamin requirement

$$\min_{P,R} 5P + 4R$$

subject to $P,R \geq 0$, $3P + 5R \geq 20$ and $5P + 2R \geq 24$

Linear Programming

- Minimise $5P + 4R$
- Subject to
  - $3P + 5R \geq 20$
  - $5P + 2R \geq 24$
  - $P,R \geq 0$



Lagrangian

- We can write the problem as a Lagrange problem

$$\min_{P,R}\max_{A,C} \quad 5P+4R-A(3P+5R-20)-C(5P+2R-24)$$

- subject to $P,R,A,C \geq 0$
- A and C are Lagrange multipliers for vitamin A and C
- We can rearrange the Lagrangian to obtain

$$\max_{A,C}\min_{P,R} \quad 20A+24C-P(3A+5C-5)-R(5A+2C-4)$$

Dual Problem

- The Lagrangian

$$\max_{A,C}\min_{P,R} \quad 20A+24C-P(3A+5C-5)-R(5A+2C-4)$$

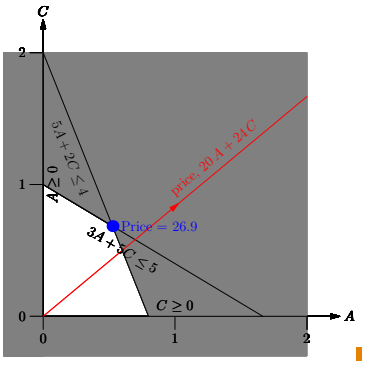
leads to the dual problem

$$\max_{A,C} 20A+24C$$
$$\text{subject to} \quad 3A+5C\leq 5 \quad 5A+2C\leq 4 \quad A,C\geq 0$$

- Consider someone selling vitamins A and C. They want to maximise the price of vitamins A and C, but their prices cannot exceed the price of the vitamins in potatoes or rice

Dual Linear Programme

- Maximise $20A+24C$
- Subject to
  - $3A+5C\leq 5$
  - $5A+2C\leq 4$
  - $A,C\geq 0$



Why?

- Why are we bothered about translating one linear programme into another?
- Sometimes one form is massively easier to solve than the other
- This is because the first linear programme depends on the dimensionality of $\boldsymbol{x}$ while the second linear programme depends on the number of constraints (or dimensionality of $\boldsymbol{\alpha}$)
- This is important, for example, in Wasserstein GANs

Quadratic Programming

- A quadratic programme involves minimising a quadratic function $\boldsymbol{x}^\mathsf{T}\mathbf{Q}\boldsymbol{x}$ (with $\mathbf{Q}\succ 0$) subject to linear constraints $\mathbf{M}\boldsymbol{x}=\boldsymbol{b}$ (or $\mathbf{M}\boldsymbol{x}\geq\boldsymbol{b}$)
- We can define the Lagrangian

$$\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha})=\boldsymbol{x}^\mathsf{T}\mathbf{Q}\boldsymbol{x}-\boldsymbol{\alpha}^\mathsf{T}(\mathbf{M}\boldsymbol{x}-\boldsymbol{b})$$

- Where the solution is given by $\max_{\boldsymbol{\alpha}}\min_{\boldsymbol{x}}\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha})$
- If the constraints are inequality constraints then $\alpha_i\geq 0$

Solution to Quadratic Programming Problem

- Using

$$\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha})=\boldsymbol{x}^\mathsf{T}\mathbf{Q}\boldsymbol{x}-\boldsymbol{\alpha}^\mathsf{T}(\mathbf{M}\boldsymbol{x}-\boldsymbol{b})$$

- Then

$$\boldsymbol{\nabla}_{\boldsymbol{x}}\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha})=2\mathbf{Q}\boldsymbol{x}-\mathbf{M}^\mathsf{T}\boldsymbol{\alpha}$$

- So $\boldsymbol{\nabla}_{\boldsymbol{x}}\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha})=0$ implies

$$\boldsymbol{x}^*=\frac{1}{2}\mathbf{Q}^{-1}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha}$$

Dual Quadratic Programming Problem

- Substituting $\boldsymbol{x}^*=\frac{1}{2}\mathbf{Q}^{-1}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha}$ into

$$\mathcal{L}(\boldsymbol{x},\boldsymbol{\alpha})=\boldsymbol{x}^\mathsf{T}\mathbf{Q}\boldsymbol{x}-\boldsymbol{\alpha}^\mathsf{T}(\mathbf{M}\boldsymbol{x}-\boldsymbol{b})$$

- We get the dual problem

$$\max_{\boldsymbol{\alpha}} \quad -\frac{1}{4}\boldsymbol{\alpha}^\mathsf{T}\mathbf{M}\mathbf{Q}^{-1}\mathbf{M}^\mathsf{T}\boldsymbol{\alpha}+\boldsymbol{\alpha}^\mathsf{T}\boldsymbol{b}$$

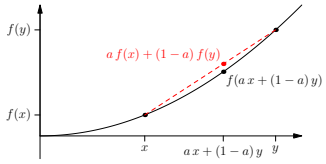
- If the constraints were inequality constraints then we have $\alpha_i\geq 0$
- We have exchanged one quadratic programme for another, but sometimes that is very useful (e.g. SVMs)

Lessons

- A useful tool for performing constrained optimisation is the introduction of Lagrange multipliers
- This is particularly useful for problems with unique solutions (it will work when there are multiple solutions, but finding many saddle points is a pain)
- For inequality constraints we need to satisfy KKT conditions
- For simple situations (linear and quadratic programming) we can eliminate the original variables to obtain the dual problem

Mathematics of Machine Learning

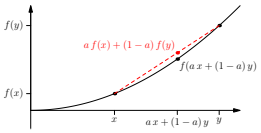
Convexity



Convex sets, convex functions, Jensen's inequality

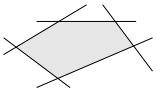
Outline

1. **Convex sets**
2. Convex functions
3. Jensen's inequality



Convex Regions

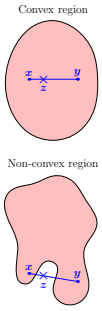
- Convex regions are familiar



- For any two points $\boldsymbol{x}$ and $\boldsymbol{y}$ in a region $\mathcal{R}$ then for any $a \in [0,1]$ if

$$\boldsymbol{z} = a\boldsymbol{x} + (1-a)\boldsymbol{y} \in \mathcal{R}$$

- then $\mathcal{R}$ is a convex region



Convex Sets

- For any set, $\mathcal{S}$, where addition and scalar multiplication are defined (e.g. a vector space) then:

  If for any two elements $\boldsymbol{x},\boldsymbol{y} \in \mathcal{S}$ and any $a \in [0,1]$

$$\boldsymbol{z} = a\boldsymbol{x} + (1-a)\boldsymbol{y} \in \mathcal{S}$$

  then $\mathcal{S}$ is said to be a convex set

Positive Semi-Definite Matrices

- Recall that a matrix $\mathbf{M}$ is positive semi-definite if for any vector $\boldsymbol{v}$

$$\boldsymbol{v}^\mathsf{T}\mathbf{M}\boldsymbol{v} \geq 0$$

  (i.e. any quadratic form of the matrix is non-negative)

- (We showed this also implies that all the eigenvalues are non-negative)

- We denote the fact that $\mathbf{M}$ is positive semi-definite by $\mathbf{M} \succeq 0$, and $\mathbf{M} \succ 0$ if it is positive definite

- The set of positive semi-definite (PSD) matrices (or kernels) form a convex set

Proof

- Consider any two arbitrarily chosen PSD matrices $\mathbf{M}_1$ and $\mathbf{M}_2$ and any $a \in [0,1]$ then let

$$\mathbf{M}_3 = a\mathbf{M}_1 + (1-a)\mathbf{M}_2$$

- Then for any vector $\boldsymbol{v}$

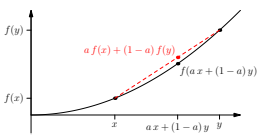
$$\begin{aligned}\boldsymbol{v}^\mathsf{T}\mathbf{M}_3\boldsymbol{v} &= \boldsymbol{v}^\mathsf{T}(a\mathbf{M}_1 + (1-a)\mathbf{M}_2)\boldsymbol{v}\\ &= a\boldsymbol{v}^\mathsf{T}\mathbf{M}_1\boldsymbol{v} + (1-a)\boldsymbol{v}^\mathsf{T}\mathbf{M}_2\boldsymbol{v}\\ &= am_1 + (1-a)m_2\end{aligned}$$

  where $m_1 = \boldsymbol{v}^\mathsf{T}\mathbf{M}_1\boldsymbol{v}$ and $m_2 = \boldsymbol{v}^\mathsf{T}\mathbf{M}_2\boldsymbol{v}$

- But $m_1, m_2 \geq 0$ since $\mathbf{M}_1, \mathbf{M}_2 \succeq 0$. Thus $am_1 + (1-a)m_2 \geq 0$ and so $\mathbf{M}_3 \succeq 0$ □

Outline

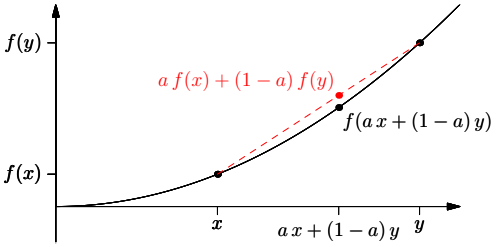
1. Convex sets
2. **Convex functions**
3. Jensen's inequality



Convex Functions

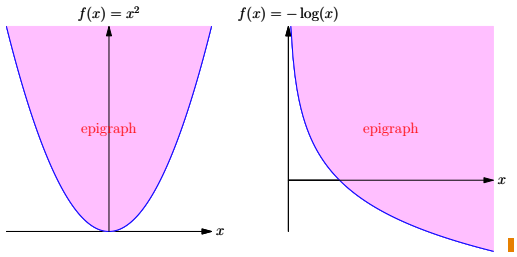
- Any function $f(x)$ is said to be a **convex function** if for any two points x and y and any $a \in [0,1]$

$$f(ax + (1-a)y) \leq af(x) + (1-a)f(y)$$



Epigraph

- The **epigraph** of a function is the area that lies above the function
- The epigraph of a convex function is a convex region



Convex-Down or Concave Functions

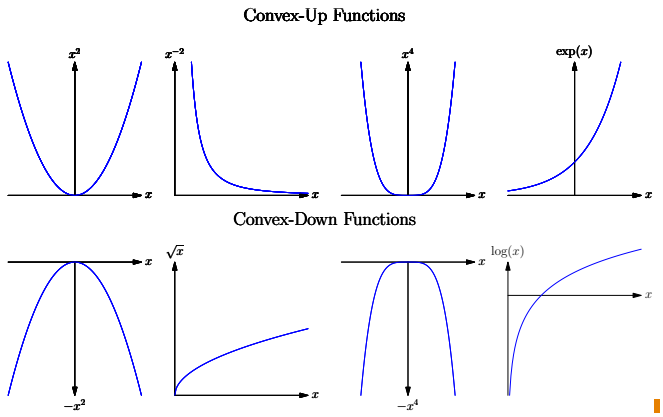
- Any function, $f(x)$, that satisfies the reverse inequality

$$f(ax + (1-a)y) \geq af(x) + (1-a)f(y)$$

for any points x and y and any $a \in [0,1]$ is said to be a **convex-down** or **concave** function

- Everything true for a convex(-up) function carries over to a convex-down function with a small modification
- If $f(x)$ is a convex-up function then $g(x) = -f(x)$ is a convex-down function
- The area that lies below a convex-down function is a convex region

Examples



Linear Functions

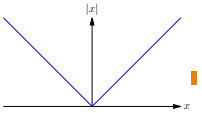
- Linear functions are given by

$$f(x) = mx + c$$

- They satisfy the **equality**

$$f(ax + (1-a)y) = af(x) + (1-a)f(y)$$

- As such they are both convex(-up) and convex-down functions
- $|x|$ is a convex-up function



Strictly Convex Function

- Functions that satisfy the strict inequality (for $0 < a < 1$ and $x \neq y$)

$$f(ax + (1-a)y) < af(x) + (1-a)f(y)$$

are said to be **strictly convex functions**

- A strictly convex-down function satisfies the reverse strict inequality
- Strictly convex-(up or down) functions don't contain any linear regions

Convexity in High Dimensions

- If $f : \mathbb{R}^n \to \mathbb{R}$ (i.e. $f(\boldsymbol{x})$ maps high dimensional point $\boldsymbol{x} \in \mathbb{R}^n$ to a real value) satisfies

$$f(a\boldsymbol{x} + (1-a)\boldsymbol{y}) \leq af(\boldsymbol{x}) + (1-a)f(\boldsymbol{y})$$

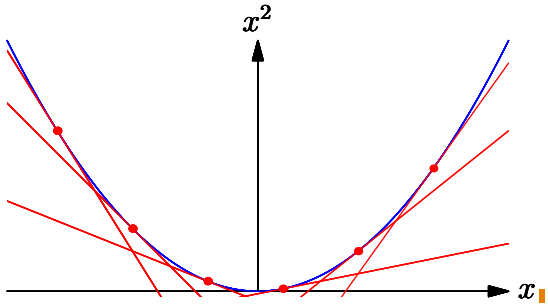
for any $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$ and any $a \in [0,1]$ then $f(\boldsymbol{x})$ is a convex function

- $\|\boldsymbol{x}\|_2^2 = \sum_i x_i^2$ is a (strictly) convex function
- $\|\boldsymbol{x}\|_1 = \sum_i |x_i|$ is a convex function

Properties of Convex Functions

- Differentiable convex functions lie on or above any tangent line

$$f(x) \geq f(x^*) + (x - x^*)f'(x^*)$$



Second Derivatives

- Adding the tangent-line inequalities at x and $x + \epsilon$, for any $\epsilon > 0$

$$f'(x + \epsilon) \geq f'(x)$$

therefore $f''(x) = \lim_{\epsilon \to 0}(f'(x+\epsilon) - f'(x))/\epsilon \geq 0$ at all points x

- In high dimensions a convex function lies above its tangent plane

$$f(\boldsymbol{x}) \geq f(\boldsymbol{x}^*) + (\boldsymbol{x} - \boldsymbol{x}^*)^\mathsf{T} \boldsymbol{\nabla} f(\boldsymbol{x}^*)$$

- The matrix of second derivatives (the Hessian) must be positive semi-definite at all points $\boldsymbol{x}$

$$\mathbf{H} = \begin{pmatrix} \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1^2} & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_1 \partial x_2} & \cdots \\ \frac{\partial^2 f(\boldsymbol{x})}{\partial x_2 \partial x_1} & \frac{\partial^2 f(\boldsymbol{x})}{\partial x_2^2} & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix} \succeq 0$$

Proving Convexity

- $f(x) = x^2$ is strictly convex as $f''(x) = 2 > 0$
- $f(x) = \mathrm{e}^{cx}$ $(c \neq 0)$ is strictly convex as $f''(x) = c^2\mathrm{e}^{cx} > 0$
- $f(x) = \log(x)$ is strictly convex-down as $f''(x) = -\frac{1}{x^2} < 0$
- $f(x,y) = \frac{x^2}{y}$ is convex for $y > 0$ as

$$\mathbf{H} = \begin{pmatrix} \frac{\partial^2 f(x,y)}{\partial x^2} & \frac{\partial^2 f(x,y)}{\partial x \partial y} \\ \frac{\partial^2 f(x,y)}{\partial y \partial x} & \frac{\partial^2 f(x,y)}{\partial y^2} \end{pmatrix} = \begin{pmatrix} \frac{2}{y} & -\frac{2x}{y^2} \\ -\frac{2x}{y^2} & \frac{2x^2}{y^3} \end{pmatrix} = \frac{2}{y^3}\begin{pmatrix} y^2 & -xy \\ -xy & x^2 \end{pmatrix}$$

- Now $T = \mathrm{tr}\mathbf{H} = \frac{2}{y^3}(x^2 + y^2)$, $D = \det(\mathbf{H}) = 0$
- $\lambda_{1,2} = T/2 \pm \sqrt{T^2/4 - D} = \{0, T\} = \{0, \frac{2(x^2+y^2)}{y^3}\} \geq 0 \Rightarrow \mathbf{H} \succeq 0$

Sums of Convex Functions

- For any set of convex functions $f_1(x)$, $f_2(x)$, … and any set of non-negative scalars a_1, a_2, … then

$$g(x) = \sum_i a_i f_i(x)$$

is convex

- Proof

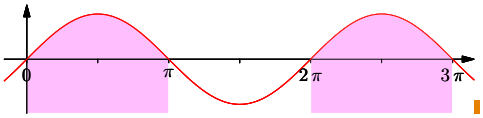
$$g''(x) = \sum_i a_i f_i''(x)$$

but $f_i''(x) \geq 0$ so $g''(x)$ is a sum of non-negative terms

- This generalises to higher dimensions as the sum of PSD matrices times non-negative factors is a PSD matrix

Convex Functions Defined on Convex Sets

- All the properties we have discussed hold for functions defined on a convex set
- $\sin(x)$ is neither convex-up nor convex-down on the whole real line
- $\sin(x)$ for $x \in [0,\pi]$ is convex-down (its second derivative $-\sin(x)$ is less than or equal to 0 in this range)



- For a convex function defined on a non-convex set, $\mathcal{S}$, there exist points $\boldsymbol{x},\boldsymbol{y} \in \mathcal{S}$ such that for some $a \in [0,1]$ the point $\boldsymbol{z} = a\boldsymbol{x} + (1-a)\boldsymbol{y} \notin \mathcal{S}$ (the function isn't defined at such points)

Constraints

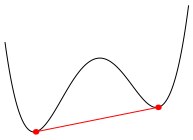
- Often we impose constraints on the set of points, e.g.

$$x_i > 0 \qquad \boldsymbol{a}^\mathsf{T}\boldsymbol{x} = b \qquad \boldsymbol{x}^\mathsf{T}\mathbf{M}\boldsymbol{x} \leq 1$$

- Linear constraints (e.g. $x_i > 0$ or $\boldsymbol{a}^\mathsf{T}\boldsymbol{x} = b$ or $\boldsymbol{a}^\mathsf{T}\boldsymbol{x} \leq b$) always define a convex region
- Multiple linear constraints always define a convex region
- Non-linear constraints may or may not define a convex region ($\{\boldsymbol{x} \in \mathbb{R}^n | \boldsymbol{x}^\mathsf{T}\mathbf{M}\boldsymbol{x} \leq 1, \mathbf{M} \succeq 0\}$ does while $\{\boldsymbol{x} \in \mathbb{R}^n | \boldsymbol{x}^\mathsf{T}\mathbf{M}\boldsymbol{x} \geq 1, \mathbf{M} \succeq 0\}$ doesn't)

Unique Minimum

- A strictly convex function has at most one minimum (e^x has none)
- A local minimum that is not global would break convexity
  - ⋆ On the line from a local minimum to a lower global minimum f lies on or below the chord, which strictly decreases
  - ⋆ Thus there are points next to the local minimum with lower values
  - ⋆ This is a contradiction



- This remains true if we consider convex functions that are constrained to live in a convex set

Convex Set of Minima

- If $f(\boldsymbol{x})$ is **convex** but not **strictly convex** then there might exist a convex set $\mathcal{M} \subset \mathcal{X}$ of minima such that for all $\boldsymbol{x},\boldsymbol{y} \in \mathcal{M}$ and any $\boldsymbol{z} \in \mathcal{X}$ we have $f(\boldsymbol{x}) = f(\boldsymbol{y}) \leq f(\boldsymbol{z})$
- This set of minima is convex, that is, if $\boldsymbol{x},\boldsymbol{y} \in \mathcal{M}$ then for any $a \in [0,1]$ the point $\boldsymbol{z} = a\boldsymbol{x} + (1-a)\boldsymbol{y} \in \mathcal{M}$
- The sum of a convex function, $f(x)$, and a strictly convex function $g(x)$ will always be strictly convex: adding the inequality that defines convexity for f to the strict inequality for g gives a strict inequality

Linear Regression

- For linear regression the loss function

$$L(\boldsymbol{w}) = \|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2 = \boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{w} - 2\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y} + \boldsymbol{y}^\mathsf{T}\boldsymbol{y}$$

is convex

- Since the Hessian $\mathbf{H} = 2\mathbf{X}^\mathsf{T}\mathbf{X} \succeq 0$ (positive semi-definite) (For any vector $\boldsymbol{v}$ then $\boldsymbol{v}^\mathsf{T}\mathbf{H}\boldsymbol{v} = 2\boldsymbol{v}^\mathsf{T}\mathbf{X}^\mathsf{T}\mathbf{X}\boldsymbol{v} = 2\|\mathbf{X}\boldsymbol{v}\|^2 \geq 0$)
- If $\mathbf{H} \succ 0$ there will be a unique minimum, while if $\mathbf{H}$ has some zero eigenvalues there will be a family of solutions

Regularised Linear Regression

- In ridge regression we minimise a loss

$$L(\boldsymbol{w}) = \|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2 + \eta\|\boldsymbol{w}\|^2 = \boldsymbol{w}^\mathsf{T}(\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I})\boldsymbol{w} - 2\boldsymbol{w}^\mathsf{T}\mathbf{X}^\mathsf{T}\boldsymbol{y} + \boldsymbol{y}^\mathsf{T}\boldsymbol{y}$$

- Because $\|\boldsymbol{w}\|^2$ is strictly convex the loss function is strictly convex and so will have a unique solution
- Using an L_1 regulariser (Lasso)

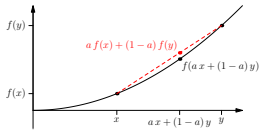
$$L(\boldsymbol{w}) = \|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2 + \eta\|\boldsymbol{w}\|_1$$

again $\|\boldsymbol{w}\|_1$ is convex and so $L(\boldsymbol{w})$ will be convex

- Using an L_1 and an L_2 regulariser (elastic net) also gives a unique solution

Outline

1. Convex sets
2. Convex functions
3. **Jensen's inequality**



Jensen's Inequality

- In proving many properties of learning machines inequalities are really useful
- One of the most useful inequalities involves expectations of convex functions, this is known as **Jensen's Inequality**
- If $f(\boldsymbol{x})$ is a convex(-up) function then

$$\mathbb{E}[f(\boldsymbol{X})] \geq f(\mathbb{E}[\boldsymbol{X}])$$

- If $f(\boldsymbol{x})$ is a convex(-down) function then

$$\mathbb{E}[f(\boldsymbol{X})] \leq f(\mathbb{E}[\boldsymbol{X}])$$

Proof

- We said before that a (differentiable) convex function must lie on or above its tangent plane at any point $\boldsymbol{x}^*$

$$f(\boldsymbol{x}) \geq f(\boldsymbol{x}^*) + (\boldsymbol{x}-\boldsymbol{x}^*)^\mathsf{T}\boldsymbol{\nabla} f(\boldsymbol{x}^*)$$

- Taking $\boldsymbol{x}^* = \mathbb{E}[\boldsymbol{X}]$

$$f(\boldsymbol{X}) \geq f(\mathbb{E}[\boldsymbol{X}]) + (\boldsymbol{X}-\mathbb{E}[\boldsymbol{X}])^\mathsf{T}\boldsymbol{\nabla} f(\mathbb{E}[\boldsymbol{X}])$$

- Taking expectations of both sides

$$\begin{aligned}\mathbb{E}[f(\boldsymbol{X})] &\geq f(\mathbb{E}[\boldsymbol{X}]) + (\mathbb{E}[\boldsymbol{X}]-\mathbb{E}[\boldsymbol{X}])^\mathsf{T}\boldsymbol{\nabla} f(\mathbb{E}[\boldsymbol{X}]) \\ &= f(\mathbb{E}[\boldsymbol{X}]) \qquad \square\end{aligned}$$

Simple Proofs with Jensen's Inequality

- Since $f(x) = x^2$ is convex by Jensen's inequality

$$\mathbb{E}\left[X^2\right] \geq (\mathbb{E}[X])^2 \quad \text{or} \quad \mathbb{E}\left[X^2\right] - \mathbb{E}[X]^2 \geq 0$$

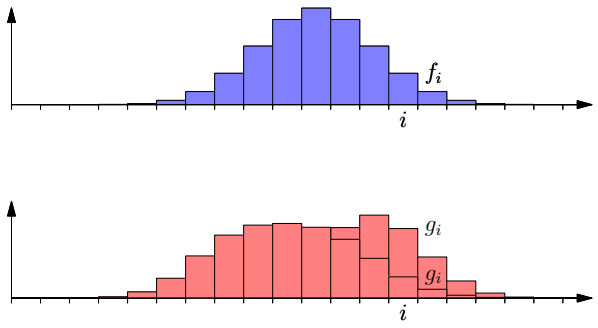
(i.e. variances are non-negative)

- The KL-divergence $\mathrm{KL}(f\|g)$ between two categorical probability distributions $(f_1, f_2, \ldots)$ and $(g_1, g_2, \ldots)$ is defined as

$$\mathrm{KL}(f\|g) = -\sum_i f_i \log\left(\frac{g_i}{f_i}\right)$$

(note $f_i, g_i \geq 0$ and $\sum_i f_i = \sum_i g_i = 1$)

Kullback-Leibler Divergence



$$\mathrm{KL}(\boldsymbol{f}\|\boldsymbol{g}) = -\sum_{i=1}^{n} f_i \log\left(\frac{g_i}{f_i}\right) = 0.2355$$

Proof of Gibbs' Inequality

- To show that $\mathrm{KL}(f\|g) \geq 0$ (Gibbs' inequality) we note that since the logarithm is a convex-down function

$$\begin{aligned}\mathrm{KL}(f\|g) &= -\sum_i f_i \log\left(\frac{g_i}{f_i}\right) = \mathbb{E}_f\left[-\log\left(\frac{g_i}{f_i}\right)\right] \\ &\geq -\log\left(\mathbb{E}_f\left[\frac{g_i}{f_i}\right]\right) \\ &= -\log\left(\sum_i f_i \frac{g_i}{f_i}\right) = -\log\left(\sum_i g_i\right) = -\log(1) = 0\end{aligned}$$

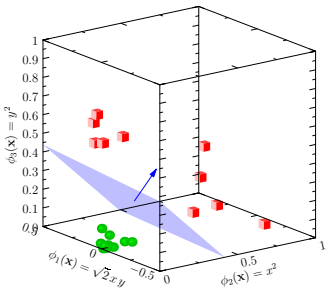
- If some $f_i = 0$ the sums run over $f_i > 0$ only (as $0\log(0) = 0$), so $\sum_i g_i \leq 1$: still ≥ 0
- We will meet KL-divergences later on

Lessons

- Although we haven't talked much about machine learning, convexity is heavily used in many machine learning applications
- A lot of ML algorithms involve convex functions e.g. SVMs
- As such they will have a unique minimum (or a convex set of minima)
- Convexity is an elegant idea which is relatively easy to prove theorems about
- One of the most useful tools is Jensen's inequality

Mathematics of Machine Learning

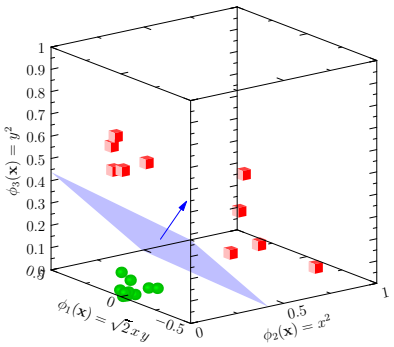
Support Vector Machines



Support Vector Machines, maximum margins

Outline

1. **The Big Picture**
2. Maximum Margins
3. Duality
4. Practice

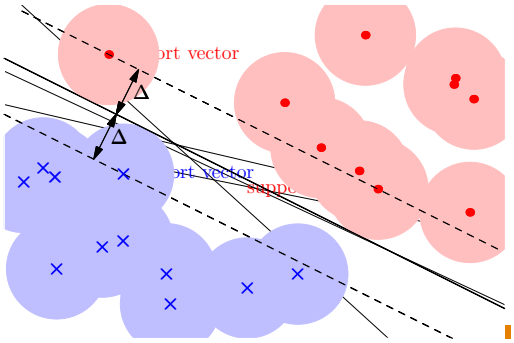


Support Vector Machines

- Support vector machines, when used right, often have the best generalisation results
- They are typically used on numerical data, but can and have been adapted to text, sequences, etc.
- Although not as trendy as deep learning, they will often be the method of choice on small data sets
- They subtly regularise themselves, choosing a solution that generalises well from a host of different solutions

Linear Separation of Data

- SVMs classify linearly separable data



- Finds maximum-margin separating plane

Extended Feature Space

- To increase the likelihood of linear-separability we often use a high-dimensional mapping

$$\boldsymbol{x} = (x_1, x_2, \ldots, x_p)^\mathsf{T} \rightarrow \boldsymbol{\phi}(\boldsymbol{x}) = (\phi_1(\boldsymbol{x}), \phi_2(\boldsymbol{x}), \ldots, \phi_{p'}(\boldsymbol{x}))^\mathsf{T}$$

  with $p' \gg p$
- Finding the maximum margin hyper-plane is time consuming in "primal" form if p' is large
- We can work in the "dual" space of patterns, then we only need to compute inner-products

$$\langle \boldsymbol{\phi}(\boldsymbol{x}_i), \boldsymbol{\phi}(\boldsymbol{x}_j) \rangle = \boldsymbol{\phi}(\boldsymbol{x}_i)^\mathsf{T} \boldsymbol{\phi}(\boldsymbol{x}_j) = \sum_{k=1}^{p'} \phi_k(\boldsymbol{x}_i) \phi_k(\boldsymbol{x}_j)$$

Kernel Trick

- If we choose a **positive semi-definite** kernel function $K(\boldsymbol{x}, \boldsymbol{y})$ then there exist functions $\boldsymbol{\phi}(\boldsymbol{x}) = (\phi_k(\boldsymbol{x}) | k = 1, 2, \ldots, p')$, such that

$$K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \langle \boldsymbol{\phi}(\boldsymbol{x}_i), \boldsymbol{\phi}(\boldsymbol{x}_j) \rangle$$

  (like an eigenvector decomposition of a matrix)
- Never need to compute $\phi_k(\boldsymbol{x}_i)$ explicitly as we only need the inner-product $\langle \boldsymbol{\phi}(\boldsymbol{x}_i), \boldsymbol{\phi}(\boldsymbol{x}_j) \rangle = K(\boldsymbol{x}_i, \boldsymbol{x}_j)$ to compute maximum margin separating hyper-plane
- Sometimes $\boldsymbol{\phi}(\boldsymbol{x}_i)$ is an infinite dimensional vector so it is good we don't have to compute all the elements!

Kernel Functions

- Kernel functions are symmetric functions of two variables
- Strong restriction: *positive semi-definite*
- Examples

Quadratic kernel: $K(\boldsymbol{x}_1, \boldsymbol{x}_2) = \left(\boldsymbol{x}_1^\mathsf{T} \boldsymbol{x}_2\right)^2$

Gaussian (RBF) kernel: $K(\boldsymbol{x}_1, \boldsymbol{x}_2) = \mathrm{e}^{-\gamma \|\boldsymbol{x}_1 - \boldsymbol{x}_2\|^2}$

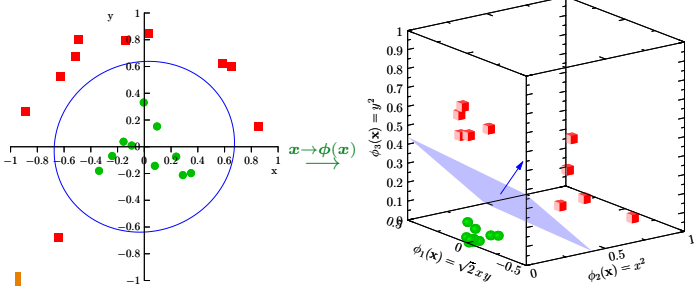
- Consider the mapping

$$\boldsymbol{x}_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \rightarrow \boldsymbol{\phi}(\boldsymbol{x}_i) = \begin{pmatrix} x_i^2 \\ y_i^2 \\ \sqrt{2} x_i y_i \end{pmatrix}$$

Non-linear Separation of Data

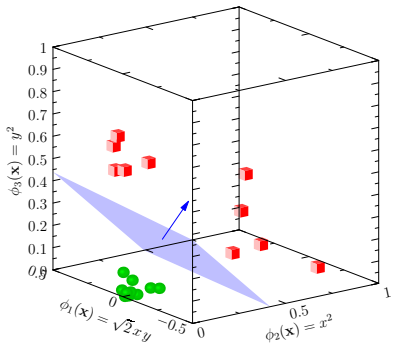
$$K(\boldsymbol{x}_1, \boldsymbol{x}_2) = \begin{pmatrix} x_1^2 & y_1^2 & \sqrt{2} x_1 y_1 \end{pmatrix} \begin{pmatrix} x_2^2 \\ y_2^2 \\ \sqrt{2} x_2 y_2 \end{pmatrix} = x_1^2 x_2^2 + y_1^2 y_2^2 + 2 x_1 y_1 x_2 y_2$$

$$= (x_1 x_2 + y_1 y_2)^2 = \left(\boldsymbol{x}_1^\mathsf{T} \boldsymbol{x}_2\right)^2$$



Outline

1. The Big Picture
2. **Maximum Margins**
3. Duality
4. Practice

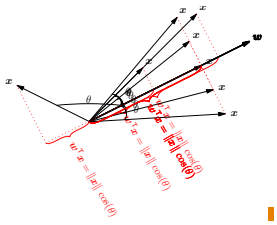


Inner Product

- Recall the inner or dot product in $\mathbb{R}^n$

$$\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \boldsymbol{x} \cdot \boldsymbol{y} = \boldsymbol{x}^\mathsf{T}\boldsymbol{y} = \sum_{i=1}^{n} x_i y_i = \|\boldsymbol{x}\| \|\boldsymbol{y}\| \cos(\theta)$$

- If $\|\boldsymbol{w}\| = 1$ then $\langle \boldsymbol{x}, \boldsymbol{w} \rangle = \|\boldsymbol{x}\| \cos(\theta)$



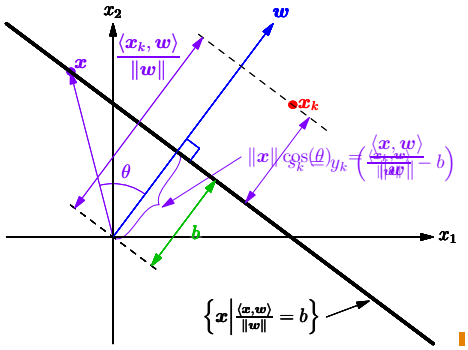
Maximise Margin

- Consider a linearly separable set of data
  - $\mathcal{D} = \{(\boldsymbol{x}_k, y_k)\}_{k=1}^m$
  - $y_k \in \{-1, 1\}$
- Our task is to find a separating plane defined by the orthogonal vector $\boldsymbol{w}$ and a threshold b such that

$$y_k \left(\frac{\langle \boldsymbol{w}, \boldsymbol{x}_k \rangle}{\|\boldsymbol{w}\|} - b \right) \geq \Delta$$

where Δ is the margin

Distance to Hyperplanes



Constrained Optimisation

- Wish to find $\boldsymbol{w}$ and b to maximise Δ subject to constraints

$$y_k \left(\frac{\langle \boldsymbol{w}, \boldsymbol{x}_k \rangle}{\|\boldsymbol{w}\|} - b \right) \geq \Delta \quad \text{for all } k = 1, 2, \ldots, m$$

- If we divide through by Δ

$$y_k \left(\frac{\langle \boldsymbol{w}, \boldsymbol{x}_k \rangle}{\Delta \|\boldsymbol{w}\|} - \frac{b}{\Delta} \right) \geq 1 \quad \text{for all } k = 1, 2, \ldots, m$$

- Define $\hat{\boldsymbol{w}} = \boldsymbol{w}/(\Delta\|\boldsymbol{w}\|)$ and $\hat{b} = b/\Delta$

$$y_k \left(\langle \hat{\boldsymbol{w}}, \boldsymbol{x}_k \rangle - \hat{b} \right) \geq 1$$

Quadratic Programming Problem

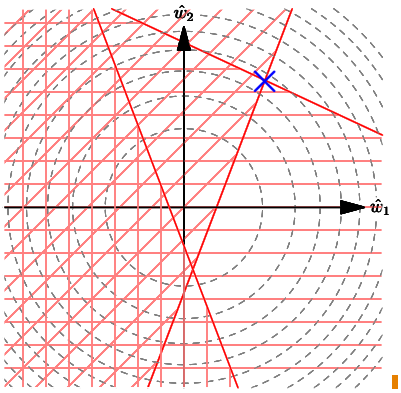
- Note that as $\hat{\boldsymbol{w}} = \boldsymbol{w}/(\Delta\|\boldsymbol{w}\|)$

$$\|\hat{\boldsymbol{w}}\| = \left\| \frac{\boldsymbol{w}}{\Delta \|\boldsymbol{w}\|} \right\| = \frac{1}{\Delta \|\boldsymbol{w}\|} \|\boldsymbol{w}\| = \frac{1}{\Delta}$$

- Minimising $\|\hat{\boldsymbol{w}}\|^2$ is equivalent to maximising the margin Δ
- Can write the optimisation problem as a *quadratic programming problem*

$$\min_{\hat{\boldsymbol{w}}, \hat{b}} \frac{\|\hat{\boldsymbol{w}}\|^2}{2} \quad \text{subject to } y_k \left(\langle \hat{\boldsymbol{w}}, \boldsymbol{x}_k \rangle - \hat{b} \right) \geq 1 \text{ for all } k = 1, 2, \ldots, m$$

Quadratic Programming in SVMs

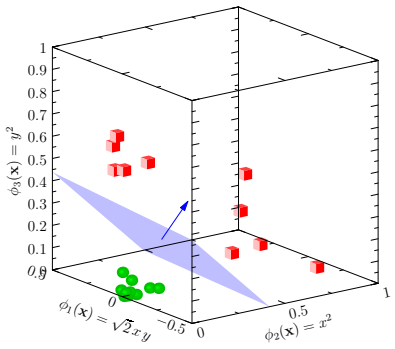


Quadratic Programming

- We have a quadratic programming problem for the weights $\hat{\boldsymbol{w}} = (\hat{w}_1, \hat{w}_2, \ldots, \hat{w}_p)$ and bias $\hat{b}$ and m constraints
- This is a classic but fiddly optimisation problem
- It is a **convex** optimisation problem (strictly convex in $\hat{\boldsymbol{w}}$) so the optimal $\hat{\boldsymbol{w}}$ is unique
- It can be solved in roughly $O(p^3)$ time (it involves solving $p \times p$ linear systems) (phew it is not NP-complete!)
- We will see that there is an equivalent dual problem which allows us to use the kernel trick with time complexity $O(m^3)$

Outline

1. The Big Picture
2. Maximum Margins
3. Duality
4. Practice



Lagrangian

- In the extended feature space, $\phi(\boldsymbol{x})$, the maximum-margin plane $(\boldsymbol{w}, b)$ solves

$$\min_{\boldsymbol{w},b} \frac{\|\boldsymbol{w}\|^2}{2} \quad \text{subject to } y_k(\langle \boldsymbol{w},\boldsymbol{\phi}(\boldsymbol{x}_k)\rangle - b) \geq 1 \text{ for all } k = 1,2,\ldots,m$$

- We can write this as a Lagrange problem

$$\min_{\boldsymbol{w},b} \max_{\boldsymbol{\alpha}\geq \mathbf{0}} \mathcal{L}(\boldsymbol{w},b,\boldsymbol{\alpha})$$

where

$$\mathcal{L}(\boldsymbol{w},b,\boldsymbol{\alpha}) = \frac{\|\boldsymbol{w}\|^2}{2} - \sum_{k=1}^m \alpha_k \Big(y_k\big(\langle \boldsymbol{w},\boldsymbol{\phi}(\boldsymbol{x}_k)\rangle - b\big) - 1 \Big)$$

subject to $\alpha_k \geq 0$

Obtaining the Dual Form of the Problem

- Differentiating the Lagrangian with respect to $\boldsymbol{w}$

$$\mathcal{L}(\boldsymbol{w},b,\boldsymbol{\alpha}) = \frac{1}{2}\|\boldsymbol{w}\|^2 - \sum_{k=1}^m \alpha_k \Big(y_k\big(\langle \boldsymbol{w},\boldsymbol{\phi}(\boldsymbol{x}_k)\rangle - b\big) - 1 \Big)$$

- $\boldsymbol{\nabla}_{\boldsymbol{w}}\mathcal{L} = \boldsymbol{w} - \sum_{k=1}^m \alpha_k y_k \boldsymbol{\phi}(\boldsymbol{x}_k) = 0 \implies \boldsymbol{w}^* = \sum_{k=1}^m \alpha_k y_k \boldsymbol{\phi}(\boldsymbol{x}_k)$
- $\frac{\partial \mathcal{L}}{\partial b} = \sum_{k=1}^m \alpha_k y_k = 0$
- Substituting back into the Lagrangian

$$\max_{\boldsymbol{\alpha}\geq \mathbf{0}} \mathcal{L}(\boldsymbol{w}^*,b^*,\boldsymbol{\alpha}) = \max_{\boldsymbol{\alpha}\geq \mathbf{0}} \sum_{k=1}^m \alpha_k - \frac{1}{2}\sum_{k,l=1}^m \alpha_k\alpha_l y_k y_l \langle \boldsymbol{\phi}(\boldsymbol{x}_k),\boldsymbol{\phi}(\boldsymbol{x}_l)\rangle$$

The Dual Problem

- The dual problem is now to find α_k's that maximise

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{k=1}^m \alpha_k - \frac{1}{2}\sum_{k,l=1}^m \alpha_k\alpha_l y_k y_l \langle \boldsymbol{\phi}(\boldsymbol{x}_k),\boldsymbol{\phi}(\boldsymbol{x}_l)\rangle$$

- subject to constraints

$$\sum_{k=1}^m \alpha_k y_k = 0 \qquad \forall k = 1,2,\ldots,m \quad \alpha_k \geq 0$$

- The Hessian of $\mathcal{L}(\boldsymbol{\alpha})$ has elements $H_{kl} = -y_k y_l \langle \boldsymbol{\phi}(\boldsymbol{x}_k),\boldsymbol{\phi}(\boldsymbol{x}_l)\rangle$ so $\boldsymbol{v}^{\mathsf{T}}\mathbf{H}\boldsymbol{v} = -\|\sum_k v_k y_k \boldsymbol{\phi}(\boldsymbol{x}_k)\|^2 \leq 0$ (negative semi-definite, so $\mathcal{L}(\boldsymbol{\alpha})$ is concave: every local maximum is a global maximum)

Kernel Trick

- We will show in the next lecture that if $K(\boldsymbol{x},\boldsymbol{y})$ is a positive semi-definite function then it can always be written as

$$K(\boldsymbol{x},\boldsymbol{y}) = \langle \boldsymbol{\phi}(\boldsymbol{x}),\boldsymbol{\phi}(\boldsymbol{y})\rangle$$

- As $\langle \boldsymbol{\phi}(\boldsymbol{x}_k),\boldsymbol{\phi}(\boldsymbol{x}_l)\rangle$ appears in the dual problem we can express the dual problem as finding α_k's that maximise

$$\mathcal{L}(\boldsymbol{\alpha}) = \sum_{k=1}^m \alpha_k - \frac{1}{2}\sum_{k,l=1}^m \alpha_k\alpha_l y_k y_l K(\boldsymbol{x}_k,\boldsymbol{x}_l)$$

- We therefore never have to compute $\boldsymbol{\phi}(\boldsymbol{x})$

Classifying New Data

- Having trained the SVM we now have to use it
- Given a new input $\boldsymbol{x}$ we decide on the class

$$y = \text{sgn}(\langle \boldsymbol{w},\boldsymbol{\phi}(\boldsymbol{x})\rangle - b) \qquad \text{but} \qquad \boldsymbol{w} = \sum_{k=1}^m \alpha_k y_k \boldsymbol{\phi}(\boldsymbol{x}_k)$$

- In the dual representation this becomes

$$\text{sgn}\left(\sum_{k=1}^m \alpha_k y_k K(\boldsymbol{x}_k,\boldsymbol{x}) - b \right)$$

where we only need to sum over the non-zero α_k (i.e. the support vectors SVs)

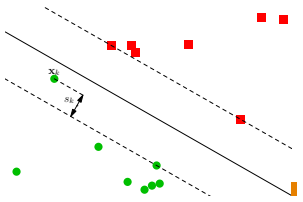
Sequential Minimal Optimisation

- One of the most efficient techniques for training SVMs is *Sequential Minimal Optimisation* or SMO
- This takes two Lagrange multipliers α_i and α_j and adjusts them to maximise the dual objective function
- This is very quick as it can be done in closed form
- Note that because $\sum_{k=1}^m y_k\alpha_k = 0$ we have to change at least two variables at the same time
- A heuristic is used to choose good pairs of α's to optimise
- Run until close to the optimum

Soft Margins

- We can relax the margin constraints by introducing *slack variables*, $s_k \geq 0$

$$y_k(\langle \boldsymbol{x}_k,\boldsymbol{w}\rangle - b) \geq 1 - s_k$$



- Minimise $\frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{k=1}^m s_k$
- Larger C punishes slack variables more

Dual Problem with Slack Variables

- The Lagrangian with slack variables is

$$\mathcal{L} = \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{k=1}^{m} s_k - \sum_{k=1}^{m}\alpha_k\Big(y_k\left(\langle \boldsymbol{w}, \boldsymbol{\phi}(\boldsymbol{x}_k)\rangle - b\right) - 1 + s_k\Big) - \sum_{k=1}^{m}\beta_k s_k$$

  where β_k are Lagrange multipliers that ensure $s_k \geq 0$ (note that $\beta_k \geq 0$—this is the KKT condition)

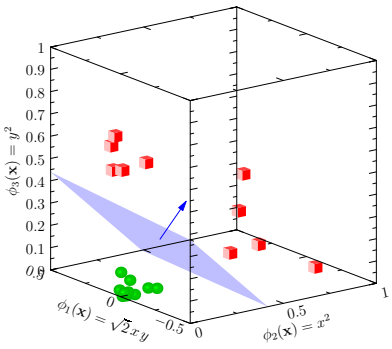
- Now minimising with respect to s_i

$$\frac{\partial \mathcal{L}}{\partial s_i} = C - \alpha_i - \beta_i = 0$$

- Or $\alpha_i = C - \beta_i$. Since $\beta_i \geq 0$ the constraint is $\alpha_i \leq C$ (recall also $\alpha_i \geq 0$)

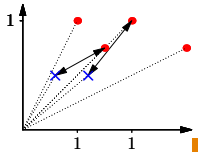
Outline

1. The Big Picture
2. Maximum Margins
3. Duality
4. **Practice**



Getting SVMs to Work Well

- SVMs rely on distances between data points
- These will change relative to each other if we rescale some features but not other—giving different maximum-margin hyper-planes



- If we don't know what features are important (most often the case), then it is worth scaling each feature (for example, so their range is between 0 and 1 or their variance is 1)

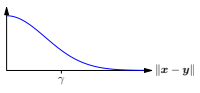
Optimising C

- Recall that we can introduce soft-margins using slack variables where we minimise $\frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{k=1}^{m} s_k$ subject to constraints
- In practice it can make a huge difference to the performance if we change C
- Optimal C values change by many orders of magnitude e.g. 2^{-5}–2^{15}
- Typically optimised by a grid search (start from 2^{-5} say and double until you reach 2^{15})
- Measure performance on a validation set

Choosing the Right Kernel Function

- There are kernels designed for particular data types (e.g. string kernels for text or biological sequences)
- For numerical data, people tend to look at using no kernel (linear SVM), a radial basis function (Gaussian) kernel or polynomial kernels
- Kernels often come with parameters, e.g. the popular radial basis function kernel

$$K(\boldsymbol{x}, \boldsymbol{y}) = \mathrm{e}^{-\gamma\|\boldsymbol{x}-\boldsymbol{y}\|^2}$$



- Optimal γ values range over 2^{-15}–2^{3}

Multi-Class Problems

- By construction SVMs separate only two classes
- If we have a multi-class problem we have to use multiple SVMs
- There are two major ways practitioners do this

  One-versus-all: for each class, train a separate classifier to determine that class versus all others; choose the class whose classifier is most certain

  All-pairs: train a classifier for every pair of classes; each votes, and the class with most votes wins

- Beware SVMs don't like imbalanced datasets

SVM Libraries

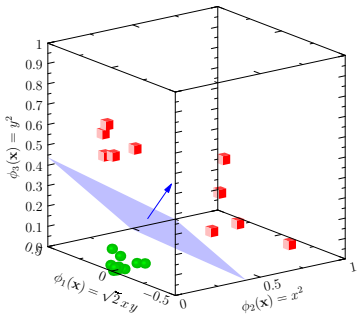
- Although SVMs have unique solutions, they require very well written optimisers
- If you have a large data set they can be very slow
- There are good libraries out there: LIBSVM, SVMlight (now old), scikit-learn (which uses LIBSVM), etc.
- These provide tools for normalising data and grid-searching parameters

Conclusions

- We've seen how SVMs work
- We've learnt how to use them
- We've seen that we can find the maximum margin hyper-plane by solving a quadratic programming problem (with a unique solution)
- This is a convex optimisation problem with a unique optimum
- The **dual problem** of an SVM is particularly simple, especially if we use a positive semi-definite kernel (we explore these in the next lecture)

Mathematics of Machine Learning

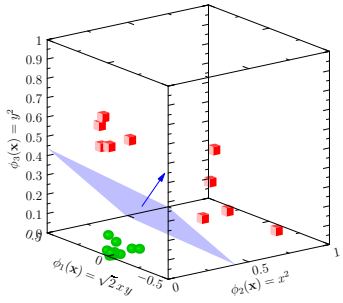
Kernel Trick



The Kernel Trick, SVMs, Regression

Outline

1. **Recap: the Kernel Trick**
2. Mercer's Theorem
3. Constructing Kernels
4. Kernels for Structured Data
5. Beyond Classification



Recap: the Kernel Trick

- SVM kernels are functions of two variables that factorise

$$K(\boldsymbol{x},\boldsymbol{y}) = \langle \boldsymbol{\phi}(\boldsymbol{x}),\boldsymbol{\phi}(\boldsymbol{y})\rangle = \sum_i \phi_i(\boldsymbol{x})\phi_i(\boldsymbol{y})$$

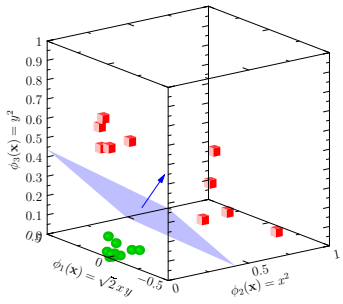
  with real-valued $\phi_i(\boldsymbol{x})$
- Such a K is positive semi-definite and, conversely, any positive semi-definite K factorises (this lecture)
- The SVM dual problem only needs the inner products

$$\max_{\boldsymbol{\alpha}} \sum_{k=1}^m \alpha_k - \frac{1}{2}\sum_{k,l=1}^m \alpha_k\alpha_l y_k y_l K(\boldsymbol{x}_k,\boldsymbol{x}_l)$$

  subject to $\sum_{k=1}^m y_k\alpha_k = 0$ and $0 \le \alpha_k \le C$
- This is the **kernel trick**: we never compute $\boldsymbol{\phi}(\boldsymbol{x})$

Outline

1. Recap: the Kernel Trick
2. **Mercer's Theorem**
3. Constructing Kernels
4. Kernels for Structured Data
5. Beyond Classification



Recap on Eigen Systems

- Recall for a symmetric $(n \times n)$ matrix $\mathbf{M}$ an eigenvector, $\boldsymbol{v}$

$$\mathbf{M}\boldsymbol{v} = \lambda\boldsymbol{v}$$

- There are n independent eigenvectors $\boldsymbol{v}^{(i)}$ with real eigenvalues $\lambda^{(i)}$
- The eigenvectors can be chosen orthonormal: $\boldsymbol{v}^{(i)\mathsf{T}}\boldsymbol{v}^{(j)} = 0$ if $i \neq j$ $(= 1$ if $i = j)$
- Forming a matrix of eigenvectors $\mathbf{V} = (\boldsymbol{v}^{(1)},\boldsymbol{v}^{(2)},\ldots\boldsymbol{v}^{(n)})$ the matrix satisfies

$$\mathbf{V}^\mathsf{T}\mathbf{V} = \mathbf{V}\mathbf{V}^\mathsf{T} = \mathbf{I}$$

- Such matrices are said to be orthogonal

Eigen Decomposition

- From the eigenvalue equation $\mathbf{M}\boldsymbol{v}^{(k)} = \lambda^{(k)}\boldsymbol{v}^{(k)}$

$$\mathbf{M}\mathbf{V} = \mathbf{V}\boldsymbol{\Lambda} \qquad \text{where} \quad \boldsymbol{\Lambda} = \begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{pmatrix}$$

- Multiplying on the right by $\mathbf{V}^\mathsf{T}$ we get

$$\mathbf{M} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^\mathsf{T} = \sum_{k=1}^n \lambda^{(k)}\boldsymbol{v}^{(k)}\boldsymbol{v}^{(k)\mathsf{T}}$$

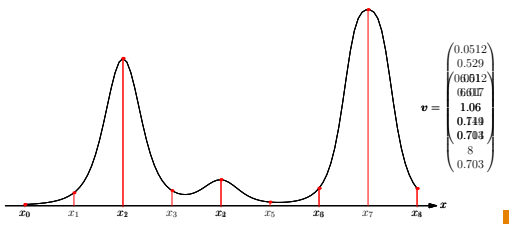
  Or

$$M_{ij} = \sum_{k=1}^n \lambda^{(k)} v_i^{(k)} v_j^{(k)} = \sum_{k=1}^n u_i^{(k)} u_j^{(k)} = \langle \boldsymbol{u}_i,\boldsymbol{u}_j\rangle$$

  $u_i^{(k)} = \sqrt{\lambda^{(k)}}\,v_i^{(k)}$ (real only if $\lambda^{(k)} \ge 0$)

Limit Process

- Consider sampling a function at a set of points

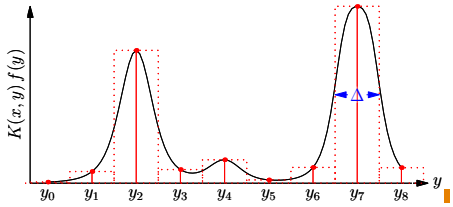


- In the limit where the number of sample points goes to infinity the vector more closely approximates a function
- The k^{th} component of the vector corresponds to $f(x_k)$

Linear Operators

- Recall a linear operator on functions, $\mathcal{T}[f(x)]$, can often be represented by a kernel

$$\mathcal{T}[f(x)] = \int_{y\in\mathcal{I}} K(x,y)f(y)\,\mathrm{d}y \approx \Delta\sum_{j=1}^n K(x,y_j)f(y_j)$$



  This is just a matrix equation with $M_{ij} = \Delta K(x_i,y_j)$

Eigenfunctions

- By analogy for a symmetric function of two variables we can define an *eigenfunction*

$$\int K(\boldsymbol{x},\boldsymbol{y})\psi(\boldsymbol{y})\mathrm{d}\boldsymbol{y} = \lambda\psi(\boldsymbol{x})$$

- In general there will be a denumerable set of eigenfunctions $\psi^{(k)}(\boldsymbol{x})$ where

$$K(\boldsymbol{x},\boldsymbol{y}) = \sum_k \lambda^{(k)}\psi^{(k)}(\boldsymbol{x})\psi^{(k)}(\boldsymbol{y})$$

- For a positive semi-definite kernel (on a bounded domain) this is **Mercer's theorem**

From Eigenfunctions to Features

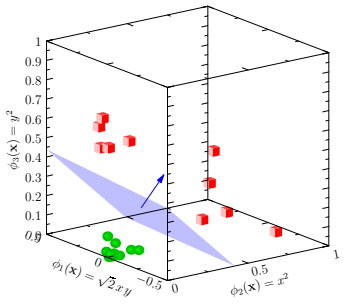
- If we define $\phi^{(k)}(\boldsymbol{x}) = \sqrt{\lambda^{(k)}}\psi^{(k)}(\boldsymbol{x})$ then

$$K(\boldsymbol{x},\boldsymbol{y}) = \sum_k \lambda^{(k)}\psi^{(k)}(\boldsymbol{x})\psi^{(k)}(\boldsymbol{y}) = \sum_k \phi^{(k)}(\boldsymbol{x})\phi^{(k)}(\boldsymbol{y})$$

- This is the definition of an SVM kernel we started with
- Note that for $\phi^{(k)}(\boldsymbol{x})$ to be real $\lambda^{(k)} \geq 0$ for all k
- If $\lambda^{(k)} < 0$ then $\langle\boldsymbol{\phi}(\boldsymbol{x}),\boldsymbol{\phi}(\boldsymbol{x})\rangle = \|\phi(\boldsymbol{x})\|^2$ might be negative and "distance" between points in the extended feature space can be negative!
- If we use a kernel that isn't positive semi-definite then the Hessian of the dual objective need not be negative semi-definite: the objective can be unbounded, with $\boldsymbol{\alpha}$ diverging

Outline

1. Recap: the Kernel Trick
2. Mercer's Theorem
3. **Constructing Kernels**
4. Kernels for Structured Data
5. Beyond Classification



Positive Semi-Definite Kernels

- Kernels (or matrices) that have eigenvalues $\lambda^{(k)} \geq 0$ are called positive semi-definite
- (If the eigenvalues are strictly positive $\lambda^{(k)} > 0$ the kernels or matrices are called positive definite)
- Positive semi-definite kernels can always be decomposed into a sum of real functions

$$K(\boldsymbol{x},\boldsymbol{y}) = \langle\boldsymbol{\phi}(\boldsymbol{x}),\boldsymbol{\phi}(\boldsymbol{y})\rangle$$

Properties of Positive Semi-Definiteness

- Since

$$K(\boldsymbol{x},\boldsymbol{y}) = \big\langle\boldsymbol{\phi}(\boldsymbol{x}),\boldsymbol{\phi}(\boldsymbol{y})\big\rangle$$

- An immediate consequence is that for any function $f(\boldsymbol{x})$

$$\begin{aligned}\int f(\boldsymbol{x})K(\boldsymbol{x},\boldsymbol{y})f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y} &= \int f(\boldsymbol{x})\big\langle\boldsymbol{\phi}(\boldsymbol{x}),\boldsymbol{\phi}(\boldsymbol{y})\big\rangle f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y}\\ &= \left\langle\int f(\boldsymbol{x})\boldsymbol{\phi}(\boldsymbol{x})\mathrm{d}\boldsymbol{x}, \int f(\boldsymbol{y})\boldsymbol{\phi}(\boldsymbol{y})\mathrm{d}\boldsymbol{y}\right\rangle\\ &= \left\|\int f(\boldsymbol{x})\boldsymbol{\phi}(\boldsymbol{x})\mathrm{d}\boldsymbol{x}\right\|^2 \geq 0\end{aligned}$$

Positive Semi-Definiteness

- The following statements are equivalent
  - $K(\boldsymbol{x},\boldsymbol{y})$ is positive semi-definite (written $K(\boldsymbol{x},\boldsymbol{y}) \succeq 0$)
  - The eigenvalues of $K(\boldsymbol{x},\boldsymbol{y})$ are non-negative
  - The kernel can be written

$$K(\boldsymbol{x},\boldsymbol{y}) = \big\langle\boldsymbol{\phi}(\boldsymbol{x}),\boldsymbol{\phi}(\boldsymbol{y})\big\rangle$$

    where the $\phi^{(k)}(\boldsymbol{x})$'s are real functions
  - For any real function $f(\boldsymbol{x})$

$$\int f(\boldsymbol{x})K(\boldsymbol{x},\boldsymbol{y})f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y} \geq 0$$

  - For any points $\boldsymbol{x}_1,\ldots,\boldsymbol{x}_n$ the **Gram matrix** with elements $K_{ij} = K(\boldsymbol{x}_i,\boldsymbol{x}_j)$ is positive semi-definite

Adding Kernels

- We can construct SVM kernels from other kernels
- If $K_1(\boldsymbol{x},\boldsymbol{y})$ and $K_2(\boldsymbol{x},\boldsymbol{y})$ are valid kernels then so is $K_3(\boldsymbol{x},\boldsymbol{y}) = K_1(\boldsymbol{x},\boldsymbol{y}) + K_2(\boldsymbol{x},\boldsymbol{y})$

$$\begin{aligned}Q &= \int f(\boldsymbol{x})K_3(\boldsymbol{x},\boldsymbol{y})f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y}\\ &= \int f(\boldsymbol{x})\big(K_1(\boldsymbol{x},\boldsymbol{y}) + K_2(\boldsymbol{x},\boldsymbol{y})\big)f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y}\\ &= \int f(\boldsymbol{x})K_1(\boldsymbol{x},\boldsymbol{y})f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y} + \int f(\boldsymbol{x})K_2(\boldsymbol{x},\boldsymbol{y})f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y} \geq 0\end{aligned}$$

- If $K(\boldsymbol{x},\boldsymbol{y})$ is a valid kernel so is $cK(\boldsymbol{x},\boldsymbol{y})$ for $c > 0$

Product of Kernels

- If $K_1(\boldsymbol{x},\boldsymbol{y})$ and $K_2(\boldsymbol{x},\boldsymbol{y})$ are valid kernels then so is $K_3(\boldsymbol{x},\boldsymbol{y}) = K_1(\boldsymbol{x},\boldsymbol{y})K_2(\boldsymbol{x},\boldsymbol{y})$
- Writing

$$K_1(\boldsymbol{x},\boldsymbol{y}) = \sum_i \phi_i^{(1)}(\boldsymbol{x})\phi_i^{(1)}(\boldsymbol{y}), \qquad K_2(\boldsymbol{x},\boldsymbol{y}) = \sum_j \phi_j^{(2)}(\boldsymbol{x})\phi_j^{(2)}(\boldsymbol{y})$$

then

$$\begin{aligned}K_3(\boldsymbol{x},\boldsymbol{y}) &= \sum_{i,j} \phi_i^{(1)}(\boldsymbol{x})\phi_i^{(1)}(\boldsymbol{y})\phi_j^{(2)}(\boldsymbol{x})\phi_j^{(2)}(\boldsymbol{y})\\ &= \sum_{i,j} \phi_{ij}^{(3)}(\boldsymbol{x})\phi_{ij}^{(3)}(\boldsymbol{y}) = \Big\langle\boldsymbol{\phi}^{(3)}(\boldsymbol{x}),\boldsymbol{\phi}^{(3)}(\boldsymbol{y})\Big\rangle\end{aligned}$$

where $\phi_{ij}^{(3)}(\boldsymbol{x}) = \phi_i^{(1)}(\boldsymbol{x})\phi_j^{(2)}(\boldsymbol{x})$

Exponentiating Kernels

- If $K(\boldsymbol{x},\boldsymbol{y})$ is a valid kernel so is $K^n(\boldsymbol{x},\boldsymbol{y})$ (by induction)
  - ⋆ Assume $K(\boldsymbol{x},\boldsymbol{y}) \succeq 0$ this satisfies base case
  - ⋆ If $K^{n-1}(\boldsymbol{x},\boldsymbol{y}) \succeq 0$ then

$$K^n(\boldsymbol{x},\boldsymbol{y}) = K^{n-1}(\boldsymbol{x},\boldsymbol{y})K(\boldsymbol{x},\boldsymbol{y}) \succeq 0$$

- and $\exp(K(\boldsymbol{x},\boldsymbol{y}))$ is also a valid kernel since

$$\mathrm{e}^{K(\boldsymbol{x},\boldsymbol{y})} = \sum_{i=0}^{\infty} \frac{1}{i!} K^i(\boldsymbol{x},\boldsymbol{y}) = 1 + K(\boldsymbol{x},\boldsymbol{y}) + \frac{1}{2}K^2(\boldsymbol{x},\boldsymbol{y}) + \cdots$$

  but each term in the sum is a kernel (the constant 1 has $\phi(\boldsymbol{x}) = 1$)

Polynomial Kernels

- A constant $c \geq 0$ is a kernel: $c = \sqrt{c}\sqrt{c}$, so $\phi(\boldsymbol{x}) = \sqrt{c}$
- $\boldsymbol{x}^\mathsf{T}\boldsymbol{y}$ is a kernel (with $\boldsymbol{\phi}(\boldsymbol{x}) = \boldsymbol{x}$), so the sum $c + \boldsymbol{x}^\mathsf{T}\boldsymbol{y}$ is too
- Hence the **polynomial kernel**

$$K(\boldsymbol{x},\boldsymbol{y}) = \left(c + \boldsymbol{x}^\mathsf{T}\boldsymbol{y}\right)^d$$

  is a kernel for $d = 1, 2, \ldots$
- Its features $\boldsymbol{\phi}(\boldsymbol{x})$ are all the monomials of the x_i of degree up to d (suitably weighted)
- $c = 0$, $d = 2$ is the quadratic kernel from the last lecture

RBF Kernel

- Now $\boldsymbol{x}^\mathsf{T}\boldsymbol{y} = \langle \boldsymbol{x},\boldsymbol{y} \rangle$ is a valid kernel because it is an inner product of functions $\boldsymbol{\phi}(\boldsymbol{x}) = \boldsymbol{x}$
- For $\gamma > 0$ we have $2\gamma\boldsymbol{x}^\mathsf{T}\boldsymbol{y} \succeq 0$
- Thus $\exp(2\gamma\boldsymbol{x}^\mathsf{T}\boldsymbol{y}) \succeq 0$
- If $K(\boldsymbol{x},\boldsymbol{y}) \succeq 0$ then $g(\boldsymbol{x})K(\boldsymbol{x},\boldsymbol{y})g(\boldsymbol{y}) \succeq 0$

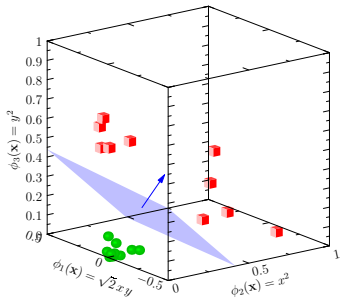
$$\int f(\boldsymbol{x})g(\boldsymbol{x})K(\boldsymbol{x},\boldsymbol{y})g(\boldsymbol{y})f(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y} = \int h(\boldsymbol{x})K(\boldsymbol{x},\boldsymbol{y})h(\boldsymbol{y})\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y} \geq 0$$

  where $f(\boldsymbol{x})g(\boldsymbol{x}) = h(\boldsymbol{x})$

$$\mathrm{e}^{-\gamma\boldsymbol{x}^\mathsf{T}\boldsymbol{x}}\mathrm{e}^{2\gamma\boldsymbol{x}^\mathsf{T}\boldsymbol{y}}\mathrm{e}^{-\gamma\boldsymbol{y}^\mathsf{T}\boldsymbol{y}} = \mathrm{e}^{-\gamma\|\boldsymbol{x}-\boldsymbol{y}\|^2} \succeq 0$$

Outline

1. Recap: the Kernel Trick
2. Mercer's Theorem
3. Constructing Kernels
4. **Kernels for Structured Data**
5. Beyond Classification



Other Kernels

- The success of SVMs has meant that researchers try to increase the area of application
- The condition that an SVM kernel must be positive semi-definite is quite restrictive
- The key to finding new kernels is to use the properties of kernels to build more complicated kernels from simpler ones
- There has been a cottage industry of researchers finding smart kernels for complicated data
- One area where SVMs were very important is document classification, which requires comparing strings

Spectrum Kernel

- A simple way to compare documents is to collect a histogram of all occurrences of substrings of length ℓ
- This is known as an ℓ-spectrum
- The ℓ-spectrum kernel is the inner product of the two histograms

$$s = \texttt{statistics} \quad \mathcal{S}_3(s) = \{\texttt{sta},\texttt{tat},\texttt{ati},\texttt{tis},\texttt{ist},\texttt{sti},\texttt{tic},\texttt{ics}\}$$
$$t = \texttt{computation} \quad \mathcal{S}_3(t) = \{\texttt{com},\texttt{omp},\texttt{mpu},\texttt{put},\texttt{uta},\texttt{tat},\texttt{ati},\texttt{tio},\texttt{ion}\}$$

- $K(s,t) = 1 \times 1 + 1 \times 1 = 2$ ("tat" and "ati" each occur once in both)

All Subsequences Kernel

- A more sophisticated kernel counts all of the common subsequences (not necessarily contiguous) of two documents
- Naively this would take an exponential amount of time to compute
- Using clever dynamic-programming techniques this can be done relatively efficiently
- Gap-weighted versions down-weight subsequences that are spread out over long gaps

Other Kernel Applications

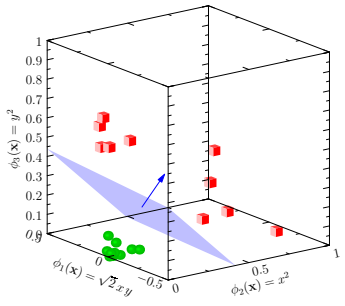
- String kernels for comparing subsequences are used in bioinformatics
- Kernels have been developed for comparing trees (e.g. for computer program evaluation, XML, etc.)
- Kernels have also been developed for comparing graphs (e.g. for comparing chemicals based on their molecular graph)

Fisher Kernels

- In an attempt to build kernels that capture more domain knowledge, kernels are constructed from other learning machines
- An example is the **Fisher kernel**, whose features are the gradients of the log-likelihood of a generative model (e.g. a Hidden Markov Model, HMM) trained on the data
- These tend to have better discriminative power than the underlying model (HMM), and a better feature set than an SVM using a generic kernel

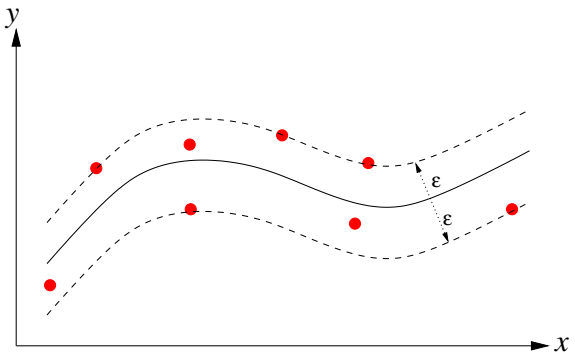
Outline

1. Recap: the Kernel Trick
2. Mercer's Theorem
3. Constructing Kernels
4. Kernels for Structured Data
5. **Beyond Classification**



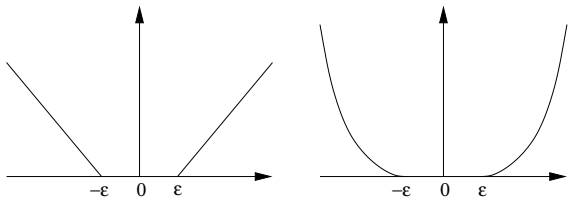
Regression with Margins

- SVMs can be modified to perform regression



Error Functions

- Can introduce slack variables with different errors



- This can be transformed to a quadratic programming problem

Ridge Regression Using Kernels

- We can also solve regression problems without using margins
- Kernel ridge regression minimises

$$\min_{\boldsymbol{w}} \eta\|\boldsymbol{w}\|^2 + \sum_{i=1}^{m}\big(y_i - \boldsymbol{w}^\top\boldsymbol{\phi}(\boldsymbol{x}_i)\big)^2$$

  where $\eta\|\boldsymbol{w}\|^2$ is a regularisation term
- The optimal $\boldsymbol{w}$ lies in the span of the $\boldsymbol{\phi}(\boldsymbol{x}_i)$ (any other component only increases $\|\boldsymbol{w}\|^2$), so $\boldsymbol{w} = \sum_i \alpha_i \boldsymbol{\phi}(\boldsymbol{x}_i)$
- This gives $\boldsymbol{\alpha} = (\mathbf{K} + \eta\mathbf{I})^{-1}\boldsymbol{y}$, with $\mathbf{K}$ the Gram matrix, and predictions $f(\boldsymbol{x}) = \sum_i \alpha_i K(\boldsymbol{x}_i, \boldsymbol{x})$

Kernel Methods

- Kernel methods where we project into an extended feature space are used with other linear algorithms
  - ⋆ Kernel Fisher discriminant analysis (KFDA)
  - ⋆ Kernel principal component analysis (KPCA)
  - ⋆ Kernel canonical correlation analysis (KCCA)
  - ⋆ Gaussian Processes
- These are also extremely powerful machine learning algorithms

Summary

- SVMs require a positive semi-definite kernel function
- These can be built from simpler functions
- There was a cottage industry of people creating new kernels for different applications
- SVMs are just one example of a host of machines that
  - ⋆ use the kernel trick
  - ⋆ often use linear constraints
  - ⋆ tend to be convex optimisation problems

Mathematics of Machine Learning

Probability

$Y = g(X)$ Ω

$y_{13} = g(x_{13})$	$y_{14} = g(x_{14})$	$y_{15} = g(x_{15})$	$y_{16} = g(x_{16})$
$y_9 = g(x_9)$	$y_{10} = g(x_{10})$	$y_{11} = g(x_{11})$	$y_{12} = g(x_{12})$
$y_5 = g(x_5)$	$y_6 = g(x_6)$	$y_7 = g(x_7)$	$y_8 = g(x_8)$
$y_1 = g(x_1)$	$y_2 = g(x_2)$	$y_3 = g(x_3)$	$y_4 = g(x_4)$

Probability, Random Variables, Expectations

Outline

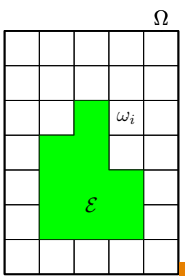
1. **Random Variables**
2. Expectations
3. Calculus of Probabilities

Ω

x_{31}	x_{32}	x_{33}	x_{34}	x_{35}
x_{26}	x_{27}	x_{28}	x_{29}	x_{30}
x_{21}	x_{22}	x_{23}	x_{24}	x_{25}
x_{16}	x_{17}	x_{18}	x_{19}	x_{20}
x_{11}	x_{12}	x_{13}	x_{14}	x_{15}
x_6	x_7	x_8	x_9	x_{10}
x_1	x_2	x_3	x_4	x_5

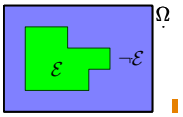
Modelling Uncertainty

- To model a world with uncertainty we consider some set of **elementary events** or **outcomes** Ω
- For the outcome of rolling a dice $\Omega = \{1,2,3,4,5,6\}$
- Outcomes are **mutually exclusive** (exactly one happens) and **exhaustive** (together they make up Ω)
- An **event** is a set of outcomes, $\mathcal{E} \subseteq \Omega$
- E.g. For a dice throw $\mathcal{E} = \{2,4,6\}$



Probabilities

- We attribute a **probability**, $\mathbb{P}(\mathcal{E})$, to an event, $\mathcal{E}$, with the requirements
  - $0 \leq \mathbb{P}(\mathcal{E}) \leq 1$, $\mathbb{P}(\Omega) = 1$
  - $\mathbb{P}(\mathcal{A} \cup \mathcal{B}) = \mathbb{P}(\mathcal{A}) + \mathbb{P}(\mathcal{B})$ if $\mathcal{A} \cap \mathcal{B} = \emptyset$
  - Hence $\mathbb{P}(\mathcal{E}) + \mathbb{P}(\neg\mathcal{E}) = 1$ where $\neg\mathcal{E} = \Omega \setminus \mathcal{E}$



- In some cases we can interpret $\mathbb{P}(\mathcal{E})$ as the expected frequency of occurrence of a repetitive trial
- But $\mathbb{P}$(Pass AICE3001 exam) is something you do once
- Think of probability as an informed belief, which changes when our knowledge changes

Probability Is Not Counting

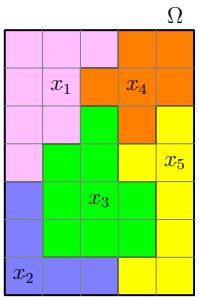
- At school probability is often taught as counting: if all outcomes are equally likely then $\mathbb{P}(\mathcal{E}) = |\mathcal{E}|/|\Omega|$
- But "equally likely" is an assumption about the world (a fair dice, a well-shuffled pack), not part of probability
- Most problems have no equally likely outcomes: tomorrow's weather, whether a network misclassifies an image
- Probability is about the **rules** for combining and updating beliefs: adding probabilities of exclusive events, conditioning, marginalising
- Combinatorics is just a tool for counting outcomes when symmetry tells us they are equally likely

Random Variables

- A **random variable**, X, assigns a number to each outcome in Ω: it partitions Ω and gives each part a number
- E.g. for a dice

$$X = \begin{cases} 0 & \text{if } \omega \in \{1,3,5\} \\ 1 & \text{if } \omega \in \{2,4,6\} \end{cases}$$

- $\mathbb{P}(X = x_i) = \mathbb{P}(\mathcal{E}_i)$ where $\mathcal{E}_i$ is the event corresponding to the part with value x_i



What's In A Name

- We denote random variables with capital letters, X, Y, Z, etc.
- The symbol denotes an object that can take one of a number of different values, but which one is still to be decided by chance
- When we write $\mathbb{P}(X)$ we can view this as short-hand for

$$(\mathbb{P}(X = x) \mid x \in \mathcal{X}) = (\mathbb{P}(X = x_1), \mathbb{P}(X = x_2), \ldots \mathbb{P}(X = x_n))$$

  where $\mathcal{X}$ is the set of possible values that X can take
- We treat random variables very differently to normal numbers (scalars) when we consider taking expectations

Function of Random Variables

- Any function, $Y = g(X)$, of a random variable, X, is a random variable

$Y = g(X)$ Ω

$y_{13} = g(x_{13})$	$y_{14} = g(x_{14})$	$y_{15} = g(x_{15})$	$y_{16} = g(x_{16})$
$y_9 = g(x_9)$	$y_{10} = g(x_{10})$	$y_{11} = g(x_{11})$	$y_{12} = g(x_{12})$
$y_5 = g(x_5)$	$y_6 = g(x_6)$	$y_7 = g(x_7)$	$y_8 = g(x_8)$
$y_1 = g(x_1)$	$y_2 = g(x_2)$	$y_3 = g(x_3)$	$y_4 = g(x_4)$

Continuous Spaces

- If the space of elementary events is continuous (e.g. for darts $\boldsymbol{x} = (x,y)$) then $\mathbb{P}(\boldsymbol{X} = \boldsymbol{x}) = 0$
- But if we consider a region, $\mathcal{R}$, then we can assign a probability to landing in the region $\mathbb{P}(\boldsymbol{X} \in \mathcal{R})$
- It is useful to work with the **probability density function** (PDF)

$$f_{\boldsymbol{X}}(\boldsymbol{x}) = \lim_{\epsilon \to 0} \frac{\mathbb{P}(\boldsymbol{X} \in \mathcal{B}(\boldsymbol{x}, \epsilon))}{\mathrm{Vol}(\mathcal{B}(\boldsymbol{x}, \epsilon))}$$

where $\mathcal{B}(\boldsymbol{x}, \epsilon)$ is a ball of radius ϵ around the point $\boldsymbol{x}$ and $\mathrm{Vol}(\mathcal{B}(\boldsymbol{x}, \epsilon))$ is the volume of the ball

- If we make a change of variable the volume $\mathrm{Vol}(\mathcal{B}(\boldsymbol{x}, \epsilon))$ might change so $f_{\boldsymbol{X}}(\boldsymbol{x})$ will change

Change of Variables

- Consider a region $\mathcal{R}$—we can describe this using different coordinate systems x or $y = g(x)$
- But

$$\mathbb{P}(X \in \mathcal{R}) = \int_{\mathcal{R}} f_X(x)\mathrm{d}x = \mathbb{P}(Y \in \mathcal{R}) = \int_{\mathcal{R}} f_Y(y)\mathrm{d}y$$

- As this is true for any region $\mathcal{R}$: $f_X(x)|\mathrm{d}x| = f_Y(y)|\mathrm{d}y|$
- Or

$$f_X(x) = f_Y(y)\left|\frac{\mathrm{d}y}{\mathrm{d}x}\right| = f_Y(g(x))|g'(x)|$$

- A probability density measured in units of probability per cm is different to that measured in units of probability per inch

Jacobian

- In high dimension if we make a change of variables $\boldsymbol{x} \to \boldsymbol{y}(\boldsymbol{x})$ (which can be seen as a change of random variables $\boldsymbol{X} \to \boldsymbol{Y}(\boldsymbol{X})$)
- Then

$$f_{\boldsymbol{X}}(\boldsymbol{x}) = f_{\boldsymbol{Y}}(\boldsymbol{y})|\det(\mathbf{J})|$$

where $\mathbf{J}$ is the Jacobian matrix

$$\mathbf{J} = \begin{pmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \cdots & \frac{\partial y_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial y_n}{\partial x_1} & \frac{\partial y_n}{\partial x_2} & \cdots & \frac{\partial y_n}{\partial x_n} \end{pmatrix}$$

- Ensures integrals over volumes are the same

Meaning of Probability Densities

- Probability densities are not probabilities
- They are non-negative, but can be greater than 1
- Note that

$$f_X(x) = \lim_{\delta x \to 0} \frac{\mathbb{P}(x \le X < x + \delta x)}{\delta x}$$

- We can think of $f_X(x)\delta x$ as $\mathbb{P}(x \le X < x + \delta x)$
- Note that $f_X(x)\delta x \le 1$

Cumulative Distribution Functions

- We can define the **cumulative distribution function** (CDF)

$$F_X(x) = \mathbb{P}(X \le x) = \begin{cases} \sum_{i:x_i \le x} \mathbb{P}(X = x_i) \\ \int_{-\infty}^{x} f_X(y)\mathrm{d}y \end{cases}$$

- This is a function that goes from 0 to 1 as x goes from $-\infty$ to ∞
- We note that for continuous random variables

$$f_X(x) = \frac{\mathrm{d}F_X(x)}{\mathrm{d}x}$$

The Language of Probability

- Mathematical texts use a standard vocabulary
  - ⋆ **sample space** Ω, **outcome** ω, **event** $\mathcal{E} \subseteq \Omega$
  - ⋆ **random variable** X: a function from outcomes to numbers
  - ⋆ its **distribution**: a **probability mass function** $\mathbb{P}(X = x)$ (discrete) or **density** $f_X(x)$ (continuous), or the CDF $F_X(x)$
  - ⋆ $X \sim \mathcal{N}(\mu, \sigma^2)$: "$X$ is distributed as" a normal
  - ⋆ **support**: the values with non-zero probability (or density)
  - ⋆ **i.i.d.**: independent and identically distributed (e.g. repeated, well conducted experiments)
  - ⋆ **almost surely**: with probability one

Outline

1. Random Variables
2. **Expectations**
3. Calculus of Probabilities

Ω

x_{31}	x_{32}	x_{33}	x_{34}	x_{35}
x_{26}	x_{27}	x_{28}	x_{29}	x_{30}
x_{21}	x_{22}	x_{23}	x_{24}	x_{25}
x_{16}	x_{17}	x_{18}	x_{19}	x_{20}
x_{11}	x_{12}	x_{13}	x_{14}	x_{15}
x_6	x_7	x_8	x_9	x_{10}
x_1	x_2	x_3	x_4	x_5

Expectation

- We can define the expectation of $\boldsymbol{Y} = g(\boldsymbol{X})$ as

$$\mathbb{E}_{\boldsymbol{X}}[g(\boldsymbol{X})] = \begin{cases} \sum_{\boldsymbol{x} \in \mathcal{X}} g(\boldsymbol{x})\mathbb{P}(\boldsymbol{X} = \boldsymbol{x}) \\ \int g(\boldsymbol{x}) f_{\boldsymbol{X}}(\boldsymbol{x})\mathrm{d}\boldsymbol{x} \end{cases}$$

- The expectation of a constant c is

$$\mathbb{E}_{\boldsymbol{X}}[c] = \begin{cases} \sum_{\boldsymbol{x} \in \mathcal{X}} c\mathbb{P}(\boldsymbol{X} = \boldsymbol{x}) = c\sum_{\boldsymbol{x} \in \mathcal{X}} \mathbb{P}(\boldsymbol{X} = \boldsymbol{x}) = c \\ \int c f_{\boldsymbol{X}}(\boldsymbol{x})\mathrm{d}\boldsymbol{x} = c\int f_{\boldsymbol{X}}(\boldsymbol{x})\mathrm{d}\boldsymbol{x} = c \end{cases}$$

- Note $\mathbb{E}_X[\mathbb{E}_X[g(X)]] = \mathbb{E}_X[g(X)]$

Linearity of Expectation

- Because sums and integrals are linear operators

$$\sum_i (ax_i + by_i) = a\left(\sum_i x_i\right) + b\left(\sum_i y_i\right)$$
$$\int (af(\boldsymbol{x}) + bg(\boldsymbol{x}))\mathrm{d}\boldsymbol{x} = a\left(\int f(\boldsymbol{x})\mathrm{d}\boldsymbol{x}\right) + b\left(\int g(\boldsymbol{x})\mathrm{d}\boldsymbol{x}\right)$$

  then expectations are linear

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y]$$

- Beware: in general $\mathbb{E}[XY] \neq \mathbb{E}[X]\mathbb{E}[Y]$ (equality holds if X and Y are independent, or more generally uncorrelated)

Indicator Functions

- An indicator function has the property

$$[\![predicate]\!] = \begin{cases} 1 & \text{if } predicate \text{ is True} \\ 0 & \text{if } predicate \text{ is False} \end{cases}$$

  (sometimes written $\boldsymbol{I}_A(x)$ where $A(x)$ is the predicate)

- We can obtain probabilities from expectations

$$\mathbb{P}(predicate) = \mathbb{E}[[\![predicate]\!]]$$

- E.g. The CDF is given by

$$F_X(x) = \mathbb{P}(X \leq x) = \mathbb{E}[[\![X \leq x]\!]]$$

Outline

1. Random Variables
2. Expectations
3. **Calculus of Probabilities**

Ω

x_{31}	x_{32}	x_{33}	x_{34}	x_{35}
x_{26}	x_{27}	x_{28}	x_{29}	x_{30}
x_{21}	x_{22}	x_{23}	x_{24}	x_{25}
x_{16}	x_{17}	x_{18}	x_{19}	x_{20}
x_{11}	x_{12}	x_{13}	x_{14}	x_{15}
x_6	x_7	x_8	x_9	x_{10}
x_1	x_2	x_3	x_4	x_5

Joint Probabilities

- Often we want to model complex processes where we have multiple random variables
- We can define the joint probability

$$p_{X,Y}(x,y) = \mathbb{P}(X = x, Y = y)$$

  i.e. the probability of the event where both $X = x$ and $Y = y$
- Clearly $\mathbb{P}(X,Y) = \mathbb{P}(Y,X)$

Marginalisation

- Probabilities are extremely easy to manipulate (although lots of people struggle)
- One of the most useful properties is known as **marginalisation**

$$\mathbb{P}(X) = \sum_{y \in \mathcal{Y}} \mathbb{P}(X, Y = y)$$

  where $\mathcal{Y}$ is the set of values that the random variable Y takes
- Note that when we write $\mathbb{P}(X)$ we are saying this is true for all values that X can take
- Although obvious and easy this is extremely useful

Conditional Probability

- We can also define the probability of an event X given that $Y = y$ has occurred

$$\mathbb{P}(X \mid Y = y) = \frac{\mathbb{P}(X, Y = y)}{\mathbb{P}(Y = y)}$$

  (for $\mathbb{P}(Y = y) > 0$)
- In constructing a model it is often much easier to specify conditional probabilities (because you know something) rather than joint probabilities
- When manipulating probabilities it is often easier to work with joint probabilities because we can simplify them by marginalising out random variables we are not interested in

Basic Calculus

- To obtain the joint probability we can use

$$\mathbb{P}(X,Y) = \mathbb{P}(X|Y)\mathbb{P}(Y) = \mathbb{P}(Y|X)\mathbb{P}(X)$$

- This generalises to more random variables

$$\mathbb{P}(X,Y,Z) = \mathbb{P}(X,Y|Z)\mathbb{P}(Z) = \mathbb{P}(X|Y,Z)\mathbb{P}(Y|Z)\mathbb{P}(Z)$$

- We can do this in a number of different ways

$$\mathbb{P}(X,Y,Z) = \mathbb{P}(Y,Z|X)\mathbb{P}(X) = \mathbb{P}(Z|Y,X)\mathbb{P}(Y|X)\mathbb{P}(X)$$

- Note that $\mathbb{P}(A,B \mid X,Y)$ means the probability of random variables A and B given that X and Y take particular values

Beware

- Conditional probabilities, $\mathbb{P}(X \mid Y)$ are probabilities for X, but not Y

$$\sum_{x \in \mathcal{X}} \mathbb{P}(X = x \mid Y) = 1$$
$$\sum_{y \in \mathcal{Y}} \mathbb{P}(X \mid Y = y) \neq 1$$

  (in general)
- Note that

$$\mathbb{E}_Y[\mathbb{P}(X \mid Y)] = \sum_{y \in \mathcal{Y}} \mathbb{P}(Y = y)\mathbb{P}(X|Y = y)$$
$$= \sum_{y \in \mathcal{Y}} \mathbb{P}(X, Y = y) = \mathbb{P}(X)$$

Causality

- Conditional probabilities do not imply causality
- We might have causal relationships

$$\mathbb{P}(\text{pass} \mid \text{study}) = 0.9 \qquad \mathbb{P}(\text{pass} \mid \neg\text{study}) = 0.2$$

- But if we know $\mathbb{P}(\text{study}) = 0.8$ then we can compute

$$\mathbb{P}(\text{pass},\text{study}) = \mathbb{P}(\text{pass} \mid \text{study})\mathbb{P}(\text{study}) = 0.9 \times 0.8 = 0.72$$
$$\mathbb{P}(\text{pass},\neg\text{study}) = \mathbb{P}(\text{pass} \mid \neg\text{study})\mathbb{P}(\neg\text{study}) = 0.2 \times 0.2 = 0.04$$

and

$$\mathbb{P}(\text{study} \mid \text{pass}) = \frac{\mathbb{P}(\text{pass},\text{study})}{\mathbb{P}(\text{pass})} = \frac{\mathbb{P}(\text{pass},\text{study})}{\mathbb{P}(\text{pass},\text{study}) + \mathbb{P}(\text{pass},\neg\text{study})} = \frac{0.72}{0.72 + 0.04} \approx 0.947$$

Independence

- Random variables X and Y are said to be **independent** if

$$\mathbb{P}(X,Y) = \mathbb{P}(X)\mathbb{P}(Y)$$

- Because $\mathbb{P}(X,Y) = \mathbb{P}(X|Y)\mathbb{P}(Y)$ and $\mathbb{P}(X,Y) = \mathbb{P}(Y|X)\mathbb{P}(X)$ independence implies

$$\mathbb{P}(X|Y) = \mathbb{P}(X) \qquad \mathbb{P}(Y|X) = \mathbb{P}(Y)$$

- Probabilistic independence is a mathematical property, not necessarily causal independence
- Variables with no causal link can be dependent through a shared cause (ice-cream sales and drownings)
- If $X \in \{0,1\}$ represents the outcome of tossing a coin and $Y \in \{1,2,3,4,5,6\}$ the outcome of rolling a dice then X and Y are independent

Well Conducted Experiments

- In well conducted experiments we expect the results we obtain are independent
- Let $\mathcal{D} = (X_1,X_2,\ldots,X_m)$ represents possible outcomes from a set of m well conducted experiments then

$$\mathbb{P}(\mathcal{D}) = \prod_{i=1}^{m} \mathbb{P}(X_i)$$

- Denoting a possible sentence I might say by $\mathcal{S} = (W_1,W_2,\ldots,W_m)$ then

$$\mathbb{P}(\mathcal{S}) \neq \prod_{i=1}^{m} \mathbb{P}(W_i)$$

otherwise it's time I retired

Conditional Independence

- Let $K(d)$ be a random variable measuring the amount you know about ML on day d of your revision
- From your revision schedule you can write down your belief

$$\mathbb{P}(K(d) \mid K(d-1),K(d-2),\ldots K(1))$$

- But a very reasonable model is

$$\mathbb{P}(K(d) \mid K(d-1),K(d-2),\ldots K(1)) = \mathbb{P}(K(d) \mid K(d-1))$$

what you are going to know today will just depend on what you knew yesterday

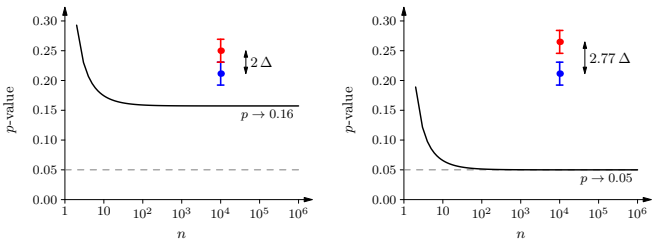
- We say that $K(d)$ is **conditionally independent** of $K(d-2)$, $K(d-3)$, etc. given $K(d-1)$

Conclusion

- To work with probabilities you need to know
  - How to go back and forward between joint probabilities and conditional probabilities
  - How to marginalise out variables
- You need to understand that for continuous outcomes, it makes sense to talk about the probability density
- You need to know that expectations are linear operators and the expectation of a constant is the constant
- You need to understand independence

Mathematics of Machine Learning

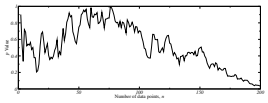
On Becoming a Scientist



interpreting data, standard errors, confidence tests

Outline

1. **Reporting Experiments**
2. Experimental Error
3. Confidence Estimates



Experiments and Reproducibility

- One of the fundamental requirements of science is that experiments can be reproduced
- This applies to machine learning and deep-learning as it does to particle physics, molecular biology, etc.
- We need to describe completely what procedures were carried out
- However, experiments have natural variability so what do we mean by repeatability?

Describing Research

- When describing research you need to find a story (narrative)—why is the work you are doing interesting
- You need to give a general motivation as well as a more detailed explanation of what you are specifically going to test
- You then need to give enough details that someone can take your report/paper and reproduce what you are doing
- These days it is also common practice to put the model you are using on-line—but you should still describe everything in your report

Describing Your Experiments

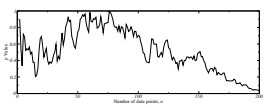
- You need to tell the reader
  - What your deep-learning architecture looks like
  - Your training set
  - How you partitioned your training set into train, validation and test sets
  - Any preprocessing you performed
  - The optimiser you used, mini-batch size, learning rate
  - The number of epochs you trained for
  - Any hyper-parameter tuning you carried out
  - How long it took in clock time on the platform you are using
- You can put some of this into an appendix if necessary

Results

- A paper builds a story (thesis) based on experimental evidences
- It is not enough to present results, it is also your job to interpret the results
- Your results provide evidence in support of your thesis
- But results all come with uncertainty. If you tested on a different test set, or used a different training set then the answer would be different
- **Results are meaningless unless the uncertainty is quantified!**

Outline

1. Reporting Experiments
2. **Experimental Error**
3. Confidence Estimates



Experimental Error

- Any empirical result is useless without some estimate of error
- If you tell me algorithm A has a performance of 95% and algorithm B has a performance of 97%, how do I know that this didn't happen by chance?
- Maybe algorithm A is actually better, but you got lucky (or unlucky)
- We need to get a handle on the estimated experimental error

Datasets

- We assume that our test set consists of typical examples that we might encounter and that they are independent (e.g. we are not seeing the same example over and over again)
- These are assumptions that are not always true
- We need to think about it and if they are not true you may need to collect new data or be very cautious of the claims that you are making
- However, if the person curating the training set did a good job then usually these are reasonable assumptions

Collecting Statistics

- Given some experimental measurements $\{X_i | i = 1,2,\ldots,n\}$ we can estimate the mean and variance

$$\hat{\mu}_n = \frac{1}{n}\sum_{i=1}^n X_i \qquad \hat{\sigma}_n^2 = \frac{1}{n-1}\sum_{i=1}^n (X_i - \hat{\mu}_n)^2$$

- These are not the true mean, $\mu = \mathbb{E}[X_i]$, and true variance $\sigma^2 = \mathbb{E}\big[(X_i - \mu)^2\big]$, but they are unbiased estimates in that

$$\mathbb{E}[\hat{\mu}_n] = \mu \qquad \mathbb{E}\big[\hat{\sigma}_n^2\big] = \sigma^2$$

Variance in Mean Estimator

- Consider the mean squared error (MSE) of the estimator $\hat{\mu}_n$ (independent X_i)

$$\begin{aligned}
\mathbb{E}\big[(\hat{\mu}_n - \mu)^2\big] &= \mathbb{E}\left[\left(\left(\frac{1}{n}\sum_{i=1}^n X_i\right) - \mu\right)^2\right] = \mathbb{E}\left[\left(\frac{1}{n}\sum_{i=1}^n (X_i - \mu)\right)^2\right] \\
&= \frac{1}{n^2}\mathbb{E}\left[\left(\sum_{i=1}^n (X_i - \mu)\right)\left(\sum_{j=1}^n (X_j - \mu)\right)\right] \\
&= \frac{1}{n^2}\mathbb{E}\left[\sum_{i=1}^n (X_i - \mu)^2 + \sum_{i=1}^n \sum_{\substack{j=1 \\ j\neq i}}^n (X_i - \mu)(X_j - \mu)\right] \\
&= \frac{1}{n^2}\sum_{i=1}^n \left(\mathbb{E}\big[(X_i - \mu)^2\big] + \sum_{\substack{j=1 \\ j\neq i}}^n \mathbb{E}[X_i - \mu]\mathbb{E}[X_j - \mu]\right) \\
&= \frac{1}{n^2}\sum_{i=1}^n \sigma^2 = \frac{1}{n}\sigma^2
\end{aligned}$$

Estimated Error in the Mean

- The root mean squared error (RMSE) is equal to

$$\sqrt{\mathbb{E}[(\hat{\mu}_n - \mu)^2]} = \frac{\sigma}{\sqrt{n}}$$

- The **estimated error in the mean** or **standard error** is defined as $\Delta = \frac{\hat{\sigma}_n}{\sqrt{n}}$ where $\hat{\sigma}_n$ is the empirical estimate of the standard deviation
- Traditionally we would write an experimental result as $\hat{\mu}_n \pm \Delta$
- Also we only quote $\hat{\mu}_n$ up to the number of significant figures consistent with Δ
- Not 1.6582356 ± 0.122543 but 1.7 ± 0.1

Repeated Bernoulli Trials

- Suppose we are measuring a binary outcome, success or failure
- Assuming the trials are independent (so called Bernoulli trials) then the only unknown is the probability, p, of a success
- If we make n trials then probability of k successes is given by the binomial distribution

$$\mathbb{P}(K = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

- The mean and variance of K are

$$\mathbb{E}[K] = np \qquad \mathbb{E}\big[(K - np)^2\big] = np(1-p)$$

Mean Squared Error

- For the case of n Bernoulli trials we can estimate the mean success rate

$$\hat{p} = \frac{K}{n}$$

- The mean squared error (MSE) in this estimate is

$$\begin{aligned}
\mathbb{E}\big[(\hat{p} - p)^2\big] &= \mathbb{E}\left[\left(\frac{K}{n} - p\right)^2\right] = \frac{1}{n^2}\mathbb{E}\big[(K - np)^2\big] \\
&= \frac{1}{n^2} np(1-p) = \frac{p(1-p)}{n}
\end{aligned}$$

Standard Error for Bernoulli Trials

- The estimated error in the mean is given by the root mean squared error (RMSE) $\sqrt{\mathbb{E}[(\hat{p} - p)^2]} = \sqrt{\frac{p(1-p)}{n}}$
- We don't know p, but we can use an estimate, $\hat{p} = \frac{K}{n}$

$$\Delta \approx \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

- We can also use a worst-case bound (when $p = \frac{1}{2}$)

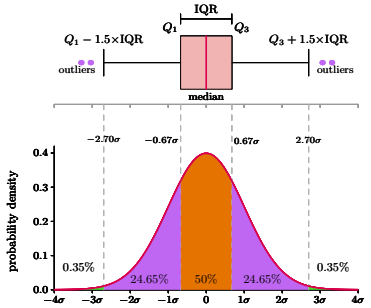
$$\Delta \approx \frac{1}{2\sqrt{n}}$$

This is pessimistic if p is small, although the previous estimate is optimistic if $\hat{p} = 0$

Errorbars

- There are different conventions on errorbars
- Traditionally errorbars show the estimated error in the mean (standard error, $\Delta = \hat{\sigma}_n/\sqrt{n}$)
- For roughly normal data the true mean is outside the errorbar about 32% of the time
- Some people use 95% confidence intervals as errorbars, saying 95% of the time the true mean lies within the errorbars—this corresponds to errorbars showing twice the standard error
- Some people show the standard deviation as errorbars, this tells you nothing about the confidence of the point you plot—in my opinion it is just wrong

Box Plots

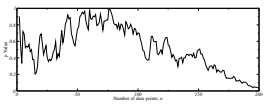


Box plots summarise the *spread* of the data.

They show no error bars on the median, so you cannot tell from them whether two results are statistically significantly different.

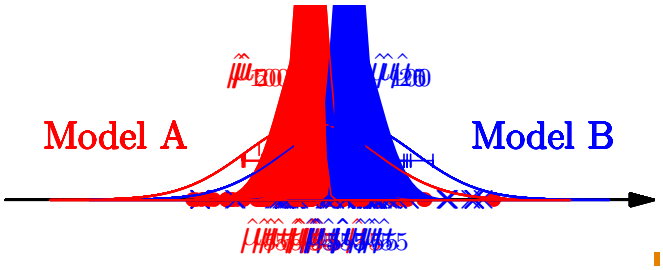
Outline

1. Reporting Experiments
2. Experimental Error
3. **Confidence Estimates**



Know What's Best

- How do we know if model B is better than model A?



The more data we have the less likely we will make a mistake

Quantifying Uncertainty

- We want a way of quantifying the chance that model B is better than model A
- To do this we can consider a **null hypothesis**

  We suppose that model A and B have the same performance
- We then ask, under the null hypothesis, what is the chance of obtaining a gap in performance at least as big as the one we observed
- This will depend on the number of observations and the empirical variance

T-tests

- The original t-test was invented by William Gosset who worked for the Guinness brewery in Dublin, but published under the pseudonym *Student*
- He assumed A and B are normally distributed with the same standard deviation; the null hypothesis is that their means are equal
- The more common test now is Welch's t-test, which drops the assumption that the standard deviations are equal
- The gap is equal to the t value

$$t = \frac{\hat{\mu}_A - \hat{\mu}_B}{\hat{\sigma}_{A,B}}$$

Welch's T-Test

- For Welch's t-test

$$t = \frac{\hat{\mu}_A - \hat{\mu}_B}{\hat{\sigma}_{A,B}} \qquad \hat{\sigma}_{A,B} = \sqrt{\frac{\hat{\sigma}_A^2}{n} + \frac{\hat{\sigma}_B^2}{m}}$$

- The p-value depends on t and the *number of degrees of freedom*

$$\nu = \frac{\hat{\sigma}_{A,B}^4}{(\hat{\sigma}_A^2/n)^2/(n-1) + (\hat{\sigma}_B^2/m)^2/(m-1)}$$

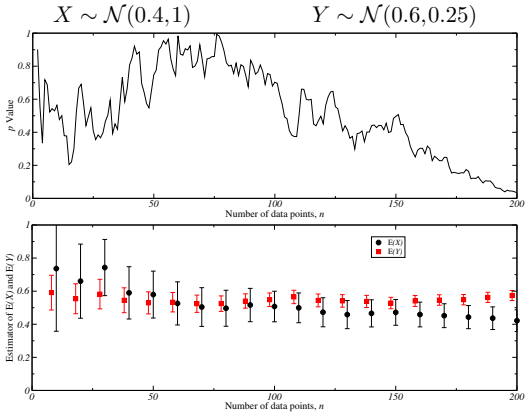
- The beauty of the t-test is that from t and ν we can compute p
- Small p is good: the gap would be surprising if the null hypothesis were true (p is not the probability that the null hypothesis is true)

Statistical Significance

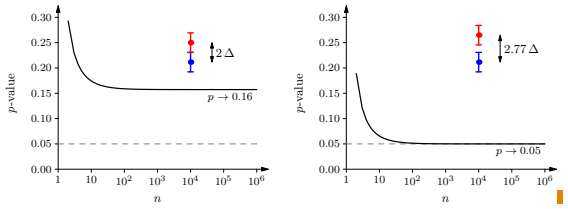
- It is common to say that A is *statistically significantly* better than B if, under the null hypothesis, the probability of a t at least as extreme as the one observed (the p-value) is less than 0.05
- Statistically significant difference in performance does not mean that the difference is anything to care about (that is a judgement call)
- We are assuming the data is normally distributed. We can check but no one does. Usually it's not a bad approximation if the data is not normal, but there are exceptions. . .

Example

$X \sim \mathcal{N}(0.4,1)$ $\quad$ $Y \sim \mathcal{N}(0.6,0.25)$



Error Bars and Significance



- Separate errorbars (1 standard error) are **not** enough: if they just touch, $p \approx 0.16$
- For $p < 0.05$ the gap must be about 2.8 standard errors

Paired T-Test

- Sometimes the fluctuations in performance will come both from the test example and from the difference in performance
- For example model A might always be better than model B, but the performance fluctuates depending on the example
- A normal t-test might not show any significant difference
- A paired t-test uses the per-example differences $X_i - Y_i$, whose standard error is

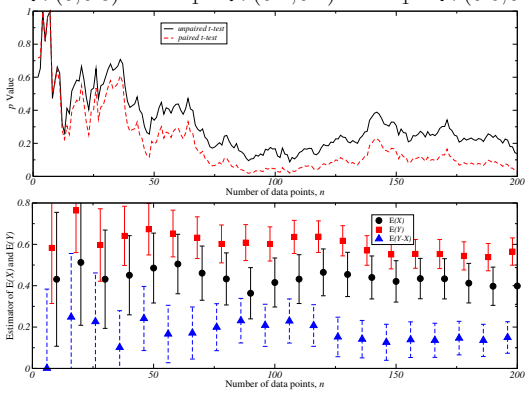
$$\hat{\sigma}^2_{X-Y} = \frac{\hat{\sigma}^2_X + \hat{\sigma}^2_Y - 2\hat{c}_{X,Y}}{n}$$

$$\hat{c}_{X,Y} = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \hat{\mu}_X)(Y_i - \hat{\mu}_Y)$$

Paired T-Test Example

$$X = Z + X_1 \qquad Y = Z + Y_1$$

$$Z \sim \mathcal{N}(0,0.8) \qquad X_1 \sim \mathcal{N}(0.4,0.2) \qquad Y_1 \sim \mathcal{N}(0.6,0.2)$$



Cheating Significance Tests

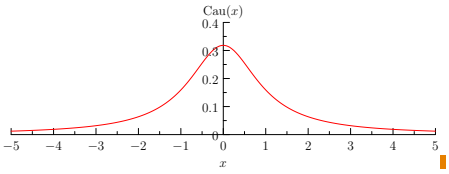
- You should only run your t-test once
- If you run multiple t-tests until you get one with $p < 0.05$ then you have cheated
- With a p value of 0.05 you expect to find the observed t value given the null hypothesis once every 20 trials—if you do the t-test twice you should expect this to happen every 10 trials (i.e. $p = 0.1$)
- If you run t-tests until you find a p value of 0.05 you are cheating
- If you want to compare many different models then use a method designed for it (e.g. ANOVA) or correct for the number of tests

Computing Significance Ab-Initio

- The t-test results follow from a beautiful, but difficult calculation
- However, you can simulate the null-hypothesis multiple times and measure empirically the number of times the t-value exceeds the observed t-values
- This isn't necessary under the assumptions that the data is normally distributed (done by Student and Welch)
- However, there are other times where you want to show that what you have found has not happened by chance. Often it is easy to simulate a null hypothesis

Good, Bad and the Ugly

- What I've told you is true of data that follows well behaved distributions
- Just occasionally you come across data that has long (thick) tails



- Here taking the average of many samples might not give you reliable estimates of the mean
- This isn't that common, but when nothing seems to make sense plot a histogram of the data and check for thick tails

Lessons

- You need to understand the errors in your data
- Computing the standard error is always necessary
- In some cases you can quantify your certainty that A is better than B using statistical confidence tests—t-tests (other tests are available)
- You need to apply common sense in interpreting data
- **The person you cheat when you don't do this is yourself**

Mathematics of Machine Learning

Bayesian Inference

Bayes, Conjugate Priors, Uninformative Priors

Outline

1. **Bayes' Rule**
2. Conjugate Priors
3. Uninformative Priors

Dealing with Uncertainty

- In machine learning we are attempting to make inference under uncertainty
- The natural language for discussing uncertainty is probability
- The natural framework for making inferences is Bayesian statistics
- However, this requires that we encode our prior knowledge of the problem and specify a likelihood
- In consequence, probabilistic methods tend to be bespoke, rather than general purpose black boxes

Revision on Bayes

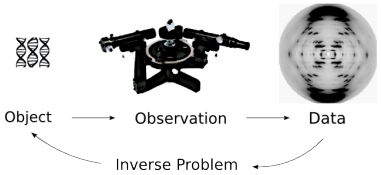
- Bayes' rule

$$\mathbb{P}(\mathcal{H}_i|\mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|\mathcal{H}_i)\mathbb{P}(\mathcal{H}_i)}{\mathbb{P}(\mathcal{D})}$$

  - $\mathbb{P}(\mathcal{H}_i|\mathcal{D})$ is the **posterior** probability of a hypothesis $\mathcal{H}_i$ (i.e. the probability of $\mathcal{H}_i$ **after** we see the data)
  - $\mathbb{P}(\mathcal{D}|\mathcal{H}_i)$ is the **likelihood** of the data given the hypothesis. Note, that we calculated this from the forward problem
  - $\mathbb{P}(\mathcal{H}_i)$ is the **prior** probability (i.e. the probability of $\mathcal{H}_i$ **before** we see the data)
  - $\mathbb{P}(\mathcal{D})$ is the **evidence** or **marginal likelihood**

$$\mathbb{P}(\mathcal{D}) = \sum_{i=1}^{n} \mathbb{P}(\mathcal{H}_i, \mathcal{D}) = \sum_{i=1}^{n} \mathbb{P}(\mathcal{D}|\mathcal{H}_i)\mathbb{P}(\mathcal{H}_i)$$

Solving Inverse Problems



- We want the posterior $\mathbb{P}(\mathcal{H}_i|\mathcal{D})$ (i.e. the probability of what happened given some evidence)
- The Bayesian formalism converts this into the forward problem

$$\mathbb{P}(\mathcal{H}_i|\mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|\mathcal{H}_i)\mathbb{P}(\mathcal{H}_i)}{\mathbb{P}(\mathcal{D})}$$

Bayesian Inference

- Bayes' rule says $\mathbb{P}(\mathcal{H}_i|\mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|\mathcal{H}_i)\mathbb{P}(\mathcal{H}_i)}{\mathbb{P}(\mathcal{D})}$
- We calculate the likelihood $\mathbb{P}(\mathcal{D}|\mathcal{H}_i)$ (i.e. assuming the hypothesis, what is the chance of obtaining the data?)
- We consider the process of how the data is generated
- This uses the data we have (doesn't care about missing data)
- But we also need to know the prior $\mathbb{P}(\mathcal{H}_i)$
- Also, this can get difficult when we have many hypotheses

Evidence

- The normalisation term

$$\mathbb{P}(\mathcal{D}) = \sum_{i=1}^{n} \mathbb{P}(\mathcal{H}_i, \mathcal{D}) = \sum_{i=1}^{n} \mathbb{P}(\mathcal{D}|\mathcal{H}_i)\mathbb{P}(\mathcal{H}_i)$$

  tells you how likely the data is (given the prior and likelihood function)
- It is called the **marginal likelihood** or **evidence**
- At one level this is just a normalisation condition, which is often a pain to compute
- But it also plays an important role in **model selection**

Model Selection

- Suppose we have two models of the likelihood or prior; M_1 and M_2. Then the posterior will depend on the model

$$\mathbb{P}(\mathcal{H} \mid \mathcal{D}, M_i) = \frac{\mathbb{P}(\mathcal{D} \mid \mathcal{H}, M_i)\mathbb{P}(\mathcal{H} \mid M_i)}{\mathbb{P}(\mathcal{D} \mid M_i)}$$

- We can use $\mathbb{P}(\mathcal{D} \mid M_1)$ or $\mathbb{P}(\mathcal{D} \mid M_2)$ to select which model is most likely!
- E.g. is my likelihood best captured by a normal or gamma distribution
- This also allows us to select hyperparameters for a model

Probability Density

- When we are working with continuous variables it is more natural to work with probability densities

$$f_X(x) = \lim_{\delta x \to 0} \frac{\mathbb{P}(x \leq X < x + \delta x)}{\delta x}$$

- Note that densities are non-negative, but can be greater than 1 (they are not probabilities)
- However

$$\mathbb{P}(a \leq X \leq b) = \int_a^b f_X(x) \mathrm{d}x$$

is a probability and is less than or equal to 1

Densities and Bayes

- Bayes' rule also applies to densities

$$\mathbb{P}(x \leq X < x + \delta x|Y) = \frac{\mathbb{P}(Y|x)\mathbb{P}(x \leq X < x + \delta x)}{\mathbb{P}(Y)}$$

- Dividing by δx and taking the limit $\delta x \to 0$

$$f_{X|Y}(x|Y) = \frac{\mathbb{P}(Y|x) f_X(x)}{\mathbb{P}(Y)}$$

- Similarly if X is discrete and Y continuous

$$\mathbb{P}(X|y) = \frac{f_{Y|X}(y|X)\mathbb{P}(X)}{f_Y(y)}$$

- If both X and Y are continuous

$$f_{X|Y}(x|y) = \frac{f_{Y|X}(y|x) f_X(x)}{f_Y(y)}$$

Practical Bayesian Inference

- Often consider learning parameters $\boldsymbol{\theta}$

$$p(\boldsymbol{\theta}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\theta})p(\boldsymbol{\theta})}{p(\mathcal{D})}$$

- This can be hard for large data sets as the posterior, $p(\boldsymbol{\theta}|\mathcal{D})$, is often a mess
- If we are lucky and have a simple likelihood then if we choose the right prior we end up with a posterior of the same form as the prior
- This occurs in some classic probabilistic inference problems, but as we will see soon it is also true for Gaussian Processes

Outline

1. Bayes' Rule
2. **Conjugate Priors**
3. Uninformative Priors

Learning a Probability

- Suppose we have a coin and we want to establish the probability of a head
- We want to learn this from a series of independent trials
- (Independent trials with two possible outcomes are known in probability theory as Bernoulli trials)
- Let X_i equal 1 if the i^{th} trial is a head and 0 otherwise
- If the probability of a head is p then the **likelihood** of a X_i is

$$\mathbb{P}(X_i|p) = p^{X_i}(1-p)^{1-X_i} = \begin{cases} p & \text{if } X_i = 1 \\ (1-p) & \text{if } X_i = 0 \end{cases}$$

Prior

- We may have a prior belief (e.g. we have made a few trials or we see the coin looks like a normal penny)
- We will suppose we can model our prior belief in terms of a **Beta distribution**

$$f(p) = \mathrm{Beta}(p|a,b) = \frac{p^{a-1}(1-p)^{b-1}}{B(a,b)}$$

- $B(a,b)$ is just a normalisation constant

$$B(a,b) = \int_0^1 p^{a-1}(1-p)^{b-1}\mathrm{d}p = \frac{\Gamma(a)\Gamma(b)}{\Gamma(a+b)}$$

- This is a useful function for modelling the distribution of a random variable in the range 0 to 1

Uninformative Prior

- Suppose we have no idea about p what should we do?
- Laplace (one of the first Bayesians) suggested giving equal weighting to all values of p
- This corresponds to a beta distribution with $a = b = 1$
- (Surprisingly other arguments suggest using $a = b = 0$ which provides a strong bias towards $p = 0$ and $p = 1$)
- Given enough data the prior is not so important and we will stick with Laplace for now

Independent Trials

- Using Bayes' rule

$$f(p|\mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|p) f(p)}{\mathbb{P}(\mathcal{D})}$$

- Assuming the trials are independent (a reasonably fair assumption for tossing coins) then the likelihood factorises

$$\begin{aligned}\mathbb{P}(\mathcal{D}|p) &= \prod_{i=1}^n p^{X_i}(1-p)^{1-X_i} \\ &= p^{X_1}(1-p)^{1-X_1} p^{X_2}(1-p)^{1-X_2} \cdots p^{X_n}(1-p)^{1-X_n} \\ &= p^{\sum_i X_i}(1-p)^{\sum_i (1-X_i)} = p^s(1-p)^{n-s}\end{aligned}$$

$s = \sum_i X_i$ (number of successes/heads)

Posterior

- Plugging in a prior $f(p) = \text{Beta}(p|a_0,b_0)$

$$f(p|\mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|p)f(p)}{\mathbb{P}(\mathcal{D})} = \frac{p^s(1-p)^{n-s} \times p^{a_0-1}(1-p)^{b_0-1}}{\mathbb{P}(\mathcal{D})B(a_0,b_0)}$$

- The denominator is a normalising factor

$$\begin{aligned}\mathbb{P}(\mathcal{D}) &= \int_0^1 \mathbb{P}(\mathcal{D}|p)f(p)\mathrm{d}p = \int_0^1 \frac{p^{s+a_0-1}(1-p)^{n-s+b_0-1}}{B(a_0,b_0)}\mathrm{d}p \\ &= \frac{B(s+a_0,n-s+b_0)}{B(a_0,b_0)}\end{aligned}$$

Conjugate Priors

- The posterior distribution is Beta distribution

$$f(p|\mathcal{D}) = \frac{p^{s+a_0-1}(1-p)^{n-s+b_0-1}}{B(s+a_0,n-s+b_0)} = \text{Beta}(p|s+a_0,n-s+b_0)$$

- Something rather nice happened
- Starting with a beta distributed prior $f(p) = \text{Beta}(p|a_0,b_0)$ for a set of Bernoulli trials we obtain a beta distributed posterior $f(p|\mathcal{D}) = \text{Beta}(p|a_0+s,b_0+n-s)$
- This is not always the case (often the posterior will be very complicated) but it happens for a few likelihoods and priors
- When the posterior is in the same family as the prior then the likelihood and prior distributions are said to be **conjugate**

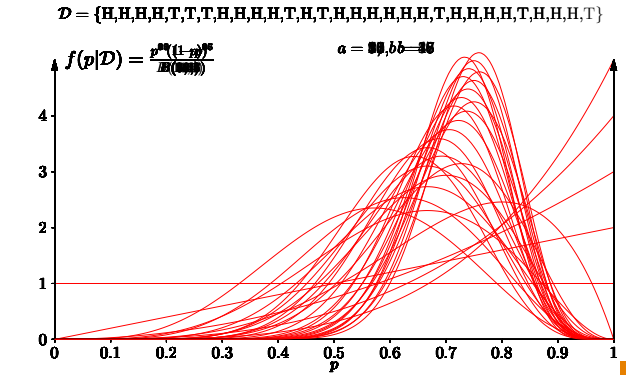
Incremental Updating

- The X_i are independent **given** p, so we can update incrementally, $\mathcal{D} = (X_1,X_2,\ldots,X_n)$

$$\begin{aligned}f(p|X_1) &= \frac{\mathbb{P}(X_1|p)f(p)}{\mathbb{P}(X_1)} \\ f(p|X_1,X_2) &= \frac{\mathbb{P}(X_2|p)f(p|X_1)}{\mathbb{P}(X_2|X_1)} \\ \vdots &= \vdots \\ f(p|X_1,X_2,\ldots,X_n) &= \frac{\mathbb{P}(X_n|p)f(p|X_1,\ldots,X_{n-1})}{\mathbb{P}(X_n|X_1,\ldots,X_{n-1})}\end{aligned}$$

- The posterior becomes the prior for the next piece of data
- For our problem the posterior is always Beta distributed

Example (p=0.7)



Estimating Prediction Errors

- A full Bayesian treatment gives a prediction of its own error
- Assuming $f(p|\mathcal{D}) = \text{Beta}(p|a,b)$ (in the example $a = 23$, $b = 9$ after 30 trials)
- The expected value of p is given by $a/(a+b) = 23/32 = 0.72$
- The standard deviation is

$$\sqrt{\frac{ab}{(a+b)^2(a+b+1)}} = 0.08$$

Poisson Likelihoods

- Let's look at a second example of conjugate priors
- Suppose we want to find the rate of traffic along a road between 1:00pm and 2:00pm
- We assume the number of cars is given by a Poisson distribution

$$\mathbb{P}(N) = \text{Pois}(N|\mu) = \frac{\mu^N}{N!}\mathrm{e}^{-\mu}$$

- μ is the rate of traffic per hour which we want to infer from observation taken on different days

Using Bayes

- Let us assume a Gamma distributed prior

$$p(\mu) = \text{Gamma}(\mu|a_0,b_0) = \frac{b_0^{a_0}\mu^{a_0-1}\mathrm{e}^{-b_0\mu}}{\Gamma(a_0)}$$

- We will assume that we know nothing. The uninformative prior is $a_0 = b_0 = 0$
- The data is $\mathcal{D} = \{N_1,N_2,\ldots,N_n\}$
- The likelihood is $\text{Pois}(N_i|\mu)$

Posterior

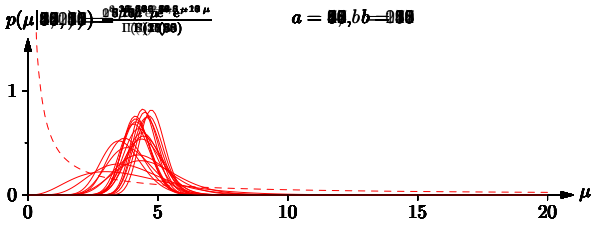
- The posterior after seeing the first piece of data is

$$\begin{aligned}p(\mu|N_1) &\propto \mathbb{P}(N_1|\mu)p(\mu) \\ &\propto \frac{\mu^{N_1}}{N_1!}\mathrm{e}^{-\mu}\mu^{a_0-1}\mathrm{e}^{-b_0\mu} \\ &\propto \mu^{N_1+a_0-1}\mathrm{e}^{-(b_0+1)\mu}\end{aligned}$$

- The posterior is also a Gamma distribution $\text{Gamma}(\mu|a_1,b_1)$ with $a_1 = a_0 + N_1$, $b_1 = b_0 + 1$

Example ($\mu = 5$)

$\mathcal{D} = \{4, 4, 6, 4, 2, 2, 5, 9, 5, 4, 3, 2, 5, 4, 4, 11, 6, 2, 3, 11\}$



$$\mathbb{E}[\mu] = \frac{a}{b} = \frac{96}{20} = 4.8 \qquad \sqrt{\mathbb{V}\mathrm{ar}(\mu)} = \sqrt{\frac{a}{b^2}} = 0.5$$

Outline

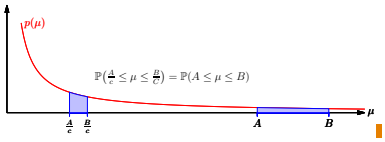
1. Bayes' Rule
2. Conjugate Priors
3. Uninformative Priors

Uninformative Priors

- What if we have no prior knowledge, what should we do?
- OK usually we know whether we should make a measurement using a micrometer, ruler or car mileage, but we might still know almost nothing
- This led to Bayesian statistics being labelled as *subjective*
- However E. T. Jaynes (the greatest proponent of Bayesian methods) argued that we could answer this using symmetry arguments

Uninformative Priors for Scale Parameter

- Why did we choose $a_0 = b_0 = 0$ implying a prior $p(\mu) = 1/\mu$?



- That is, we have no idea on what scale to measure μ

$$\int_A^B p(\mu)\mathrm{d}\mu = \int_{A/c}^{B/c} p(\mu)\mathrm{d}\mu = \int_A^B \frac{1}{c} p(\frac{\nu}{c})\mathrm{d}\nu = \int_A^B \frac{1}{c} p(\frac{\mu}{c})\mathrm{d}\mu$$

making a change of variables $\mu = \nu/c$

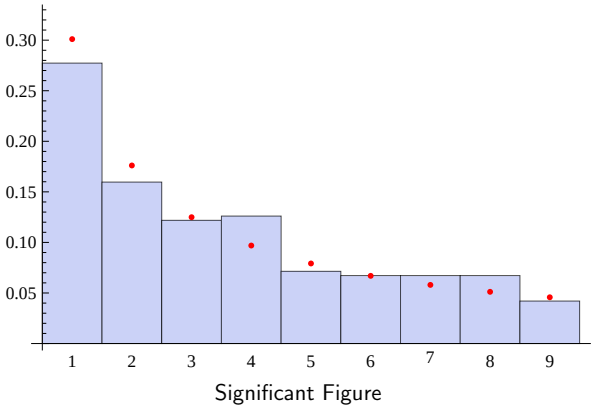
- Or $p(\mu) = \frac{1}{c}p(\frac{\mu}{c})$ implying $p(\mu) \propto \frac{1}{\mu}$

Benford's Law

- Numbers occurring in life (physical constants, amounts of money) should not depend on the units (scale) measuring them
- They should then be distributed as $p(x) \propto 1/x$
- A curious consequence of this is that the first significant digit has a distribution

$$\mathbb{P}(\text{most s.f. of } x = n) = \frac{\int_n^{n+1} \frac{1}{x}\mathrm{d}x}{\int_1^{10} \frac{1}{x}\mathrm{d}x} = \frac{\int_{10n}^{10n+10} \frac{1}{x}\mathrm{d}x}{\int_{10}^{100} \frac{1}{x}\mathrm{d}x}$$
$$= \frac{\log(n+1) - \log(n)}{\log(10)} = \log_{10}\left(\frac{n+1}{n}\right)$$

Population Size of 238 Countries

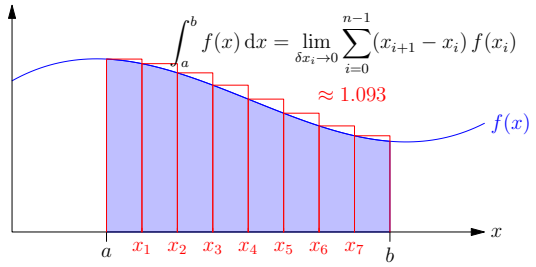


Conclusion

- Bayesian inference provides a coherent framework which we can use for machine learning
- However, it requires a model of what is happening
- In practice Bayesian methods are easy if the data is generated from a likelihood with a conjugate prior distribution—we have to be clever to choose the right prior
- We will see later that much more frequently we will have likelihoods with no conjugate prior and we have to work much harder
- When we have no knowledge there are consistent ways to express our ignorance

Mathematics of Machine Learning

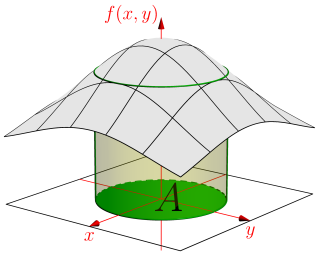
Integral Calculus



Riemann integration, integration by parts, substitution, change of variables

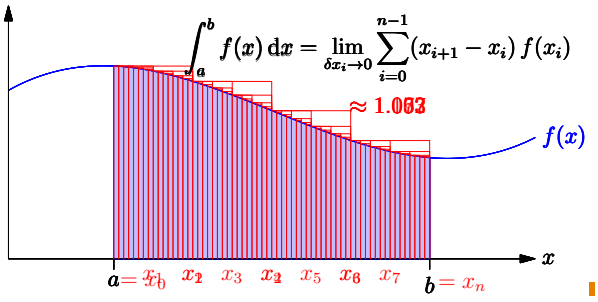
Outline

1. **Defining Integrals**
2. Doing Integrals
3. Changing Variables



Riemann Integral

- Integrals represent area beneath a curve



Linearity of Integration

- Integration is a linear operator

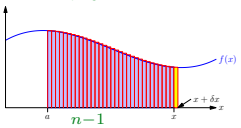
$$\begin{aligned}\int_a^b (rf(x) + sg(x))\mathrm{d}x &= \lim_{\delta x_i \to 0} \sum_{i=0}^{n-1} (x_{i+1} - x_i)(rf(x_i) + sg(x_i)) \\ &= \lim_{\delta x_i \to 0} \left(\sum_{i=0}^{n-1} (x_{i+1} - x_i) r f(x_i) + \sum_{i=0}^{n-1} (x_{i+1} - x_i) s g(x_i) \right) \\ &= \lim_{\delta x_i \to 0} \left(r \sum_{i=0}^{n-1} (x_{i+1} - x_i) f(x_i) + s \sum_{i=0}^{n-1} (x_{i+1} - x_i) g(x_i) \right) \\ &= r \lim_{\delta x_i \to 0} \sum_{i=0}^{n-1} (x_{i+1} - x_i) f(x_i) + s \lim_{\delta x_i \to 0} \sum_{i=0}^{n-1} (x_{i+1} - x_i) g(x_i) \\ &= r \int_a^b f(x)\mathrm{d}x + s \int_a^b g(x)\mathrm{d}x\end{aligned}$$

Fundamental Theorem of Calculus

- Let

$$I(a,x) = \int_a^x f(z)\mathrm{d}z = \lim_{\delta z_i \to 0} \sum_{i=0}^{n-1} (z_{i+1} - z_i) f(z_i)$$

- Now for small δx



$$I(a,x+\delta x) = \int_a^{x+\delta x} f(z)\mathrm{d}z = \lim_{\delta z_i \to 0} \sum_{i=0}^{n-1} (z_{i+1} - z_i) f(z_i) + \delta x f(x)$$

- Thus

$$\frac{\mathrm{d}I(a,x)}{\mathrm{d}x} = \lim_{\delta x \to 0} \frac{I(x+\delta x) - I(x)}{\delta x} = \lim_{\delta x \to 0} \frac{\delta x f(x)}{\delta x} = f(x)$$

The Other Way Around

- Consider

$$\begin{aligned}\int_a^b \frac{\mathrm{d}f(x)}{\mathrm{d}x}\mathrm{d}x &= \int_a^b \lim_{\delta x \to 0} \frac{f(x+\delta x) - f(x)}{\delta x}\mathrm{d}x \\ &= \lim_{x_{i+1} - x_i \to 0} \sum_{i=0}^{n-1} (x_{i+1} - x_i) \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i} \\ &= \lim_{x_{i+1} - x_i \to 0} \sum_{i=0}^{n-1} (f(x_{i+1}) - f(x_i)) \\ &= (f(x_1) - f(x_0)) + (f(x_2) - f(x_1)) + (f(x_3) - f(x_2)) + \cdots \\ &\quad + (f(x_{n-1}) - f(x_{n-2})) + (f(x_n) - f(x_{n-1})) \\ &= f(x_n) - f(x_0) = f(b) - f(a)\end{aligned}$$

- We can think of integration as an **anti-derivative**—it undoes differentiation

Indefinite Integrals

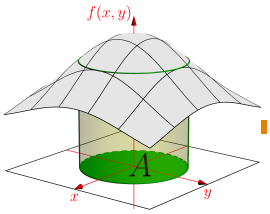
- So far we have considered **definite integrals** where we integrate between two points (a and b)
- However, when think about integration as an anti-derivative, it is useful to think of a function $F(x) = \int f(x)\mathrm{d}x$ (really $\int^x f(z)\mathrm{d}z$)
- So that $F'(x) = f(x)$
- However the function $F(x)$, $F(x)+1$, $F(x)+\pi$, etc. all have the same derivative so $F(x)$ is only defined up to an additive constant
- Note that the definite integral is given by

$$\int_a^b f(x)\mathrm{d}x = F(b) - F(a)$$

Multiple Integrals

- For functions involving many independent variables (e.g. $f(x,y)$, $f(x,y,z)$, $f(\boldsymbol{x})$) we can integrate over multiple dimensions
- For example

$$\iint_A f(x,y)\mathrm{d}x\mathrm{d}y$$

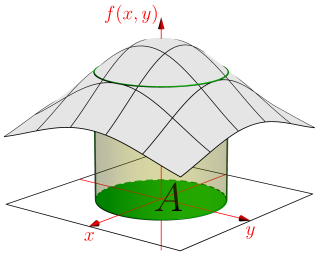


- It gets tedious writing multiple integral signs and I tend to write just one

$$\int \cdots \int f(x_1, x_2, \ldots, x_n)\mathrm{d}x_1\mathrm{d}x_2\cdots\mathrm{d}x_n = \int f(\boldsymbol{x})\mathrm{d}\boldsymbol{x}$$

Outline

1. Defining Integrals
2. **Doing Integrals**
3. Changing Variables



Performing Integration

- A key method for performing integrals is through knowledge of the anti-derivative
- If we know $F'(x) = f(x)$ then $F(x) + c = \int f(x)\mathrm{d}x$
- E.g. we know that $\mathrm{d}x^n/\mathrm{d}x = nx^{n-1}$ therefore

$$\int x^{n-1}\mathrm{d}x = \frac{1}{n}\int \frac{\mathrm{d}x^n}{\mathrm{d}x}\mathrm{d}x = \frac{x^n}{n} + c$$

and

$$\int_a^b x^{n-1}\mathrm{d}x = \frac{b^n}{n} - \frac{a^n}{n}$$

Is Integration Straightforward?

- We saw, due to the product and chain rules, that we can differentiate almost anything. Given integration is the anti-derivative can we integrate anything?
- Products and compositions

$$\int f(x)g(x)\mathrm{d}x = ? \qquad \int f(g(x))\mathrm{d}x = ?$$

- Unfortunately, unlike differentiation we don't have a small parameter we can expand in
- In general integration is hard

Integration by Parts

- Recall the product rule $\frac{\mathrm{d}f(x)g(x)}{\mathrm{d}x} = \frac{\mathrm{d}f(x)}{\mathrm{d}x}g(x) + f(x)\frac{\mathrm{d}g(x)}{\mathrm{d}x}$
- Integrating we get

$$\begin{aligned}\int_a^b \frac{\mathrm{d}f(x)g(x)}{\mathrm{d}x}\mathrm{d}x &= \int_a^b \frac{\mathrm{d}f(x)}{\mathrm{d}x}g(x)\mathrm{d}x + \int_a^b f(x)\frac{\mathrm{d}g(x)}{\mathrm{d}x}\mathrm{d}x \\ &= [f(x)g(x)]_a^b = f(b)g(b) - f(a)g(a)\end{aligned}$$

- Unfortunately we get two integrals, but we can turn this around

$$\int_a^b f(x)\frac{\mathrm{d}g(x)}{\mathrm{d}x}\mathrm{d}x = [f(x)g(x)]_a^b - \int_a^b \frac{\mathrm{d}f(x)}{\mathrm{d}x}g(x)\mathrm{d}x$$

whether this is helpful depends on $f(x)$ and $g(x)$

Example of Integration by Parts

- Consider

$$\begin{aligned}\Pi(z) &= \int_0^\infty x^z \mathrm{e}^{-x}\mathrm{d}x = \int_0^\infty x^z \frac{\mathrm{d}(-\mathrm{e}^{-x})}{\mathrm{d}x}\mathrm{d}x \\ &= \left[x^z(-\mathrm{e}^{-x})\right]_0^\infty - \int_0^\infty \frac{\mathrm{d}x^z}{\mathrm{d}x}(-\mathrm{e}^{-x})\mathrm{d}x \\ &= \int_0^\infty (zx^{z-1})\mathrm{e}^{-x}\mathrm{d}x = z\int_0^\infty x^{z-1}\mathrm{e}^{-x}\mathrm{d}x = z\Pi(z-1)\end{aligned}$$

- Thus $\Pi(z) = z\Pi(z-1)$, but

$$\Pi(0) = \int_0^\infty \mathrm{e}^{-x}\mathrm{d}x = \left[-\mathrm{e}^{-x}\right]_0^\infty = -\mathrm{e}^{-\infty} - (-\mathrm{e}^0) = 1$$

- Now
$\Pi(n) = n\Pi(n-1) = n(n-1)\Pi(n-2) = n(n-1)(n-2)\ldots 1 = n!$

Differentiating Through the Integral

- A trick that sometimes works is differentiating through an integral, e.g. consider finding moments

$$M_n = \mathbb{E}[X^n] = \int_{-\infty}^\infty x^n f_X(x)\mathrm{d}x$$

- We can define a moment generating function

$$Z(\ell) = \int_{-\infty}^\infty \mathrm{e}^{\ell x} f_X(x)\mathrm{d}x$$

- Then $M_n = Z^{(n)}(0)$ (the n^{th} derivative of $Z(\ell)$ at $\ell = 0$)

$$\left.\frac{\mathrm{d}^n Z(\ell)}{\mathrm{d}\ell^n}\right|_{\ell=0} = \int_{-\infty}^\infty \left.\frac{\mathrm{d}^n \mathrm{e}^{\ell x}}{\mathrm{d}\ell^n}\right|_{\ell=0} f_X(x)\mathrm{d}x = \int_{-\infty}^\infty x^n f_X(x)\mathrm{d}x = M_n$$

Cumulant Generating Function

- Note that $\mathrm{e}^{\ell x} = 1 + \ell x + \frac{1}{2}\ell^2 x^2 + \frac{1}{3!}\ell^3 x^3 + \cdots$
- So

$$Z(\ell) = \int_{-\infty}^\infty \mathrm{e}^{\ell x} f_X(x)\mathrm{d}x = 1 + \ell M_1 + \frac{1}{2}\ell^2 M_2 + \frac{1}{3!}\ell^3 M_3 + \cdots$$

- Now using $\log(1+\epsilon) = \epsilon - \frac{1}{2}\epsilon^2 + \frac{1}{3}\epsilon^3 + \cdots$

$$G(\ell) = \log(Z(\ell)) = \ell M_1 + \frac{1}{2}\ell^2(M_2 - M_1^2) + \frac{1}{3!}\ell^3(M_3 - 3M_2M_1 + 2M_1^3) + \cdots$$

- So that $\kappa_n = G^{(n)}(0)$, with $\kappa_1 = M_1$ (the mean), $\kappa_2 = M_2 - M_1^2$ (the variance), $\kappa_3 = M_3 - 3M_2M_1 + 2M_1^3$ (the third cumulant related to the skewness)

More Integration

- Although we have a few tricks, integration is hard
- Surprisingly integration sometimes is easier when carried out in the complex plane
- This is a beautiful part of mathematics (due largely to Cauchy)—but beyond the scope of this course
- Interestingly, also there is an algorithm that allows us to integrate a lot of functions. It is sufficiently complicated that you need to write a computer algorithm of considerable complexity to implement it. Most symbolic manipulation packages (e.g. Mathematica) have implemented some part of this algorithm

Special Functions

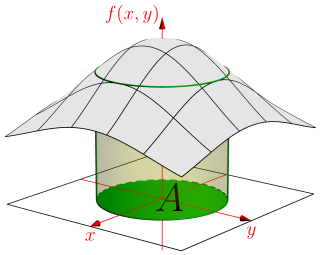
- There are integrals with no known closed form solution
- We saw that $\Pi(z) = \int_0^\infty x^z \mathrm{e}^{-x} \mathrm{d}x$ satisfies $\Pi(z) = z\Pi(z-1)$
- For integer n then $\Pi(n) = n!$ but for general z, the integal $\Pi(z)$ can't be written in terms of elementary functions
- We consider $\Pi(z)$ as a special function in its own right
- Although, history has left us with the gamma function instead

$$\Gamma(z) = \int_0^\infty x^{z-1} \mathrm{e}^{-x} \mathrm{d}x = \Pi(z-1)$$

- Other special function defined by integrals exist (e.g. the Bessel , Aire, hypergeometric, elliptic, error functions, . . .)

Outline

1. Defining Integrals
2. Doing Integrals
3. **Changing Variables**



Substitution

- We can make a transformation from x to $u = u(x)$

$$\begin{aligned}\int_a^b f(x)\mathrm{d}x &= \lim_{\delta x_i \to 0} \sum_{i=0}^{n-1} f(x_i)(x_{i+1} - x_i) \\ &= \lim_{\delta u_i \to 0} \sum_{i=0}^{n-1} f(x(u_i)) \frac{x(u_{i+1}) - x(u_i)}{u_{i+1} - u_i}(u_{i+1} - u_i) \\ &= \int_{u(a)}^{u(b)} f(x(u)) \frac{\mathrm{d}x(u)}{\mathrm{d}u} \mathrm{d}u\end{aligned}$$

  - where u_i is such that $x(u_i) = x_i$ or $u_i = u(x_i)$ where $u(x)$ is the inverse of $x(u)$
  - using $\lim_{\delta u_i \to 0} \frac{x(u_{i+1}) - x(u_i)}{u_{i+1} - u_i} = \frac{\mathrm{d}x(u_i)}{\mathrm{d}u}$

Substitution in Practice

- A recipe
  1. Choose $u(x)$: often an "inner" function whose derivative also appears in the integrand
  2. Write $\mathrm{d}u = u'(x)\mathrm{d}x$ and replace every x by u
  3. Change the limits: $x = a$ becomes $u = u(a)$
  4. Integrate in u
- Example: $\int_0^1 2x(1+x^2)^3 \mathrm{d}x$

$$u = 1 + x^2, \quad \mathrm{d}u = 2x\mathrm{d}x, \quad x : 0 \to 1 \;\Rightarrow\; u : 1 \to 2$$

$$\int_0^1 2x(1+x^2)^3 \mathrm{d}x = \int_1^2 u^3 \mathrm{d}u = \left[\frac{u^4}{4}\right]_1^2 = \frac{16-1}{4} = \frac{15}{4}$$

More Substitutions

- The limits can swap over: for $\int_0^{\pi/2} \sin(x)\cos^2(x)\mathrm{d}x$ take $u = \cos(x)$, $\mathrm{d}u = -\sin(x)\mathrm{d}x$, and $x : 0 \to \pi/2$ gives $u : 1 \to 0$

$$\int_0^{\pi/2} \sin(x)\cos^2(x)\mathrm{d}x = \int_1^0 -u^2 \mathrm{d}u = \int_0^1 u^2 \mathrm{d}u = \frac{1}{3}$$

- A linear change of variable stretches $\mathrm{d}x$: with $z = (x-\mu)/\sigma$, $\mathrm{d}x = \sigma \mathrm{d}z$

$$\int_{-\infty}^{\infty} \mathrm{e}^{-(x-\mu)^2/(2\sigma^2)} \mathrm{d}x = \sigma \int_{-\infty}^{\infty} \mathrm{e}^{-z^2/2} \mathrm{d}z$$

- The factor σ is the one-dimensional version of the Jacobian determinant (see later slide)

Example of Integration by Substitution

- We consider $I(n) = \int_0^\infty x^n \mathrm{e}^{-x^2/2} \mathrm{d}x$
- Let $u(x) = x^2/2$ or $x(u) = \sqrt{2u}$ so that

$$\frac{\mathrm{d}x(u)}{\mathrm{d}u} = \frac{1}{\sqrt{2u}} \qquad u(0) = 0 \qquad u(\infty) = \infty$$

- Thus

$$\begin{aligned} I(n) &= \int_0^\infty \left(\sqrt{2u}\right)^n \mathrm{e}^{-u} \frac{1}{\sqrt{2u}} \mathrm{d}u \\ &= 2^{\frac{n-1}{2}} \int_0^\infty u^{\frac{n-1}{2}} \mathrm{e}^{-u} \mathrm{d}u = 2^{\frac{n-1}{2}} \Pi\left(\frac{n-1}{2}\right) \end{aligned}$$

- $I(1) = 1$, $I(3) = 2 \times 1! = 2$, $I(5) = 2^2 \times 2! = 8$, but $I(0) = \Pi(-1/2)/\sqrt{2}$, $I(2) = \sqrt{2}\Pi(1/2) = \Pi(-1/2)/\sqrt{2}$

Gamma Distribution

- Last lecture we met the Gamma distribution

$$\mathrm{Gamma}(x|a,b) = cx^{a-1}\mathrm{e}^{-bx}, \qquad x \geq 0$$

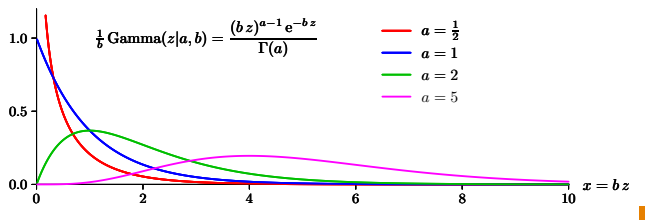
  Let's compute the normalisation factor c

- We need $c \int_0^\infty x^{a-1}\mathrm{e}^{-bx}\mathrm{d}x = 1$
- Substitute $z = bx$, so $\mathrm{d}x = \mathrm{d}z/b$ and the limits stay 0 and ∞

$$c\int_0^\infty \left(\frac{z}{b}\right)^{a-1} \mathrm{e}^{-z} \frac{\mathrm{d}z}{b} = \frac{c}{b^a} \int_0^\infty z^{a-1}\mathrm{e}^{-z}\mathrm{d}z = \frac{c\Gamma(a)}{b^a}$$

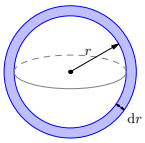
- So $c = \frac{b^a}{\Gamma(a)} = \frac{b^a}{\Pi(a-1)}$; b only sets the scale, a sets the shape

Shapes of the Gamma Distribution



- $a < 1$: infinite at 0; $a = 1$: the exponential distribution; $a > 1$: a peak at $bz = a - 1$
- In terms of $x = bz$ every Gamma distribution is $\mathrm{Gamma}(x|a,1)$: one curve for each a

Lengths of Gaussian Vectors



- Take $\boldsymbol{x} \in \mathbb{R}^n$ with $\boldsymbol{x} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$: the density $\propto \mathrm{e}^{-r^2/2}$ depends only on $r = \|\boldsymbol{x}\|$
- The density is largest at $\boldsymbol{x} = \mathbf{0}$, but there is more space at large r: a shell of radius r and thickness $\mathrm{d}r$ has volume $\propto r^{n-1}\mathrm{d}r$
- So, normalising with $I(n-1)$ from the last example,

$$f_R(r) = \frac{r^{n-1}\mathrm{e}^{-r^2/2}}{I(n-1)}, \qquad I(n-1) = 2^{\frac{n}{2}-1}\Gamma\left(\frac{n}{2}\right)$$

- The peak is at $r = \sqrt{n-1}$: in high dimensions almost all the probability is far from the most probable point
- With $s = r^2$, $f_S(s) = \mathrm{Gamma}(s|\frac{n}{2}, \frac{1}{2})$: the **chi-squared** distribution with n degrees of freedom

Gamma Distributions Everywhere

- **Chi-squared**: a sum of squares of n independent standard normals is $\mathrm{Gamma}(\frac{n}{2}, \frac{1}{2})$
- For m normal samples the sample variance has $(m-1)\hat{\sigma}^2/\sigma^2 \sim \chi^2_{m-1}$, which lies behind the t-test
- **Maxwell–Boltzmann**: each velocity component of a gas molecule of mass M is $\mathcal{N}(0, kT/M)$, so the speed is the $n = 3$ case, $f(v) \propto v^2\mathrm{e}^{-Mv^2/(2kT)}$, and the energy $\frac{1}{2}Mv^2 \sim \mathrm{Gamma}(\frac{3}{2}, \frac{1}{kT})$
- **Waiting times**: $a = 1$ is the exponential distribution; the time to the a-th event of a Poisson process of rate b is $\mathrm{Gamma}(a, b)$
- And it is the conjugate prior for a Poisson rate (Bayes lecture)

Changing Variables in Multidimensional Space

- When changing variables in many dimensions $\boldsymbol{x} \to \boldsymbol{u}$ the change of variables involves the Jacobian

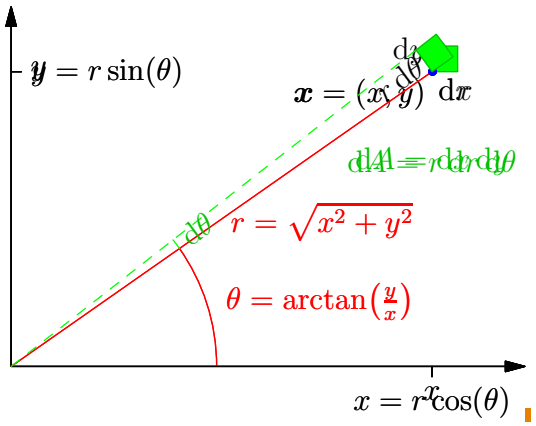
$$\int f(\boldsymbol{x})\mathrm{d}\boldsymbol{x} = \int f(\boldsymbol{x}(\boldsymbol{u}))|\det(\mathbf{J})|\mathrm{d}\boldsymbol{u}, \qquad \boldsymbol{J} = \begin{pmatrix} \frac{\partial x_1}{\partial u_1} & \frac{\partial x_1}{\partial u_2} & \cdots & \frac{\partial x_1}{\partial u_n} \\ \frac{\partial x_2}{\partial u_1} & \frac{\partial x_2}{\partial u_2} & \cdots & \frac{\partial x_2}{\partial u_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial x_n}{\partial u_1} & \frac{\partial x_n}{\partial u_2} & \cdots & \frac{\partial x_n}{\partial u_n} \end{pmatrix}$$

- E.g. transforming from Cartesian coordinates (x,y) to polar coordinates (r,θ) then $x = r\cos(\theta)$ and $y = r\sin(\theta)$

$$|\det(\mathbf{J})| = \left|\det\begin{pmatrix} \frac{\partial r\cos(\theta)}{\partial r} & \frac{\partial r\cos(\theta)}{\partial \theta} \\ \frac{\partial r\sin(\theta)}{\partial r} & \frac{\partial r\sin(\theta)}{\partial \theta} \end{pmatrix}\right| = \left|\det\begin{pmatrix} \cos(\theta) & -r\sin(\theta) \\ \sin(\theta) & r\cos(\theta) \end{pmatrix}\right|$$
$$= r\left(\cos^2(\theta) + \sin^2(\theta)\right) = r$$

- That is, $\mathrm{d}x\mathrm{d}y = r\mathrm{d}r\mathrm{d}\theta$

Change of Variables in Pictures

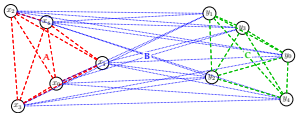


Summary

- Integration is extra-ordinarily useful as a tool of analysis
- It occurs when you work with probabilities densities for continuous random variables
- Integration is beautiful, but hard—often impossible
- Changing variables turns many hard integrals into easy ones
- Normal distributions, luckily, give integrals that can be done in closed form, although it takes some work: that is the next lecture
- Making friends with integration will give you a super-power that not too many people share

Mathematics of Machine Learning

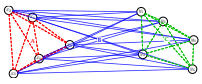
Gaussian Integrals



Gaussian integrals, completing the square, moment generating functions, marginals, conditionals, Kalman filter

Outline

1. One Dimension
2. Many Dimensions
3. Marginals and Conditionals
4. Inference with Gaussians



Why Gaussians?

- Gaussians (normal distributions) are everywhere in machine learning: Gaussian processes, Kalman filters, linear regression, the noise in diffusion models, . . .
- The reason is that we can do all their integrals exactly
- Marginalising, conditioning, adding or linearly transforming Gaussians gives another Gaussian
- The algebra is long-winded, but it uses only three tricks: the basic Gaussian integral, completing the square, and diagonalising a matrix
- Know the results and when to use them; the derivations are there so you can trust them

The Gaussian Integral

- We cannot find an antiderivative of $\mathrm{e}^{-x^2/2}$, but there is a trick: square the integral

$$I^2 = \int_{-\infty}^{\infty} \mathrm{e}^{-x^2/2}\mathrm{d}x \int_{-\infty}^{\infty} \mathrm{e}^{-y^2/2}\mathrm{d}y = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} \mathrm{e}^{-(x^2+y^2)/2}\mathrm{d}x\mathrm{d}y$$

- In polar coordinates ($x^2+y^2=r^2$, $\mathrm{d}x\mathrm{d}y = r\mathrm{d}r\mathrm{d}\theta$)

$$I^2 = \int_0^{2\pi}\mathrm{d}\theta \int_0^{\infty} r\mathrm{e}^{-r^2/2}\mathrm{d}r = 2\pi\int_0^{\infty}\mathrm{e}^{-u}\mathrm{d}u = 2\pi$$

using $u = r^2/2$, $\mathrm{d}u = r\mathrm{d}r$

- So

$$\int_{-\infty}^{\infty} \mathrm{e}^{-x^2/2}\mathrm{d}x = \sqrt{2\pi}$$

Normal Distribution

- Changing variables, $z = (x-\mu)/\sigma$ so $\mathrm{d}x = \sigma\mathrm{d}z$,

$$\int_{-\infty}^{\infty} \mathrm{e}^{-(x-\mu)^2/(2\sigma^2)}\mathrm{d}x = \sigma\int_{-\infty}^{\infty}\mathrm{e}^{-z^2/2}\mathrm{d}z = \sqrt{2\pi}\sigma$$

- So the normal density

$$\mathcal{N}(x \mid \mu,\sigma^2) = \frac{1}{\sqrt{2\pi}\sigma}\mathrm{e}^{-(x-\mu)^2/(2\sigma^2)}$$

integrates to one

- (For small σ the density exceeds one at $x=\mu$: densities are not probabilities)

Completing the Square

- Any quadratic in the exponent can be rewritten (for $a>0$)

$$-\frac{a}{2}x^2 + bx = -\frac{a}{2}\left(x - \frac{b}{a}\right)^2 + \frac{b^2}{2a}$$

- So the integral is a shifted Gaussian times a constant

$$\int_{-\infty}^{\infty}\mathrm{e}^{-\frac{a}{2}x^2+bx}\mathrm{d}x = \mathrm{e}^{\frac{b^2}{2a}}\int_{-\infty}^{\infty}\mathrm{e}^{-\frac{a}{2}\left(x-\frac{b}{a}\right)^2}\mathrm{d}x = \sqrt{\frac{2\pi}{a}}\,\mathrm{e}^{\frac{b^2}{2a}}$$

- This one result does most of the work in this lecture

Moment Generating Function

- The **moment generating function** of X is $M(t) = \mathbb{E}\left[\mathrm{e}^{tX}\right]$
- For $X \sim \mathcal{N}(\mu,\sigma^2)$, completing the square gives

$$M(t) = \int_{-\infty}^{\infty}\mathrm{e}^{tx}\mathcal{N}(x \mid \mu,\sigma^2)\mathrm{d}x = \mathrm{e}^{\mu t + \frac{1}{2}\sigma^2 t^2}$$

- Expanding $\mathrm{e}^{tX} = 1 + tX + \frac{1}{2}t^2X^2 + \cdots$ shows $M^{(n)}(0) = \mathbb{E}[X^n]$

$$M'(0) = \mu, \qquad M''(0) = \mu^2 + \sigma^2$$

- The cumulant generating function $\log M(t) = \mu t + \frac{1}{2}\sigma^2 t^2$ stops at t^2: a Gaussian has no cumulants beyond the mean and variance

Sums of Gaussians

- A moment generating function determines its distribution
- If X and Y are independent, $\mathbb{E}\left[\mathrm{e}^{t(X+Y)}\right] = \mathbb{E}\left[\mathrm{e}^{tX}\right]\mathbb{E}\left[\mathrm{e}^{tY}\right]$
- So for $X \sim \mathcal{N}(\mu_1,\sigma_1^2)$ and $Y \sim \mathcal{N}(\mu_2,\sigma_2^2)$

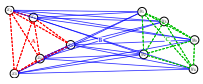
$$\mathbb{E}\left[\mathrm{e}^{t(X+Y)}\right] = \mathrm{e}^{(\mu_1+\mu_2)t + \frac{1}{2}(\sigma_1^2+\sigma_2^2)t^2}$$

which is the MGF of $\mathcal{N}(\mu_1+\mu_2, \sigma_1^2+\sigma_2^2)$

- Means add and variances add, and the sum is still Gaussian
- No integral needed: this is why MGFs are so useful

Outline

1. One Dimension
2. **Many Dimensions**
3. Marginals and Conditionals
4. Inference with Gaussians



Independent Variables

- With $\boldsymbol{x} = (x_1,\ldots,x_n)^\mathsf{T}$ and $\|\boldsymbol{x}\|^2 = \sum_i x_i^2$, the exponential of a sum is a product of exponentials

$$\int \mathrm{e}^{-\frac{1}{2}\|\boldsymbol{x}\|^2}\mathrm{d}\boldsymbol{x} = \prod_{i=1}^n \int_{-\infty}^{\infty} \mathrm{e}^{-x_i^2/2}\mathrm{d}x_i = (2\pi)^{n/2}$$

- Similarly with a different variance λ_i in each direction

$$\int \mathrm{e}^{-\frac{1}{2}\sum_i x_i^2/\lambda_i}\mathrm{d}\boldsymbol{x} = \prod_{i=1}^n \sqrt{2\pi\lambda_i}$$

- The general case can be rotated into this one

The Multivariate Normal

- For a positive definite covariance $\boldsymbol{\Sigma} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^\mathsf{T}$ (eigenvalues $\lambda_i > 0$) consider $I = \int \mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}\mathrm{d}\boldsymbol{x}$
- Rotate, $\boldsymbol{y} = \boldsymbol{V}^\mathsf{T}(\boldsymbol{x}-\boldsymbol{\mu})$: the Jacobian $\boldsymbol{V}$ has $|\det \boldsymbol{V}| = 1$ (rotations keep volumes)

$$I = \int \mathrm{e}^{-\frac{1}{2}\boldsymbol{y}^\mathsf{T}\boldsymbol{\Lambda}^{-1}\boldsymbol{y}}\mathrm{d}\boldsymbol{y} = \prod_i \sqrt{2\pi\lambda_i} = \sqrt{\det(2\pi\boldsymbol{\Sigma})}$$

as $\det\boldsymbol{\Sigma} = \prod_i \lambda_i$ and $\det(c\boldsymbol{\Sigma}) = c^n \det\boldsymbol{\Sigma}$

- So the multivariate normal density is

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu},\boldsymbol{\Sigma}) = \frac{1}{\sqrt{\det(2\pi\boldsymbol{\Sigma})}}\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\mathsf{T}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}$$

Completing the Square in Many Dimensions

- For symmetric positive definite $\boldsymbol{A}$

$$-\frac{1}{2}\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}^\mathsf{T}\boldsymbol{x} = -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{A}^{-1}\boldsymbol{b})^\mathsf{T}\boldsymbol{A}(\boldsymbol{x}-\boldsymbol{A}^{-1}\boldsymbol{b}) + \frac{1}{2}\boldsymbol{b}^\mathsf{T}\boldsymbol{A}^{-1}\boldsymbol{b}$$

(multiply out the right-hand side to check)

- So, by the last slide with $\boldsymbol{\Sigma} = \boldsymbol{A}^{-1}$,

$$\int \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^\mathsf{T}\boldsymbol{A}\boldsymbol{x}+\boldsymbol{b}^\mathsf{T}\boldsymbol{x}}\mathrm{d}\boldsymbol{x} = \sqrt{\frac{(2\pi)^n}{\det\boldsymbol{A}}}\,\mathrm{e}^{\frac{1}{2}\boldsymbol{b}^\mathsf{T}\boldsymbol{A}^{-1}\boldsymbol{b}}$$

- This is the central result: every Gaussian integral in this course is an instance of it

Multivariate Moment Generating Function

- For $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu},\boldsymbol{\Sigma})$, the central result with $\boldsymbol{A} = \boldsymbol{\Sigma}^{-1}$ gives

$$M(\boldsymbol{t}) = \mathbb{E}\left[\mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{x}}\right] = \mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{\mu}+\frac{1}{2}\boldsymbol{t}^\mathsf{T}\boldsymbol{\Sigma}\boldsymbol{t}}$$

- Differentiating at $\boldsymbol{t} = \boldsymbol{0}$

$$\mathbb{E}[\boldsymbol{x}] = \boldsymbol{\mu}, \qquad \mathbb{E}\left[(\boldsymbol{x}-\boldsymbol{\mu})(\boldsymbol{x}-\boldsymbol{\mu})^\mathsf{T}\right] = \boldsymbol{\Sigma}$$

- A Gaussian is completely described by its mean and its covariance

Linear Transformations

- Let $\boldsymbol{z} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{c}$ with $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu},\boldsymbol{\Sigma})$
- Its MGF is

$$\mathbb{E}_{\boldsymbol{z}}\left[\mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{z}}\right] = \mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{c}}\mathbb{E}_{\boldsymbol{x}}\left[\mathrm{e}^{(\boldsymbol{W}^\mathsf{T}\boldsymbol{t})^\mathsf{T}\boldsymbol{x}}\right] = \mathrm{e}^{\boldsymbol{t}^\mathsf{T}(\boldsymbol{W}\boldsymbol{\mu}+\boldsymbol{c})+\frac{1}{2}\boldsymbol{t}^\mathsf{T}\boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^\mathsf{T}\boldsymbol{t}}$$

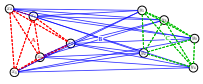
- So

$$\boldsymbol{z} \sim \mathcal{N}\big(\boldsymbol{W}\boldsymbol{\mu}+\boldsymbol{c},\, \boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^\mathsf{T}\big)$$

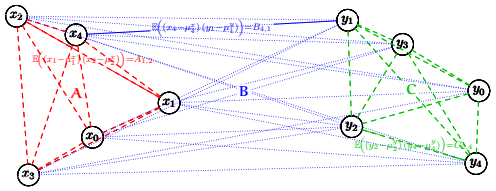
- Linear maps of Gaussians are Gaussian, and we only need to track the mean and covariance ($\boldsymbol{W}$ need not be square)

Outline

1. One Dimension
2. Many Dimensions
3. **Marginals and Conditionals**
4. Inference with Gaussians



Joint Gaussians



$$\begin{pmatrix}\boldsymbol{z}_1\\ \boldsymbol{z}_2\end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix}\boldsymbol{\mu}_1\\ \boldsymbol{\mu}_2\end{pmatrix}, \begin{pmatrix}\boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12}\\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22}\end{pmatrix}\right)$$

With $\boldsymbol{\Sigma}_{21} = \boldsymbol{\Sigma}_{12}^\mathsf{T}$. From now on we write $\boldsymbol{z}_1$ for $\boldsymbol{x}$ and $\boldsymbol{z}_2$ for $\boldsymbol{y}$, and $\boldsymbol{\Sigma}_{11}$, $\boldsymbol{\Sigma}_{12}$ and $\boldsymbol{\Sigma}_{22}$ for $\boldsymbol{A}$, $\boldsymbol{B}$ and $\boldsymbol{C}$

Marginalising

- We want $p(\boldsymbol{z}_1) = \int p(\boldsymbol{z}_1, \boldsymbol{z}_2) \mathrm{d}\boldsymbol{z}_2$
- Doing the integral directly takes a page of block-matrix algebra (see the notes)
- Now set $\boldsymbol{t}_2 = \boldsymbol{0}$, so $\boldsymbol{t}^\mathsf{T}\boldsymbol{z} = \boldsymbol{t}_1^\mathsf{T}\boldsymbol{z}_1$

$$\mathbb{E}\left[\mathrm{e}^{\boldsymbol{t}_1^\mathsf{T}\boldsymbol{z}_1}\right] = \mathrm{e}^{\boldsymbol{t}_1^\mathsf{T}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{t}_1^\mathsf{T}\boldsymbol{\Sigma}_{11}\boldsymbol{t}_1}$$

- That is the MGF of $\mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_{11})$, so $\boldsymbol{z}_1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_{11})$
- To marginalise, just strike out the rows and columns of the variables you integrate away

Conditioning: a Light Derivation

- We want $p(\boldsymbol{z}_1 \mid \boldsymbol{z}_2)$
- Subtract the best linear prediction of $\boldsymbol{z}_1$ from $\boldsymbol{z}_2$

$$\boldsymbol{w} = \boldsymbol{z}_1 - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{z}_2$$

- $\boldsymbol{w}$ is a linear map of $\boldsymbol{z}$, so $(\boldsymbol{w}, \boldsymbol{z}_2)$ are jointly Gaussian, and

$$\mathrm{Cov}(\boldsymbol{w}, \boldsymbol{z}_2) = \boldsymbol{\Sigma}_{12} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{22} = \boldsymbol{0}$$

- For jointly Gaussian variables, uncorrelated means independent (the MGF factorises)
- So knowing $\boldsymbol{z}_2$ tells us nothing about $\boldsymbol{w}$, and $\boldsymbol{z}_1 = \boldsymbol{w} + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{z}_2$ with $\boldsymbol{z}_2$ now fixed

The Conditional Distribution

- From the linear-transformation rule, $\boldsymbol{w}$ has mean $\boldsymbol{\mu}_1 - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\mu}_2$ and covariance $\mathrm{Cov}(\boldsymbol{w}, \boldsymbol{w}) = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}$
- Hence

$$\boldsymbol{z}_1 \mid \boldsymbol{z}_2 \sim \mathcal{N}\big(\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\boldsymbol{z}_2 - \boldsymbol{\mu}_2),\ \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}\big)$$

- The mean moves linearly with what we observe
- The covariance shrinks, by an amount that does not depend on the value we observe
- This one formula is the engine behind Gaussian processes and Kalman filters

The Long Way Round

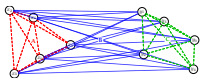
- The same results follow by brute force from the block factorisation

$$\begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix} = \begin{pmatrix} \mathbf{I} & \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1} \\ \mathbf{0} & \mathbf{I} \end{pmatrix} \begin{pmatrix} \mathbf{S} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{22} \end{pmatrix} \begin{pmatrix} \mathbf{I} & \mathbf{0} \\ \boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21} & \mathbf{I} \end{pmatrix}$$

  where $\mathbf{S} = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}$ (the **Schur complement**)
- Inverting each factor splits the exponent into a part for $\boldsymbol{z}_2$ and a part for $\boldsymbol{z}_1$ given $\boldsymbol{z}_2$
- It also gives $\det\boldsymbol{\Sigma} = \det\mathbf{S}\det\boldsymbol{\Sigma}_{22}$
- The full derivation is in the notes: worth reading once, not memorising

Outline

1. One Dimension
2. Many Dimensions
3. Marginals and Conditionals
4. **Inference with Gaussians**



Bayes with Gaussians

- We want to infer $\boldsymbol{x}$, with a Gaussian **prior** $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}_0, \boldsymbol{\Sigma}_0)$
- We measure $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b} + \boldsymbol{\epsilon}$, with noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma}_\epsilon)$—the **likelihood** is Gaussian
- $(\boldsymbol{x}, \boldsymbol{y})$ is a linear map of $(\boldsymbol{x}, \boldsymbol{\epsilon})$, so jointly Gaussian

$$p(\boldsymbol{x}, \boldsymbol{y}) = \mathcal{N}\left(\begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{pmatrix} \middle| \begin{pmatrix} \boldsymbol{\mu}_0 \\ \boldsymbol{\mu}_y \end{pmatrix}, \begin{pmatrix} \boldsymbol{\Sigma}_0 & \boldsymbol{\Sigma}_{0y} \\ \boldsymbol{\Sigma}_{y0} & \boldsymbol{\Sigma}_{yy} \end{pmatrix}\right)$$

  $\boldsymbol{\mu}_y = \boldsymbol{W}\boldsymbol{\mu}_0 + \boldsymbol{b}$, $\boldsymbol{\Sigma}_{0y} = \boldsymbol{\Sigma}_0\boldsymbol{W}^\mathsf{T} = \boldsymbol{\Sigma}_{y0}^\mathsf{T}$, $\boldsymbol{\Sigma}_{yy} = \boldsymbol{W}\boldsymbol{\Sigma}_0\boldsymbol{W}^\mathsf{T} + \boldsymbol{\Sigma}_\epsilon$
- The **posterior** $p(\boldsymbol{x}|\boldsymbol{y}) = p(\boldsymbol{x}, \boldsymbol{y})/p(\boldsymbol{y})$ is just the conditional distribution

$$p(\boldsymbol{x}|\boldsymbol{y}) = \mathcal{N}\big(\boldsymbol{x}\big|\boldsymbol{\mu}_0 + \boldsymbol{\Sigma}_{0y}\boldsymbol{\Sigma}_{yy}^{-1}(\boldsymbol{y} - \boldsymbol{\mu}_y), \boldsymbol{\Sigma}_0 - \boldsymbol{\Sigma}_{0y}\boldsymbol{\Sigma}_{yy}^{-1}\boldsymbol{\Sigma}_{y0}\big)$$

A Noisy Measurement

- Prior belief about a quantity: $x \sim \mathcal{N}(\mu_0, s_0^2)$
- We measure $y = x + \epsilon$ with noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$
- (x, y) is a linear map of (x, ϵ), so jointly Gaussian with

$$\begin{pmatrix} x \\ y \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_0 \\ \mu_0 \end{pmatrix}, \begin{pmatrix} s_0^2 & s_0^2 \\ s_0^2 & s_0^2 + \sigma^2 \end{pmatrix}\right)$$

- Conditioning on y

$$\mathbb{E}[x \mid y] = \mu_0 + \frac{s_0^2}{s_0^2 + \sigma^2}(y - \mu_0), \qquad \frac{1}{\mathbb{V}\mathrm{ar}(x \mid y)} = \frac{1}{s_0^2} + \frac{1}{\sigma^2}$$

- The estimate moves towards the measurement, and the precisions (inverse variances) add

Combining Sensors

- Two sensors measure the same x, with noise variances σ_1^2 and σ_2^2, giving y_1 and y_2
- With a flat prior ($s_0 \to \infty$), applying the last result twice gives

$$\mathbb{E}[x \mid y_1, y_2] = \frac{y_1/\sigma_1^2 + y_2/\sigma_2^2}{1/\sigma_1^2 + 1/\sigma_2^2}, \qquad \frac{1}{\mathbb{V}\mathrm{ar}(x \mid y_1, y_2)} = \frac{1}{\sigma_1^2} + \frac{1}{\sigma_2^2}$$

- A precision-weighted average: trust the better sensor more
- With n equally good measurements the variance falls as σ^2/n

Predicting One Variable from Another

- Two quantities (e.g. height and weight) with correlation ρ

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}, \begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} \right)$$

- Conditioning on x_1 gives a straight line (linear regression)

$$\mathbb{E}[x_2 \mid x_1] = \mu_2 + \rho\frac{\sigma_2}{\sigma_1}(x_1 - \mu_1), \qquad \mathbb{V}\mathrm{ar}(x_2 \mid x_1) = \sigma_2^2(1-\rho^2)$$

- In standard units

$$\frac{\mathbb{E}[x_2 \mid x_1] - \mu_2}{\sigma_2} = \rho\frac{x_1 - \mu_1}{\sigma_1}$$

so if x_1 is k standard deviations out, the prediction is only ρk out ($|\rho| < 1$): **regression to the mean**

Tracking: the Kalman Filter

- A position drifts, $x_{t+1} = x_t + \eta_t$ with $\eta_t \sim \mathcal{N}(0, q)$, and is observed as $y_t = x_t + \epsilon_t$ with $\epsilon_t \sim \mathcal{N}(0, \sigma^2)$
- Our belief stays Gaussian, $x_t \sim \mathcal{N}(m_t, v_t)$. **Predict** (a sum of Gaussians): $x_{t+1} \sim \mathcal{N}(m_t,\ v_t + q)$
- **Update:** a noisy measurement, with prior $\mathcal{N}(m_t, v_t + q)$

$$m_{t+1} = m_t + k(y_{t+1} - m_t), \qquad v_{t+1} = (1-k)(v_t + q),$$
$$k = \frac{v_t + q}{v_t + q + \sigma^2}$$

- That is the Kalman filter (with vectors and matrices it guides aircraft and phones)

Towards Gaussian Processes

- Suppose the function values at some inputs are jointly Gaussian, and we observe some of them with noise
- Put the unknown value f_* in $\boldsymbol{z}_1$ and the observations $\boldsymbol{y}$ in $\boldsymbol{z}_2$

$$\begin{pmatrix} f_* \\ \boldsymbol{y} \end{pmatrix} \sim \mathcal{N}\left(\boldsymbol{0}, \begin{pmatrix} k_* & \boldsymbol{k}_*^\mathsf{T} \\ \boldsymbol{k}_* & \mathbf{K} + \sigma^2\mathbf{I} \end{pmatrix} \right)$$

- The conditional formula gives the prediction immediately

$$f_* \mid \boldsymbol{y} \sim \mathcal{N}\big(\boldsymbol{k}_*^\mathsf{T}(\mathbf{K} + \sigma^2\mathbf{I})^{-1}\boldsymbol{y},\ k_* - \boldsymbol{k}_*^\mathsf{T}(\mathbf{K} + \sigma^2\mathbf{I})^{-1}\boldsymbol{k}_*\big)$$

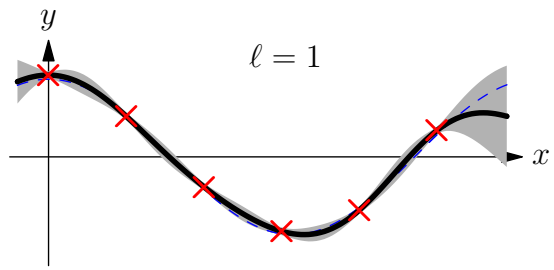
- Where the covariances come from is the subject of the next lecture

Summary

- The toolkit
  - $\int \mathrm{e}^{-\frac{1}{2}\boldsymbol{x}^\mathsf{T}\mathbf{A}\boldsymbol{x} + \boldsymbol{b}^\mathsf{T}\boldsymbol{x}}\mathrm{d}\boldsymbol{x} = \sqrt{(2\pi)^n/\det\mathbf{A}}\ \mathrm{e}^{\frac{1}{2}\boldsymbol{b}^\mathsf{T}\mathbf{A}^{-1}\boldsymbol{b}}$
  - MGF: $\mathbb{E}\left[\mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{x}}\right] = \mathrm{e}^{\boldsymbol{t}^\mathsf{T}\boldsymbol{\mu} + \frac{1}{2}\boldsymbol{t}^\mathsf{T}\boldsymbol{\Sigma}\boldsymbol{t}}$
  - Linear maps: $\boldsymbol{W}\boldsymbol{x} + \boldsymbol{c} \sim \mathcal{N}(\boldsymbol{W}\boldsymbol{\mu} + \boldsymbol{c}, \boldsymbol{W}\boldsymbol{\Sigma}\boldsymbol{W}^\mathsf{T})$
  - Marginals: strike out rows and columns
  - Conditionals: $\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\boldsymbol{z}_2 - \boldsymbol{\mu}_2)$ and $\boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}$
- Gaussians stay Gaussian, so inference reduces to linear algebra on means and covariances

Mathematics of Machine Learning

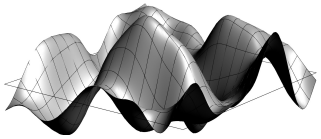
Gaussian Processes



Gaussian Processes, regression

Outline

1. **ML with Gaussians**
2. Gaussian Processes
3. Bayesian Inference
4. Hyper-parameters



Machine Learning with Gaussians

- Due to the fact that we can analytically integrate over multivariate Gaussians they are heavily used in machine learning
- Examples include multivariate modelling, Kalman filters and Gaussian Processes
- Despite being analytically tractable, they are very fiddly
- This is mainly because it is easy to specify means, $\boldsymbol{\mu}$, and covariance, $\mathbf{C}$, but Gaussians involve the inverse covariance matrix

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \mathbf{C}) = \frac{\mathrm{e}^{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\mathbf{C}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}}{\sqrt{\det(2\pi\mathbf{C})}}$$

Recap: the Gaussian Toolkit

- From the last lecture, for jointly Gaussian

$$\begin{pmatrix} \boldsymbol{z}_1 \\ \boldsymbol{z}_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix}, \begin{pmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{pmatrix} \right)$$

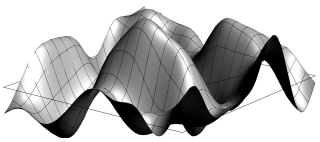
- Marginal: strike out rows and columns, $\boldsymbol{z}_1 \sim \mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_{11})$
- Conditional:

$$\boldsymbol{z}_1 \mid \boldsymbol{z}_2 \sim \mathcal{N}\big(\boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\boldsymbol{z}_2 - \boldsymbol{\mu}_2),\ \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}\big)$$

- Gaussian processes are built entirely on these two results

Outline

1. ML with Gaussians
2. **Gaussian Processes**
3. Bayesian Inference
4. Hyper-parameters



Gaussian Processes

- Gaussian processes (GPs) are a mathematically defined ensemble of functions
- They can be combined with Bayesian inference to give one of the most powerful regression techniques
- Although Bayesian they can be used in a black-box fashion due to the ubiquity of the prior
- Mathematically they are a bit complicated (because Gaussians involve the inverse of matrices which are a real pain to work with)
- In practice they aren't that difficult to use

Regression

- In regression we try to fit a multi-dimensional function to our data
- (You can use Gaussian Processes for classification, e.g. by inferring the probabilities of being in a class, but we ignore this as regression is where GP excel)
- In regression we have some p dimensional feature vectors $\boldsymbol{x}_i$ and some target $y_i \in \mathbb{R}$
- Our task is to fit a function through all the data points

Priors on Functions

- We can think of a solution as a function $f(\boldsymbol{x})$
- We can put a prior probability distribution, $p(f)$, on a function, f, that prefers smooth functions
- We can then compute a posterior probability distribution on functions given the data, $p(f|\mathcal{D})$
- As a likelihood, $p(y_i|f(\boldsymbol{x}_i))$, we use the probability of observing y_i given the true function value is $f(\boldsymbol{x}_i)$
- In general, this would be next to impossible to compute except in the special case where everything is Gaussian (normally) distributed

Gaussian Processes

- Gaussian Processes are probability distributions over functions
- (Functions can be viewed as vectors in an infinite dimensional vector space)
- In the Gaussian Process, $\mathcal{GP}(m,k)$, the probability of a function, f, is proportional

$$p(f|m,k) \propto \mathrm{e}^{-\frac{1}{2}\int (f(\boldsymbol{x})-m(\boldsymbol{x}))k^{-1}(\boldsymbol{x},\boldsymbol{y})(f(\boldsymbol{y})-m(\boldsymbol{y}))\mathrm{d}\boldsymbol{x}\mathrm{d}\boldsymbol{y}}$$

- The function $m(\boldsymbol{x})$ is the mean $\mathbb{E}[f(\boldsymbol{x})]$ (usually taken to be zero in most inference problems)

Meaning of GP

- To understand GP's we can discretise space, $\boldsymbol{x}$, into a lattice of points $\{\boldsymbol{x}_i\}$
- Then (assuming $m(\boldsymbol{x})=0$)

$$p(f|m,k) \propto \prod_i \mathrm{e}^{-\frac{f_i^2 k^{-1}(\boldsymbol{x}_i,\boldsymbol{x}_i)}{2} + f_i \sum_j k^{-1}(\boldsymbol{x}_i,\boldsymbol{x}_j) f_j}$$

  where $f_i = f(\boldsymbol{x}_i)$
- We see that the value of the function at each point is normally distributed with a mean that depends on functions at neighbouring points

Covariance function

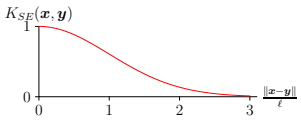
- $k(\boldsymbol{x},\boldsymbol{y})$ is a covariance function

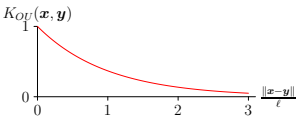
$$\mathbb{E}\big[\big(f(\boldsymbol{x})-m(\boldsymbol{x})\big)\big(f(\boldsymbol{y})-m(\boldsymbol{y})\big)\big] = k(\boldsymbol{x},\boldsymbol{y})$$

- This is sometimes know as a **kernel**—it must be positive semi-definite (just like in SVMs)
- It is a free "parameter" that the user gets to choose (although we can learn its parameters too)
- If $k(\boldsymbol{x},\boldsymbol{y})$ is a function of $\boldsymbol{x}-\boldsymbol{y}$ it is **"stationary"**
- If $k(\boldsymbol{x},\boldsymbol{y})$ is a function of $\|\boldsymbol{x}-\boldsymbol{y}\|$ it is also **"isometric"**

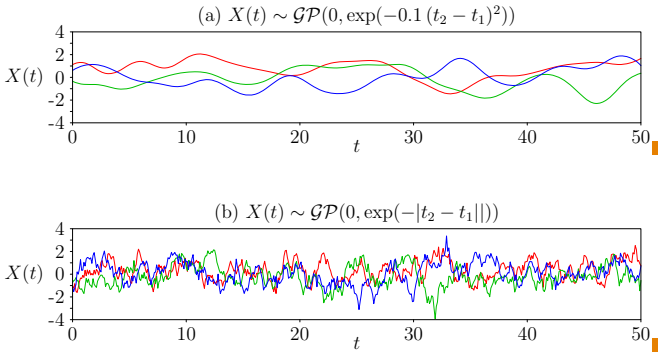
Popular Choices of GP Kernel Function

- Constant: $k_{\mathrm{C}}(\boldsymbol{x},\boldsymbol{y}) = C$
- Gaussian noise: $k_{\mathrm{GN}}(\boldsymbol{x},\boldsymbol{y}) = \sigma^2\delta_{\boldsymbol{x},\boldsymbol{y}}$
- Squared exponential: $k_{\mathrm{SE}}(\boldsymbol{x},\boldsymbol{y}) = \exp\left(-\frac{\|\boldsymbol{x}-\boldsymbol{y}\|^2}{2\ell^2}\right)$
- Ornstein–Uhlenbeck: $k_{\mathrm{OU}}(\boldsymbol{x},\boldsymbol{y}) = \exp\left(-\frac{\|\boldsymbol{x}-\boldsymbol{y}\|}{\ell}\right)$

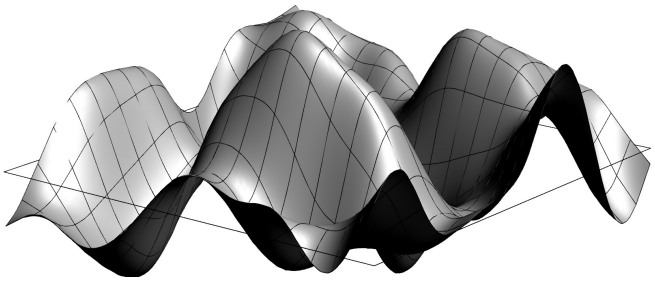




Gaussian Process Worlds

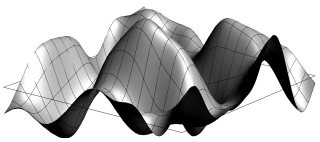


2-D Gaussian Processes



Outline

1. ML with Gaussians
2. Gaussian Processes
3. **Bayesian Inference**
4. Hyper-parameters



Observed Gaussian Processes

- Given some data points $\mathcal{D} = \big((\boldsymbol{x}_i,y_i)\big|i=1,\ldots,m\big)$ the likelihood (assuming Gaussian error are independence of the data point) is given by

$$p(\mathcal{D}|f) = \prod_{i=1}^{m} \mathcal{N}\big(y_i\big|f(\boldsymbol{x}_i),\sigma^2\big)$$

- Using a Gaussian Process prior we can compute a posterior using Bayes's rule
- The posterior is a Gaussian Process with a shifted mean and variance depending on the data-points
- This direct Bayesian derivation gives the answer involving the inverse matrix of the correlation function, $k^{-1}(\boldsymbol{x},\boldsymbol{y})$—this is a pain to work with

Alternative Derivation

- Denoting the target values as a vector $\boldsymbol{y}$ with elements y_i
- Denoting the matrices of covariances between data points as $\mathbf{K}$ with elements $k(\boldsymbol{x}_i,\boldsymbol{x}_j)$
- Denoting the covariance between the data points and a particular position, $\boldsymbol{x}_*$ as $\boldsymbol{k}_*$ with elements $k(\boldsymbol{x}_i,\boldsymbol{x}_*)$
- Denoting the variance a point $\boldsymbol{x}_*$ as $k_* = k(\boldsymbol{x}_*,\boldsymbol{x}_*)$
- Then the distribution of function values at points at $\boldsymbol{x}_i$ and $\boldsymbol{x}_*$ is

$$p(\boldsymbol{y},f_*) = \mathcal{N}\left(\begin{pmatrix}\boldsymbol{y}\\ f_*\end{pmatrix}\middle|\mathbf{0},\begin{pmatrix}\mathbf{K}+\sigma^2\mathbf{I} & \boldsymbol{k}_*\\ \boldsymbol{k}_*^\mathsf{T} & k_*\end{pmatrix}\right)$$

Conditional Probability

- To compute the posterior $p(f_*|\boldsymbol{y})$ we use

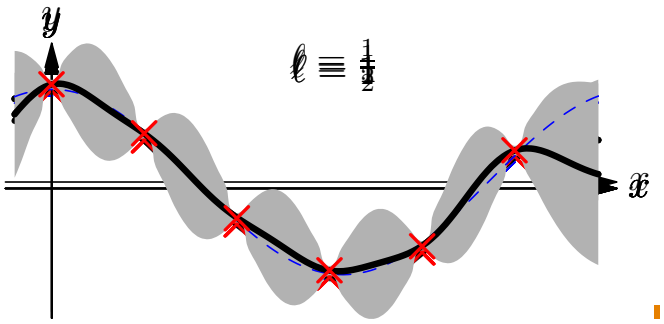
$$p(f_*|\boldsymbol{y}) = \frac{p(f_*,\boldsymbol{y})}{p(\boldsymbol{y})}$$

- where $p(\boldsymbol{y}) = \int p(f_*,\boldsymbol{y})\mathrm{d}f_*$
- The conditional formula from the last lecture gives

$$p(f_*|\boldsymbol{y}) = \mathcal{N}\left(f_*\middle|\boldsymbol{k}_*^\mathsf{T}(\mathbf{K}+\sigma^2\mathbf{I})^{-1}\boldsymbol{y}, k_* - \boldsymbol{k}_*^\mathsf{T}(\mathbf{K}+\sigma^2\mathbf{I})^{-1}\boldsymbol{k}_*\right)$$

- Looks complicated, but numerically easy to evaluate

$$K(x,x') = \exp(-(x-x')^2/(2\ell^2))$$

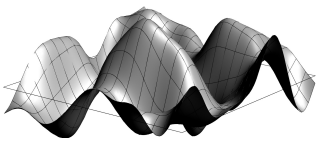


Multi-dimensional Regression

- I've shown a 1-D regression example because it is easy to visualise
- This might be used with a time series
- The much more typical situation in machine learning is for $\boldsymbol{x}$ to have many features so we are doing multi-dimensional regression
- Gaussian process inference were first used in spatial problems where it was known as **krigging**
- It was re-invented by the machine learning community who call it Gaussian Processes (GP)

Outline

1. ML with Gaussians
2. Gaussian Processes
3. Bayesian Inference
4. **Hyper-parameters**



Choosing the Correct Covariance Function

- Choosing the correct covariance function is critical
- Most covariance functions include a continuous **hyper-parameter** (e.g. the correlation length ℓ) that we have to choose correctly
- This is typical of many Bayesian problems where we have some set of hyper-parameters, $\boldsymbol{\phi}$, describing the model
- These are different to the normal parameters we learn (e.g. weights $\boldsymbol{w}$ or in GP the functions $f(\boldsymbol{x})$)
- In Bayesian inference we learn the posterior for these normal parameters

$$p(f|\mathcal{D},\boldsymbol{\phi}) = \frac{p(\mathcal{D}|f,\boldsymbol{\phi})p(f|\boldsymbol{\phi})}{p(\mathcal{D}|\boldsymbol{\phi})}$$

Evidence Framework

- The normalisation factor, $p(\mathcal{D}|\boldsymbol{\phi})$ is known as the **marginal likelihood** or **evidence**

$$p(\mathcal{D}|\boldsymbol{\phi}) = \int p(\mathcal{D}|f,\boldsymbol{\phi})p(f|\boldsymbol{\phi})\mathrm{d}f$$

- We can perform a Bayesian calculation at a second level by putting a prior on $\boldsymbol{\phi}$

$$p(\boldsymbol{\phi}|\mathcal{D}) = \frac{p(\mathcal{D}|\boldsymbol{\phi})p(\boldsymbol{\phi})}{p(\mathcal{D})}$$

- From this we can now marginalise out the hyper-parameters

$$p(f|\mathcal{D}) = \int p(f|\mathcal{D},\boldsymbol{\phi})p(\boldsymbol{\phi}|\mathcal{D})\mathrm{d}\boldsymbol{\phi}$$

Maximum-Likelihood-II

- The integral

$$p(f|\mathcal{D}) = \int p(f|\mathcal{D},\boldsymbol{\phi})p(\boldsymbol{\phi}|\mathcal{D})\mathrm{d}\boldsymbol{\phi}$$

usually can't be computed analytically and we have to use Monte Carlo methods (see later lecture)

- An alternative is to use the most likely hyper-parameter
- We can find this by using gradient search of $p(\mathcal{D}|\boldsymbol{\phi})$
- This is sometimes referred to as ML-II
- Normally even this can be difficult, but for GP its not too difficult

Evidence for GP

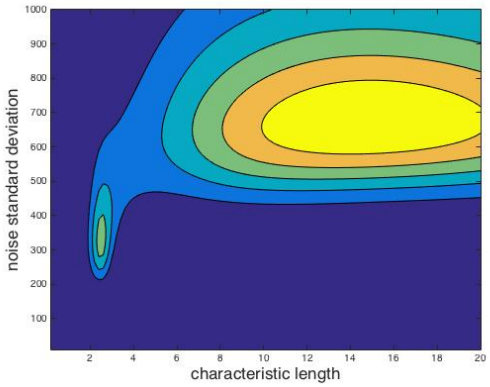
- For GP the (log)-evidence can be computed in closed form

$$\log(p(\mathcal{D}|\boldsymbol{\phi})) = -\frac{1}{2}\boldsymbol{y}^\top(\mathbf{K}+\sigma^2\mathbf{I})\boldsymbol{y} - \frac{1}{2}\log(|\mathbf{K}+\sigma^2\mathbf{I}|) - \frac{m}{2}\log(2\pi)$$

  - ⋆ First term measures goodness of fit
  - ⋆ Second term measure complexity of model
  - ⋆ Last term is a common normalisation constant

- Can efficiently compute derivatives and find best parameters
- Could overfit!

Example (slightly pathological)

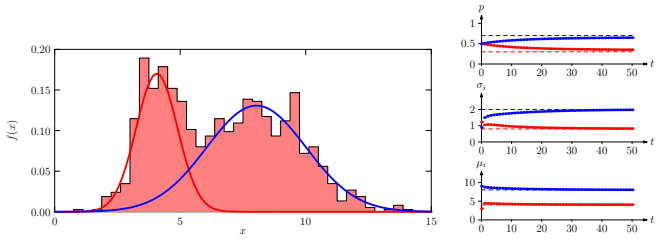


Conclusions

- Gaussian processes are very powerful for regression (and classification?)
- Because all calculations involve Gaussian integrals we can compute everything in closed form
- (Actually its a pain to do the mathematics because you end up working with inverse of matrices)
- Fairly generic (black-box) technique because the prior captures many continuity constraints
- We can use the evidence framework (probability of data) to do model selection and hyper-parameter optimisations

Mathematics of Machine Learning

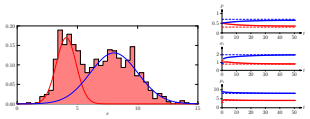
Probabilistic Inference



Generative Models, Hierarchical Models, MAP, Empirical Bayes, Mixture of Gaussians, Expectation Maximisation

Outline

1. **Building Probabilistic Models**
2. Hierarchical Models
3. Solution Strategies
4. Mixtures and EM



Beyond Conjugate Priors

- In the Bayes lecture we chose priors so that the posterior had the same form (conjugate priors)
- Real problems have many interacting random variables, some of which we never observe, and the posterior has no closed form
- This lecture: how to set up a probabilistic model, and simple strategies for using it (MAP, empirical Bayes, EM)
- The MCMC lecture shows how to compute the full posterior

Two Rules

- Sum rule (marginalisation; see the probability lecture)

$$\mathbb{P}(X) = \sum_Y \mathbb{P}(X,Y) \qquad f(x) = \int f(x,y)\mathrm{d}y$$

- Product rule (the definition of conditional probability)

$$\mathbb{P}(X,Y) = \mathbb{P}(X|Y)\mathbb{P}(Y) = \mathbb{P}(Y|X)\mathbb{P}(X)$$

- Bayes' rule is the product rule rearranged: $\mathbb{P}(X|Y) = \mathbb{P}(Y|X)\mathbb{P}(X)/\mathbb{P}(Y)$
- Everything else follows from these two rules

Discriminative and Generative Models

- A **discriminative model** gives the prediction directly, e.g. $\mathbb{P}(C|\boldsymbol{x})$ for a class C given features $\boldsymbol{x}$ (as for the Bayes' optimal discrimination surface in *foundations of machine learning*)
- A **generative model** describes how the features and the output are generated together: $\mathbb{P}(\boldsymbol{X},Y)$
- Generative models are often more natural to think about
- And we can still use them to discriminate

$$\mathbb{P}(Y|\boldsymbol{X}) = \frac{\mathbb{P}(\boldsymbol{X},Y)}{\mathbb{P}(\boldsymbol{X})} = \frac{\mathbb{P}(\boldsymbol{X},Y)}{\sum_Y \mathbb{P}(\boldsymbol{X},Y)}$$

Latent Variables

- Sometimes we have models that involve random variables that we don't observe and we don't care about
- These are called **latent variables**
- They are not observed and not our final target, but they make the model easy to write down
- If we have a latent variable Z and observed variable $\boldsymbol{X}$ and we are predicting a variable Y then we would **marginalise** over the latent variable

$$\mathbb{P}(\boldsymbol{X},Y) = \sum_Z \mathbb{P}(\boldsymbol{X},Y,Z)$$

Generative Stories

- Build a model by writing down how the data could have been generated (a simulator), each line drawing one variable given earlier ones

```
theta = rng.beta(a, b, size=J)     # a rate for each product
k = rng.binomial(n, theta)         # observed positive reviews
```

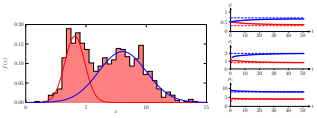
- The joint distribution is the product of the lines (product rule)

$$f(\boldsymbol{\theta},\boldsymbol{k}) = \prod_{j=1}^{J} \mathrm{Beta}(\theta_j|a,b)\,\mathrm{Bin}(k_j|n_j,\theta_j)$$

- **Probabilistic programming languages** (e.g. Stan, PyMC) take such a programme and do the inference for you

Outline

1. Building Probabilistic Models
2. **Hierarchical Models**
3. Solution Strategies
4. Mixtures and EM



Rating Products

- A shop lists products with the fraction of positive reviews: which is better?

	positive	fraction
Product A	3 of 3	100%
Product B	1840 of 2000	92%

- Ranking by the fraction puts every product with a handful of good reviews at the top
- Intuitively we should trust A's 100% less, and something like "most products are rated about 80%" should pull it down: a hierarchical model makes this precise

A Model for Ratings

- Product j has an unknown probability θ_j that a review is positive; k_j of its n_j reviews are positive

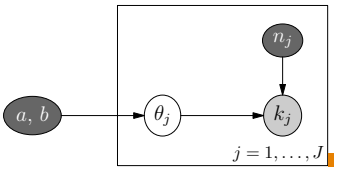
$$k_j \sim \mathrm{Bin}(n_j, \theta_j)$$

- The products are different but similar: their θ_j come from a common population

$$\theta_j \sim \mathrm{Beta}(a, b)$$

- The **hyperparameters** a, b describe the population: $a/(a+b)$ is the typical rate and $a+b$ measures how alike the products are
- The model has levels: population → product → reviews (a **hierarchical model**)

Graphical Models



- Nodes are variables (dark: fixed; shaded: observed; white: latent); arrows point from the variables each one is drawn from
- A box (a **plate**) means "repeat for each j"
- The joint distribution is the product over nodes of each node given its parents
- Diagrams like this are a good way to design and explain models

One Product at a Time

- If we knew a and b, each product is a beta-binomial problem from the Bayes lecture

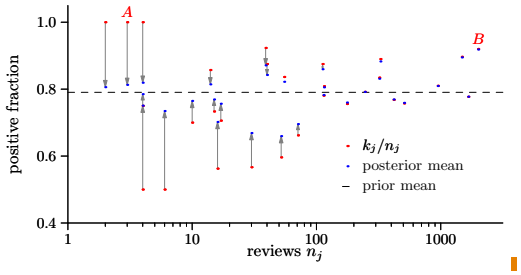
$$f(\theta_j|k_j) = \mathrm{Beta}(\theta_j|a+k_j, b+n_j-k_j)$$

- The posterior mean is a weighted average: the prior counts as $a+b$ extra reviews

$$\mathbb{E}[\theta_j|k_j] = \frac{a+k_j}{a+b+n_j} = \frac{a+b}{a+b+n_j}\frac{a}{a+b} + \frac{n_j}{a+b+n_j}\frac{k_j}{n_j}$$

- With $a=20$, $b=5$ (typical rate 0.8, worth 25 reviews): A gets $23/28 = 0.82$ and B gets $1860/2025 = 0.92$: B is better

Shrinkage



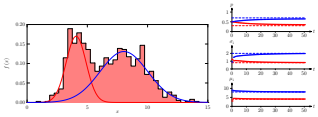
Few reviews: pulled (**shrunk**) to the typical rate; many: barely move. Products **borrow strength** from each other.

Partial Pooling

- Two extremes
  - ⋆ no pooling: estimate each product alone (k_j/n_j)—noisy when n_j is small
  - ⋆ complete pooling: assume all products are the same—ignores real differences
- The hierarchical model sits in between, and the data decide where (through $a+b$)
- The same idea appears everywhere: schools, hospitals, adverts, players, users of a recommender
- But we don't know a and b, and with them unknown the posterior $f(a,b,\boldsymbol{\theta}|\boldsymbol{k})$ has no closed form

Outline

1. Building Probabilistic Models
2. Hierarchical Models
3. **Solution Strategies**
4. Mixtures and EM



Full Bayesian Inference

- Bayes' rule gives the posterior over the parameters

$$\mathbb{P}(\boldsymbol{\Theta}|\mathcal{D}) = \frac{\mathbb{P}(\mathcal{D}|\boldsymbol{\Theta})\mathbb{P}(\boldsymbol{\Theta})}{\mathbb{P}(\mathcal{D})}$$

- It describes its own uncertainty and is much less prone to over-fitting (we average over parameters rather than choosing the best)
- But the posterior is rarely a nice function, and its normalisation, the *evidence* $\mathbb{P}(\mathcal{D}) = \sum_{\boldsymbol{\Theta}} \mathbb{P}(\mathcal{D}|\boldsymbol{\Theta})\mathbb{P}(\boldsymbol{\Theta})$, is a sum over far too many values
- Monte Carlo methods (the MCMC lecture) can sample from the posterior without computing $\mathbb{P}(\mathcal{D})$

Maximum A Posteriori (MAP) Solution

- Work around: the mode of the posterior (no need for $f(\mathcal{D})$)

$$\boldsymbol{\Theta}_{\text{MAP}} = \operatorname*{argmax}_{\boldsymbol{\Theta}} f(\mathcal{D}|\boldsymbol{\Theta}) f(\boldsymbol{\Theta}) = \operatorname*{argmax}_{\boldsymbol{\Theta}} \log(f(\mathcal{D}|\boldsymbol{\Theta})) + \log(f(\boldsymbol{\Theta}))$$

- $-\log(f(\boldsymbol{\Theta}))$ acts as a regulariser: a Gaussian prior on weights gives $\|\boldsymbol{w}\|^2$, i.e. ridge regression or weight decay (we use this in the next lecture)
- With a uniform prior MAP is **maximum likelihood**: often sufficient for *government work*
- Neither is Bayesian (despite what you are sometimes told): they can overfit and give no error estimate

Empirical Bayes

- A half-way house: integrate out the many θ_j, optimise the few hyperparameters, then use the conjugate posteriors
- For the ratings model the integral is in closed form (the **beta-binomial**)

$$\begin{aligned}\mathbb{P}(k_j|a,b) &= \int_0^1 \text{Bin}(k_j|n_j,\theta)\,\text{Beta}(\theta|a,b)\,\text{d}\theta \\ &= \binom{n_j}{k_j}\frac{B(a+k_j,b+n_j-k_j)}{B(a,b)}\end{aligned}$$

- Choose a, b to maximise the evidence $\prod_j \mathbb{P}(k_j|a,b)$ (as for the GP hyperparameters): for the data in the figure $a = 19.9$, $b = 5.3$
- Cheap and often good, but ignores uncertainty in a, b

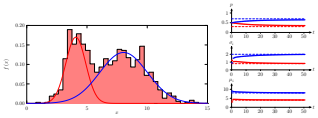
A Ladder of Strategies

Strategy	What you get	Cost
ML	a point estimate	optimiser
MAP	a regularised point	optimiser
Empirical Bayes	posteriors given a,b	an integral
EM	ML/MAP with latents	iterate
Full Bayes	all the uncertainty	MCMC

- Choose the cheapest that answers your question
- The rest of this lecture: EM

Outline

1. Building Probabilistic Models
2. Hierarchical Models
3. Solution Strategies
4. **Mixtures and EM**



You Already Know EM

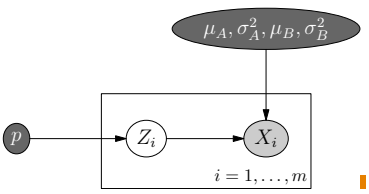
- k-means clustering
  1. assign each point to its nearest centre
  2. move each centre to the mean of its points
  3. repeat
- Step 1 guesses the hidden cluster labels; step 2 fits the parameters as if those guesses were right
- The EM algorithm does the same with **probabilities** of belonging to each cluster
- It comes from a probabilistic model: a mixture

Mixture of Gaussians

- Suppose we detect two types of short-lived particle, A or B, and measure each particle's mass, X_i, with a noisy detector, but not its type
- Let $Z_i \in \{0,1\}$ indicate that particle i is of type A, with $\mathbb{P}(Z_i = 1) = p$
- X_i is normal with unknown means and variances, so the parameters are $\boldsymbol{\Theta} = (\mu_A,\sigma_A^2,\mu_B,\sigma_B^2,p)$
- The probability of X_i is given by

$$f(X_i|Z_i,\boldsymbol{\Theta}) = Z_i\mathcal{N}\big(X_i\big|\mu_A,\sigma_A^2\big) + (1-Z_i)\mathcal{N}\big(X_i\big|\mu_B,\sigma_B^2\big)$$

Mixture as a Graphical Model

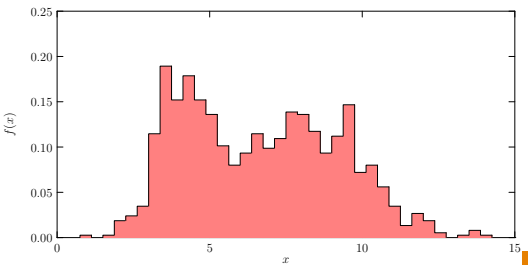


- For each data point draw a type Z_i, then a mass X_i from that type's normal distribution
- Z_i is latent: we must marginalise it out
- The parameters (dark) are shared by all data points, so they sit outside the plate

Data

- Note that

$$\begin{aligned}f(X_i|\boldsymbol{\Theta}) &= \sum_{Z_i\in\{0,1\}} f(X_i,Z_i|\boldsymbol{\Theta}) = \sum_{Z_i\in\{0,1\}} f(X_i|Z_i,\boldsymbol{\Theta})\mathbb{P}(Z_i) \\ &= \mathbb{E}_{Z_i}[f(X_i|Z_i,\boldsymbol{\Theta})] = p\mathcal{N}\big(X_i\big|\mu_A,\sigma_A^2\big) + (1-p)\mathcal{N}\big(X_i\big|\mu_B,\sigma_B^2\big)\end{aligned}$$



Maximum Likelihood

- A full Bayesian treatment needs priors on $\boldsymbol{\Theta}$ and would give a distribution over parameters
- Often we only want a reasonable estimate for some of our parameters (e.g. the masses μ_A and μ_B)
- A reasonable approach is to seek those parameters that maximise the likelihood of our observed data (assumed independent)

$$f(\mathcal{D}|\boldsymbol{\Theta}) = \prod_{X \in \mathcal{D}} f(X|\boldsymbol{\Theta})$$

- Strictly, we seek a good **local** maximum: putting $\mu_A = X_1$ and letting $\sigma_A \to 0$ makes the likelihood infinite

EM Algorithm

- If we knew each particle's type, Z_i, maximising the likelihood would be easy, but estimating $\mathbb{P}(Z_i = 1|X_i)$ needs the parameters
- The **expectation-maximisation** (EM) algorithm iterates: maximise the log-likelihood of the data **and** latent variables, averaged over the latent variables using the current parameters

$$\boldsymbol{\Theta}^{(t+1)} = \operatorname*{argmax}_{\boldsymbol{\Theta}} \sum_{\boldsymbol{Z}} \mathbb{P}\Big(\boldsymbol{Z}\Big|\mathcal{D},\boldsymbol{\Theta}^{(t)}\Big) \log\big(f(\mathcal{D},\boldsymbol{Z}|\boldsymbol{\Theta})\big)$$

- E-step: compute $\mathbb{P}\big(\boldsymbol{Z}|\mathcal{D},\boldsymbol{\Theta}^{(t)}\big)$; M-step: maximise
- No step decreases the likelihood (the proof is in the notes)

E-Step: Responsibilities

- We need the distribution of the latent variables conditioned on the data and current estimated parameters
- For our problem

$$\mathbb{P}\Big(\boldsymbol{Z}\Big|\mathcal{D},\boldsymbol{\Theta}^{(t)}\Big) = \prod_{i=1}^{m} \mathbb{P}\Big(Z_i\Big|X_i,\boldsymbol{\Theta}^{(t)}\Big)$$

where, by Bayes' rule, the **responsibility** of type A for point i is

$$\gamma_i^{(t)} = \mathbb{P}\Big(Z_i = 1\Big|X_i,\boldsymbol{\Theta}^{(t)}\Big) = \frac{p^{(t)}\mathcal{N}\Big(X_i|\mu_A^{(t)},\sigma_A^{2(t)}\Big)}{p^{(t)}\mathcal{N}\Big(X_i|\mu_A^{(t)},\sigma_A^{2(t)}\Big) + (1-p^{(t)})\mathcal{N}\Big(X_i|\mu_B^{(t)},\sigma_B^{2(t)}\Big)}$$

and $\mathbb{P}\big(Z_i = 0\big|X_i,\boldsymbol{\Theta}^{(t)}\big) = 1 - \gamma_i^{(t)}$

M-Step

- The data points are independent, so

$$Q(\boldsymbol{\Theta}|\boldsymbol{\Theta}^{(t)}) = \sum_{i=1}^{m}\Big[\gamma_i^{(t)}\Big(\log(p) - \frac{(X_i-\mu_A)^2}{2\sigma_A^2} - \tfrac{1}{2}\log\big(2\pi\sigma_A^2\big)\Big) + (1-\gamma_i^{(t)})\Big(\log(1-p) - \frac{(X_i-\mu_B)^2}{2\sigma_B^2} - \tfrac{1}{2}\log\big(2\pi\sigma_B^2\big)\Big)\Big]$$

- For example, only the first bracket involves μ_A

$$\frac{\partial Q}{\partial \mu_A} = \sum_{i=1}^{m} \gamma_i^{(t)} \frac{X_i - \mu_A}{\sigma_A^2} = 0 \quad \Rightarrow \quad \mu_A^{(t+1)} = \frac{\sum_{i=1}^m \gamma_i^{(t)} X_i}{\sum_{i=1}^m \gamma_i^{(t)}}$$

- The other parameters work the same way (see the notes)

Update Equations

- Fraction of type A: the average responsibility

$$p^{(t+1)} = \frac{1}{m}\sum_{i=1}^{m} \gamma_i^{(t)}$$

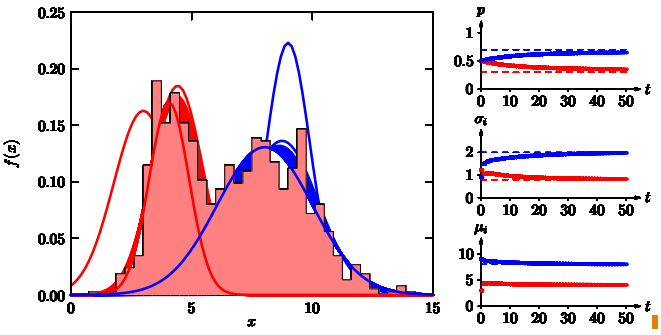
- Means: weighted averages of the data

$$\mu_A^{(t+1)} = \frac{\sum_{i=1}^m \gamma_i^{(t)} X_i}{\sum_{i=1}^m \gamma_i^{(t)}}, \qquad \mu_B^{(t+1)} = \frac{\sum_{i=1}^m (1-\gamma_i^{(t)}) X_i}{\sum_{i=1}^m (1-\gamma_i^{(t)})}$$

- Variances: weighted variances (if every γ_i were 0 or 1, these would all be the ordinary group statistics)

$$\sigma_A^{2(t+1)} = \frac{\sum_{i=1}^m \gamma_i^{(t)} (X_i - \mu_A^{(t+1)})^2}{\sum_{i=1}^m \gamma_i^{(t)}}$$

Example



EM in Practice

- With equal, fixed variances and $\sigma \to 0$ the responsibilities become 0 or 1: EM becomes k-means
- EM finds a local maximum: run it from several starts
- A component can collapse onto a single point ($\sigma \to 0$, likelihood $\to \infty$); a prior on σ (i.e. MAP) prevents this
- Convergence can be slow when the components overlap (the latent variables are very uncertain)
- Simple updates, no step size, and it works for any model with latent variables (missing data, hidden Markov models, . . .)

Summary

- Build a model by writing down how the data were generated: a generative story or graphical model
- Hierarchical models let similar groups share strength (shrinkage)
- Beyond conjugate priors there is a ladder of strategies: ML, MAP, empirical Bayes, EM, full Bayes with MCMC
- The EM algorithm maximises likelihoods (or MAP) when there are latent variables
- It gives nice update equations, but finds local optima and can converge slowly

Mathematics of Machine Learning

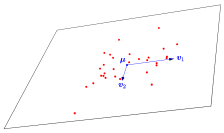
Matrix Factorisation

Name	Titanic	Dr No	The Matrix	Spartacus	Ten
Alice	2	Ø	4	5	2
Bob	5	3	Ø	3	Ø
Chas	Ø	2	3	5	3

Matrix completion, low-rank approximation, recommender systems, alternating least squares, non-negative matrix factorisation

Outline

1. **Missing Data and Recommenders**
2. Low-Rank Approximation
3. Probabilistic Approach
4. Non-Negative Matrix Factorisation



Matrix Completion

- It is extremely common to be given a matrix of data with missing entries (particularly when the data comes from humans filling in forms)
- Most learning machines need a complete set of data, so we either throw away examples or features with missing values (sometimes leaving nothing) or do our best to infer them
- Inferring them only works if there is structure in the matrix that we can exploit
- The structure we use in this lecture: **structured data typically lies close to a low-dimensional subspace** (the idea behind PCA and the SVD)

Recommender Systems

- The best-known example: recommending items to users, which can be financially important
  - Recommendations are reported to drive a large share of Amazon's sales
  - Netflix (then a DVD rental company, now a streaming service) ran a $1 million competition (2006–09) to improve the quality of its recommendations
- **Content-based filtering** uses (often text) features to find similar items; **collaborative filtering** uses ratings supplied (implicitly or explicitly) by users
- We concentrate on collaborative filtering

Recommending Movies

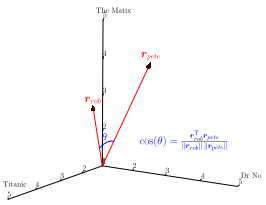
- The classic example is to recommend a movie given some ratings from users

Name	Titanic	Dr No	The Matrix	Spartacus	Ten
Alice	2	Ø	4	5	2
Bob	5	3	Ø	3	Ø
Chas	Ø	2	3	5	3

- Ø means there are no ratings
- The task is to predict the unknown ratings and make a recommendation
- Typically data sets are huge and very sparse

Classic Collaborative Filtering

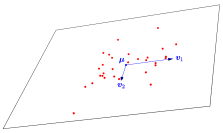
- The classic collaborative filtering technique is to find similar users (those giving similar ratings)
- Use the recommendation of similar users weighted by their similarity
- Can also recommend items that are similar (i.e. items that have received similar ratings) to those that a user rates highly



$$\hat{R}_{ui} = \frac{\sum_{u'} \text{sim}(u,u') R_{u'i}}{\sum_{u'} \text{sim}(u,u')}$$

Outline

1. Missing Data and Recommenders
2. **Low-Rank Approximation**
3. Probabilistic Approach
4. Non-Negative Matrix Factorisation



Matrix Completion

- A different (often better) method is to think of recommendation as a matrix completion task

Name	Titanic	Dr No	The Matrix	Spartacus	Ten
Alice	2	Ø	4	5	2
Bob	5	3	Ø	3	Ø
Chas	Ø	2	3	5	3

- Complete the matrix

$$\mathbf{R} = \begin{pmatrix} 2 & & 4 & 5 & 2 \\ 5 & 3 & & 3 & \\ & 2 & 3 & 5 & 3 \end{pmatrix}$$

Replace by Means

- The simplest guess uses the means, column-wise and row-wise

$$R_{ui} \approx \bar{R}_u + \bar{R}_i - \bar{R}$$

where

  - $\bar{R}_u$ is the mean rating of user u
  - $\bar{R}_i$ is the mean rating for item i
  - $\bar{R}$ is the mean overall rating
- Gets the means right and gives a reasonable rating
- (For general missing data: replace by the feature's mean)
- But this ignores structure: similar users give similar ratings

Best Low-Rank Approximation

- Recall the SVD of a complete matrix, with $s_1 \geq s_2 \geq \cdots \geq 0$

$$\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T} = \sum_k s_k \boldsymbol{u}_k \boldsymbol{v}_k^\mathsf{T}$$

- Keeping the K largest singular values gives a rank-K matrix $\mathbf{X}_K = \sum_{k=1}^K s_k \boldsymbol{u}_k \boldsymbol{v}_k^\mathsf{T}$
- **Eckart–Young theorem**: $\mathbf{X}_K$ is the best rank-K approximation to $\mathbf{X}$, with error $\|\mathbf{X} - \mathbf{X}_K\|_F^2 = \sum_{k>K} s_k^2$ (also best in the spectral norm, with error s_{K+1})
- Data close to a K-dimensional subspace has small discarded s_k (this is effectively PCA)

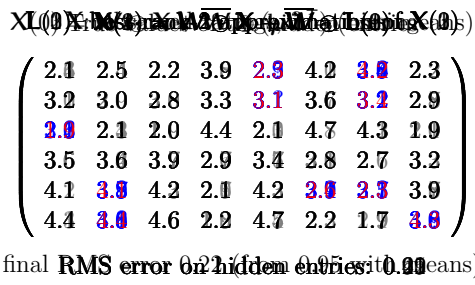
Completing a Matrix with the SVD

- The SVD needs a complete matrix, so we iterate
  1. Fill in the missing entries (e.g. with the user means)
  2. Compute the best rank-K approximation $\mathbf{L}$ of the filled-in matrix
  3. Replace the missing entries (only) by the values in $\mathbf{L}$
  4. Repeat from 2
- With a mask $\boldsymbol{W}$ ($W_{ij} = 1$ where X_{ij} is observed, 0 otherwise) and $\odot$ the element-by-element product

$$\mathbf{X}(t+1) = \boldsymbol{W} \odot \mathbf{X} + \overline{\boldsymbol{W}} \odot \mathbf{L}(t)$$

where $\overline{\boldsymbol{W}} = 1 - \boldsymbol{W}$

SVD in Operation



final RMS error on hidden entries: 0.20

Low-Rank Approximation

- More directly, approximate an $m \times n$ matrix $\mathbf{X}$ by

$$\mathbf{X} \approx \mathbf{A}\,\mathbf{B}$$

where $\mathbf{A}$ is $m \times K$ and $\mathbf{B}$ is $K \times n$

- $\mathbf{AB}$ has *rank* at most K
- We replace $m \times n$ elements by $(m+n) \times K$ free parameters
- K controls the complexity of the model: large K fits the data better but may over-fit
- Choose K by cross-validation (hide some of the observed entries and see how well we predict them)

Low-Rank Updates: LoRA

- **LoRA** (low-rank adaptation, Hu et al. 2021) uses the same idea to fine-tune a large network: a pretrained weight matrix $\boldsymbol{W}_0$ ($m \times n$) becomes $\boldsymbol{W}_0 + \Delta\boldsymbol{W}$
- LoRA freezes $\boldsymbol{W}_0$ and learns only a low-rank change

$$\Delta\boldsymbol{W} = \mathbf{AB}, \qquad \mathbf{A}\ (m \times K),\ \mathbf{B}\ (K \times n),\ K \ll m,n$$

with one factor starting at zero (so training starts from $\boldsymbol{W}_0$)
- For $m = n = 4096$ and $K = 8$: $(m+n)K \approx 6.6 \times 10^4$ parameters to train instead of $mn \approx 1.7 \times 10^7$ (0.4%)
- The bet: the change a new task needs is close to low rank; each task stores only its own $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{AB}$ can be added into $\boldsymbol{W}_0$ afterwards

Least-Squares

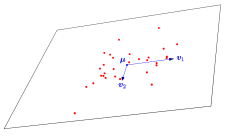
- Assume we have elements of $\mathbf{X}$ only at the locations $\mathcal{D}$
- We can seek a least-squares solution for $\mathbf{A}$ and $\mathbf{B}$

$$(\mathbf{A},\mathbf{B}) = \underset{(\mathbf{A},\mathbf{B})}{\operatorname{argmin}} \sum_{(i,j)\in\mathcal{D}} \left(X_{ij} - \sum_{k=1}^K A_{ik} B_{kj} \right)^2$$

- OK, but it can over-fit
- $\mathbf{A}$ and $\mathbf{B}$ are never unique: $\mathbf{AB} = (\mathbf{AG})(\mathbf{G}^{-1}\mathbf{B})$ for any invertible $K \times K$ matrix $\mathbf{G}$

Outline

1. Missing Data and Recommenders
2. Low-Rank Approximation
3. **Probabilistic Approach**
4. Non-Negative Matrix Factorisation



Bayesian Approach

- We can use a Bayesian approach to infer $\mathbf{A}$ and $\mathbf{B}$
- The likelihood is

$$\mathbb{P}(X_{ij}|\mathbf{A},\mathbf{B}) = \mathcal{N}\left(X_{ij}\middle|\sum_{k=1}^{K} A_{ik}B_{kj},\sigma^2\right)$$
$$\mathcal{N}(x|\mu,\sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma}\mathrm{e}^{-(x-\mu)^2/(2\sigma^2)}$$

- σ measures the natural error in the ratings

Priors

- We also need priors

$$\mathbb{P}(\mathbf{A}) = \prod_{i,k}\mathcal{N}(A_{ik}|0,\sigma_A^2), \qquad \mathbb{P}(\mathbf{B}) = \prod_{k,j}\mathcal{N}(B_{kj}|0,\sigma_B^2)$$

- σ_A^2 and σ_B^2 explain the variation in the original matrix $\mathbf{X}$
- Note that

$$\mathbb{E}\left[\sum_{k=1}^{K} A_{ik}B_{kj}\right] = 0, \qquad \mathbb{V}\mathrm{ar}\left(\sum_{k=1}^{K} A_{ik}B_{kj}\right) = K\sigma_A^2\sigma_B^2$$

  which should be equal to the variance in X_{ij}

MAP Solution

- Calculating the posterior probability for $\mathbf{A}$ and $\mathbf{B}$ is intractable
- We can, however, compute the *maximum a posteriori* (MAP) solution, as in the last lecture

$$\begin{aligned}\mathcal{E} &= \sum_{(i,j)\in\mathcal{D}} \log(\mathbb{P}(X_{ij}|\mathbf{A},\mathbf{B})) + \log(\mathbb{P}(\mathbf{A})) + \log(\mathbb{P}(\mathbf{B})) + C\\ &= -\frac{1}{2\sigma^2}\sum_{(i,j)\in\mathcal{D}}\left(X_{ij} - \sum_{k=1}^{K} A_{ik}B_{kj}\right)^2 - \frac{1}{2\sigma_A^2}\sum_{i,k} A_{ik}^2\\ &\quad - \frac{1}{2\sigma_B^2}\sum_{k,j} B_{kj}^2 + C'\end{aligned}$$

Iterative Learning

- We can maximise the log-posterior by first maximising the log-posterior with respect to $\mathbf{A}$ keeping $\mathbf{B}$ fixed

$$\frac{\partial\mathcal{E}}{\partial A_{ik}} = \frac{1}{\sigma^2}\sum_{j|(i,j)\in\mathcal{D}}\left(X_{ij} - \sum_{l=1}^{K} A_{il}B_{lj}\right)B_{kj} - \frac{1}{\sigma_A^2}A_{ik} = 0$$

- This is equivalent to

$$\sum_{l=1}^{K}\left(\frac{\sigma^2}{\sigma_A^2}\delta_{k,l} + \sum_{j|(i,j)\in\mathcal{D}} B_{lj}B_{kj}\right)A_{il} = \sum_{j|(i,j)\in\mathcal{D}} B_{kj}X_{ij}$$

Solving for A

- We got

$$\sum_{l=1}^{K}\left(\frac{\sigma^2}{\sigma_A^2}\delta_{k,l} + \sum_{j|(i,j)\in\mathcal{D}} B_{lj}B_{kj}\right)A_{il} = \sum_{j|(i,j)\in\mathcal{D}} B_{kj}X_{ij}$$

- Which has the form $\mathbf{M}^i\boldsymbol{a}^i = \boldsymbol{c}^i$

$$M_{kl}^i = \frac{\sigma^2}{\sigma_A^2}\delta_{k,l} + \sum_{j|(i,j)\in\mathcal{D}} B_{lj}B_{kj}, \qquad a_l^i = A_{il}, \qquad c_k^i = \sum_{j|(i,j)\in\mathcal{D}} B_{kj}X_{ij}$$

- Solution is $\boldsymbol{a}^i = (\mathbf{M}^i)^{-1}\boldsymbol{c}^i$ (one $K\times K$ solve for each row i)

Each Step is Ridge Regression

- Write $\boldsymbol{b}_j$ for column j of $\mathbf{B}$ and $\boldsymbol{a}_i$ for row i of $\mathbf{A}$ (as a column vector)
- With $\mathbf{B}$ fixed, row i of $\mathbf{A}$ minimises

$$\sum_{j|(i,j)\in\mathcal{D}}\left(X_{ij} - \boldsymbol{a}_i^\mathsf{T}\boldsymbol{b}_j\right)^2 + \eta\|\boldsymbol{a}_i\|^2, \qquad \eta = \frac{\sigma^2}{\sigma_A^2}$$

- This is ridge regression: features $\boldsymbol{b}_j$, targets X_{ij}, regularisation strength η
- Its solution $\boldsymbol{a}_i = \left(\eta\mathbf{I} + \sum_j \boldsymbol{b}_j\boldsymbol{b}_j^\mathsf{T}\right)^{-1}\sum_j \boldsymbol{b}_j X_{ij}$ is exactly $(\mathbf{M}^i)^{-1}\boldsymbol{c}^i$
- Alternating between $\mathbf{A}$ and $\mathbf{B}$ is called **alternating least squares** (ALS): each step is convex, even though the whole problem is not

Converging to a Solution

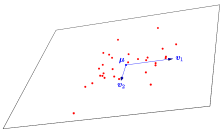
- Then solve for $\mathbf{B}$ (almost identical to solving for $\mathbf{A}$)
- Iterate between $\mathbf{A}$ and $\mathbf{B}$ until they converge
- No step can decrease the log-posterior, so it converges, usually rapidly (to a local optimum: the full problem is not convex)
- Used to predict an X_{ij} where $(i,j)\notin\mathcal{D}$
- Matrix factorisation was at the heart of the Netflix Prize winners and is still a strong baseline for recommenders

Embeddings

- Row i of $\mathbf{A}$ is a K-dimensional vector describing user i; column j of $\mathbf{B}$ describes item j
- The predicted rating is their inner product, $\boldsymbol{a}_i^\mathsf{T}\boldsymbol{b}_j$
- Items with similar vectors are rated alike: we have learnt an **embedding** of users and items
- Modern recommenders learn embeddings with neural networks, mostly from **implicit feedback** (clicks, views, purchases): a missing entry is then weak evidence of not liking, not simply unknown
- Matrix completion is also used to fill in missing survey, sensor and medical data

Outline

1. Missing Data and Recommenders
2. Low-Rank Approximation
3. Probabilistic Approach
4. **Non-Negative Matrix Factorisation**



Matrix Factorisation

- Although we have treated matrix factorisation as a matrix completion task we can also think of it as a data modelling task
- Consider a set of books in mathematics, physics and biology
- We can build a matrix of books (columns) versus words (rows), with elements being the word count
- Can use the SVD, taking $\mathbf{A}$ as the first three columns of $\mathbf{US}$ and $\mathbf{B}$ as the first three rows of $\mathbf{V}^\mathsf{T}$

SVD

	Euclid	Gauss	Feynman	Newton	Darwin	Mendel
theorem	51	101	5	10	3	1
proof	51	103	2	8	0	0
sum	22	51	60	72	5	7
vector	0	70	90	50	3	3
wave	0	1	51	92	2	0
species	0	4	5	1	161	89
fitness	0	1	3	1	35	2

$\approx$

	1	2	3
theorem	9.0	-2.2	-6.6
proof	8.8	-2.5	-7.0
sum	10.2	-1.7	3.4
vector	11.2	-2.3	3.4
wave	6.5	-1.3	7.7
species	5.2	17.7	-0.5
fitness	1.0	3.0	0.0

	Euclid	Gauss	Feynman	Newton	Darwin	Mendel
1	2.5	6.9	4.5	4.5	2.2	1.3
2	-0.8	-1.9	-0.9	-1.1	8.5	4.5
3	-3.5	-5.6	4.9	5.7	-0.3	-0.1

- Best rank-3 approximation, but the factors mix signs: hard to interpret

Non-Negative Matrix Factorisation

- When $\mathbf{X}$ is non-negative we can get a sparse model by minimising $E(\mathbf{A},\mathbf{B}) = \|\mathbf{X} - \mathbf{AB}\|_F^2$ with all elements of $\mathbf{A}$ and $\mathbf{B}$ non-negative
- Projected gradient descent: take a gradient step, then set negative elements to zero
- Or Lee and Seung's **multiplicative updates** ($\oslash$ is element-by-element division)

$$\mathbf{B} \leftarrow \mathbf{B} \odot (\mathbf{A}^\mathsf{T}\mathbf{X}) \oslash (\mathbf{A}^\mathsf{T}\mathbf{AB}), \quad \mathbf{A} \leftarrow \mathbf{A} \odot (\mathbf{XB}^\mathsf{T}) \oslash (\mathbf{ABB}^\mathsf{T})$$

which never make an element negative

Non-Negative Matrix Factorisation

	Euclid	Gauss	Feynman	Newton	Darwin	Mendel
theorem	51	101	5	10	3	1
proof	51	103	2	8	0	0
sum	22	51	60	72	5	7
vector	0	70	90	50	3	3
wave	0	1	51	92	2	0
species	0	4	5	1	161	89
fitness	0	1	3	1	35	2

$\approx$

	1	2	3
theorem	10.3	0.3	0.2
proof	10.5	0.1	0.0
sum	4.4	6.7	0.6
vector	5.3	7.1	0.4
wave	0.0	7.5	0.0
species	0.0	0.0	16.2
fitness	0.0	0.1	2.8

	Euclid	Gauss	Feynman	Newton	Darwin	Mendel
1	4.4	10.0	0.6	0.2	0.0	0.0
2	0.0	1.1	9.1	10.0	0.0	0.2
3	0.0	0.3	0.3	0.0	10.0	5.4

- The three factors pick out mathematics, physics and biology

Non-Negative Matrix Factorisation: Caveats

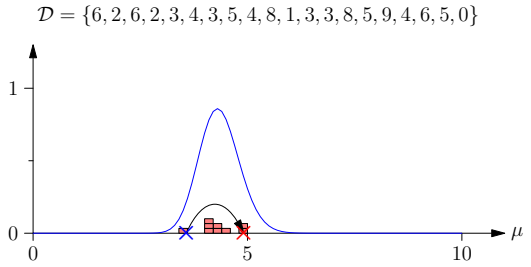
- NMF is very popular because its factors are often easier to interpret than those of the SVD
- But
  - ⋆ It cannot fit better than the SVD: by Eckart–Young the truncated SVD is the best rank-K fit (here relative errors 0.164 against 0.158)
  - ⋆ The problem is not convex: the answer depends on the starting point and on K
  - ⋆ The factors are not unique, and a "parts-based" reading of them is not guaranteed
- Always compare against simple baselines (e.g. the SVD or PCA)

Conclusion

- Structured data typically lies close to a low-dimensional subspace, and we can exploit this
- Matrix factorisation has many applications including recommender systems, data completion, and data modelling
- The truncated SVD gives the best low-rank approximation; iterating it fills in missing entries
- A Bayesian (MAP) approach helps with over-fitting, and each step of the resulting ALS is ridge regression
- Non-negative matrix factorisation gives more interpretable, though not necessarily better, factors

Mathematics of Machine Learning

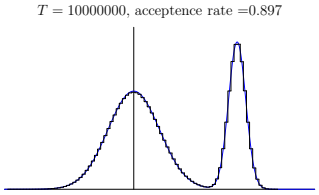
MCMC



Monte Carlo methods, MCMC, Metropolis-Hastings, Gibbs sampling, Hamiltonian Monte Carlo, Convergence diagnostics

Outline

1. **Monte Carlo**
2. MCMC
3. Modern MCMC



Bayesian Inference Gets Hard

- With a simple likelihood (normal, binomial, Poisson, multinomial) a conjugate prior gives a posterior of the same form
- Most real models (e.g. hierarchical models) have no conjugate prior
- We can still compute the top of Bayes' rule

$$f(\boldsymbol{\theta}|\mathcal{D}) = \frac{f(\mathcal{D}|\boldsymbol{\theta})\, f(\boldsymbol{\theta})}{f(\mathcal{D})}$$

- But the evidence $f(\mathcal{D})$ is an integral over all $\boldsymbol{\theta}$, which we usually cannot do: this lecture shows how to draw samples from the posterior anyway

Histograms, Samples and Means

- We could represent our posterior as a histogram, although for multivariate distributions (i.e. when we are modelling more than one variable) a histogram can be unwieldy
- A sample from the posterior distribution is often sufficient e.g. in a topic model of documents a typical set of topics is what we are after
- However, when samples vary a lot, often the most useful quantities are expectations, e.g.

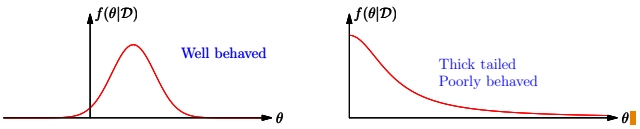
$$\mathbb{E}[\boldsymbol{\Theta}] \qquad \mathbb{E}\left[\Theta_i^2\right] - \mathbb{E}[\Theta_i]^2$$
$$\mathbb{E}[\Theta_i \Theta_j] - \mathbb{E}[\Theta_i]\mathbb{E}[\Theta_j] \qquad \mathbb{E}\left[\boldsymbol{\Theta}\boldsymbol{\Theta}^{\mathsf{T}}\right] - \mathbb{E}[\boldsymbol{\Theta}]\mathbb{E}[\boldsymbol{\Theta}]^{\mathsf{T}}$$

Sample Estimation

- If we can draw independent samples (aka **deviates** or **variates**), $\boldsymbol{\Theta}_i$, from our posterior distribution then we can obtain an estimate of our expectation

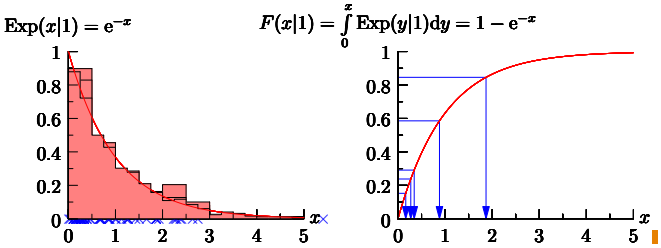
$$\mathbb{E}[g(\boldsymbol{\Theta})] \approx \frac{1}{n}\sum_{i=1}^{n} g(\boldsymbol{\Theta}_i)$$

- Provided our posterior distribution is well behaved (the variance of $g(\boldsymbol{\Theta})$ is finite) the error in our estimate will drop off as $1/\sqrt{n}$



Drawing Random Samples

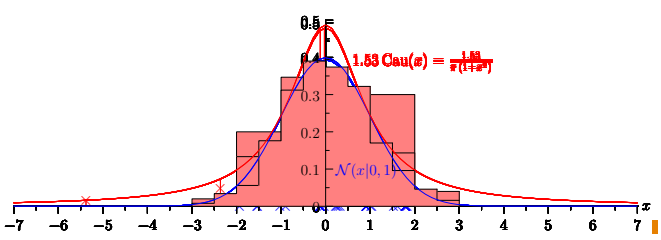
- Computing with (pseudo) random samples drawn from a distribution is known as **Monte Carlo**
- For some very simple distributions we can use the **transformation methods** to transform a uniform distribution



Rejection Method

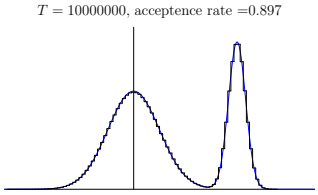
- The transformation method only works when we can easily compute the inverse *cumulative distribution function* (CDF)
- The **rejection method** draws Y from a density g_Y with $cg_Y(x) \geq f_X(x)$ for all x, and accepts it with probability $f_X(Y)/(cg_Y(Y))$
- A deviate is accepted with probability $1/c$, so on average we need c tries for each deviate
- Very general, but for complicated or multivariate distributions it is hard to find a good g_Y and c can be enormous (it typically grows exponentially with the dimension)

Drawing Normal Deviates



Outline

1. Monte Carlo
2. MCMC
3. Modern MCMC



Detailed Balance

- Suppose we have a set of states $\mathcal{S}$ and want to draw samples from a probability distribution $\boldsymbol{\pi} = (\pi_i | i \in \mathcal{S})$
- We invent a dynamical system with a transition probability M_{ij} from state j to state i such that

$$M_{ij}\pi_j = M_{ji}\pi_i$$

- This is known as **detailed balance**
- Summing both sides over j (using $\sum_j M_{ji} = 1$: from i we must go somewhere)

$$\sum_j M_{ij}\pi_j = \sum_j M_{ji}\pi_i = \pi_i \qquad \mathbf{M}\boldsymbol{\pi} = \boldsymbol{\pi}$$

Convergence of MCMC

- Start from a probability vector $\boldsymbol{p}(0) = \sum_i c_i \boldsymbol{v}^{(i)}$, where $\boldsymbol{v}^{(i)}$ are eigenvectors of $\mathbf{M}$ with eigenvalues λ_i
- If I apply $\mathbf{M}$ many times then

$$\boldsymbol{p}(t) = \mathbf{M}^t\boldsymbol{p}(0) = \mathbf{M}^t\sum_i c_i\boldsymbol{v}^{(i)} = \sum_i \lambda_i^t c_i \boldsymbol{v}^{(i)}$$

- As $t \to \infty$ only the eigenvector(s) with the largest $|\lambda_i|$ survive
- Now $\|\mathbf{M}\boldsymbol{v}\|_1 \le \|\mathbf{M}\|_1\|\boldsymbol{v}\|_1 = \|\boldsymbol{v}\|_1$ so $|\lambda_i| \le 1$ and $\lambda = 1$ has eigenvector $\boldsymbol{\pi}$ ($\mathbf{M}$ is known as a **stochastic matrix**)

(Detailed balance makes $\mathbf{M}$ diagonalisable; if every state can be reached and the chain is not periodic, all other $|\lambda_i| < 1$, so $\boldsymbol{p}(t) \to \boldsymbol{\pi}$)

Metropolis Algorithm

- A very easy way to achieve detailed balance: from state j propose a "neighbouring" state i, each neighbour with the same probability q
- Accept the move if $\pi_i \ge \pi_j$; otherwise accept it with probability π_i/π_j
- If $\pi_i \ge \pi_j$ then $M_{ij} = q$ and $M_{ji} = q\pi_j/\pi_i$. Thus

$$M_{ij}\pi_j = q\pi_j \qquad M_{ji}\pi_i = q\frac{\pi_j}{\pi_i}\pi_i = q\pi_j$$

- Note that we require the state i to have the same number of neighbours as state j so that detailed balance is satisfied

Continuous Variables and Metropolis-Hastings

- For continuous $\boldsymbol{\theta}$, detailed balance for the transition density W is

$$W(\boldsymbol{\theta} \to \boldsymbol{\theta}')\pi(\boldsymbol{\theta}) = W(\boldsymbol{\theta}' \to \boldsymbol{\theta})\pi(\boldsymbol{\theta}')$$

- Propose $\boldsymbol{\theta}'$ from a **proposal distribution** $p(\boldsymbol{\theta}'|\boldsymbol{\theta})$ and accept with probability

$$\min\left(1, \frac{p(\boldsymbol{\theta}|\boldsymbol{\theta}')\pi(\boldsymbol{\theta}')}{p(\boldsymbol{\theta}'|\boldsymbol{\theta})\pi(\boldsymbol{\theta})}\right)$$

- For a symmetric proposal (e.g. $\boldsymbol{\theta}' = \boldsymbol{\theta} + s\boldsymbol{\epsilon}$) the p's cancel: the Metropolis algorithm
- Otherwise (e.g. when $\theta_i \ge 0$) we need the correction: **Metropolis-Hastings**

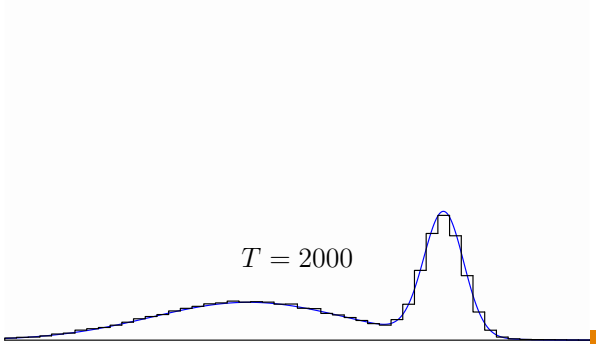
What Makes MCMC Nice

- Because we are free to choose where we move (and choose close by neighbours) $\pi(\boldsymbol{\theta}') \approx \pi(\boldsymbol{\theta})$ so that moves are not too infrequent (but tiny steps explore slowly: the step size is a trade-off)
- Also very importantly the updates depend only on the ratio $\pi(\boldsymbol{\theta}')/\pi(\boldsymbol{\theta})$
- We only need to know our probabilities up to a multiplicative scaling factor
- For sampling from the posterior we only need to know the likelihood and prior $\mathbb{P}(\mathcal{D}|\boldsymbol{\theta})\mathbb{P}(\boldsymbol{\theta})$ (or $f(\mathcal{D}|\boldsymbol{\theta})f(\boldsymbol{\theta})$)
- We don't need to know $\mathbb{P}(\mathcal{D})$ which we generally don't know

What Makes MCMC Nasty

- It can take a long time until our states occur with the probability $\boldsymbol{\pi}$ (i.e. we have forgotten our initial state)
- We don't even know how long we have to wait
- Even when we have reached this *equilibration time* each sample is correlated with the previous sample
- To get a good approximation to the posterior expectation requires running for many times the equilibration time
- Note, if we are just finding sample averages then we can use all samples after equilibrating even if they are not independent

Burn-In



Gibbs Sampling

- With many parameters we can update one component, θ_k, at a time, keeping the others, $\boldsymbol{\theta}_{-k}$, fixed
- **Gibbs sampling** draws the new θ'_k from its conditional distribution $\pi(\theta'_k|\boldsymbol{\theta}_{-k})$
- This is Metropolis-Hastings with proposal $p(\boldsymbol{\theta}'|\boldsymbol{\theta}) = \pi(\theta'_k|\boldsymbol{\theta}_{-k})$, and the acceptance probability is always 1
- Useful when the conditionals are standard distributions (often thanks to conjugate priors), even though the joint posterior is not
- When parameters are strongly correlated, moving one at a time gives small steps and the chain mixes slowly

Traffic Rate

- Consider monitoring the flow of traffic where we have data

$$\mathcal{D} = (N_1, N_2, \ldots, N_n)$$

where N_i is the number of cars that passed on day i

- We assume $N_i \sim \text{Poi}(\mu)$ and want to infer μ
- The Poisson distribution has a gamma conjugate prior
- We don't have any prior knowledge on μ so we use a non-informative prior $\text{Gam}(\mu|0,0) \propto 1/\mu$ (improper: it does not normalise)
- Note that we can solve this problem exactly—however, let's compare with MCMC

Proposal Distribution

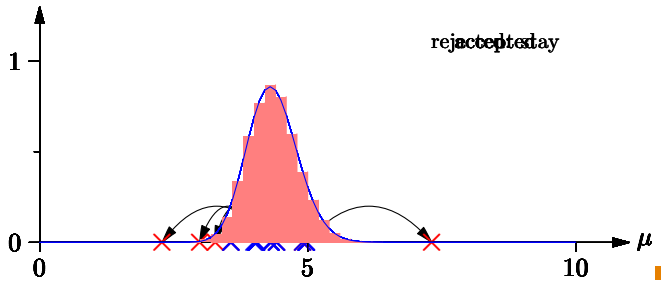
- If we can choose our proposal distribution $p(\mu'|\mu)$ to be close to the posterior distribution then our acceptance rate would be close to 1
- We choose $p(\mu'|\mu) = \text{Gam}(\mu'|\mu^2,\mu)$ which has $\mathbb{E}[\mu'] = \mu$ and variance 1
- We update with probability $\min(1,r)$ where

$$r = \frac{\text{Gam}(\mu|\mu'^2,\mu')\frac{1}{\mu'}\prod_{i=1}^n \text{Poi}(N_i|\mu')}{\text{Gam}(\mu'|\mu^2,\mu)\frac{1}{\mu}\prod_{i=1}^n \text{Poi}(N_i|\mu)}$$

$$= \frac{\mu\text{Gam}(\mu|\mu'^2,\mu')}{\mu'\text{Gam}(\mu'|\mu^2,\mu)} \text{e}^{-n(\mu'-\mu)+\sum_{i=1}^n N_i \log\left(\frac{\mu'}{\mu}\right)}$$

MCMC in Practice

$\mathcal{D} = \{6,2,6,2,3,4,3,5,4,8,1,3,3,8,5,9,4,6,5,0\}$

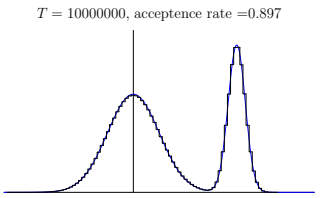


Checking Convergence

- Run several chains from widely different starting points and discard each chain's burn-in
- Plot the traces: after burn-in the chains should overlap and look alike
- $\hat{R}$ compares the variance between chains with the variance within them: close to 1 (say below 1.01) when they agree
- Successive samples are correlated: the **effective sample size** is the number of independent samples they are worth
- Count a state again whenever a move is rejected; libraries do all of this for you

Outline

1. Monte Carlo
2. MCMC
3. **Modern MCMC**



Random Walks Are Slow

- In traditional MCMC choosing a random proposal makes the search into something close to a random walk
- This is often acceptable for a few hundred parameters
- In high dimensional spaces with long shallow valleys it can take an a hugely long time to navigate the high probability landscape
- There is one commonly used trick to speed things up. . .

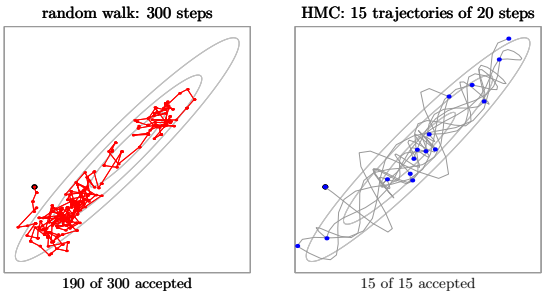
Hamiltonian Monte Carlo

- Treat $U(\boldsymbol{\theta}) = -\log\pi(\boldsymbol{\theta})$ as a landscape (it needs only likelihood $\times$ prior)
- Give the sampler a random momentum $\boldsymbol{p} \sim \mathcal{N}(\mathbf{0},\mathbf{I})$ and let it roll without friction, conserving the energy

$$H(\boldsymbol{\theta},\boldsymbol{p}) = U(\boldsymbol{\theta}) + \tfrac{1}{2}\|\boldsymbol{p}\|^2$$

- It glides along the contours and travels a long way
- We simulate the motion in small steps, which is slightly inexact, so accept the end point with probability $\min(1,\text{e}^{H_{\text{start}}-H_{\text{end}}})$ a Metropolis-Hastings step, so the samples are exact
- H hardly changes, so distant moves are almost always accepted

Comparison of MCMC with HMC



Same start and the same work (300 evaluations) on a strongly correlated Gaussian. The random walk crawls along the ridge; HMC crosses it in every trajectory.

HMC in Practice

- HMC needs the gradient $\nabla \log \pi(\boldsymbol{\theta})$: automatic differentiation (PyTorch, JAX) gives it for free
- Like momentum in optimisation, the gradient says where to go
- It needs a step size and a number of steps: too few and it is a random walk, too many and it wastes time coming back
- The **No-U-Turn Sampler** (NUTS, Hoffman and Gelman 2014) tunes these, stopping when the path's ends stop moving apart
- NUTS is the default in Stan, PyMC and NumPyro: write down the generative story and get samples

The Wider Landscape

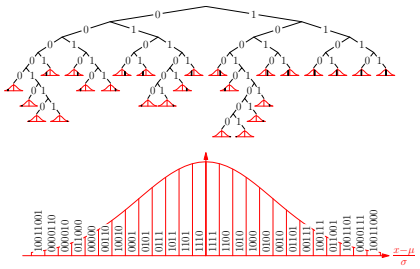
- **Variational inference**: fit an approximate posterior by optimisation (the ELBO of the probabilistic inference lecture) —fast, but approximate
- **Stochastic-gradient MCMC**: noisy gradient steps on minibatches, for very large data sets and Bayesian deep learning
- **Parallel tempering** and **sequential Monte Carlo**: for posteriors with several well-separated peaks
- Diffusion models in generative AI draw samples with Langevin-type dynamics, a close relative of MCMC

Conclusions

- As soon as we use complex models we can no longer compute the posterior in closed form
- MCMC samples from the posterior using only likelihood $\times$ prior
- Simple Metropolis-Hastings is a few lines of code, and on modern computers fast for problems with a modest number of parameters
- For larger problems use HMC (NUTS) through a probabilistic programming language
- Always check convergence: several chains, $\hat{R}$, effective sample size

Mathematics of Machine Learning

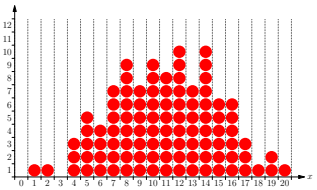
Entropy



Entropy, Coding, Maximum Entropy

Outline

1. Measuring Uncertainty
2. Code Length
3. Maximum Entropy



Measuring Uncertainty

- What is more uncertain: tossing a coin three times or throwing a dice?
- The answer depends on whether you care about the order of the coin tosses
- But, how do we answer such a question?
- Let X be a random variable denoting the possible outcomes
- Interestingly, Shannon entropy gives a precise answer

$$H_X = -\sum_{x\in\mathcal{X}} \mathbb{P}(X=x)\log_2(\mathbb{P}(X=x))$$

Let's Calculate

- For an honest dice $D \in \{1,2,3,4,5,6\}$ and $\mathbb{P}(D=i)=1/6$ so

$$H_D = -\sum_{i=1}^{6}\frac{1}{6}\log_2\left(\frac{1}{6}\right) = -\log_2\left(\frac{1}{6}\right) = \log_2(6) \approx 2.585\text{bits}$$

- For an honest coin where we care about the order so $C \in \{000,001,\ldots,111\}$ the $\mathbb{P}(C=i)=\frac{1}{8}$ and

$$H_C = -\sum_{i=0}^{7}\frac{1}{8}\log_2\left(\frac{1}{8}\right) = -\log_2\left(\frac{1}{8}\right) = \log_2(8) = 3\text{bits}$$

- This clearly makes sense: there are more possible outcomes; all equally likely

Unordered Coin Toss

- What if we don't care about the order of the outcome? Then $\mathbb{P}(HHH)=\mathbb{P}(TTT)=1/8$, while two heads (or two tails) in any order have $\mathbb{P}(HHT)=\mathbb{P}(HTT)=3/8$ so

$$H_U = -\frac{1}{4}\log_2\left(\frac{1}{8}\right) - \frac{3}{4}\log_2\left(\frac{3}{8}\right) \approx 1.811\text{bits}$$

- This seems reasonable, although it is not obvious how you would determine this without using entropy
- But why Shannon entropy?

Additive Entropy

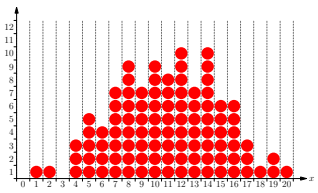
- If H_X and H_Y are the uncertainties of two independent random variables X and Y, what is the uncertainty of the combined event (X,Y)?

$$\begin{aligned}
H_{(X,Y)} &= -\sum_{X,Y}\mathbb{P}(X,Y)\log_2(\mathbb{P}(X,Y)) \\
&= -\sum_{X,Y}\mathbb{P}(X)\mathbb{P}(Y)\log_2(\mathbb{P}(X)\mathbb{P}(Y)) \\
&= -\sum_{X,Y}\mathbb{P}(X)\mathbb{P}(Y)\big(\log_2(\mathbb{P}(X))+\log_2(\mathbb{P}(Y))\big) \\
&= -\sum_{X}\mathbb{P}(X)\log_2(\mathbb{P}(X)) - \sum_{Y}\mathbb{P}(Y)\log_2(\mathbb{P}(Y)) = H_X + H_Y
\end{aligned}$$

- With a few other natural requirements, Shannon's entropy is the only function with this property

Outline

1. Measuring Uncertainty
2. Code Length
3. Maximum Entropy



Why Measure Entropy in Bits

- Suppose we had to communicate a message with 2^n equally likely outputs (e.g. the result of n-coin tosses)
- We can do this with a binary string with n bits (011..0)
- If there were 5 possible outcomes I could do this with 3 bits, but waste 3/8 of the message
- However if we have a batch of 3 independent messages each with 5 outcomes then there are 125 possible outcomes. We could communicate this with 7 bits ($2^7=128$), wasting only 3/128 of the message
- By batching together enough messages with N outcomes then we asymptotically need just $\log_2(N)$ bits

Different Probabilities

- We "showed" that if we had N events, X_i, each with probability, $\mathbb{P}(X_i) = 1/N$, we can code the outcomes with a message of length $-\log_2(\mathbb{P}(X_i)) = \log_2(N)$
- With a shorter message we would not be able to distinguish all possible outcomes from the message
- What happens if some of the outcomes occur with a different probability

X_i:	1	2	3	4	5	6
$p(X_i)$:	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{8}$	$\frac{1}{4}$	$\frac{1}{4}$
Code:	000	001	010	011	10	11
$L = -\log_2(p(X_i))$:	3	3	3	3	2	2

Shannon's Entropy

- If the probabilities are not equal to 2^{-n} we can still find a code with a length very close to $-\log_2(\mathbb{P}(X))$ per message by transmitting a large number of messages
- The length of the message measures the amount of **surprise** on receiving the message
- Shannon's entropy is the minimum expected length of the message to communicate a random variable X

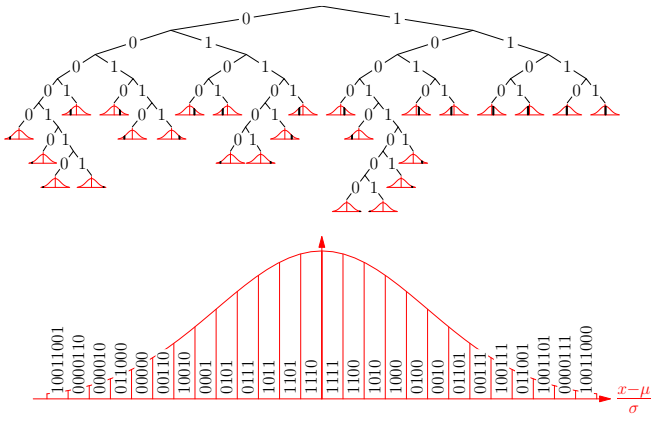
$$H_X = \mathbb{E}_X[-\log_2(\mathbb{P}(X))] = -\sum_{x\in\mathcal{X}} \mathbb{P}(X=x)\log_2(\mathbb{P}(X=x))$$

- The expected length is a measure of the uncertainty (how much information on average we need to convey the outcome)

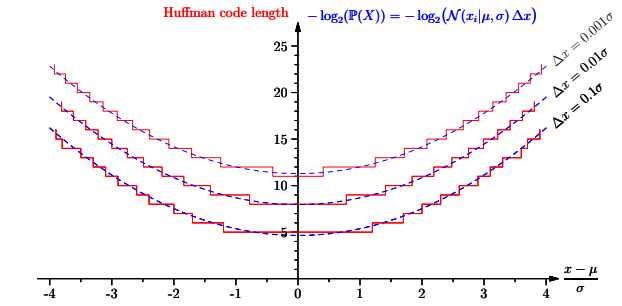
Real Codes

- Those of you with some computer science background will realise that with code words of different lengths we must know where each code word ends
- A **prefix code** (no code word is the start of another) can always be decoded
- An optimal prefix code is Huffman encoding where we associate the leaf of a tree with each code word (its expected length is between H_X and $H_X + 1$ bits)
- Using the tree we can decode any message constructed using the tree
- There is a greedy algorithm for constructing the optimal tree

Coding Normals



Coding Normals to Accuracy Δx



bits and nats

- We have measured entropy in **bits** using

$$H_X = -\sum_{x\in\mathcal{X}} \mathbb{P}(X=x)\log_2\big(\mathbb{P}(X=x)\big)$$

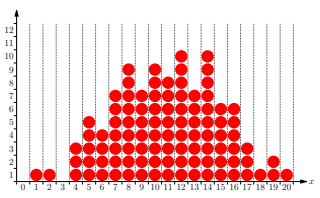
- Sometimes it is easier to use natural logarithms

$$H_X = -\sum_{x\in\mathcal{X}} \mathbb{P}(X=x)\ln\big(\mathbb{P}(X=x)\big)$$

- In this case the entropy is measured in **nats** with 1 nat equal to $\log_2(\mathrm{e})$ bits
- This is often easier when we want to do calculus on entropy

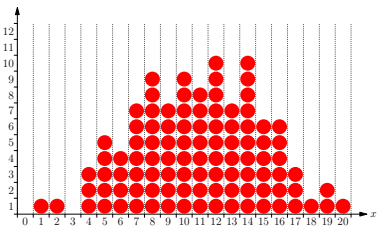
Outline

1. Measuring Uncertainty
2. Code Length
3. **Maximum Entropy**



Number of States

- Suppose I have N balls and put them in K boxes so that n_i balls finish up in box i



- Counting the number of ways to do this

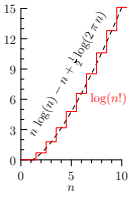
$$W(\boldsymbol{n}) = \frac{N!}{n_1!n_2!\cdots n_K!}$$

Stirling's Approximation

- We can approximate the factorial $n!$ using **Stirling's approximation**

$$n! \approx \sqrt{2\pi n} n^n \mathrm{e}^{-n}$$

$$\log(n!) \approx n\log(n) - n + \frac{1}{2}\log(2\pi n)$$



- Using this in our formula for $W(\boldsymbol{n})$ we have

$$W(\boldsymbol{n}) = \frac{N!}{n_1!n_2!\cdots n_K!} = \mathrm{e}^{\log(N!) - \sum_i \log(n_i!)}$$

$$\approx C\mathrm{e}^{N\log(N) - \sum_i n_i \log(n_i)} \approx C\mathrm{e}^{-N\sum_i \frac{n_i}{N}\log\left(\frac{n_i}{N}\right)}$$

Order n terms cancel as $N = \sum_i n_i$

Number of States and Entropy

- Let $p(x_i) = n_i/N$ be the proportion of balls in bin i then

$$W(\boldsymbol{n}) \approx C\mathrm{e}^{NH_X}$$

where

$$H_X = -\sum_i p(x_i)\log(p(x_i))$$

- That is, the "entropy" can be seen as a measure of the logarithm of the number of configurations
- As the number of balls, $N \to \infty$, the overwhelmingly likely configuration is the one that maximises the entropy subject to what we know (e.g. an observed mean)

Entropy Is Concave

- Each term $h(p) = -p\ln(p)$ has $h''(p) = -1/p < 0$, so the entropy $H = \sum_i h(p_i)$ is a concave function of the probabilities ($-H$ is convex)
- Jensen's inequality (convexity lecture) with the concave $\log$ gives, for K outcomes,

$$H_X = \mathbb{E}\left[\log_2\left(\frac{1}{\mathbb{P}(X)}\right)\right] \le \log_2\left(\mathbb{E}\left[\frac{1}{\mathbb{P}(X)}\right]\right) = \log_2(K)$$

with equality for the uniform distribution

- With linear constraints (fixed averages) the maximum entropy solution is unique
- $-H$ is a convex regulariser favouring spread-out distributions (used in optimal transport)

Maximum Entropy Method

- When we are trying to infer a distribution given some observations then we can maximise the entropy subject to constraints—the entropy acts as a prior
- This is known as the **maximum entropy method**
- We can rationalise this as this is by far the most likely set of configurations consistent with the observations
- Alternatively we can see this as maximising our uncertainty given what we know—being as unbiased as possible
- It assumes that, before we impose the constraints, every configuration is equally likely

Knowing the Mean and Variance

- Consider a continuous random variable, X, with a known mean and second moment

$$\mathbb{E}[X] = \mu, \qquad \mathbb{E}[X^2] = \mu_2 = \mu^2 + \sigma^2$$

- To maximise the entropy subject to constraints consider

$$\mathcal{L}(f) = -\int f_X(x)\log(f_X(x))\mathrm{d}x + \lambda_0\left(\int f_X(x)\mathrm{d}x - 1\right)$$

$$+ \lambda_1\left(\int f_X(x)x\mathrm{d}x - \mu\right) + \lambda_2\left(\int f_X(x)x^2\mathrm{d}x - \mu_2\right)$$

- Thus

$$\frac{\delta\mathcal{L}(f)}{\delta f_X(x)} = -\log(f_X(x)) - 1 + \lambda_0 + \lambda_1 x + \lambda_2 x^2 = 0$$

- Or

$$f_X(x) = \mathrm{e}^{-1+\lambda_0+\lambda_1 x+\lambda_2 x^2}$$

Normal Distribution

- We have three constraints

$$\int \mathrm{e}^{-1+\lambda_0+\lambda_1 x+\lambda_2 x^2}\mathrm{d}x = 1$$

$$\int \mathrm{e}^{-1+\lambda_0+\lambda_1 x+\lambda_2 x^2} x\mathrm{d}x = \mu$$

$$\int \mathrm{e}^{-1+\lambda_0+\lambda_1 x+\lambda_2 x^2} x^2\mathrm{d}x = \mu_2 = \mu^2 + \sigma^2$$

- Solving for λ_0, λ_1 and λ_2 then

$$f_X(x) = \frac{1}{\sqrt{2\pi}\sigma}\mathrm{e}^{-(x-\mu)^2/(2\sigma^2)}$$

- That is, the normal distribution is the maximum entropy distribution given we know the mean and variance

Softmax Is Maximum Entropy

- A model gives scores s_i to K classes; find the probabilities p_i with maximum entropy and a given average score $\sum_i p_i s_i = \bar{s}$
- As for the normal distribution, use Lagrange multipliers

$$\mathcal{L} = -\sum_i p_i \ln(p_i) + \lambda_0\left(\sum_i p_i - 1\right) + \beta\left(\sum_i p_i s_i - \bar{s}\right)$$

- Setting $\partial\mathcal{L}/\partial p_i = -\ln(p_i) - 1 + \lambda_0 + \beta s_i = 0$ gives the **softmax**

$$p_i = \frac{\mathrm{e}^{s_i/T}}{\sum_j \mathrm{e}^{s_j/T}}, \qquad T = 1/\beta$$

- The temperature T is (the inverse of) a Lagrange multiplier, as in statistical physics; it is the "temperature" used when sampling from language models
- With scores $s_i = \boldsymbol{w}_i^\top \boldsymbol{x}$ this is multinomial logistic regression, also called the maximum entropy classifier

Using Maximum Entropy

- Maximum entropy is often used to infer distributions
- It can be very effective, but it might not work well if there are other constraints that we have not included
- The place that they work superbly well is in statistical physics
- The whole of statistical physics is about inferring distributions making observations of volume, pressure, etc.
- Temperature appears rather strangely as a Lagrange multiplier

Historic Entropy

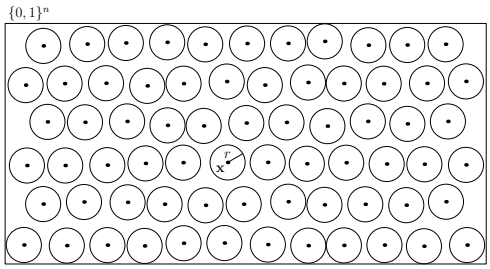
- Historically entropy was first introduced in statistical physics by Rudolf Clausius in 1865 (although Macquorn Rankine discussed it in 1850)
- Its interpretation as the number of states was introduced by Ludwig Boltzmann
- The person who got it all right was Josiah Willard Gibbs (and James Clerk Maxwell)
- Claude Shannon invented information theory based on entropy around 1948 (more on that in the next lecture)
- Ed Jaynes was the first to understand that statistical physics can be seen as an inference problem

Conclusion

- Entropy provides a measure of the disorder or uncertainty in a system
- It forms the basis of information theory which we will look at in the next lecture
- $-\log(\mathbb{P}(X = x))$ can be seen as the minimum length of a message to communicate x
- This will be used as the basis of the minimum description length formalism also discussed in the next lecture
- Entropy can be used as a prior, which we often maximise subject to constraints to obtain an unbiased estimate

Mathematics of Machine Learning

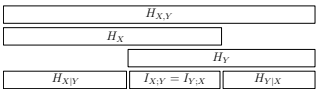
Information Theory



Information, KL-divergence, Minimum Description Length

Outline

1. **Information Theory**
2. KL-Divergence
3. Minimum Description Length
4. Variational Auto-Encoders



Communicating Via a Noisy Channel

- Information theory considers communicating down a (noisy) channel

$$X \sim \mathbb{P}(X) \xrightarrow{\text{noisy channel}} Y \sim \mathbb{P}(Y \mid X)$$

- We send a message X (with probability $\mathbb{P}(X)$) and receive a message Y with probability $\mathbb{P}(Y \mid X)$
- The uncertainty of the message sent, given we received a message y is

$$H_{X|Y=y} = -\sum_{x\in\mathcal{X}} \mathbb{P}(X=x \mid Y=y)\log(\mathbb{P}(X=x \mid Y=y))$$

- The expected uncertainty in the message sent is

$$H_{X|Y} = \sum_{y\in\mathcal{Y}} \mathbb{P}(Y=y)H_{X|Y=y} = -\sum_{x,y} \mathbb{P}(X=x,Y=y)\log(\mathbb{P}(X=x \mid Y=y))$$

Joint Entropy

- We can define the **joint entropy**

$$H_{X,Y} = -\sum_{x,y} P_{X,Y}(x,y)\log(P_{X,Y}(x,y))$$

- If the message we receive is independent of the message that is sent then $H_{X,Y} = H_X + H_Y$ (we saw this in the last lecture)
- $H_{X,Y} < H_X + H_Y$ if X and Y are dependent
- Since $\mathbb{P}(X,Y) = \mathbb{P}(Y|X)\mathbb{P}(X) = \mathbb{P}(X|Y)\mathbb{P}(Y)$ it follows

$$H_{X,Y} = H_X + H_{Y|X} = H_Y + H_{X|Y}$$

- Or $H_X - H_{X|Y} = H_Y - H_{Y|X}$

Mutual Information

- The amount of uncertainty about the message being sent, X, before receiving the message is $H_X = -\mathbb{E}_X[\log\mathbb{P}(X)]$
- Shannon defined the *mutual information* to be the expected loss in uncertainty when we receive a message

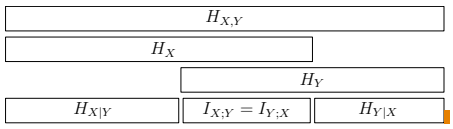
$$I_{X;Y} = H_X - H_{X|Y}$$

- Since $H_X - H_{X|Y} = H_Y - H_{Y|X}$ it follows

$$I_{X;Y} = I_{Y;X}$$

Channel Capacity

- We can summarise these relationships diagrammatically



- Shannon defined the *capacity* of a noisy channel as

$$C = \max_{\mathbb{P}(X)} I_{X;Y}$$

- That is, you choose the probability distribution of the message to maximise the information gain

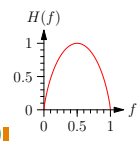
Independent Noise

- The simplest model of a noisy channel is a binary channel where each symbol is corrupted independently with a probability f

$$\mathbb{P}(Y=1|X=0) = \mathbb{P}(Y=0|X=1) = f$$

- An elementary calculation shows that

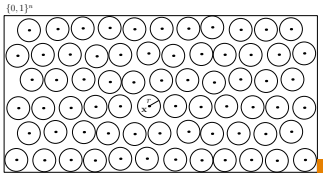
$$H_{Y_i|X_i} = -(1-f)\log_2(1-f) - f\log_2(f) = H(f)$$



- So $I_{X;Y} = H_Y - H_{Y|X} \leq 1 - H(f)$ per bit, with equality when 0 and 1 are sent equally often

Error Correcting Codes

- To reduce the chance of misinterpreting a message we need to build an error correcting code
- We can do this dividing the space of binary messages into a set of Hamming balls



- A Hamming ball $B(\boldsymbol{x},r)$ is the set of strings that differ from n-dimensional binary string, $\boldsymbol{x}$, by at most r digits

Volume of Coding Space

- The expected number of errors in a string of length n given an error rate of f is nf
- For sufficiently large n the number of errors is, with high probability, less than $(f+\epsilon)n$ (for $\epsilon > 0$)
- If we make the radius of the Hamming ball $r = (f+\epsilon)n$ ($\epsilon > 0$) then we would expect no error for sufficiently large n
- An upper bound on the number of code words we can send in a string of length n is

$$\frac{2^n}{|B(\boldsymbol{x}_i, r)|} = \frac{2^n}{\binom{n}{r}} = 2^n \mathrm{e}^{-\log(n!)+\log(r!)+\log((n-r)!)} \approx c\sqrt{n}2^{n(1-H(f+\epsilon))}$$

Shannon's Bound

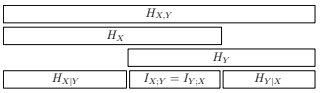
- So we can send at most about $2^{n(1-H(f))}$ distinguishable messages: a rate of $1 - H(f)$ bits per bit sent
- Shannon also showed that about this many code words chosen at random can be decoded with almost no errors
- The maximum rate is the channel capacity, here $C = 1 - H(f)$
- If $f = 0.1$ then $C = 1 - 0.469 = 0.531\text{bits}$, so we need codes about 1.9 times as long to communicate accurately over a noisy channel with a 10% corruption rate
- Random codes cannot be decoded efficiently, but modern codes (turbo, LDPC and polar codes, used in 4G, 5G and Wi-Fi) come close to Shannon's bound

Using Mutual Information

- Mutual information is used quite often in machine learning
- The **information gain** used to choose splits in decision trees is the mutual information between a feature and the label
- Suppose we want to align two images (e.g. an MRI and a CT scan of the same patient) through some non-linear transformations
- One way of doing this is to choose the non-linear transformations that maximise the mutual information (or normalised mutual information) between the two images

Outline

1. Information Theory
2. **KL-Divergence**
3. Minimum Description Length
4. Variational Auto-Encoders



KL-Divergence

- The Kullback-Leibler (KL) divergence is

$$\begin{aligned}\mathrm{KL}(p\|q) &= \mathbb{E}_{X\sim p(X)}\left[\log\left(\frac{p(X)}{q(X)}\right)\right] \\ &= \mathbb{E}_{X\sim p(X)}[-\log(q(X))] - H_X\end{aligned}$$

- Recall $-\log(q(X=x))$ is the length of code needed to send a message x with a probability $q(X=x)$
- Thus $-\mathbb{E}_{X\sim p(X)}[\log(q(X))]$ is the expected length of message needed to code $X \sim p(X)$ using the optimal code for the distribution $q(X)$ rather than $p(X)$
- $\mathrm{KL}(p\|q)$ is also known as the **relative entropy** and measures the expected extra length in coding $X \sim p(X)$ if we use the wrong distribution $q(X)$

Cross-Entropy Loss

- A classifier gives probabilities $q(y|\boldsymbol{x})$; it is usually trained by minimising the **cross-entropy**

$$\mathcal{L} = -\frac{1}{m}\sum_{k=1}^{m}\log(q(y_k|\boldsymbol{x}_k))$$

- It is the average code length for the labels using the model's probabilities
- For labels from the true $p(y|\boldsymbol{x})$, at each $\boldsymbol{x}$

$$\mathbb{E}_{Y\sim p}[-\log(q(Y|\boldsymbol{x}))] = H_p + \mathrm{KL}(p\|q)$$

- H_p does not depend on the model: minimising cross-entropy minimises $\mathrm{KL}(p\|q)$

Variational Approximation

- Recall we use MCMC in Bayesian inference because the posterior distribution is too complicated to write down in closed form
- In the variational approximation we approximate the posterior distribution by a simpler (typically factored distribution), e.g.

$$f(\boldsymbol{\theta} \mid \mathcal{D}) \approx g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) = \prod_i g(\theta_i \mid \phi_i)$$

- The standard method is to maximise the **evidence lower bound** (ELBO, minus the **variational free energy**)

$$\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta},\mathcal{D})}\right)\mathrm{d}\boldsymbol{\theta}$$

Evidence Lower Bound (ELBO)

- We can re-write the ELBO as

$$\begin{aligned}\Phi(\boldsymbol{\phi}) &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{(f(\boldsymbol{\theta},\mathcal{D})/f(\mathcal{D}))\,f(\mathcal{D})}\right)\mathrm{d}\boldsymbol{\theta} \\ &= -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\left(\log\left(\frac{g(\boldsymbol{\theta} \mid \boldsymbol{\phi})}{f(\boldsymbol{\theta} \mid \mathcal{D})}\right) - \log(f(\mathcal{D}))\right)\mathrm{d}\boldsymbol{\theta} \\ &= -\mathrm{KL}(g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\|f(\boldsymbol{\theta} \mid \mathcal{D})) + \log(f(\mathcal{D}))\end{aligned}$$

- If we maximise $\Phi(\boldsymbol{\phi})$, we end up minimising the KL divergence between g and f so that $g \approx f$ and $\Phi(\boldsymbol{\phi}) \approx \log(f(\mathcal{D}))$
- That is, we choose the parameters of our simple factorised distribution so that it is close to the true posterior

Put Another Way

- We can rewrite the ELBO as $\Phi(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\left(\frac{g(\boldsymbol{\theta}|\boldsymbol{\phi})}{f(\boldsymbol{\theta},\mathcal{D})}\right) \mathrm{d}\boldsymbol{\theta} = L_g(\boldsymbol{\phi}) + H_g(\boldsymbol{\phi})$ where

$$L_g(\boldsymbol{\phi}) = \int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \big(\log\big(f(\mathcal{D}|\boldsymbol{\theta}) f(\boldsymbol{\theta})\big)\big) \mathrm{d}\boldsymbol{\theta}$$

  is the expected log of likelihood $\times$ prior, which is large when the data is well modelled (we put $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$ where $f(\boldsymbol{\theta},\mathcal{D})$ is large, c.f. MAP)

- The second term is an entropy

$$H_g(\boldsymbol{\phi}) = -\int g(\boldsymbol{\theta} \mid \boldsymbol{\phi}) \log\big(g(\boldsymbol{\theta} \mid \boldsymbol{\phi})\big) \mathrm{d}\boldsymbol{\theta}$$

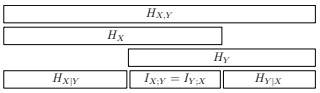
  That is, we maximise the uncertainty of the distribution $g(\boldsymbol{\theta} \mid \boldsymbol{\phi})$

Using Variational Methods

- Variational methods can be much faster than MCMC (although they tend to involve some iterations to maximise the ELBO)
- They can produce very good approximations, although this is not guaranteed (depends on the problem)
- There are variants (e.g. minimising $\mathrm{KL}(f\|g)$ rather than $\mathrm{KL}(g\|f)$ gives *expectation propagation*)
- MCMC is less elegant, but is a controlled approximation (we get better results by increasing the number of iterations)
- MCMC is slower, but on modern computers this isn't usually a problem for moderate-sized models

Outline

1. Information Theory
2. KL-Divergence
3. **Minimum Description Length**
4. Variational Auto-Encoders



Compression and Model Selection

- Outside of the Bayesian framework it is difficult to do model selection—most of ML isn't Bayesian
- When is it better to accept a more complex model for a better fit and when are we just over-fitting?
- Usually we answer this using a validation set, but this is not always possible
- One principled approach is to use the model that allows us to maximally compress the data
- If we are compressing the data then we are capturing features of the data

Alice and Bob

- Suppose Alice has data $\mathcal{D} = \{(\boldsymbol{x}_i, y_i) \mid i = 1,2,\ldots,m\}$ while Bob has only the feature vectors $\{\boldsymbol{x}_i \mid i = 1,2,\ldots,m\}$
- Alice wants to communicate y_i to Bob as efficiently as possible
- We suppose Alice & Bob have available a model $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta})$
- Rather than sending the complete list $\{y_i \mid i = 1,2,\ldots,m\}$ Alice can send Bob the parameter $\boldsymbol{\theta}$ and the errors

$$\delta_i = y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta})$$

- Assuming the δ_i's have a distribution p_δ then the cost of communicating an error to accuracy Δ is $-\log(p_\delta(\delta_i) \times \Delta)$

Description Length

- The **description length** for $\{y_i \mid i = 1,2,\ldots,m\}$ is then the cost of transmitting $\boldsymbol{\theta}$ plus the cost of transmitting the errors

$$L = \sum_k \ell(\theta_k) - \sum_{i=1}^m \Big(\log\Big(p_\delta\Big(y_i - \hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta})\Big)\Big) + \log(\Delta)\Big)$$

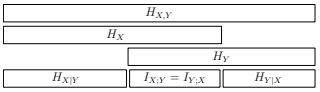
  where $\ell(\theta_k)$ is the number of bits needed to communicate θ_k (we get to choose the accuracy it is worth encoding the parameters)
- To select between models we choose the model with the **minimum description length**
- Note that the accuracy Δ will lead to the same cost, $-m\log(\Delta)$, for all models so doesn't affect which model is selected

Minimum Description Length (MDL) Method

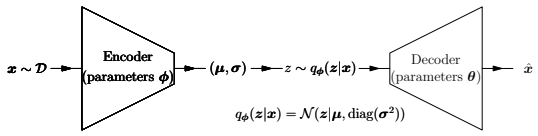
- The minimum description length method can be a powerful way of choosing between models
- Often it is the only principled method available
- It allows you to trade model accuracy against model complexity
- It can be fiddly as we need to determine the accuracy to which we should store the parameters of our model
- This isn't something we usually think about, but often we can get very good models even when we truncate the parameters to low precision

Outline

1. Information Theory
2. KL-Divergence
3. Minimum Description Length
4. **Variational Auto-Encoders**

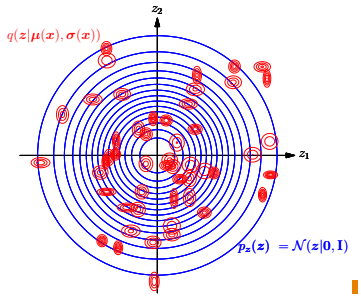


Variational Auto-Encoders VAE



$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\big[\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\big]$$

Latent Space



$$\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big)$$

Understanding the Loss Function

- The original paper derived the loss function as a variational lower bound on the likelihood of the data: it is minus an ELBO
- This is difficult to understand (at least, for me)
- It has a very natural explanation in terms of minimum description length
- Alice wants to communicate the images to Bob
- Alice uses the encoder to derive a (latent) code $q(\boldsymbol{z}|\boldsymbol{x})$ which she communicates to Bob
- She also communicates the errors $\boldsymbol{\delta} = \boldsymbol{x} - \bar{\boldsymbol{x}}$
- Bob uses the decoder to decode $q(\boldsymbol{z}|\boldsymbol{x})$ and $\boldsymbol{\delta}$ to repair the images

Description Length

- The loss

$$\mathcal{L} = \mathbb{E}_{\boldsymbol{x}\sim\mathcal{D}}\big[\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big) - \log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))\big]$$

  can be interpreted as

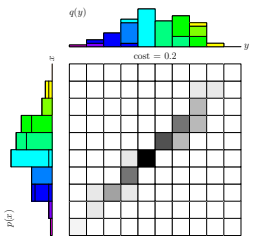
  - ⋆ The cost of communicating the code $\mathrm{KL}\big(q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})\big\|\mathcal{N}(\mathbf{0},\mathbf{I})\big)$ (using "bits-back" coding, Hinton and van Camp 1993)
  - ⋆ Plus the cost to send the repair $-\log(p_{\boldsymbol{\theta}}(\boldsymbol{x}|\boldsymbol{z}(\boldsymbol{x})))$
- We minimise the loss function equivalent to MDL
- What is really clever is that we can choose the accuracy of the code we send $q_{\boldsymbol{\theta}}(\boldsymbol{z}|\boldsymbol{x})$ to minimise the over-all cost

Conclusions

- Information theory has regularly been used in machine learning
- It requires some understanding and care to do it properly
- The KL-divergence (or relative entropy) is often used to make two probability distributions more alike
- The minimum description length is a powerful principle for model selection
- Variational Auto-Encoders have a very natural interpretation in terms of minimising a description length

Mathematics of Machine Learning

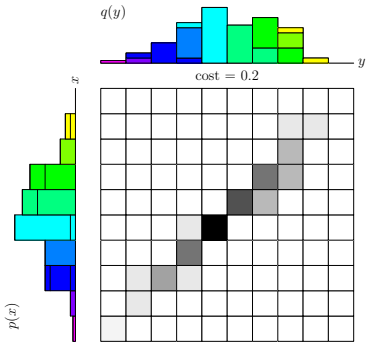
Optimal Transport and the Wasserstein Distance



Wasserstein distance, optimal transport, duality, Lipschitz functions, WGANs, Sinkhorn, FID

Outline

1. **Comparing Distributions**
2. Optimal Transport
3. Duality
4. Applications



Measuring Distances Between Distributions

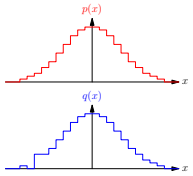
- In many machine learning tasks we want to minimise the distance between two probability distributions (e.g. make a generative model's samples look like the data)
- This requires that we can measure distances between probability distributions
- One prominent measure is the KL divergence (last lecture)

$$\mathrm{KL}(p\|q) = \int p(\boldsymbol{x}) \log\left(\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\right) \mathrm{d}\boldsymbol{x}$$

- This is very commonly used in ML (e.g. cross-entropy loss, VAEs, variational approximation)

Trouble with KL

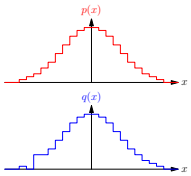
- KL-divergences are non-negative quantities that are minimised when the two probability distributions are the same
- They are not distances (they aren't symmetric and they don't satisfy the triangular inequality)
- We don't really care about this, but what we do care about is that if $q(\boldsymbol{x}) = 0$ when $p(\boldsymbol{x}) \neq 0$ then $\log\left(\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\right)$ diverges



- We can therefore have distributions that seem very similar but their KL-divergence is huge (or infinite)

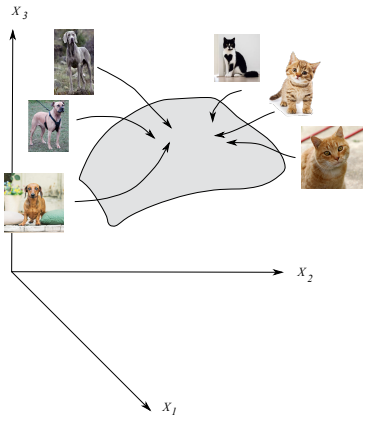
Wasserstein Distance

- A more benign measure of the differences between two probability distributions is the **Wasserstein** or **Earth Mover's** distance
- This is a true distance, but more importantly, it measures distance in a very natural way so that distributions that are close have a small Wasserstein distance



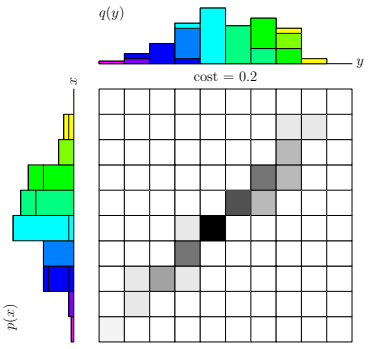
- This may seem contrived, but images (e.g. $128 \times 128 \times 3$ arrays of pixel values) lie close to a thin manifold, so two distributions of images can be close without overlapping: then the KL-divergence is useless but the Wasserstein distance is not

High Probability Manifold



Outline

1. Comparing Distributions
2. **Optimal Transport**
3. Duality
4. Applications



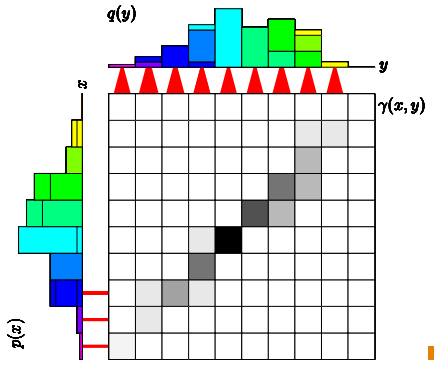
Transportation Policy

- But how do we formalise the Wasserstein distance?
- A good place to start is to define a transportation policy $\gamma(\boldsymbol{x},\boldsymbol{y})$ with

$$\int \gamma(\boldsymbol{x},\boldsymbol{y}) \mathrm{d}\boldsymbol{y} = p(\boldsymbol{x}) \qquad \int \gamma(\boldsymbol{x},\boldsymbol{y}) \mathrm{d}\boldsymbol{x} = q(\boldsymbol{y})$$

- This looks like a joint probability distribution, but we interpret $\gamma(\boldsymbol{x},\boldsymbol{y})$ as the amount of probability mass/density that we transfer from $p(\boldsymbol{x})$ to $q(\boldsymbol{y})$

Transportation Policy



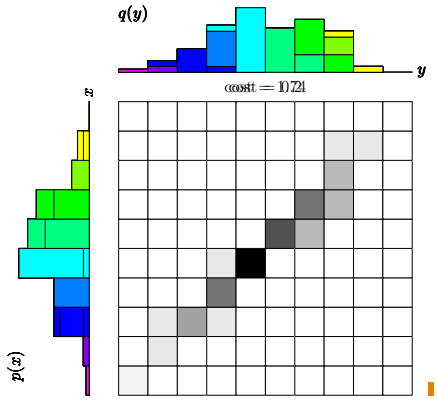
The Cost of Transport

- We want the transportation policy that minimises the total work: the probability mass moved times the distance it is moved
- Let $d(\boldsymbol{x},\boldsymbol{y}) = \|\boldsymbol{x}-\boldsymbol{y}\|$ be a distance measure then the cost of a transportation policy is

$$C(\gamma) = \int\int d(\boldsymbol{x},\boldsymbol{y})\gamma(\boldsymbol{x},\boldsymbol{y})\mathrm{d}\boldsymbol{x}\,\mathrm{d}\boldsymbol{y} = \mathbb{E}_\gamma[d(\boldsymbol{x},\boldsymbol{y})]$$

  where we interpret $\gamma(\boldsymbol{x},\boldsymbol{y})$ as a probability distribution
- Usually we take $d(\boldsymbol{x},\boldsymbol{y})$ to be the Euclidean distance, but we can choose any distance

Transportation Cost



The Wasserstein Distance

- The Wasserstein distance $W(p,q)$ between two probability distributions is defined as

$$W(p,q) = \min_{\gamma\in\Lambda(p,q)} \mathbb{E}_\gamma[d(\boldsymbol{x},\boldsymbol{y})]$$

- Where $\Lambda(p,q)$ is the set of joint distributions $\gamma(\boldsymbol{x},\boldsymbol{y})$ such that

$$\int \gamma(\boldsymbol{x},\boldsymbol{y})\mathrm{d}\boldsymbol{y} = p(\boldsymbol{x}) \qquad \int \gamma(\boldsymbol{x},\boldsymbol{y})\mathrm{d}\boldsymbol{x} = q(\boldsymbol{y})$$

- With cost d^k we get $W_k(p,q) = \left(\min_\gamma \mathbb{E}_\gamma\left[d(\boldsymbol{x},\boldsymbol{y})^k\right]\right)^{1/k}$; here we use $W = W_1$

One Dimension

- In one dimension the Wasserstein distance has a closed form in terms of the cumulative distribution functions F_p and F_q

$$W(p,q) = \int_{-\infty}^{\infty} \left|F_p(x) - F_q(x)\right| \mathrm{d}x$$

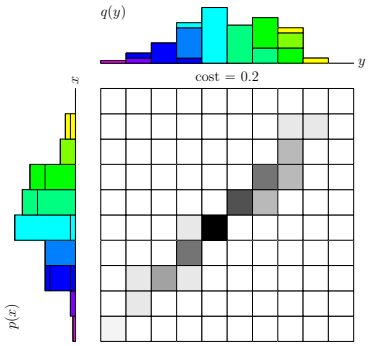
  the area between the two curves
- For two samples of the same size: sort both and match them in order

$$W = \frac{1}{n}\sum_{i=1}^{n} |x_{(i)} - y_{(i)}|$$

- In higher dimensions there is no such shortcut: we have to solve the optimisation

Outline

1. Comparing Distributions
2. Optimal Transport
3. **Duality**
4. Applications



Computing the Wasserstein Distance

- To compute the Wasserstein distance we have to solve a minimisation task!
- This looks nasty, but it is a (continuous) linear programme, like those in the constrained-optimisation lecture
- Suppose p and q were discrete distributions (i.e. $\boldsymbol{x}$ and $\boldsymbol{y}$ only take discrete points)
- Then we could treat each value of $\gamma(\boldsymbol{x},\boldsymbol{y})$ as an element of a vector $\boldsymbol{\gamma}$ and each value of $d(\boldsymbol{x},\boldsymbol{y})$ as an element of a vector $\boldsymbol{D}$
- Our objective is to choose $\boldsymbol{\gamma}$ to minimise $\boldsymbol{D}^\mathsf{T}\boldsymbol{\gamma}$

Constraints

$$\sum_j \gamma(\boldsymbol{x}_i,\boldsymbol{y}_j) = p(\boldsymbol{x}_i) \qquad \sum_i \gamma(\boldsymbol{x}_i,\boldsymbol{y}_j) = q(\boldsymbol{y}_j)$$

$$\mathbf{A}\boldsymbol{\gamma} = \boldsymbol{P}$$

$$\begin{pmatrix}
1 & 1 & \cdots & 1 & 0 & 0 & \cdots & 0 & \cdots & & & 0 & 0 & \cdots & 0 \\
0 & 0 & \cdots & 0 & 1 & 1 & \cdots & 1 & \cdots & & & 0 & 0 & \cdots & 0 \\
\vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \cdots & \vdots \\
0 & 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & \cdots & & & 1 & 1 & \cdots & 1 \\
1 & 0 & \cdots & 0 & 1 & 0 & \cdots & 0 & \cdots & & & 1 & 0 & \cdots & 0 \\
0 & 1 & \cdots & 0 & 0 & 1 & \cdots & 0 & \cdots & & & 0 & 1 & \cdots & 0 \\
\vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \vdots & \cdots & \vdots & \vdots & \cdots & \vdots \\
0 & 0 & \cdots & 1 & 0 & 0 & \cdots & 1 & \cdots & & & 0 & 0 & \cdots & 1
\end{pmatrix}
\begin{pmatrix}
\gamma(x_1,y_1) \\ \gamma(x_2,y_1) \\ \vdots \\ \gamma(x_n,y_1) \\ \gamma(x_1,y_2) \\ \gamma(x_2,y_2) \\ \vdots \\ \gamma(x_n,y_2) \\ \vdots \\ \gamma(x_1,y_n) \\ \gamma(x_2,y_n) \\ \vdots \\ \gamma(x_n,y_n)
\end{pmatrix}
=
\begin{pmatrix}
q(y_1) \\ q(y_2) \\ \vdots \\ q(y_n) \\ p(x_1) \\ p(x_2) \\ \vdots \\ p(x_n)
\end{pmatrix}$$

The Dual Problem

- With Lagrange multipliers $\alpha(\boldsymbol{x}_i)$ and $\beta(\boldsymbol{y}_j)$ for the two sets of constraints

$$\mathcal{L} = \sum_{i,j} d(\boldsymbol{x}_i,\boldsymbol{y}_j)\gamma(\boldsymbol{x}_i,\boldsymbol{y}_j) - \sum_i \alpha(\boldsymbol{x}_i)\left(\sum_j \gamma(\boldsymbol{x}_i,\boldsymbol{y}_j) - p(\boldsymbol{x}_i)\right) - \sum_j \beta(\boldsymbol{y}_j)\left(\sum_i \gamma(\boldsymbol{x}_i,\boldsymbol{y}_j) - q(\boldsymbol{y}_j)\right)$$

  subject to $\gamma(\boldsymbol{x}_i,\boldsymbol{y}_j) \geq 0$

- Rearranging

$$\mathcal{L} = \sum_i \alpha(\boldsymbol{x}_i)p(\boldsymbol{x}_i) + \sum_j \beta(\boldsymbol{y}_j)q(\boldsymbol{y}_j) - \sum_{i,j}\gamma(\boldsymbol{x}_i,\boldsymbol{y}_j)(\alpha(\boldsymbol{x}_i) + \beta(\boldsymbol{y}_j) - d(\boldsymbol{x}_i,\boldsymbol{y}_j))$$

- This is equivalent to maximising $\sum_i \alpha(\boldsymbol{x}_i)p(\boldsymbol{x}_i) + \sum_j \beta(\boldsymbol{y}_j)q(\boldsymbol{y}_j)$, subject to

$$\forall i,j \quad \alpha(\boldsymbol{x}_i) + \beta(\boldsymbol{y}_j) \leq d(\boldsymbol{x}_i,\boldsymbol{y}_j)$$

- For continuous distributions sums become integrals and $\alpha(\boldsymbol{x})$, $\beta(\boldsymbol{y})$ become functions

Dual Form Constraint

- We need $\alpha(\boldsymbol{x}) + \beta(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y})$ for all $\boldsymbol{x}$, $\boldsymbol{y}$, and want β as large as possible
- Taking $\boldsymbol{x} = \boldsymbol{y}$ (where $d(\boldsymbol{y},\boldsymbol{y}) = 0$) gives $\beta(\boldsymbol{y}) \leq -\alpha(\boldsymbol{y})$; so $-\alpha$ is the largest β could possibly be
- Try that largest value, $\beta = -\alpha$: the constraint becomes

$$\alpha(\boldsymbol{x}) - \alpha(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y}) \quad \text{for all } \boldsymbol{x},\boldsymbol{y}$$

- For any α that satisfies this, $\beta = -\alpha$ is allowed and cannot be beaten
- Restricting to such α loses nothing (the **Kantorovich–Rubinstein** theorem; see the notes)

Kantorovich–Rubinstein Duality

- Putting this together

$$W(p,q) = \max_\alpha \int \alpha(\boldsymbol{x})\big(p(\boldsymbol{x}) - q(\boldsymbol{x})\big)\,\mathrm{d}\boldsymbol{x} = \max_\alpha \Big(\mathbb{E}_p[\alpha(\boldsymbol{X})] - \mathbb{E}_q[\alpha(\boldsymbol{X})]\Big)$$

- where the maximum is over functions with

$$\alpha(\boldsymbol{x}) - \alpha(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|$$

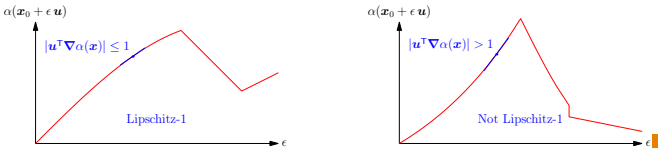
  that is, α is **1-Lipschitz** (Lipschitz-1)
- Instead of the huge object $\gamma(\boldsymbol{x},\boldsymbol{y})$ we now need one function $\alpha(\boldsymbol{x})$, and the expectations can be estimated from samples

Lipschitz-1 Functions

- We note for a Lipschitz-1 function and any unit vector $\boldsymbol{u}$

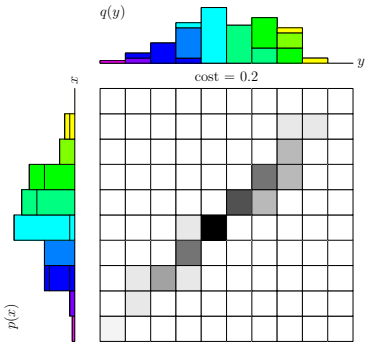
$$\boldsymbol{u}^\mathsf{T}\nabla\alpha(\boldsymbol{x}) = \lim_{\epsilon\to 0}\frac{\alpha(\boldsymbol{x}+\epsilon\boldsymbol{u}) - \alpha(\boldsymbol{x})}{\epsilon} \leq \frac{\epsilon\|\boldsymbol{u}\|}{\epsilon} = \frac{\epsilon}{\epsilon} = 1$$

- That is, at every point the gradient in every direction is at most 1 (since the gradient defines the direction of greatest increase it is both necessary and sufficient for $\|\nabla\alpha(\boldsymbol{x})\| \leq 1$ everywhere)

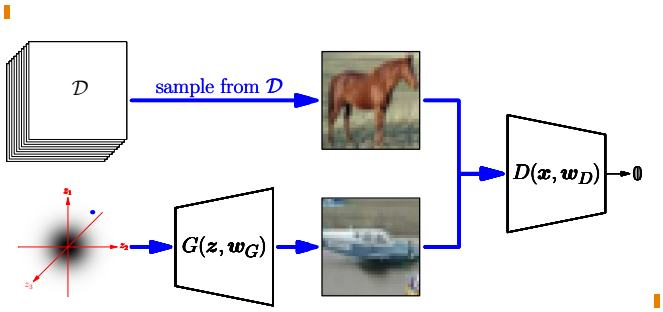


Outline

1. Comparing Distributions
2. Optimal Transport
3. Duality
4. **Applications**



How GANs Work



Wasserstein GANs

- A GAN has a **generator** $G(\boldsymbol{z},\boldsymbol{w}_G)$ that turns noise $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0},\mathbf{I})$ into images, and a **discriminator** trained to tell them from real images
- GANs are hard to train: a discriminator that is too good gives the generator no useful gradient
- A Wasserstein GAN instead trains the generator to minimise W between the data distribution p and the generator's distribution q

$$\min_{\boldsymbol{w}_G}\max_\alpha \Big(\mathbb{E}_p[\alpha(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{z}}[\alpha(G(\boldsymbol{z},\boldsymbol{w}_G))]\Big)$$

- The **critic** α is a second network in place of the discriminator; it still gives a gradient when the distributions do not overlap

The Critic

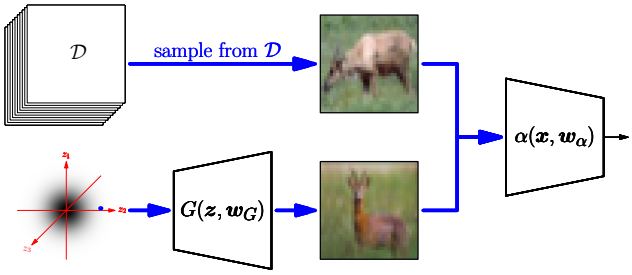
- Estimate the expectations from a minibatch $\mathcal{B}$ of real images and n generated ones

$$C = \frac{1}{|\mathcal{B}|}\sum_{\boldsymbol{x}\in\mathcal{B}}\alpha(\boldsymbol{x},\boldsymbol{w}_\alpha) - \frac{1}{n}\sum_{i=1}^n \alpha(G(\boldsymbol{z}_i,\boldsymbol{w}_G),\boldsymbol{w}_\alpha)$$

- The critic's weights $\boldsymbol{w}_\alpha$ are trained to maximise C; the generator's weights $\boldsymbol{w}_G$ to minimise it
- The critic should be 1-Lipschitz, which in practice is only roughly enforced: by clipping the weights (the original WGAN), penalising $\|\nabla\alpha\|$ away from 1, or scaling each weight matrix to spectral norm 1

Wasserstein GANs

$$\max_{\boldsymbol{w}_\alpha} \min_{\boldsymbol{w}_G} \frac{1}{|\mathcal{B}|} \sum_{\boldsymbol{x}\in\mathcal{B}} \alpha(\boldsymbol{x}, \boldsymbol{w}_\alpha) - \frac{1}{n}\sum_{i=1}^{n} \alpha(G(\boldsymbol{z}_i, \boldsymbol{w}_G), \boldsymbol{w}_\alpha)$$



Entropic Regularisation

- Solving the linear programme for n points costs roughly $O(n^3)$: too slow for large problems
- Cuturi (2013) added an entropy term, with $\epsilon > 0$

$$\min_{\gamma} \sum_{i,j} d_{ij}\gamma_{ij} + \epsilon \sum_{i,j} \gamma_{ij} \log \gamma_{ij}$$

with the same constraints

- The optimisation is given by: $\max_{\boldsymbol{\alpha},\boldsymbol{\beta}} \min_{\gamma} \mathcal{L}(\gamma, \boldsymbol{\alpha}, \boldsymbol{\beta})$

$$\mathcal{L} = \sum_{i,j} d_{ij}\gamma_{ij} + \epsilon \sum_{i,j} \gamma_{ij}\log\gamma_{ij} - \sum_i \alpha_i \left(\sum_j \gamma_{ij} - p_i\right) - \sum_j \beta_j \left(\sum_i \gamma_{ij} - q_j\right)$$

Solving the Primal Problem

- Using

$$\mathcal{L} = \sum_{i,j} d_{ij}\gamma_{ij} + \epsilon \sum_{i,j} \gamma_{ij}\log\gamma_{ij} - \sum_i \alpha_i \left(\sum_j \gamma_{ij} - p_i\right) - \sum_j \beta_j \left(\sum_i \gamma_{ij} - q_j\right)$$

- Setting the derivative of the Lagrangian to zero gives

$$\gamma_{ij} = \mathrm{e}^{(\alpha_i - d_{ij} + \beta_j)/\epsilon - 1} = u_i K_{ij} v_j$$

where $u_i = \mathrm{e}^{\alpha_i/\epsilon - 1}$, $v_j = \mathrm{e}^{\beta_j/\epsilon}$ and $K_{ij} = \mathrm{e}^{-d_{ij}/\epsilon}$

- We have to choose u_i and v_j (equivalently the Lagrange multipliers) so that

$$\sum_j u_i K_{ij} v_j = p_i \qquad \sum_i u_i K_{ij} v_j = q_j$$

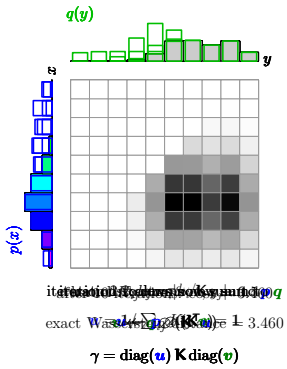
The Sinkhorn Algorithm

- Choose $\boldsymbol{u}$ and $\boldsymbol{v}$ so the row and column sums are $\boldsymbol{p}$ and $\boldsymbol{q}$: alternately rescale ($\oslash$ is element-by-element division)

$$\boldsymbol{v} \leftarrow \boldsymbol{q} \oslash (\mathbf{K}^\mathsf{T}\boldsymbol{u}), \qquad \boldsymbol{u} \leftarrow \boldsymbol{p} \oslash (\mathbf{K}\boldsymbol{v})$$

- This is Sinkhorn's matrix-scaling algorithm: each step is two matrix–vector products
- Fast on GPUs and differentiable, so it can sit inside a loss
- As $\epsilon \to 0$ we recover the exact transport cost; larger ϵ is faster but blurs the plan
- This is how optimal transport is mostly computed in ML today

Sinkhorn in Action



Evaluating Generative Models: FID

- For Gaussians, W_2 has a closed form

$$W_2^2 = \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2 + \mathrm{Tr}\Big(\boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2 - 2\big(\boldsymbol{\Sigma}_1\boldsymbol{\Sigma}_2\big)^{1/2}\Big)$$

(in one dimension $(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2$)

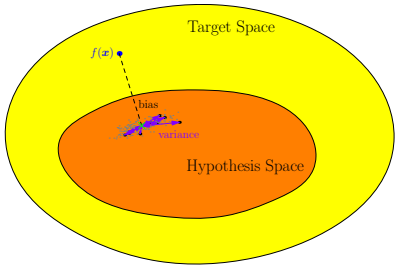
- The **Fréchet Inception Distance** (Heusel et al., 2017) fits Gaussians to a pretrained network's features of real and of generated images, and reports this distance
- It is the standard score for image generators, GANs and diffusion models alike (lower is better)
- Caveat: it only compares means and covariances of the features

Lessons

- The Wasserstein distance compares distributions by how far probability mass must move, so it behaves well when they do not overlap
- Computing it is a linear programme, whose dual is a maximum over 1-Lipschitz functions
- That dual turned GANs into Wasserstein GANs: an elegant piece of theory
- In practice optimal transport is computed with Sinkhorn, and the W_2 distance between Gaussians is the FID used to evaluate generative models

Mathematics of Machine Learning

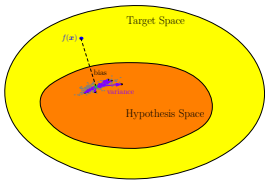
When Machine Learning Works



When ML Works, Bias Variance

Outline

1. **What Makes a Good Learning Machine?**
2. Bias-Variance Dilemma
3. Shrinkage and Ridge Regression
4. Ensembles



What Makes a Good Learning Machine?

- We want to understand why some machine learning techniques work well and others don't
- To understand why they work we need to understand what makes a good learning machine
- For this we have to get conceptual and think about **generalisation** performance

  ***generalisation**: how well do we do on unseen data as opposed to the training data*

What Makes Machine Learning Hard?

- Typically we work in high dimensions (i.e. have many features)
- The problem can be over-constrained (i.e. we have conflicting data to deal with)—solve by minimising an error function
- The problem can be under-constrained (i.e. there are many possible solutions that are consistent with the data)—need to choose a plausible solution
- Typically in machine learning the data will be over-constrained in some dimensions and under-constrained in others
- We can't visualise the data to know what is going on

Least Squares

- Suppose we want to learn some output y for a feature vector $\boldsymbol{x}$
- We construct a learning machine that makes a prediction $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta})$
- We typically choose the machine to minimise a *training loss*

$$L_T(\mathcal{D}) = \sum_{(\boldsymbol{x},y)\in\mathcal{D}} \Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}) - y\Big)^2 = \sum_{i=1}^{m} \Big(\hat{f}(\boldsymbol{x}_i|\boldsymbol{\theta}) - y_i\Big)^2$$

  where $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^m$ is a set of size m, sampled from a probability distribution $\mu(\boldsymbol{x},y)$

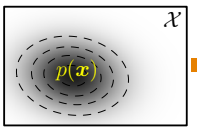
- We call this machine $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D})$

Generalisation Error

- We want to minimise the *generalisation loss* which in this case is

$$L_G(\boldsymbol{\theta}_\mathcal{D}) = \sum_{(\boldsymbol{x},y)\in\mathcal{Z}} \mu(\boldsymbol{x},y)\Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - y\Big)^2$$

$$\mu(\boldsymbol{x},y) = p(y|\boldsymbol{x})p(\boldsymbol{x})$$

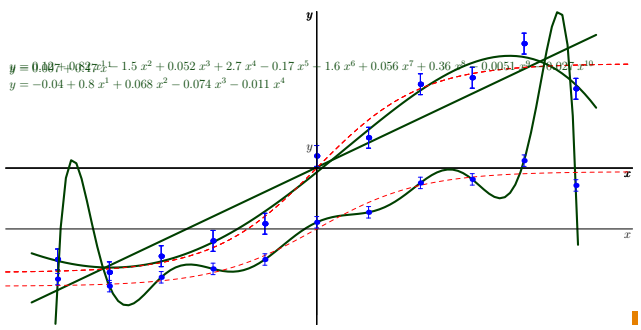


  (we can estimate this if we have some labelled examples $(\boldsymbol{x}_i, y_i)$ which we have not trained on)

- We want to minimise $L_G(\boldsymbol{\theta}_\mathcal{D})$ but in practice we areb minimising $L_T(\mathcal{D})$ *what could possibly go wrong?*

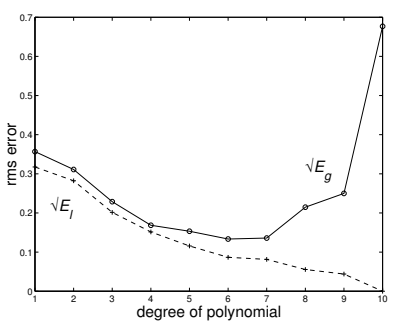
Too Simple or Too Complex?

- Fit $\hat{f}(x|\boldsymbol{\theta}_\mathcal{D})$ to data



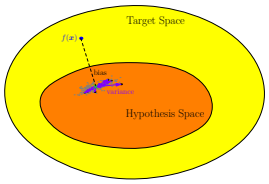
Measuring Generalisation Error for Regression

- Consider the regression example. The root mean squared error is



Outline

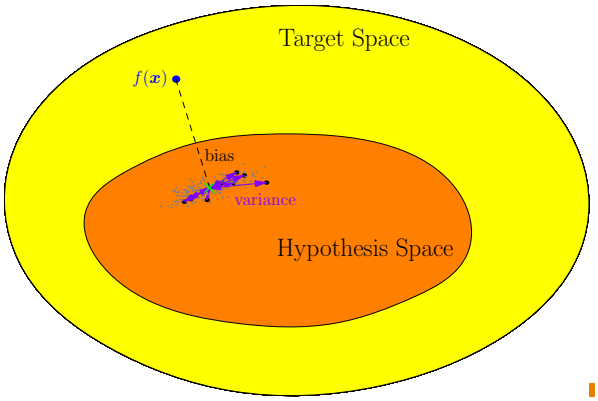
1. What Makes a Good Learning Machine?
2. **Bias-Variance Dilemma**
3. Shrinkage and Ridge Regression
4. Ensembles



Expected Generalisation Performance

- Our generalisation performance will depend on our training set, $\mathcal{D}$
- To reason about generalisation we can ask what is the *expected generalisation loss*, when we average over all different data sets of size m drawn independently from $\mu(\boldsymbol{x},y)$
- For each data set, $\mathcal{D}$, we would learn a different approximator $\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D})$
- Note that in practice we only get one data set. We might be lucky and do better than the expected generalisation or we might be unlucky and do worse

Approximation and Estimation Errors



Mean Machine

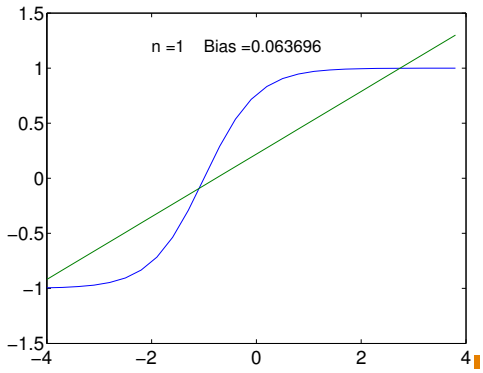
- To help understand generalisation we can consider the mean prediction with respect to machines trained with all data sets of size m

$$\hat{f}_m(\boldsymbol{x}) = \mathbb{E}_\mathcal{D}\Big[\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D})\Big]$$

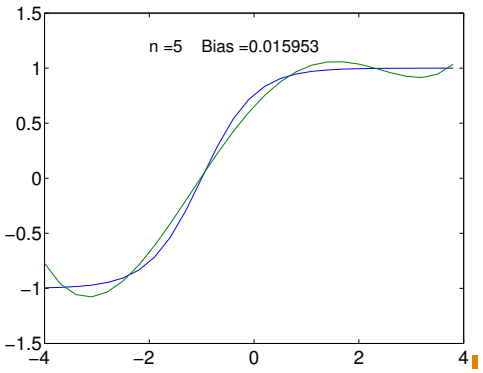
- We can define the **bias** to be generalisation performance of the mean machine

$$B = \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\Big(\hat{f}_m(\boldsymbol{x}) - y\Big)^2$$

Regression Example $n = 1$



Regression Example $n = 5$



Bias and Variance

Consider the expected generalisation for data sets of size $|\mathcal{D}| = m$

$$\begin{aligned}
\bar{L}_G = \mathbb{E}_\mathcal{D}[L_G(\boldsymbol{\theta}_\mathcal{D})] &= \mathbb{E}_\mathcal{D}\left[\sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - y\Big)^2\right] \\
&= \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\mathbb{E}_\mathcal{D}\left[\Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - y\Big)^2\right] \\
&= \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\mathbb{E}_\mathcal{D}\left[\Big(\big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - \hat{f}_m(\boldsymbol{x})\big) + \big(\hat{f}_m(\boldsymbol{x}) - y\big)\Big)^2\right] \\
&= \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\Bigg(\mathbb{E}_\mathcal{D}\left[\Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - \hat{f}_m(\boldsymbol{x})\Big)^2 + \Big(\hat{f}_m(\boldsymbol{x}) - y\Big)^2\right] \\
&\qquad + 2\mathbb{E}_\mathcal{D}\Big[\Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - \hat{f}_m(\boldsymbol{x})\Big)\Big(\hat{f}_m(\boldsymbol{x}) - y\Big)\Big]\Bigg)
\end{aligned}$$

Cross Term

- The cross term vanishes

$$\begin{aligned}
C &= \mathbb{E}_\mathcal{D}\Big[\Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - \hat{f}_m(\boldsymbol{x})\Big)\Big(\hat{f}_m(\boldsymbol{x}) - y\Big)\Big] \\
&= \mathbb{E}_\mathcal{D}\Big[\Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - \hat{f}_m(\boldsymbol{x})\Big)\Big]\Big(\hat{f}_m(\boldsymbol{x}) - y\Big) \\
&= \Big(\mathbb{E}_\mathcal{D}\Big[\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D})\Big] - \hat{f}_m(\boldsymbol{x})\Big)\Big(\hat{f}_m(\boldsymbol{x}) - y\Big) \\
&= \Big(\hat{f}_m(\boldsymbol{x}) - \hat{f}_m(\boldsymbol{x})\Big)\Big(\hat{f}_m(\boldsymbol{x}) - y\Big) = 0
\end{aligned}$$

- Thus

$$\bar{L}_G = \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\mathbb{E}_\mathcal{D}\left[\Big(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_\mathcal{D}) - \hat{f}_m(\boldsymbol{x})\Big)^2 + \Big(\hat{f}_m(\boldsymbol{x}) - y\Big)^2\right]$$

Bias and Variance

- We can write the expected generalisation loss as

$$\mathbb{E}_{\mathcal{D}}[L_G(\boldsymbol{\theta}_{\mathcal{D}})] = \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}}) - \hat{f}_m(\boldsymbol{x})\right)^2\right] + \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\left(\hat{f}_m(\boldsymbol{x}) - y\right)^2 = V + B$$

- Where B is the bias and V is the variance defined by

$$V = \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}}) - \hat{f}_m(\boldsymbol{x})\right)^2\right]$$

Where Is the Noise?

- Suppose $y = f^*(\boldsymbol{x}) + \epsilon$ where the noise ϵ has mean 0 and variance σ^2
- Then (the cross term vanishes again, as $\mathbb{E}[\epsilon] = 0$)

$$B = \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\left(\hat{f}_m(\boldsymbol{x}) - y\right)^2 = \sigma^2 + \sum_{\boldsymbol{x}} \mu(\boldsymbol{x})\left(\hat{f}_m(\boldsymbol{x}) - f^*(\boldsymbol{x})\right)^2$$

- So $\bar{L}_G = \text{noise} + \text{bias}^2 + \text{variance}$, the form you may have seen elsewhere
- The noise is the same for every machine: we can do nothing about it
- Here we keep it inside B: $\bar{L}_G = B + V$

Bias-Variance Dilemma

- The bias measures the generalisation performance of the *mean machine* and is large if the machine is too simple to capture the changes in the function we want to learn
- The variance measures the variation in the prediction of the machines as we change the data set we train on

$$V = \sum_{(\boldsymbol{x},y)} \mu(\boldsymbol{x},y)\mathbb{E}_{\mathcal{D}}\left[\left(\hat{f}(\boldsymbol{x}|\boldsymbol{\theta}_{\mathcal{D}}) - \hat{f}_m(\boldsymbol{x})\right)^2\right]$$

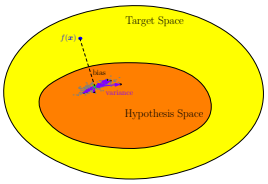
- The variance is usually large if we have a complex machine
- Striking the right balance is often the key to getting good results

Balancing Bias and Variance

- We want to choose a learning machine that is complex enough to capture the underlying function we are trying to learn, but otherwise as simple as possible
- There are a number of tricks to achieve this balance
- Some require us to preprocess the data to reduce the number of inputs
- Some machines cleverly adjust their own complexity
- The rest of this lecture: two exact examples, and averaging

Outline

1. What Makes a Good Learning Machine?
2. Bias-Variance Dilemma
3. **Shrinkage and Ridge Regression**
4. Ensembles



Shrinking an Estimate

- The simplest learning problem: estimate the mean θ of a distribution with variance σ^2 from m samples
- The sample mean $\bar{x}$ is unbiased with variance σ^2/m
- Try the deliberately biased $\hat{\theta} = c\bar{x}$ with $0 \le c \le 1$

$$\mathbb{E}\left[\left(\hat{\theta} - \theta\right)^2\right] = \underbrace{(1-c)^2\theta^2}_{\text{bias}^2} + \underbrace{c^2\frac{\sigma^2}{m}}_{\text{variance}}$$

- At $c = 1$ the derivative is $2\sigma^2/m > 0$: shrinking a little **always** reduces the error
- The best c depends on the unknown θ, so in practice we choose it by validation

Ridge Regression

- Data $\boldsymbol{y} = \mathbf{X}\boldsymbol{w}^* + \boldsymbol{\epsilon}$, with noise of variance σ^2 on each target
- Ridge regression minimises $\|\mathbf{X}\boldsymbol{w} - \boldsymbol{y}\|^2 + \eta\|\boldsymbol{w}\|^2$, giving

$$\hat{\boldsymbol{w}} = \left(\mathbf{X}^\mathsf{T}\mathbf{X} + \eta\mathbf{I}\right)^{-1}\mathbf{X}^\mathsf{T}\boldsymbol{y}$$

- $\eta = 0$ is least squares: unbiased, but how variable?
- Use the SVD $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\mathsf{T}$ and work in the basis of the columns of $\mathbf{V}$, where everything separates

Ridge Through the SVD

- Write $w_k^* = \boldsymbol{v}_k^\mathsf{T}\boldsymbol{w}^*$ for the component of $\boldsymbol{w}^*$ along $\boldsymbol{v}_k$; since $\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y} = s_k w_k^* + \boldsymbol{u}_k^\mathsf{T}\boldsymbol{\epsilon}$, the same component of the estimate is

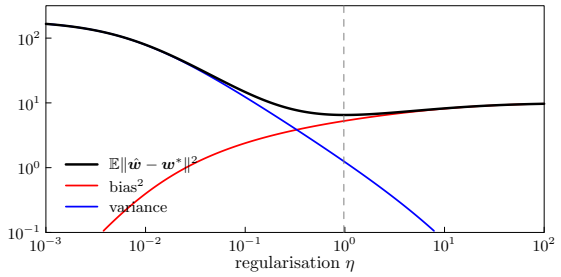
$$\hat{w}_k = \frac{s_k\boldsymbol{u}_k^\mathsf{T}\boldsymbol{y}}{s_k^2 + \eta}, \qquad \mathbb{E}[\hat{w}_k] = \frac{s_k^2}{s_k^2+\eta}w_k^*, \qquad \mathrm{Var}(\hat{w}_k) = \frac{\sigma^2 s_k^2}{(s_k^2+\eta)^2}$$

- So the expected squared error of the weights is

$$\mathbb{E}\left[\|\hat{\boldsymbol{w}} - \boldsymbol{w}^*\|^2\right] = \sum_k \underbrace{\left(\frac{\eta}{s_k^2+\eta}\right)^2 w_k^{*2}}_{\text{bias}^2} + \underbrace{\frac{\sigma^2 s_k^2}{(s_k^2+\eta)^2}}_{\text{variance}}$$

- With $\eta = 0$ the variance is $\sum_k \sigma^2/s_k^2$: small singular values (ill-conditioning) make it huge! η shrinks exactly those directions, trading a little bias for a large drop in variance

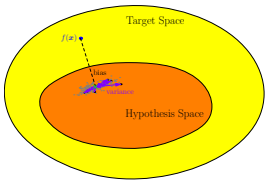
Choosing η



- Ten singular values from 3 down to 0.1, $\sigma = 1$: least squares has error 188, the best $\eta \approx 1$ gives 6.5
- In practice choose η by cross-validation

Outline

1. What Makes a Good Learning Machine?
2. Bias-Variance Dilemma
3. Shrinkage and Ridge Regression
4. **Ensembles**



Averaging Reduces Variance

- An **ensemble** averages M predictors, $\bar{f}_M(\boldsymbol{x}) = \frac{1}{M}\sum_{i=1}^M f_i(\boldsymbol{x})$, trained with different randomness
- Averaging does not change the mean, so the bias is unchanged
- If each $f_i(\boldsymbol{x})$ has variance s^2 and each pair has correlation ρ

$$\mathrm{Var}\big(\bar{f}_M(\boldsymbol{x})\big) = \rho s^2 + \frac{1-\rho}{M}s^2$$

- More members remove the second term, but correlation sets a floor ρs^2
- So we want members that are good **and** different

Bagging, Forests and Deep Ensembles

- **Bagging**: train each member on a bootstrap resample of the data (sampled with replacement)
- **Random forests**: bagged decision trees that also choose each split from a random subset of the features—each tree is worse, but the trees are less correlated
- **Deep ensembles**: train several networks on the same data from different random initialisations
- The spread of the members' predictions is a useful measure of uncertainty
- Ensembles of high-variance, low-bias machines are among the most reliable methods in practice

Lessons

- The expected generalisation loss splits into bias plus variance
- One of the most useful intuitions about what works is the bias-variance framework
- A biased estimate can beat an unbiased one: shrinkage, ridge regression
- The bias is high for simple machines that can't capture the data
- The variance is high for complex machines that are sensitive to the training set
- Averaging (ensembles) reduces variance without adding bias

Mathematics of Machine Learning

Over-Fitting

Overfitting, regularisation, feature selection

Outline

1. **Over-fitting?**
2. Controlling Complexity
3. Hidden structure
4. Regularisation
5. Double Descent

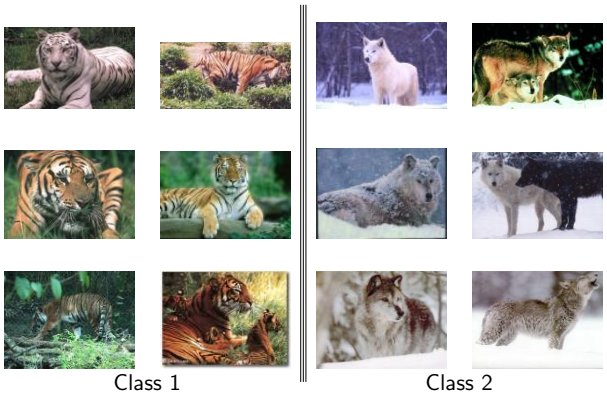
Over-fitting

- Complex machines can **over-fit**

  ***over-fitting**: fitting the training data well at the cost of getting poorer generalisation performance*

- Three red cars. . .
- If we used an infinitely flexible machine we can fit our training data perfectly, but would have no generalisation ability

Binary Classification Task for You



Class 1 Class 2

Which Category?

- Which category does the following image belong to?

Spurious Rules

- You ask a learning machine to solve a task based on data
- It will find a rule that does this, but not necessarily the rule you had in mind—machine learning can't read your mind
- Infinitely flexible machines have an infinity of spurious rules they can learn—they are useless
- What should we do?

All Binary Functions

$$\mathcal{D} = \{(000, 0), (010, 1), (110, 1), (001, 0), (101, 0)\}$$

$x_0 = 000$ $y_0 = \begin{cases} 0 \\ \text{✗} \end{cases}$

$x_1 = 100$ $y_1 = \begin{cases} 0 \\ 1 \end{cases}$ unseen

$x_2 = 010$ $y_2 = \begin{cases} \text{✗} \\ 1 \end{cases}$

$x_3 = 110$ $y_3 = \begin{cases} \text{✗} \\ 1 \end{cases}$

$x_4 = 001$ $y_4 = \begin{cases} 0 \\ \text{✗} \end{cases}$ seen

$x_5 = 101$ $y_5 = \begin{cases} 0 \\ \text{✗} \end{cases}$

$x_6 = 011$ $y_6 = \begin{cases} 0 \\ 1 \end{cases}$

$x_7 = 111$ $y_7 = \begin{cases} 0 \\ 1 \end{cases}$ unseen

Are MLPs Universal Approximators?

- Yes and No
- Yes: If you give me any (continuous) function, I can find an MLP that approximates that function to any desired accuracy (on a bounded region)
- No: If you give me an MLP, I can find a function with an arbitrary high approximation error
- Would an MLP that could approximate any function be useful?
- **Absolutely not!**

Outline

1. Over-fitting?
2. **Controlling Complexity**
3. Hidden structure
4. Regularisation
5. Double Descent

Controlling Complexity

- Infinitely flexible machines don't generalise (because any unseen data could have any value)
- **Machine learning only works because there is some structure in the data**
- A successful machine should capture this structure
- Even deep learning machines with millions or billions of parameters only work because they successfully capture the structure of images or text
- Different learning machines have different performance on different problems because the data has different structure

Training Examples

- As we increase the number of training examples, we make it harder to find a spurious rule
- Bigger data sets allow us to use more complicated machines
- Part of the success of deep learning is because it uses huge training sets—but this is only a part of their success
- (Labelled) data is often expensive to collect so we sometimes have no choice but to use a small training set
- One of the limitations of using deep learning comes because we often have limited data

Preprocessing

- Structure might often be obscure to the learning machine
- If we are trying to predict the spread of disease then a list of place names might be a lot less useful than their coordinates
- Imposing an ordering on an unordered set might **not** be useful

$$\{\text{“blue”} : 0,\ \text{“brown”} : 1,\ \text{“green”} : 2,\ \text{“black”} : 3\}$$

- Choosing an encoding that reflect meaningful structure is essential to successful machine learning

Outline

1. Over-fitting?
2. Controlling Complexity
3. **Hidden structure**
4. Regularisation
5. Double Descent

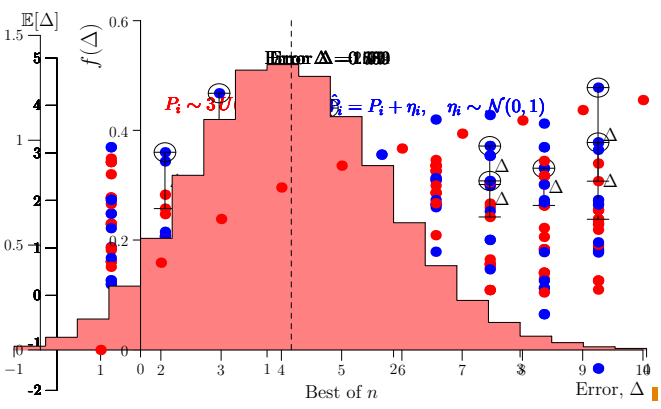
Hidden Structure

- Often the structure of data is invisible to us
- A very successful strategy is to try many different machine learning techniques and choose the best (stupid but effective)
- Often learning machines have adjustable parameters (hyper-parameters) that we have to set (they are the same for all input data, but change with the problem)
- We need to choose the hyper-parameters to fit the data in our problem
- Fine tuning hyper-parameters is important and almost always required (true in SVMs, MLP, deep learning)

Measuring Generalisation Performance

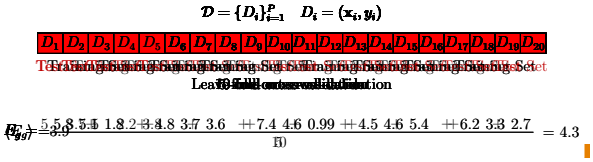
- Recall, we want to predict **unseen** data
- **You cannot use data that you have trained on!**—you will overfit
- Need to split your data up into training and validation set
- Use the validation set to choose the hyper-parameters
- You need a separate testing set if you want to measure your generalisation performance

The Overfitting Game



Cross Validation

- If you want to use more data for training then you can use cross validation
- K-fold cross validation splits the data into K groups

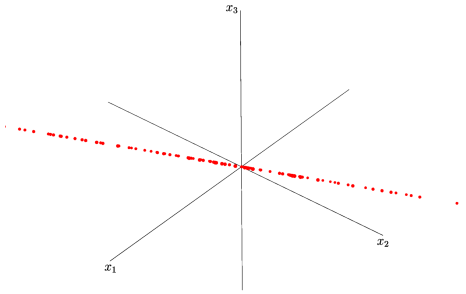


$$E_{\text{cv}} = \frac{5.5 + 8.5 + 1.8 + 4.8 + 3.7 + 3.6 + 7.4 + 4.6 + 0.99 + 4.5 + 4.6 + 5.4 + 6.2 + 3.3 + 2.7}{15} = 4.3$$

- Leave-one-out cross-validation is extreme case

Hidden Structure

Can't spot low dimensional data by looking at numbers



Feature Selection

- Spurious features will allow us to find spurious rules (**over-fitting**)
- We can try different combinations of features to find the best set, although it rapidly becomes intractable to do this in all ways
- We can use heuristics to decide which features to keep (none is fail-safe), or project onto a few principal components (PCA)
- Feature selection however can be powerful, often we can get very good results by keeping only a few variables
- As well as possibly improving generalisation we also get a more **interpretable** rule

Normalising Features

- Measuring a feature in millimeters or kilometers is going to make a lot of difference to the size of that feature
- Many learning algorithms are sensitive to the size of a feature (larger features are more important)
- If we don't know how important different features are then it makes sense to normalise our features. E.g.

$$x_i^\alpha \leftarrow \frac{x_i^\alpha - \hat{\mu}_i}{\hat{\sigma}_i}, \quad \hat{\mu}_i = \frac{1}{m}\sum_{\beta=1}^{m} x_i^\beta, \quad \hat{\sigma}_i^2 = \frac{1}{m-1}\sum_{\beta=1}^{m}(x_i^\beta - \hat{\mu}_i)^2$$

Outline

1. Over-fitting?
2. Controlling Complexity
3. Hidden structure
4. **Regularisation**
5. Double Descent

Explicit Regularisation

- As in ridge regression, we can modify our error function to choose smoother functions

$$L = \sum_{k=1}^{m}\left(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_k - y_k\right)^2 + \eta\|\boldsymbol{w}\|^2$$

(Good to normalise features)

- Second term is minimised when $w_i = 0$
- If w_i is large then

$$f(\boldsymbol{x}|\boldsymbol{w}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} = \sum_{i=1}^{p} w_i x_i$$

varies rapidly as we change x_i

Regularisers Are Priors

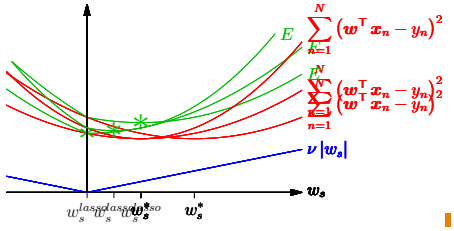
- Recall MAP: maximise $\log f(\mathcal{D}|\boldsymbol{w}) + \log f(\boldsymbol{w})$, so $-\log f(\boldsymbol{w})$ is a regulariser
- Gaussian noise of variance σ^2 and a Gaussian prior $w_i \sim \mathcal{N}(0, s^2)$ give ridge regression with $\eta = \sigma^2/s^2$
- A Laplace prior $f(w_i) \propto \mathrm{e}^{-|w_i|/b}$ gives the **lasso** penalty $\eta\sum_i |w_i|$
- The prior says what we believe before seeing data: small weights, or few non-zero weights
- Regularisation is how we tell the machine about the structure we expect

Lasso

- We can use other regularisers

$$L = \sum_{k=1}^{m}\left(\boldsymbol{w}^\mathsf{T}\boldsymbol{x}_k - y_k\right)^2 + \eta\sum_{i=1}^{p}|w_i|$$

- Spurious features (e.g. colour of flag on energy consumption) will give us a small improvement in training error



Early Stopping Is Implicit Ridge

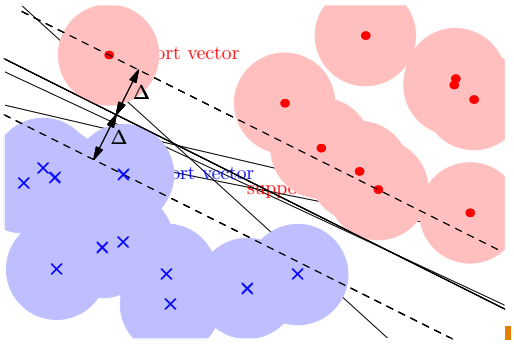
- Some machines regularise without an explicit penalty
- Gradient descent on $\frac{1}{2}\|\boldsymbol{X}\boldsymbol{w} - \boldsymbol{y}\|^2$ from $\boldsymbol{w} = \boldsymbol{0}$ with learning rate r: in the SVD basis, after t steps component k is

$$\left(1 - (1 - r s_k^2)^t\right) \times \text{(least squares component)}$$

- Ridge regression multiplies the same component by $s_k^2/(s_k^2 + \eta)$
- Both keep the directions with large s_k and suppress those with small s_k, where the variance lives
- So stopping early acts like ridge with $\eta \approx 1/(rt)$: training time is a regulariser

Maximum Margin Machines

- Perceptrons have many options to separate data



- SVMs choose the machine with the biggest margins

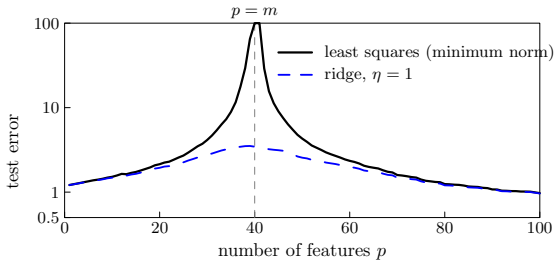
Why SVMs Generalise

- For data in a ball of radius R separated with margin γ, the capacity of the machine depends on $(R/\gamma)^2$, not on the dimension of the feature space
- That is why the kernel trick, with enormous (even infinite) feature spaces, need not over-fit
- Removing a training point that is not a support vector leaves the solution unchanged, so the leave-one-out error is at most (number of support vectors)$/m$
- Few support vectors and a large margin mean good generalisation
- But you still need to normalise the features and tune C (and the kernel parameters)

Outline

1. Over-fitting?
2. Controlling Complexity
3. Hidden structure
4. Regularisation
5. **Double Descent**

Double Descent



- Least squares on $m = 40$ points using p of 100 features
- The classical U-curve, a spike at $p = m$, then a second descent

Why Bigger Can Be Better

- At $p = m$ there is exactly one fit through every (noisy) point; the smallest singular value is near 0, so the variance $\sum_k \sigma^2/s_k^2$ explodes
- For $p > m$ there are infinitely many exact fits; we take the one with the smallest $\|\boldsymbol{w}\|$
- Gradient descent from $\boldsymbol{w} = \boldsymbol{0}$ finds exactly this minimum-norm solution: an implicit regulariser
- Modern networks have far more parameters than data, yet trained with (S)GD they often generalise well
- Counting parameters is not the same as measuring complexity

Lessons

- Machine learning isn't magic
- It works when the learning machine is well attuned to the problem
- Sometimes you can help by preprocessing your data
- Regularisers (explicit, priors, or implicit like early stopping and minimum norm) select simpler machines
- Most machines have hyper-parameters that you tune to match the machine to the data
- Complexity is not just the number of parameters: margins and norms matter

Mathematics of Machine Learning

Symmetry

Inductive Bias, Symmetry, Invariance, Group theory

Outline

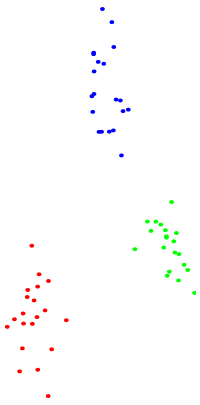
1. **Inductive Bias**
2. Invariance
3. Group Theory
4. Symmetry in Machine Learning

Inductive Bias

- Learning machines differ in how they split up the space of inputs
- Perceptrons split the space using a hyperplane, RBFs split the space depending on some set of centres, MLPs successively split the world at different layers
- These differences are known as the **inductive bias** of the machine
- The different inductive biases lead to different performance
- How can we exploit the inductive bias to get good performance?

The Real World

- The problems that we care about come from a highly structured world where the solutions inherit some of the properties of that world
- Usually similar input have similar outputs (we don't expect to change one pixel in an image and that it should be classified differently)
- The classes are often quite well separated



Smoothness

- A natural assumption is smoothness (the classification surface or regression surface changes slowly as we change the input)
- Often there is a hyperparameter that controls this
  - The kernel parameter γ in SVMs (or ℓ in Gaussian Processes)
  - k in k-nearest neighbours
  - The size of the L_2 regularisation term in neural networks
- We often have to choose these using trial and error (using a validation set)

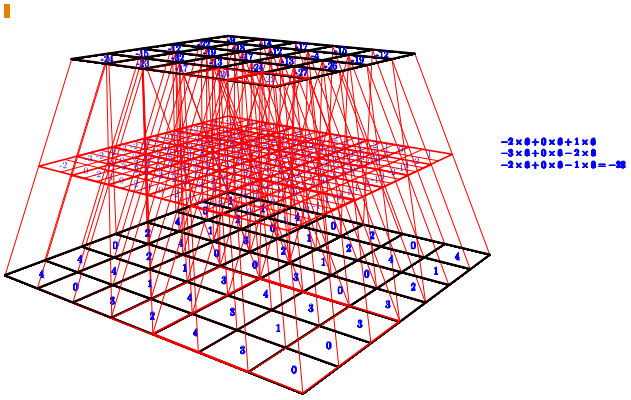
Outline

1. Inductive Bias
2. **Invariance**
3. Group Theory
4. Symmetry in Machine Learning

Symmetry

- There are sometimes symmetries in the world we can exploit
- In image classification we consider an object should be classified the same wherever it is in the image
- The classification should be invariant to translations (scaling and perhaps rotations)
- The classification function possesses a translational symmetry
- Learning machines that respect translational invariance have far less propensity to overfit—they don't need to learn that the same object in a different part of the image is the same

Convolutional Neural Networks



Equivariance

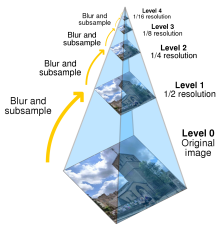
- CNNs are not translationally invariant
- The output will be different if we shift the input
- They are (up to edge effects) **equivariant** in that we would get the same result if we shifted both the input and the output in the same way
- They are not perfectly equivariant (they break at the pixel level and at the full image level—due to edges)
- But they are sufficiently good that CNNs have dominated computer vision for years (ignoring vision transformers)

Other Symmetries

- The identity of an object should not change if we move the camera further away or rotate the camera
- CNNs are not naturally scale or rotationally invariant (although there are attempts to remedy this)
- Rotation invariance doesn't always reflect "the world"—people usually stand on their feet not on their heads
- Although you want rotational invariance when interpreting aerial images

Scale Symmetry

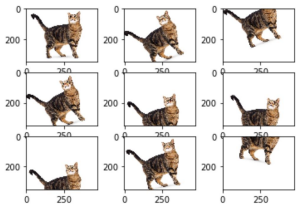
- CNNs are not scale invariant
- There are various pyramid image processing techniques to try and build in approximate scale invariance
- However, in many image classification tasks we concentrate on the object in the foreground
- Many datasets rescale objects in the image so they are roughly the same size



Learning Invariance

- Currently we are not smart enough to come up with networks that are scale invariant or rotational invariant or invariant to small distortions
- We therefore hope that our networks learns this by training on objects of different sizes, orientations, lighting conditions, etc.
- Knowing this is an invariance we want, it is now common practice to do data augmentation where we perform transformations that replicate physical plausible changes in the data
- We are removing spurious rules by (artificially) increasing our dataset size

Augmentations



- Data augmentation is standard practice in deep learning for images and makes a very considerable difference

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering (divide the length on the nose by the width between the persons eyes)
- We might do this by normalising the output—use cosine similarity rather than distance

$$\cos(\theta) = \frac{\boldsymbol{a}^\top \boldsymbol{b}}{\|\boldsymbol{a}\|\|\boldsymbol{b}\|}$$

- These make the results invariant to the distance of the camera or the length of $\boldsymbol{a}$ and $\boldsymbol{b}$

Outline

1. Inductive Bias
2. Invariance
3. **Group Theory**
4. Symmetry in Machine Learning

Group Theory

- Symmetries are very well studied in mathematics
- The language of symmetries are **groups**
- These capture the structure of transformations that represent some symmetry
- Group theory is highly abstract and it is easy to lose sight of its connection to symmetry, but it has proved itself extraordinarily powerful in revealing structures

Groups

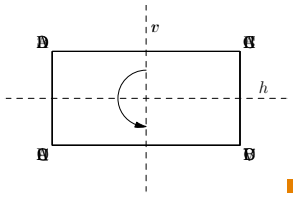
- A group consists of a set, $\mathcal{G} = \{a,b,\cdots\}$, whose elements we can think of as transformations
- And an operator "$\cdot$", usually composition: $a \cdot b$ means apply transformation b and then a
- The set and operation form a group if they satisfy four axioms
  - ⋆ closure: for all $a,b \in \mathcal{G}$ then $a \cdot b \in \mathcal{G}$
  - ⋆ associativity: $a \cdot (b \cdot c) = (a \cdot b) \cdot c$
  - ⋆ existence of an identity e such that $a \cdot e = e \cdot a = a$ for all a
  - ⋆ Every element, a, has an inverse a^{-1} such that $a^{-1} \cdot a = a \cdot a^{-1} = e$

Tossing a coin

- We consider a coin with two actions
  - ⋆ flip the coin: f
  - ⋆ leave the coin alone: e
- These form a group (the cyclic group C_2)
  - ⋆ Closure is obvious $f \cdot e = f$, $f \cdot f = e$, $e \cdot e = e$ and $e \cdot f = f$
  - ⋆ Associativity: e.g. $(e \cdot f) \cdot f = e \cdot (f \cdot f)$
  - ⋆ e is the identity
  - ⋆ $e^{-1} = e$ and $f^{-1} = f$

The Four Group

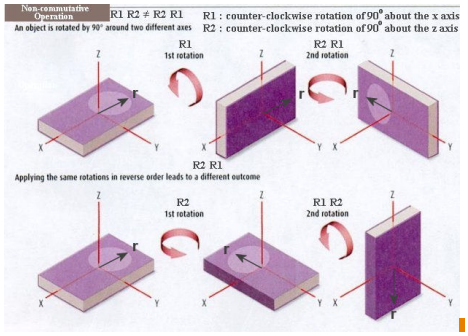
- A slightly less trivial example represents the set of transformations that leaves a rectangle invariant



	e	v	h	r
e	e	v	h	r
v	v	e	r	h
h	h	r	e	v
r	r	h	v	e

Non-commutativity

- Not all groups commute
- For 3-d rotations : $R_x \cdot R_z \neq R_z \cdot R_x$



Permutations

- The set of permutations of n objects form a group (known as the symmetric group S_n)
- The composition of a permutation is a permutation
- There is an identity (do nothing)
- For every permutation there is an inverse permutation
- Obviously S_n has $n!$ elements (you can reorder elements in n! ways)
- Permutations have a lot of structure (in terms of elements cycling, subgroups, etc.)

Infinite and Continuous Groups

- The integers under addition form a group (identity 0, inverse $-n$): the discrete shifts
- The reals under addition describe translations; the positive reals under multiplication describe scalings
- Smooth continuous groups are called **Lie groups**: rotations, translations, the symmetries of relativity
- They can be represented by matrices, and are studied through their infinitesimal transformations (Lie algebras)
- Much used in physics, and increasingly in machine learning

Rotations $SO(n)$

- The set of $n \times n$ orthogonal matrices (matrices, $\mathbf{M}$, such that $\mathbf{M}^\mathsf{T}\mathbf{M} = \mathbf{M}\mathbf{M}^\mathsf{T} = \mathbf{I}$) form a group $O(n)$
- Note that $\det(\mathbf{M}) = \pm 1$
- These correspond to rotations and reflections
- The group of matrices that don't correspond to reflections (where $\det(\mathbf{M}) = 1$) form a subgroup $SO(n)$
- $SO(2)$ consist of the 2-d rotation matrices

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

Outline

1. Inductive Bias
2. Invariance
3. Group Theory
4. **Symmetry in Machine Learning**

Invariance by Averaging

- Given any function f and a finite group G of transformations, define

$$\bar{f}(\boldsymbol{x}) = \frac{1}{|G|}\sum_{g\in G} f(g\cdot\boldsymbol{x})$$

- For any $h \in G$

$$\bar{f}(h\cdot\boldsymbol{x}) = \frac{1}{|G|}\sum_{g\in G} f(g\cdot h\cdot\boldsymbol{x}) = \frac{1}{|G|}\sum_{g'\in G} f(g'\cdot\boldsymbol{x}) = \bar{f}(\boldsymbol{x})$$

  because $g \mapsto g\cdot h$ just reorders the group (closure and inverses)
- So $\bar{f}$ is invariant—here the group axioms earn their keep
- Data augmentation approximates this in training; **test-time augmentation** does it exactly

Equivariance and Convolution

- A map is **equivariant** if transforming the input transforms the output in the same way: $f(g\cdot\boldsymbol{x}) = g\cdot f(\boldsymbol{x})$
- Convolution is shift-equivariant: shifting then convolving is the same as convolving then shifting
- In fact the linear maps that commute with all (circular) shifts are exactly the convolutions
- A 3×3 filter has 9 weights, against n^4 for a dense layer on an $n\times n$ image
- Building in the symmetry removes parameters, and with them variance (bias-variance lecture)

Permutation Symmetry

- Often the input is a **set**: points in a cloud, objects in a scene, atoms in a molecule
- Any function of the form

$$f(\{\boldsymbol{x}_1,\ldots,\boldsymbol{x}_n\}) = \rho\Big(\sum_{i=1}^{n}\boldsymbol{\phi}(\boldsymbol{x}_i)\Big)$$

  is invariant to the order (**Deep Sets**)
- Graph neural networks update each node from a sum over its neighbours: equivariant to relabelling the nodes
- The symmetric group S_n is doing the work

Transformers

- Self-attention: each token i computes a query $\boldsymbol{q}_i$, key $\boldsymbol{k}_i$ and value $\boldsymbol{v}_i$, and outputs

$$\boldsymbol{y}_i = \sum_j \mathrm{softmax}_j\Big(\frac{\boldsymbol{q}_i^{\mathsf{T}}\boldsymbol{k}_j}{\sqrt{d}}\Big)\boldsymbol{v}_j$$

- Permuting the tokens just permutes the outputs: self-attention is permutation-equivariant
- So on its own it cannot see word order—transformers add **positional encodings** to break this symmetry
- Knowing the symmetry tells you what the model cannot know

Rotations and Molecules

- The energy of a molecule does not change if we rotate or move it: it is invariant to $SO(3)$ and translations
- The forces on the atoms rotate with the molecule: they are equivariant
- Equivariant networks build this in, and are now standard for learning about molecules and materials
- AlphaFold 2's structure module is built to respect rotations and translations of the protein
- This area is called **geometric deep learning**

Summary

- Invariances are important as if a machine properly captures a true invariance it can massively limit the space of functions being approximated—reducing the chances of finding spurious rules
- Often it is hard to build-in invariances into the machine learning model and practitioners will often resort to data augmentation in the hope that these invariances can be learned
- There is a mathematical language around describing the structure of symmetries: **group theory**
- Groups explain why averaging gives invariance, and underlie CNNs, graph networks, transformers and equivariant networks
- I'm not going to expect you to know any details about group theory

The Course in One Slide

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- These ideas recur: the SVD in PCA, ridge and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

Final Thoughts

- Machine learning works when the model's structure matches the structure of the world
- Priors, regularisers and symmetries are three ways of saying what that structure is
- Mathematics is the language for saying it precisely
- The methods will keep changing; the mathematics underneath them changes far more slowly